



Queensland University of Technology
Brisbane Australia

ROBOT ARMS & FORWARD KINEMATICS



Used with permission of ABB



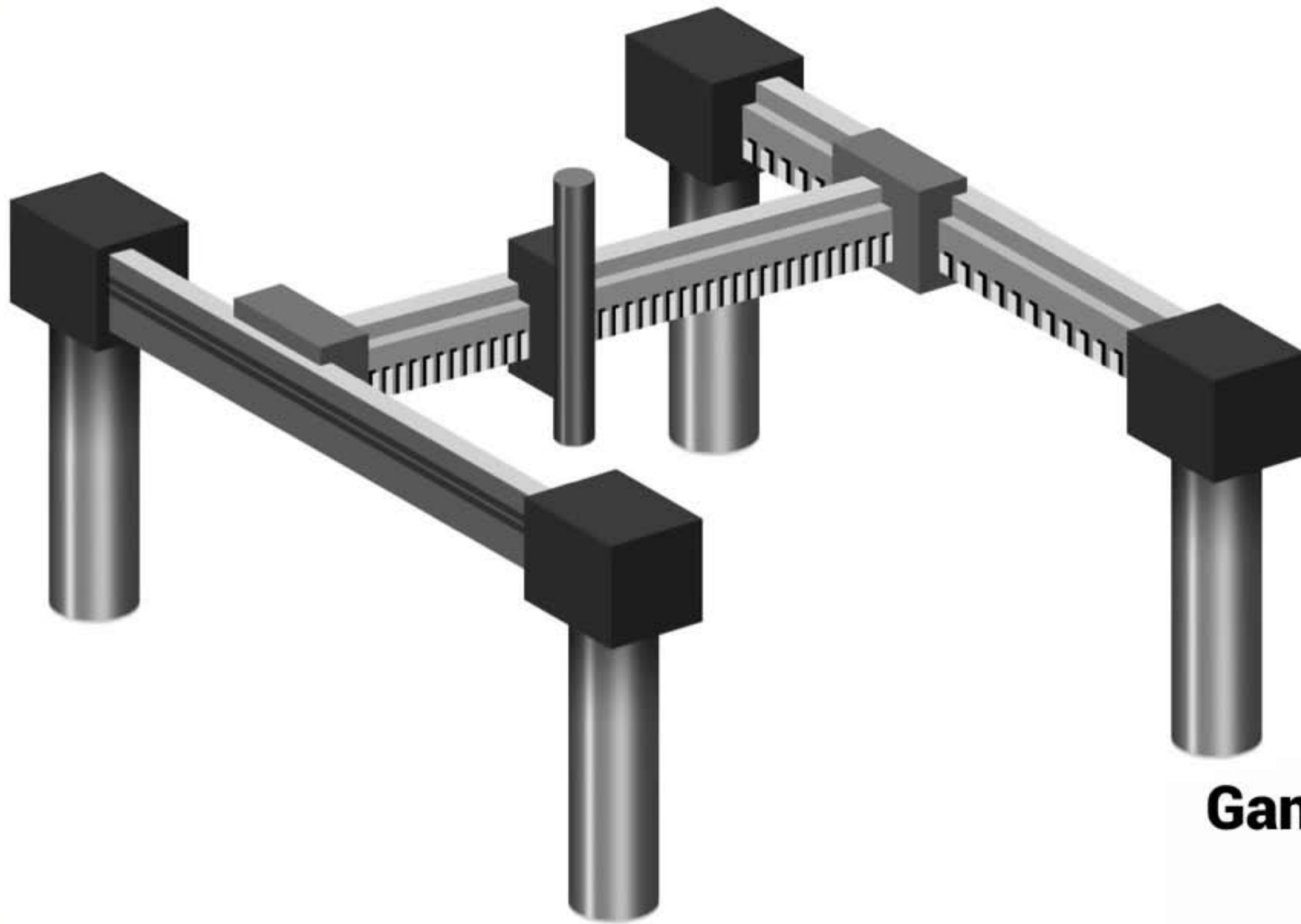
By Aldebaran Robotics via Wikimedia Commons



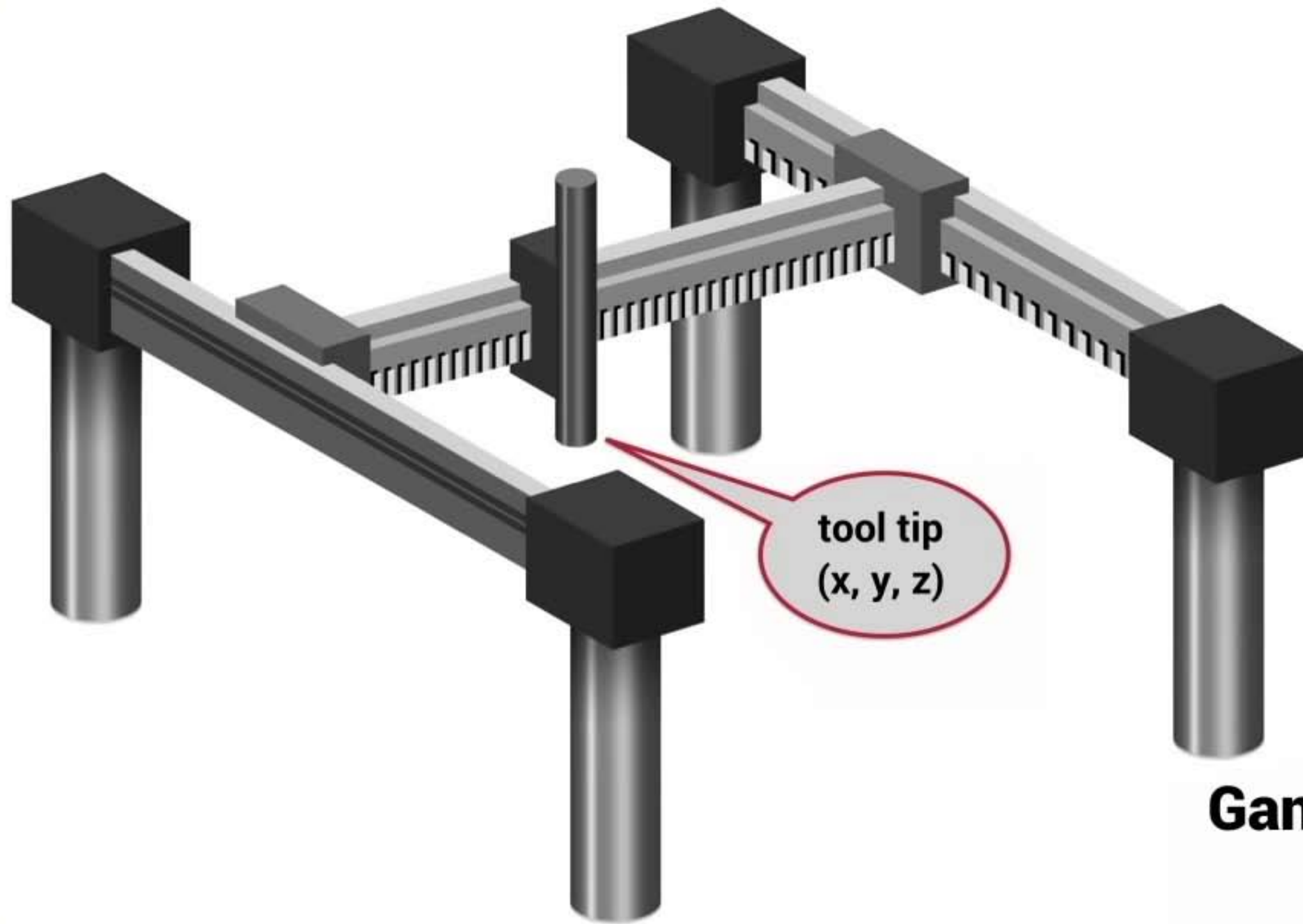
Image: NASA



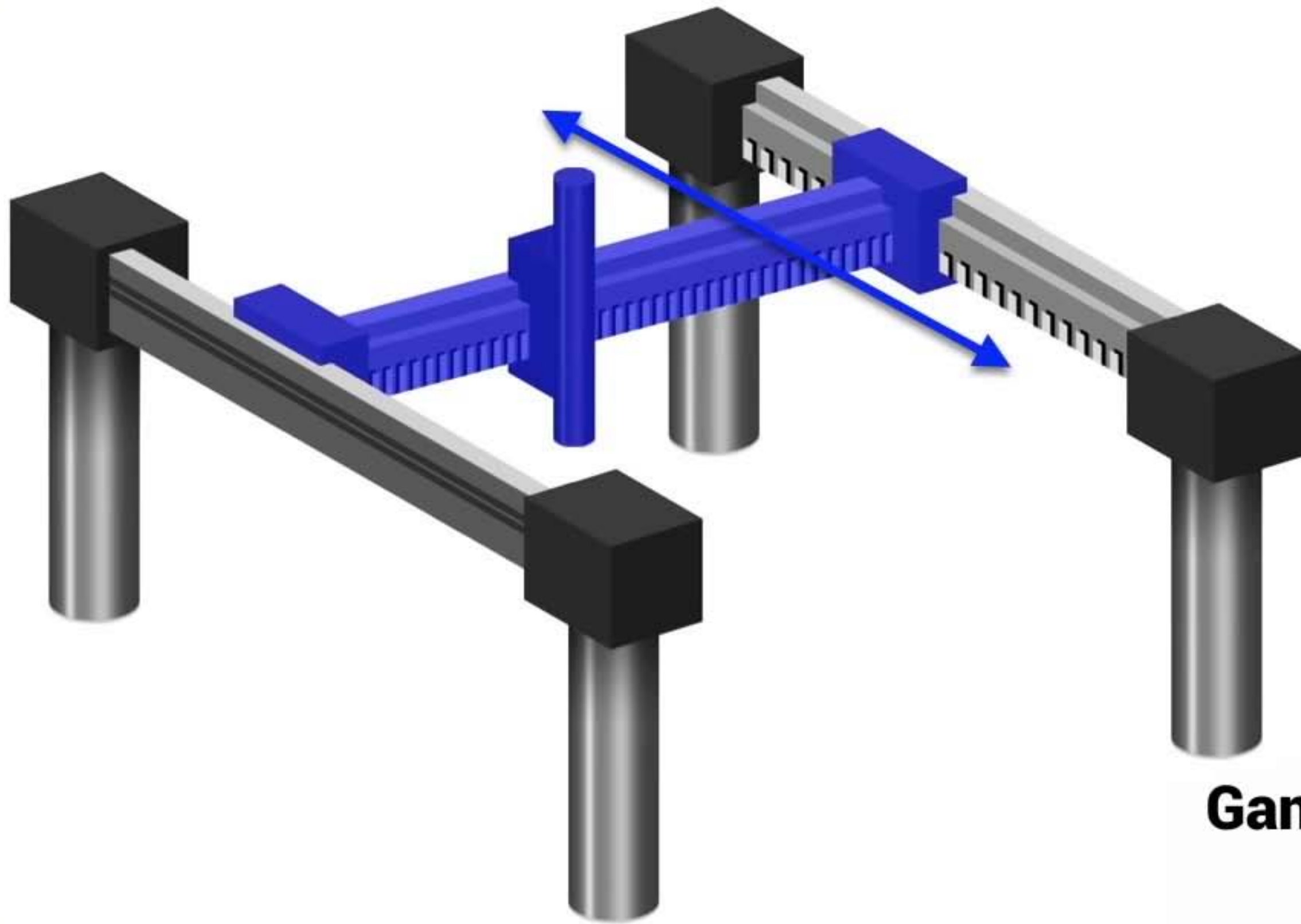
Copyright © 2014 Adept Technology, Inc.
The usage of this image in no manner is to be construed as
Adept's endorsement of QUT or any QUT product or service



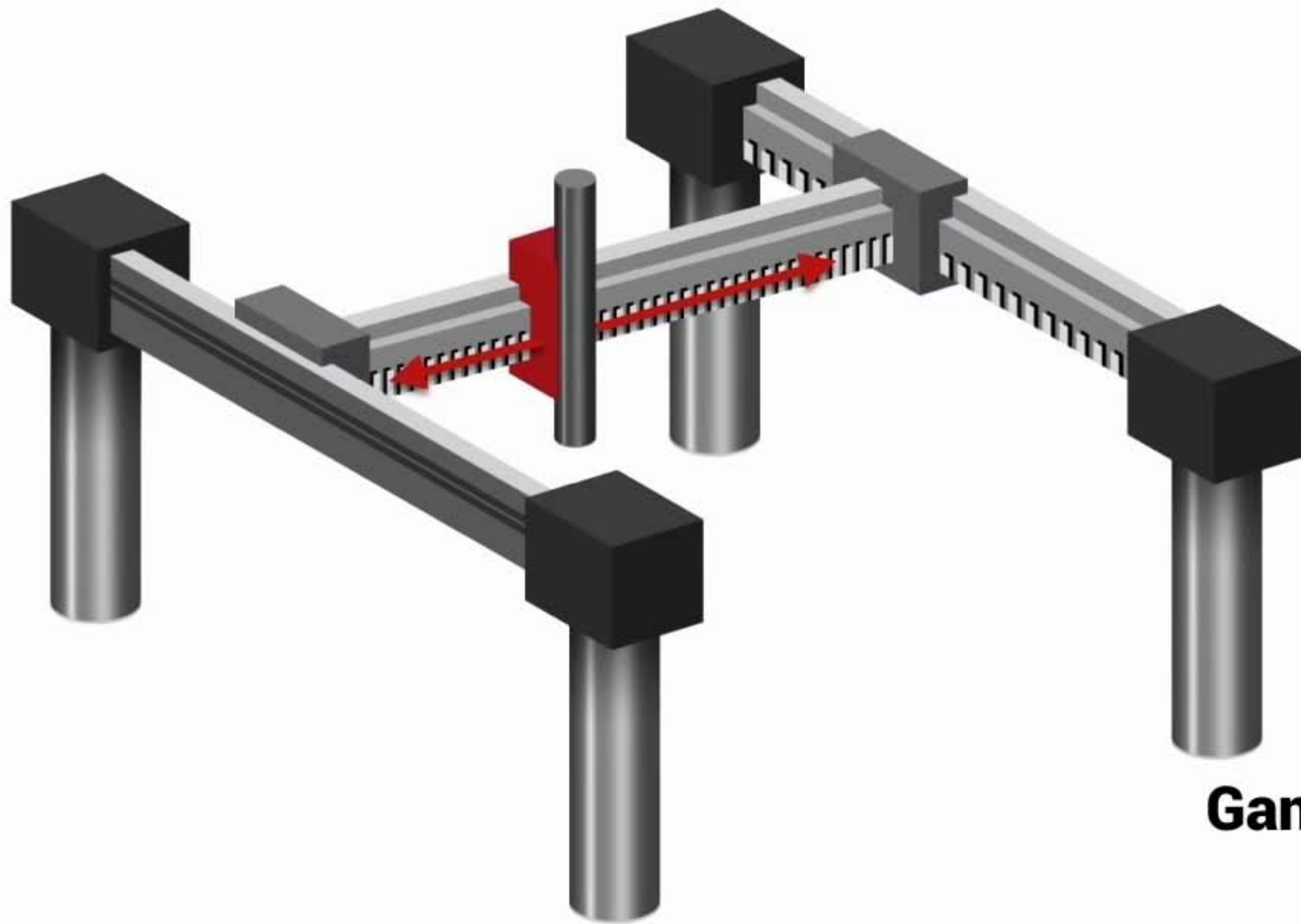
Gantry robot



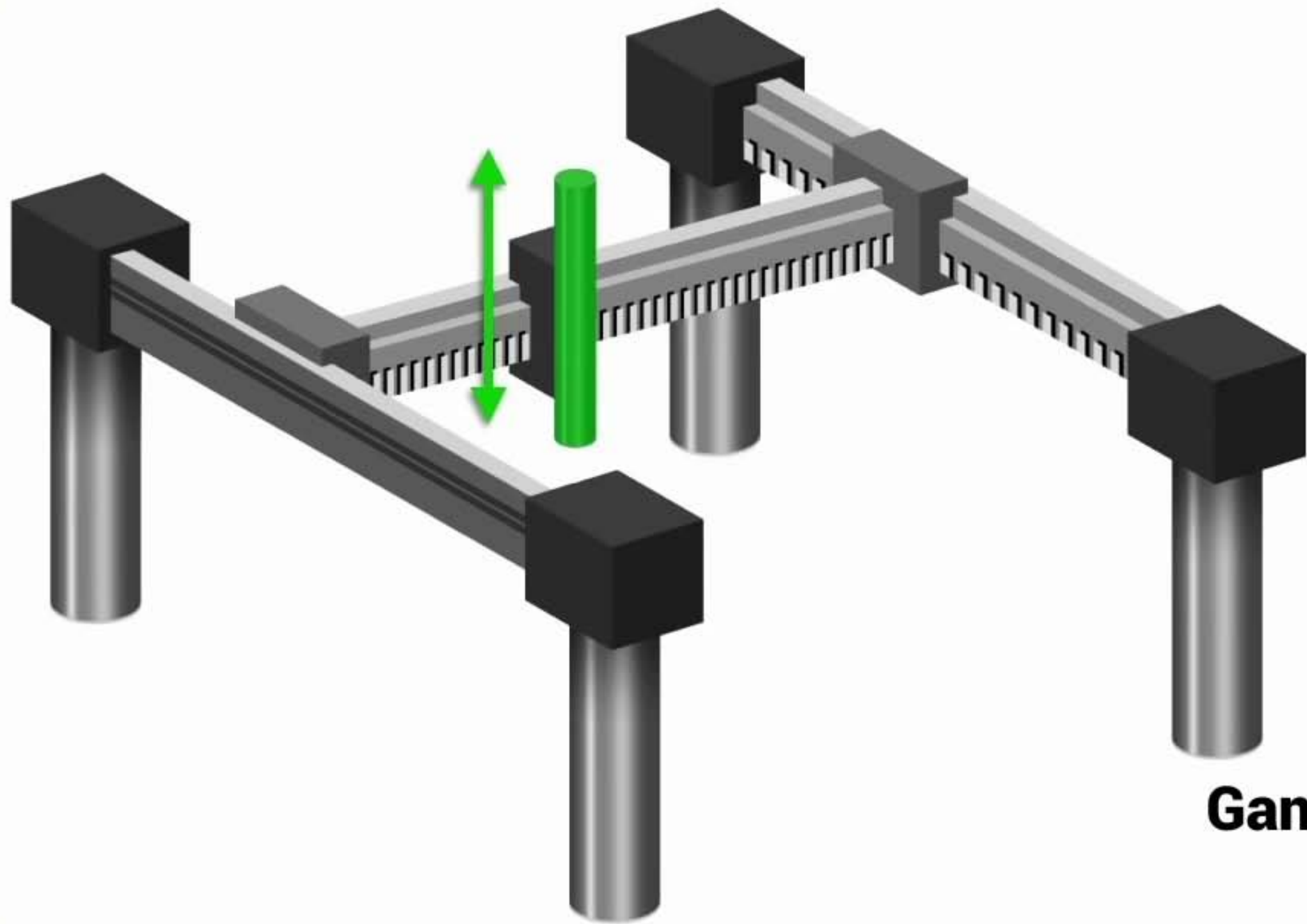
Gantry robot



Gantry robot



Gantry robot



Gantry robot



Copyright © 2014 Adept Technology, Inc.
The usage of this image in no manner is to be construed as
Adept's endorsement of QUT or any QUT product or service

**SCARA robot
1981**



Copyright © 2014 Adept Technology, Inc.
The usage of this image in no manner is to be construed as
Adept's endorsement of QUT or any QUT product or service

SCARA robot
1981



Copyright © 2014 Adept Technology, Inc.
The usage of this image in no manner is to be construed as
Adept's endorsement of QUT or any QUT product or service

SCARA robot
1981



Copyright © 2014 Adept Technology, Inc.
The usage of this image in no manner is to be construed as
Adept's endorsement of QUT or any QUT product or service

SCARA robot
1981



Selective
Compliance
Articulating
Robot

**SCARA robot
1981**

Copyright © 2014 Adept Technology, Inc.
The usage of this image in no manner is to be construed as
Adept's endorsement of QUT or any QUT product or service



Used with permission of ABB

Parallel link robot



PhantomX Pincher Robot Arm 2014
Trossen Robotics | Used with permission



By Aldebaran Robotics via Wikimedia Commons

Educational robots



PhantomX Pincher Robot Arm 2014
Trossen Robotics | Used with permission



By Aldebaran Robotics via Wikimedia Commons

Educational robots



Used with permission of ABB

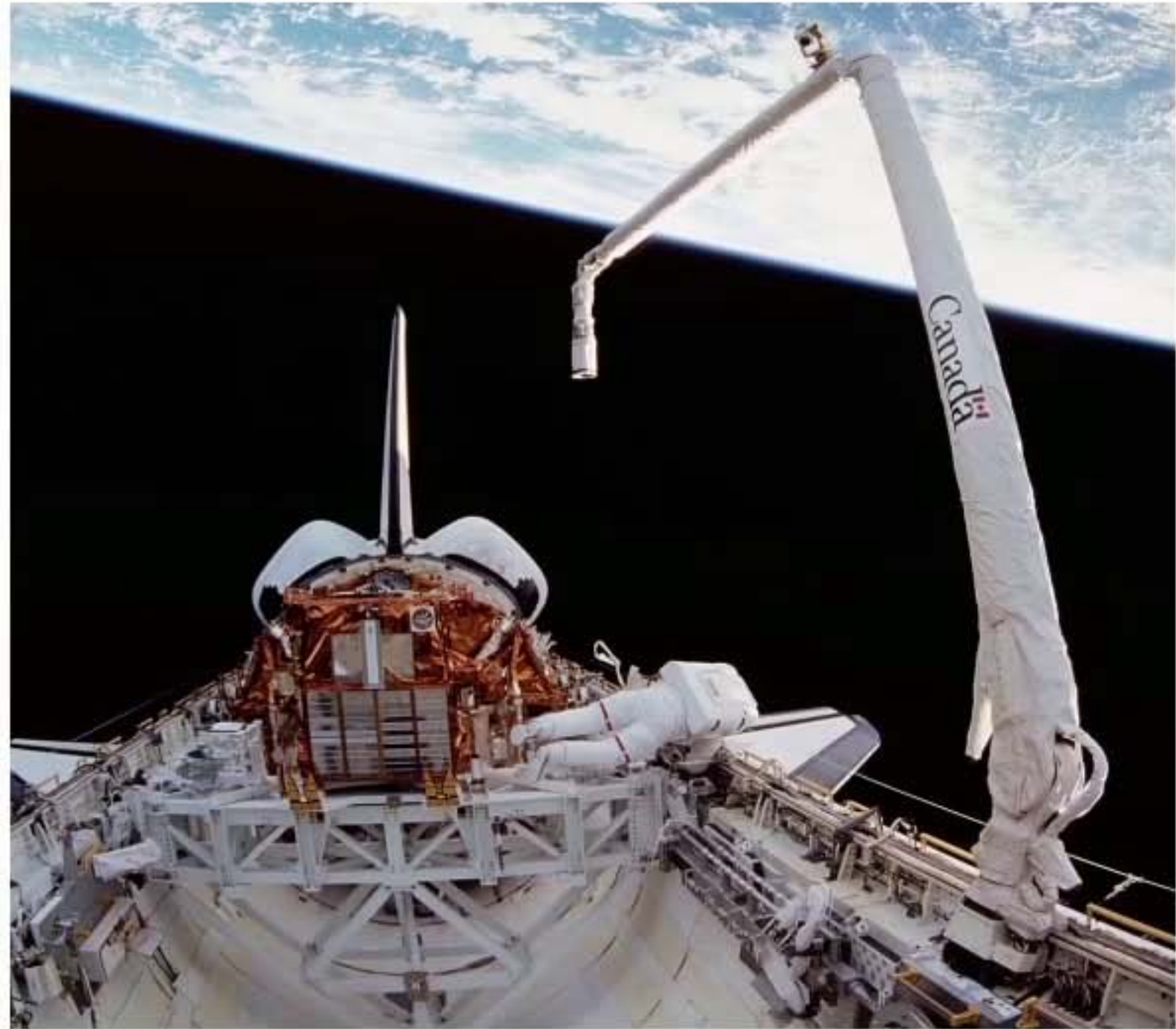


Image: NASA

6-axis serial-link robot



Stanford robotic arm
Oussama Khatib | Used with permission



SAIL Pump Assembly 1973
Stanford Artificial Intelligence Laboratory | CC BY 2.5.

Stanford arm
1969

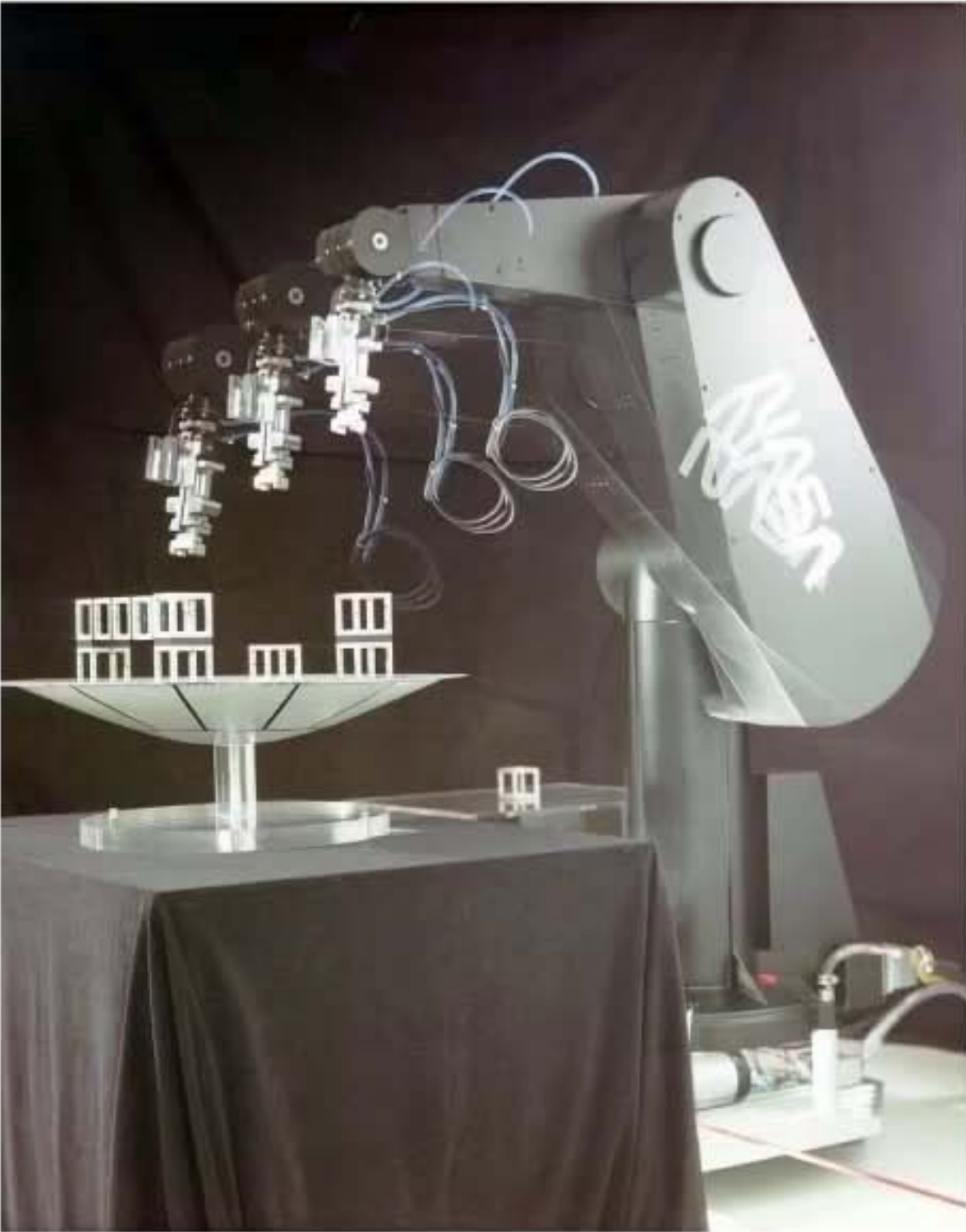


Image: NASA Dominic Hart

**Unimate robot
1978**



Unimate Puma 500
Oussama Khatib | Used with permission



Drawing with the PUMA Robot Arm 2011
Jerry Rutherford | Used with permission

**Puma 560 robot
1983**

Section 2

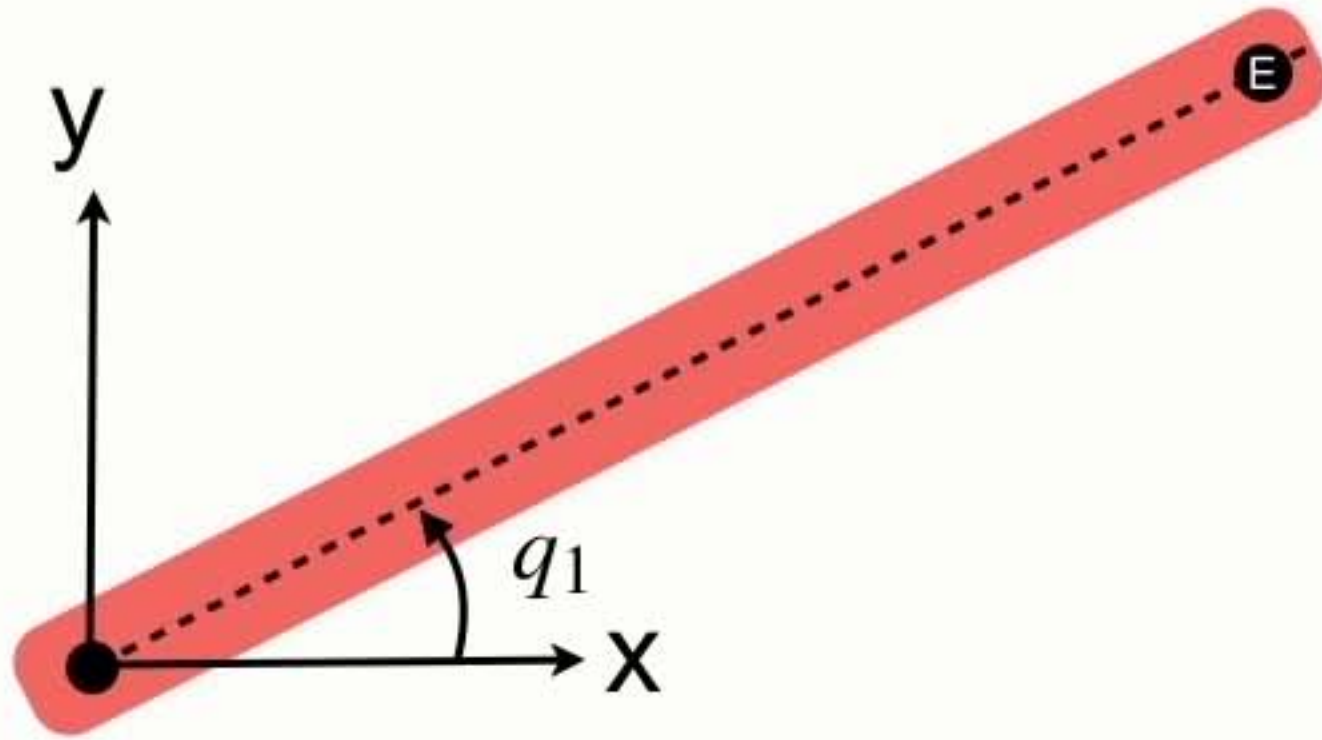
A very simple (1 joint) robot



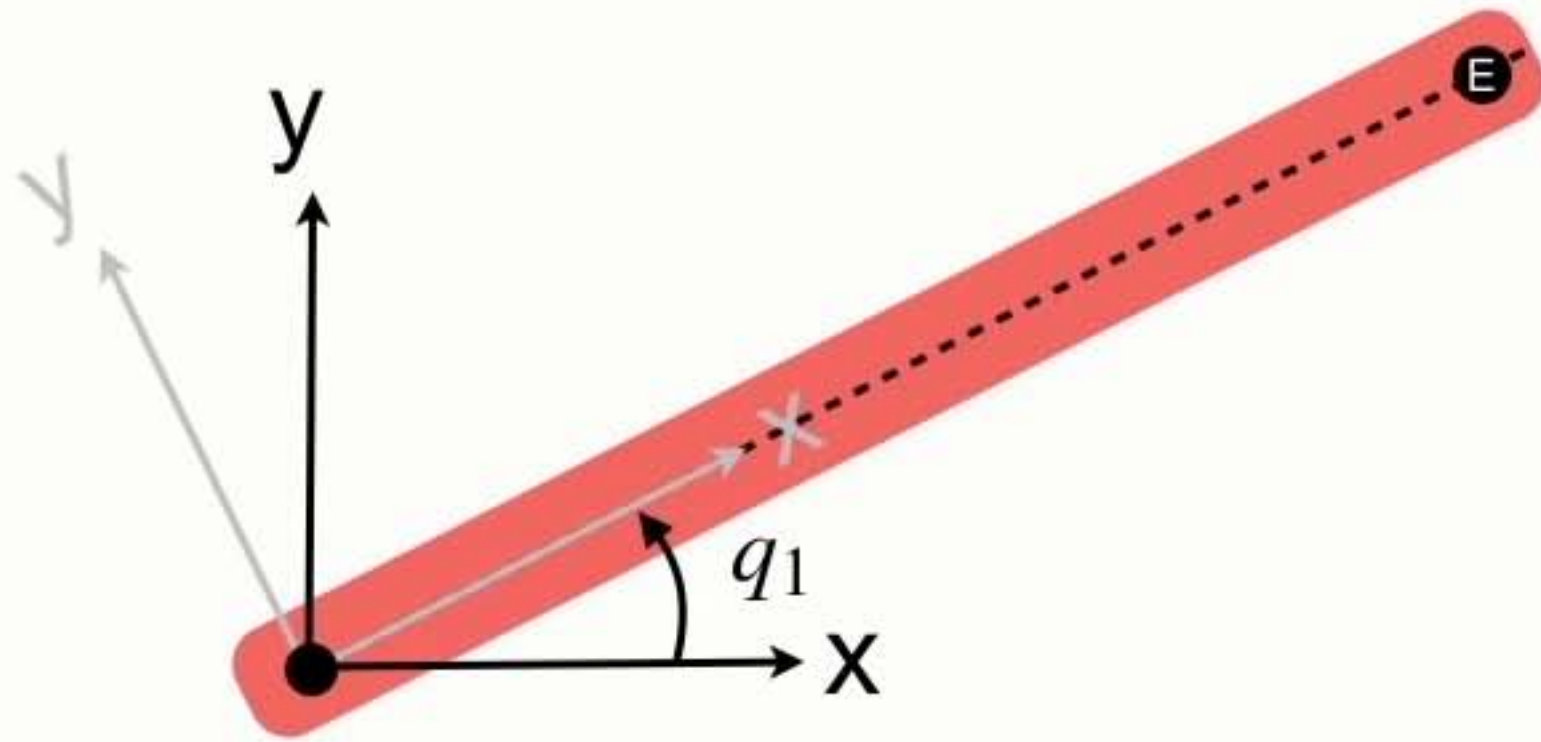




Tool tip pose

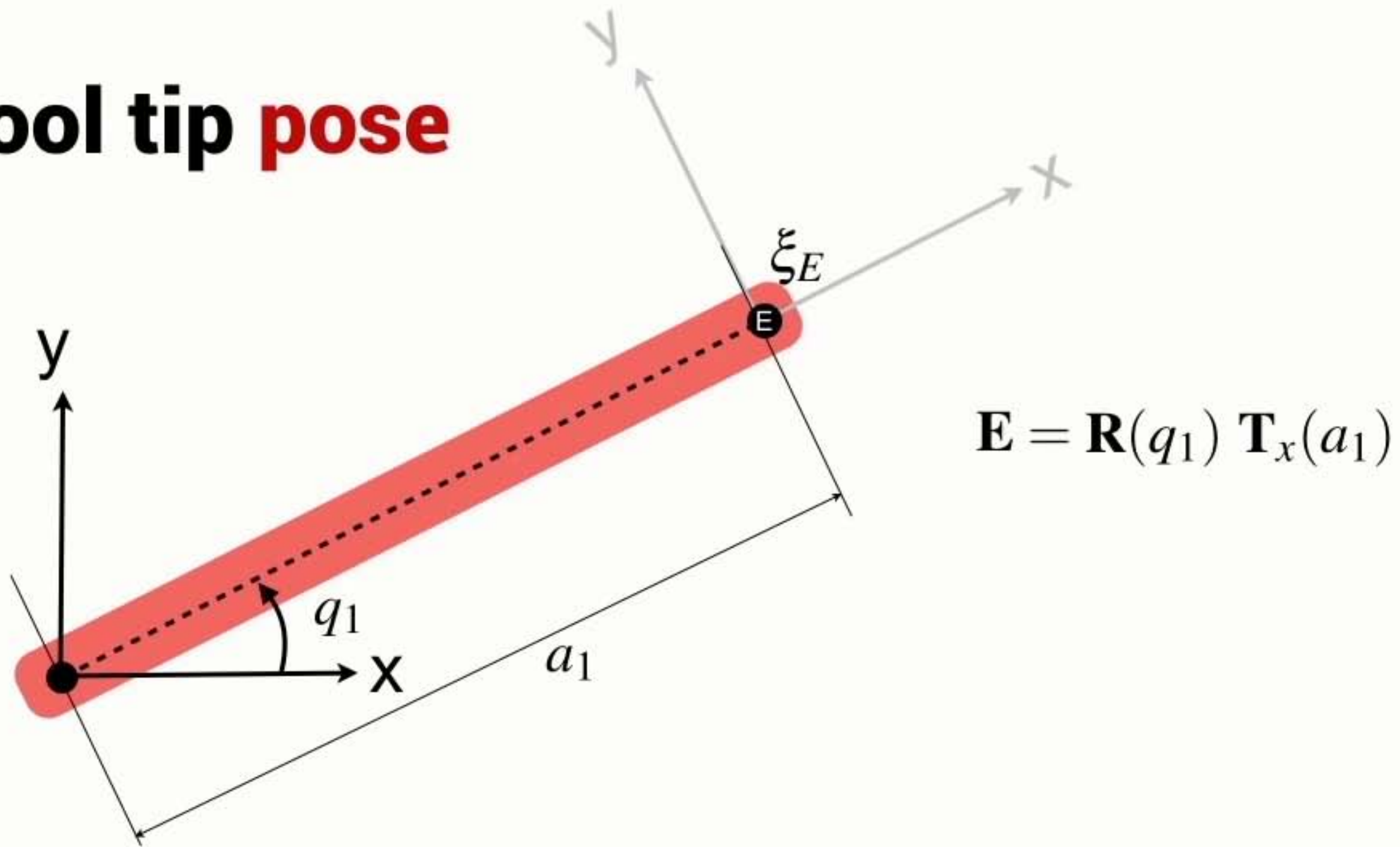


Tool tip pose

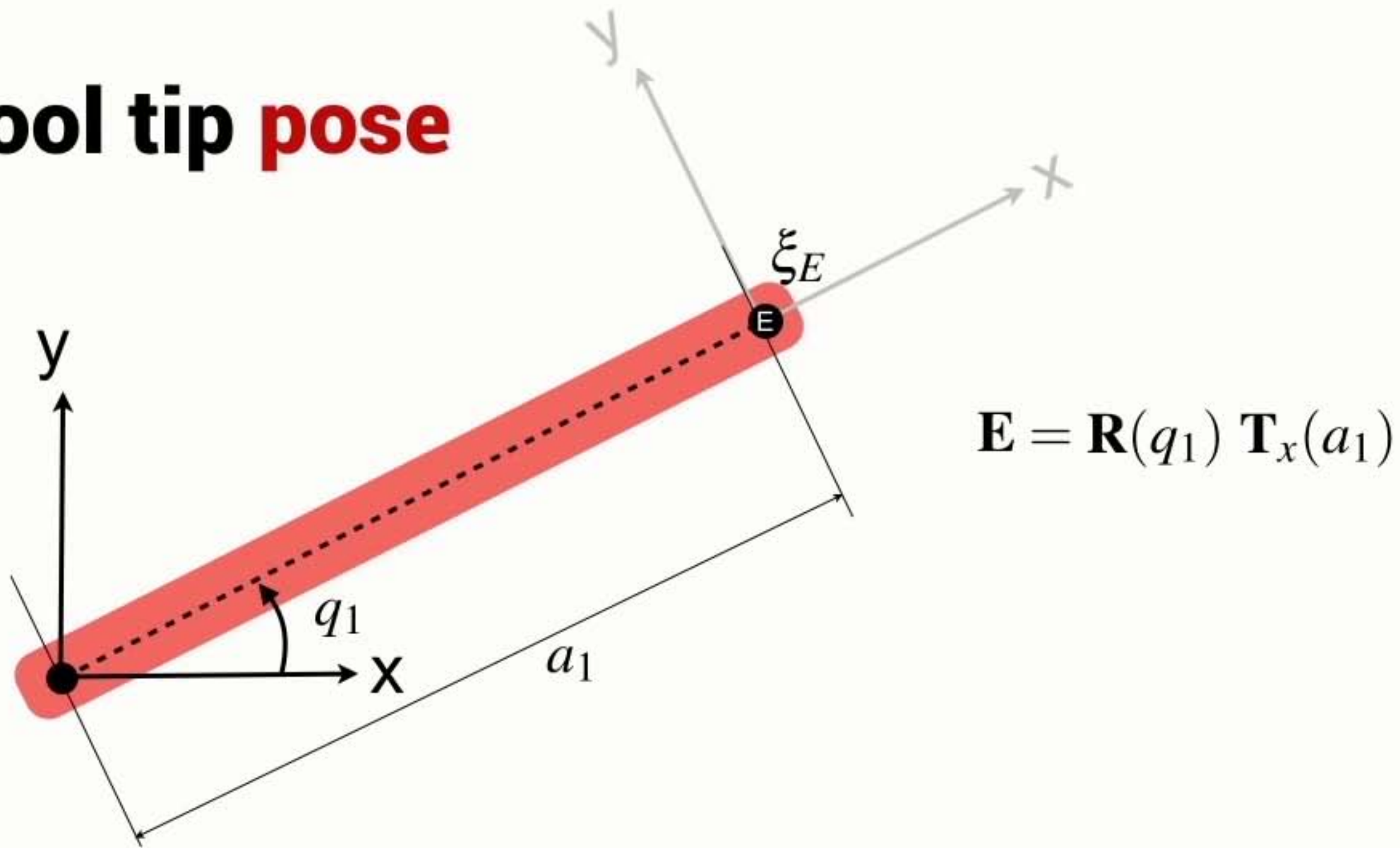


$$\mathbf{R}(q_1)$$

Tool tip pose

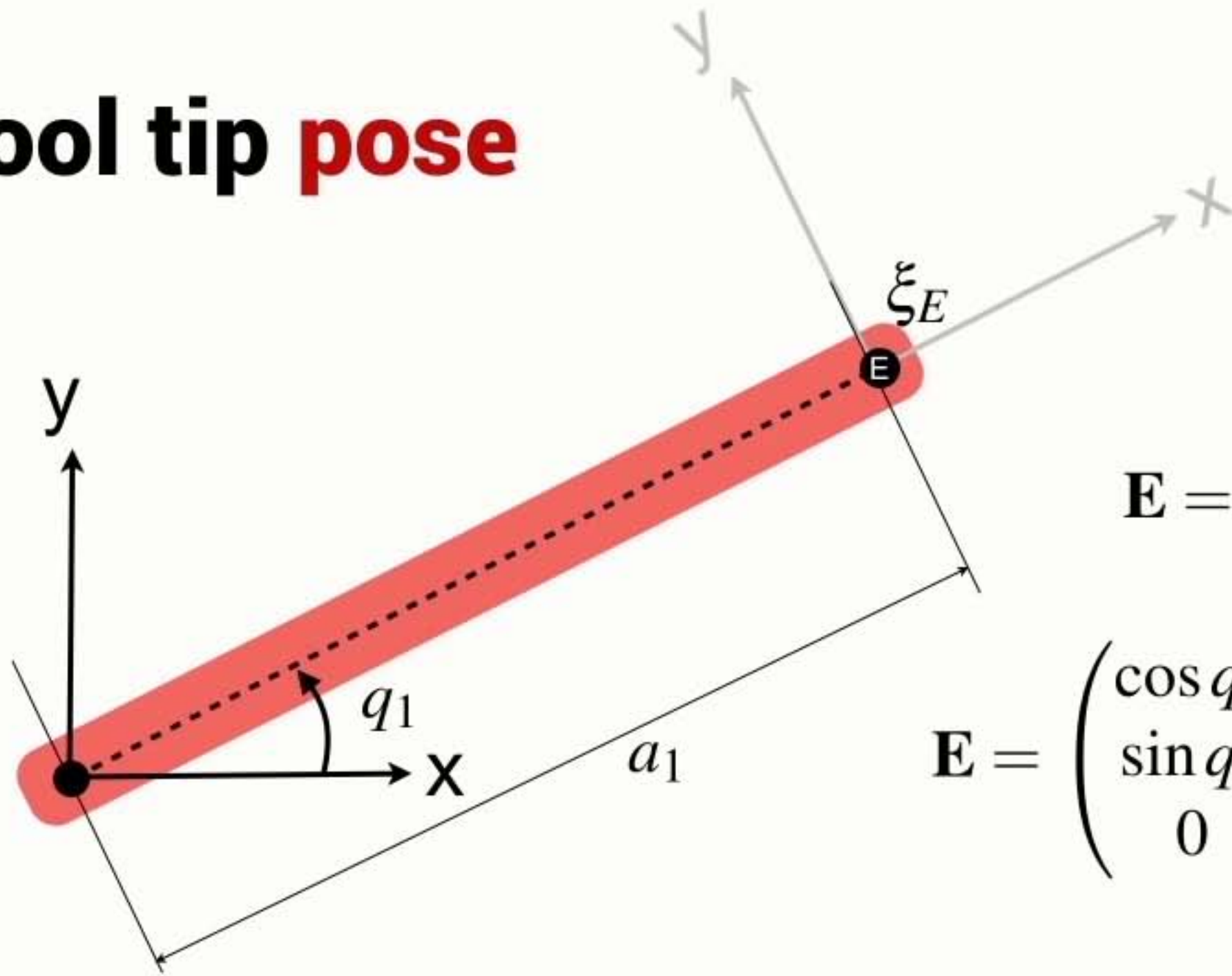


Tool tip pose



- These are all 2D transforms $\mathbf{E} \in \text{SE}(2)$

Tool tip pose

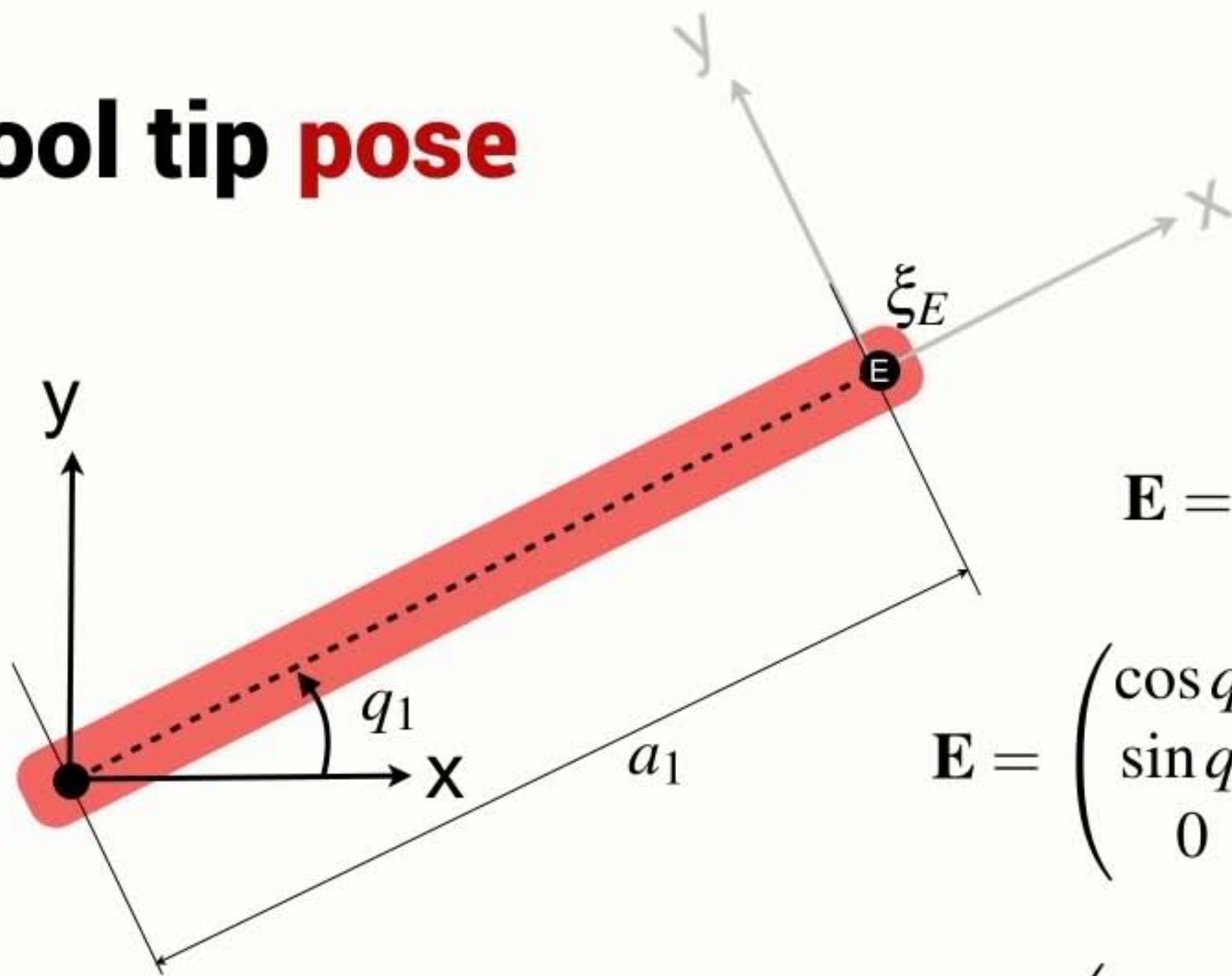


$$\mathbf{E} = \mathbf{R}(q_1) \mathbf{T}_x(a_1)$$

$$\mathbf{E} = \begin{pmatrix} \cos q_1 & -\sin q_1 & 0 \\ \sin q_1 & \cos q_1 & 0 \\ 0 & 0 & 1 \end{pmatrix} \begin{pmatrix} 1 & 0 & a_1 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{pmatrix}$$

- These are all 2D transforms $\mathbf{E} \in \text{SE}(2)$

Tool tip pose



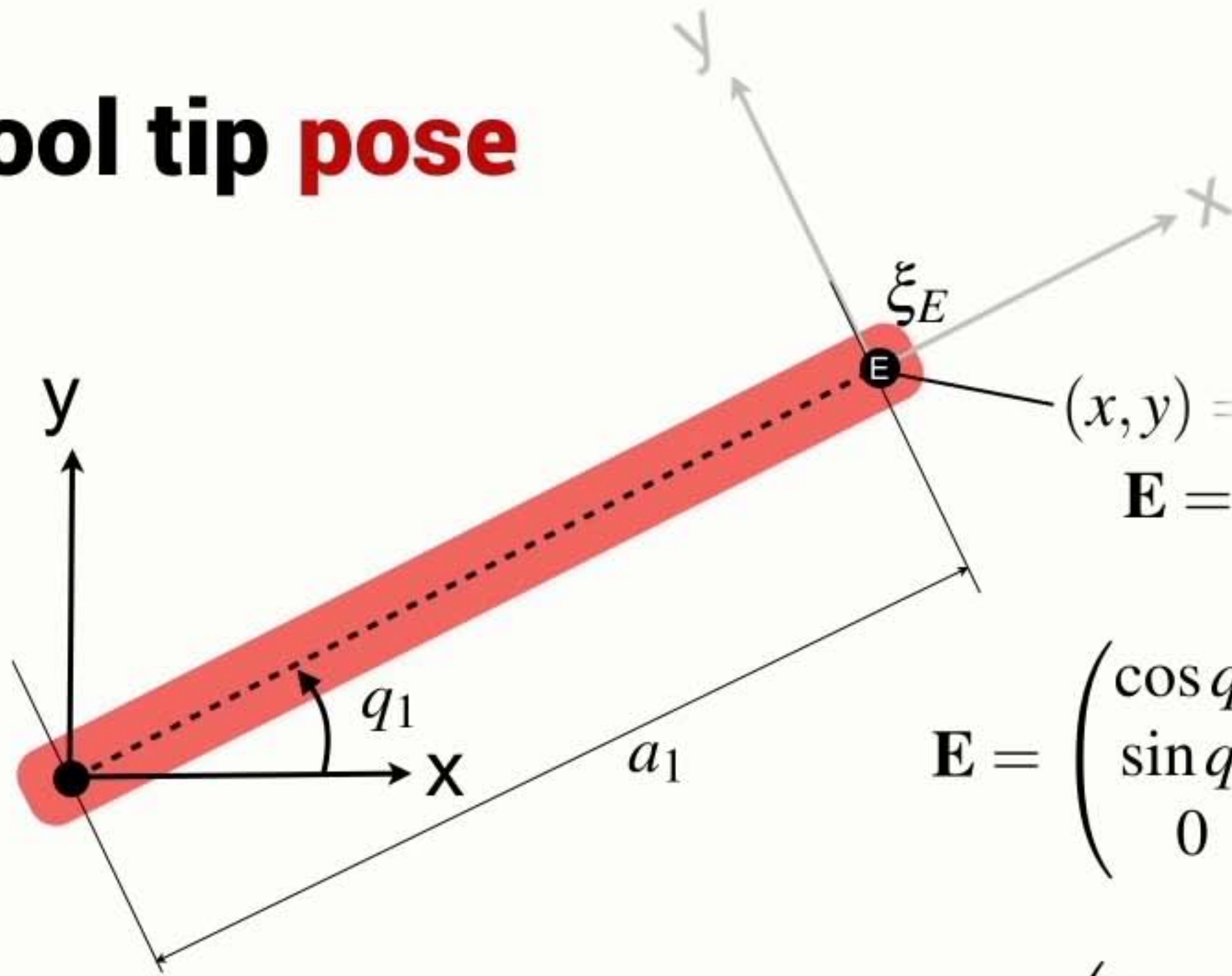
$$\mathbf{E} = \mathbf{R}(q_1) \mathbf{T}_x(a_1)$$

$$\mathbf{E} = \begin{pmatrix} \cos q_1 & -\sin q_1 & 0 \\ \sin q_1 & \cos q_1 & 0 \\ 0 & 0 & 1 \end{pmatrix} \begin{pmatrix} 1 & 0 & a_1 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{pmatrix}$$

$$\mathbf{E} = \begin{pmatrix} \cos q_1 & -\sin q_1 & a_1 \cos q_1 \\ \sin q_1 & \cos q_1 & a_1 \sin q_1 \\ 0 & 0 & 1 \end{pmatrix}$$

■ These are all 2D transforms $\mathbf{E} \in \text{SE}(2)$

Tool tip pose



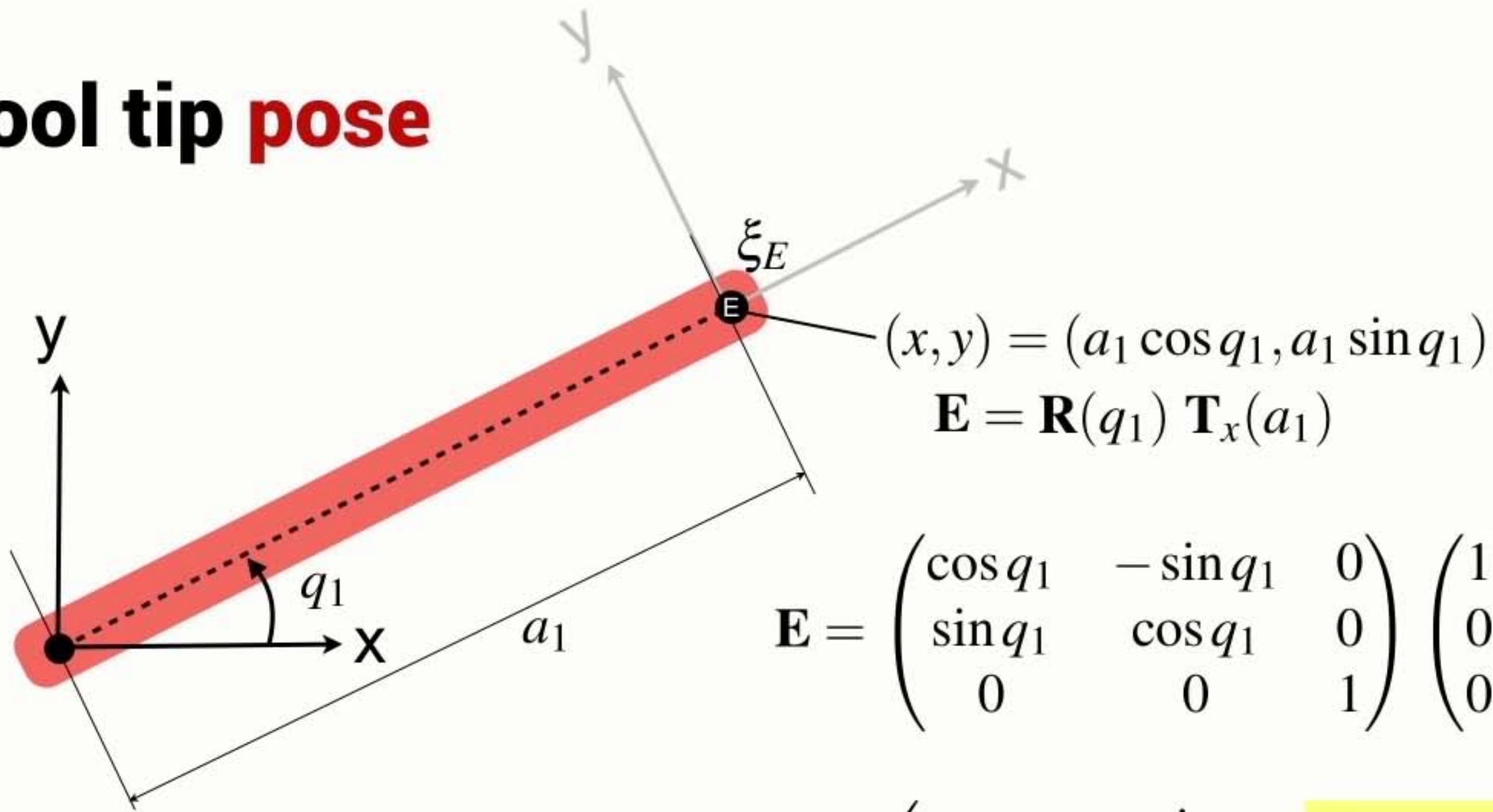
$$(x, y) = \mathbf{E} = \mathbf{R}(q_1) \mathbf{T}_x(a_1)$$

$$\mathbf{E} = \begin{pmatrix} \cos q_1 & -\sin q_1 & 0 \\ \sin q_1 & \cos q_1 & 0 \\ 0 & 0 & 1 \end{pmatrix} \begin{pmatrix} 1 & 0 & a_1 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{pmatrix}$$

$$\mathbf{E} = \begin{pmatrix} \cos q_1 & -\sin q_1 & a_1 \cos q_1 \\ \sin q_1 & \cos q_1 & a_1 \sin q_1 \\ 0 & 0 & 1 \end{pmatrix}$$

■ These are all 2D transforms $\mathbf{E} \in \text{SE}(2)$

Tool tip pose



$$(x, y) = (a_1 \cos q_1, a_1 \sin q_1)$$

$$\mathbf{E} = \mathbf{R}(q_1) \mathbf{T}_x(a_1)$$

$$\mathbf{E} = \begin{pmatrix} \cos q_1 & -\sin q_1 & 0 \\ \sin q_1 & \cos q_1 & 0 \\ 0 & 0 & 1 \end{pmatrix} \begin{pmatrix} 1 & 0 & a_1 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{pmatrix}$$

$$\mathbf{E} = \begin{pmatrix} \cos q_1 & -\sin q_1 & a_1 \cos q_1 \\ \sin q_1 & \cos q_1 & a_1 \sin q_1 \\ 0 & 0 & 1 \end{pmatrix}$$

■ These are all 2D transforms $\mathbf{E} \in \text{SE}(2)$

Command Window

```
>> a1 = 1
```

```
a1 =
```

```
1
```

```
fx >> |
```

Workspace

Name ▲	Value
 a1	1

Command Window

```
>> a1 = 1  
  
a1 =  
  
    1  
  
>> q1 = 0.2  
  
q1 =  
  
    0.2000  
  
fx >>
```

Workspace

Name ▲	Value
a1	1
q1	0.2000

Command Window

```

>> a1 = 1

a1 =

     1

>> q1 = 0.2

q1 =

    0.2000

>> trchain2('R(q1) Tx(a1)', q1)

ans =

    0.9801    -0.1987    0.9801
    0.1987     0.9801    0.1987
         0         0     1.0000
  
```

fx >>

Workspace

Name ▲	Value
a1	1
ans	[0.9801,-0.1987,...
q1	0.2000

Command Window

```

>> a1 = 1

a1 =

     1

>> q1 = 0.2

q1 =

    0.2000

>> trchain2('R(q1) Tx(a1)', q1)

ans =

    0.9801    -0.1987    0.9801
    0.1987     0.9801    0.1987
         0         0     1.0000

>> syms q1 a1
fx >>
  
```

Workspace

Name ▲	Value
a1	1x1 sym
ans	[0.9801,-0.1987,...
q1	1x1 sym

Command Window

```
>> a1 = 1

a1 =

     1

>> q1 = 0.2

q1 =

    0.2000

>> trchain2('R(q1) Tx(a1)', q1)

ans =

    0.9801    -0.1987    0.9801
    0.1987     0.9801    0.1987
         0         0     1.0000

>> syms q1 a1
>> trchain2('R(q1) Tx(a1)', q1)

ans =

[ cos(q1), -sin(q1), a1*cos(q1)]
[ sin(q1),  cos(q1), a1*sin(q1)]
[      0,      0,      1]
```

f_x >>

Workspace

Name ▲	Value
a1	1x1 sym
ans	3x3 sym
q1	1x1 sym

Command Window

```
>> a1 = 1

a1 =

     1

>> q1 = 0.2

q1 =

    0.2000

>> trchain2('R(q1) Tx(a1)', q1)

ans =

    0.9801    -0.1987    0.9801
    0.1987     0.9801    0.1987
         0         0     1.0000

>> syms q1 a1
>> trchain2('R(q1) Tx(a1)', q1)

ans =

[ cos(q1), -sin(q1), a1*cos(q1)]
[ sin(q1),  cos(q1), a1*sin(q1)]
[      0,      0,      1]
```

```
>> mdl_planar1
```

fx >> |

Workspace

Name ▲	Value
a1	1
ans	3x3 sym
p1	1x1 SerialLink
q1	1x1 sym
qz	0

Command Window

```
>> a1 = 1

a1 =

    1

>> q1 = 0.2

q1 =

    0.2000

>> trchain2('R(q1) Tx(a1)', q1)

ans =

    0.9801    -0.1987    0.9801
    0.1987     0.9801    0.1987
         0         0         1.0000

>> syms q1 a1
>> trchain2('R(q1) Tx(a1)', q1)

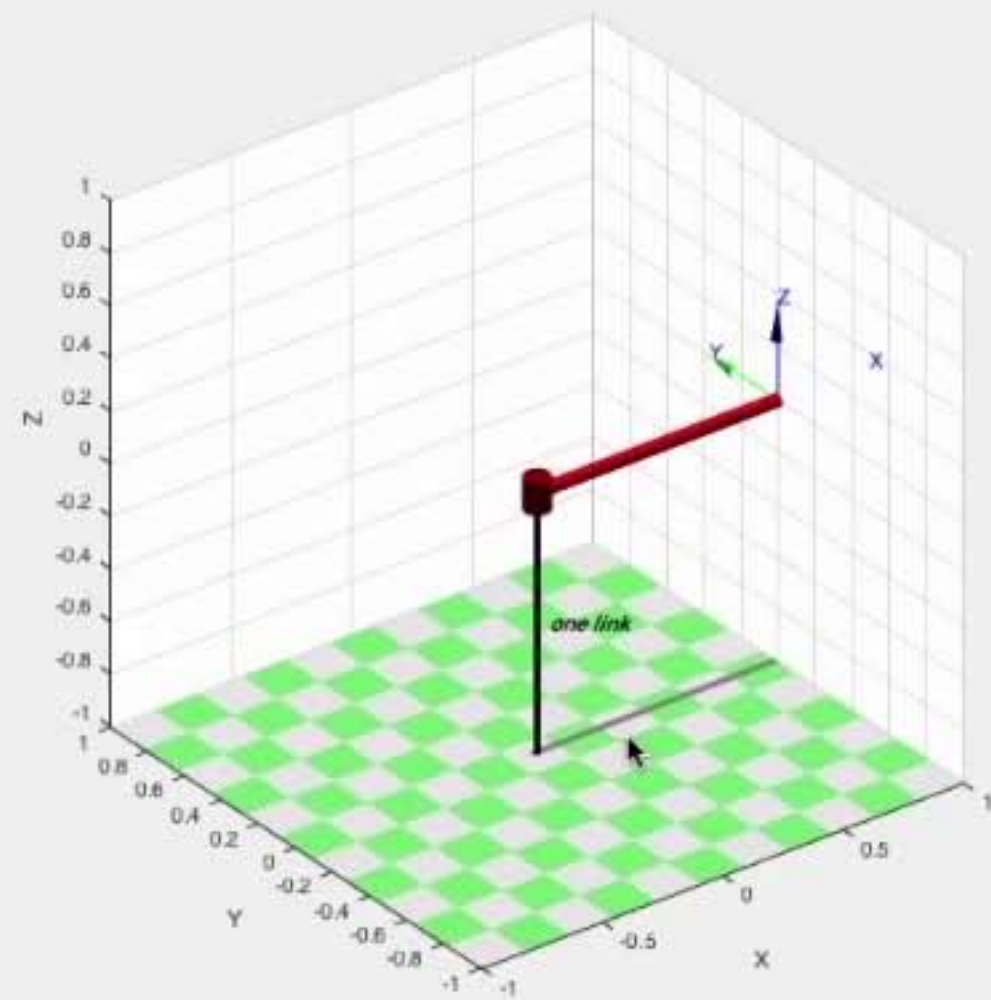
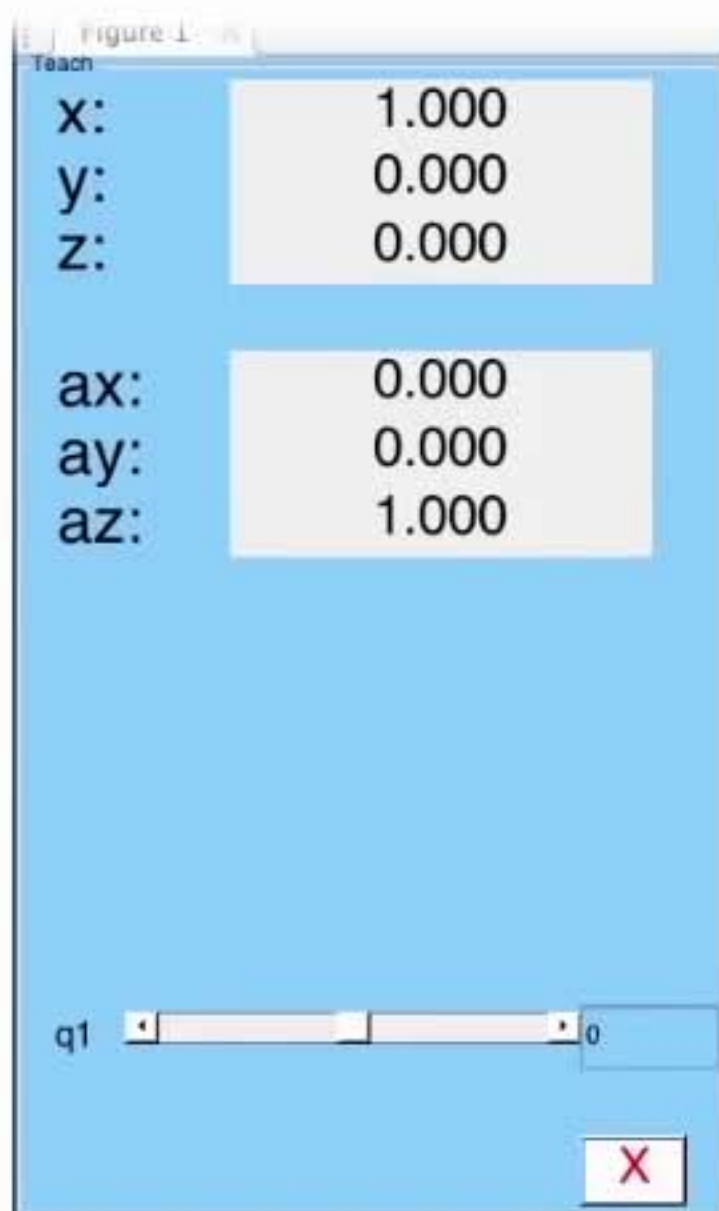
ans =

[ cos(q1), -sin(q1), a1*cos(q1)]
[ sin(q1),  cos(q1), a1*sin(q1)]
[         0,         0,         1]
```

```
>> mdl_planar1
>> p1.teach
```

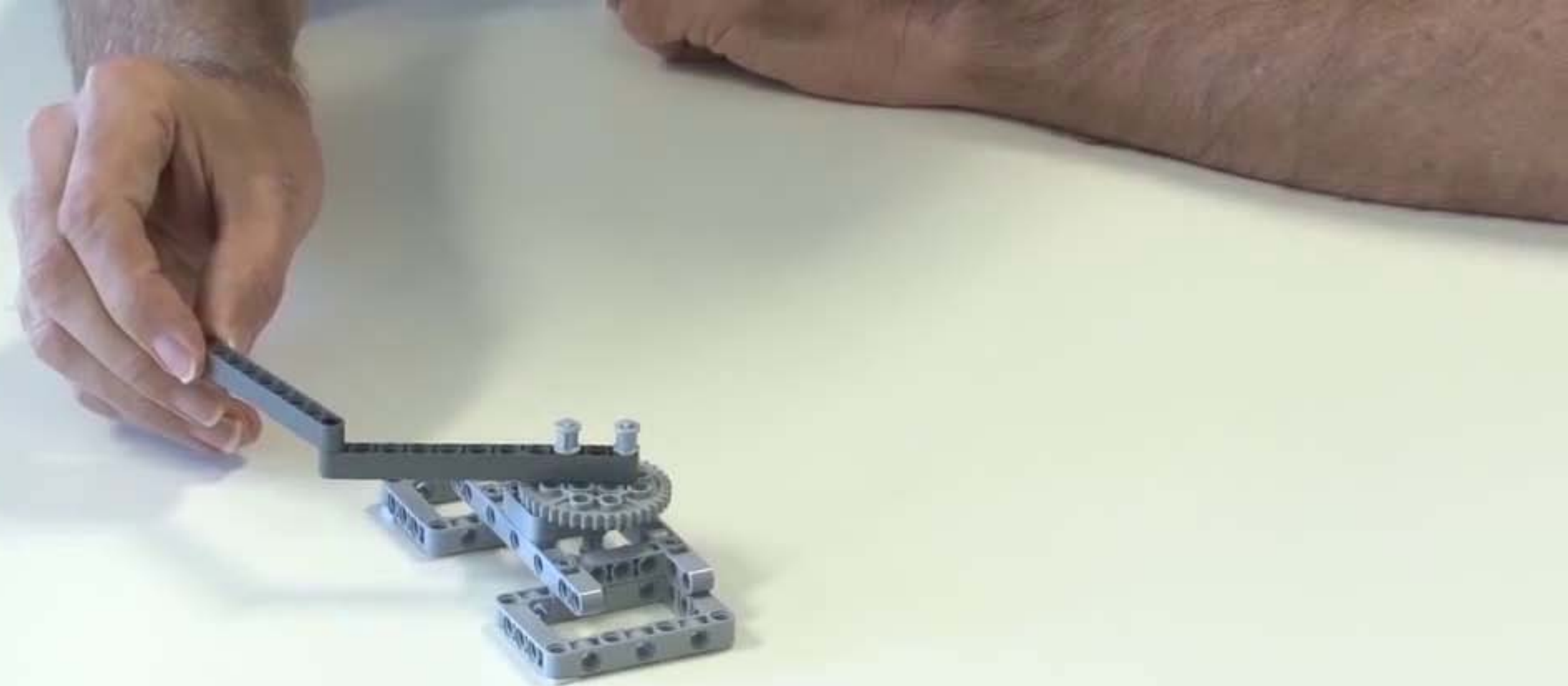
Workspace

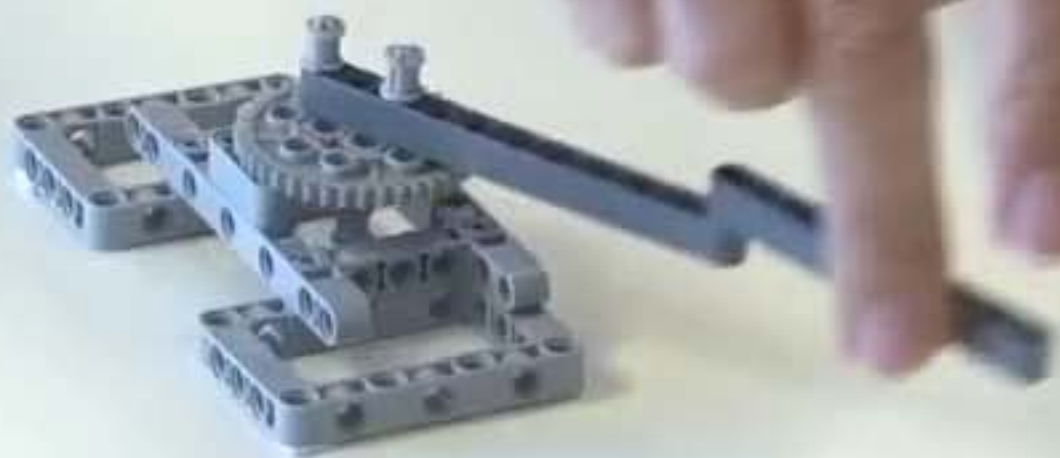
Name	Value
a1	1
ans	3x3 sym
p1	1x1 SerialLink
q1	1x1 sym
qz	0



Section3

A 2 joint planar robot

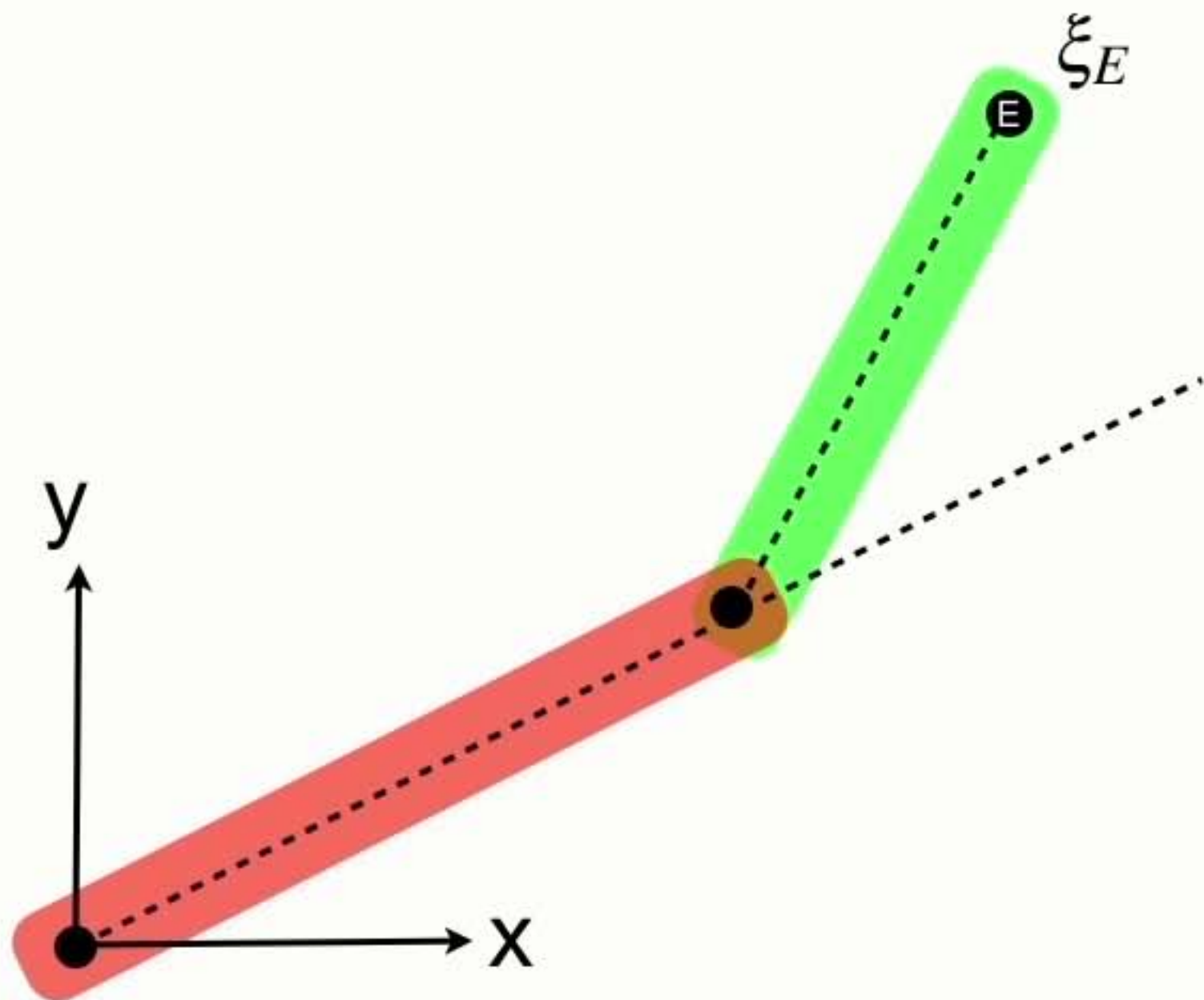






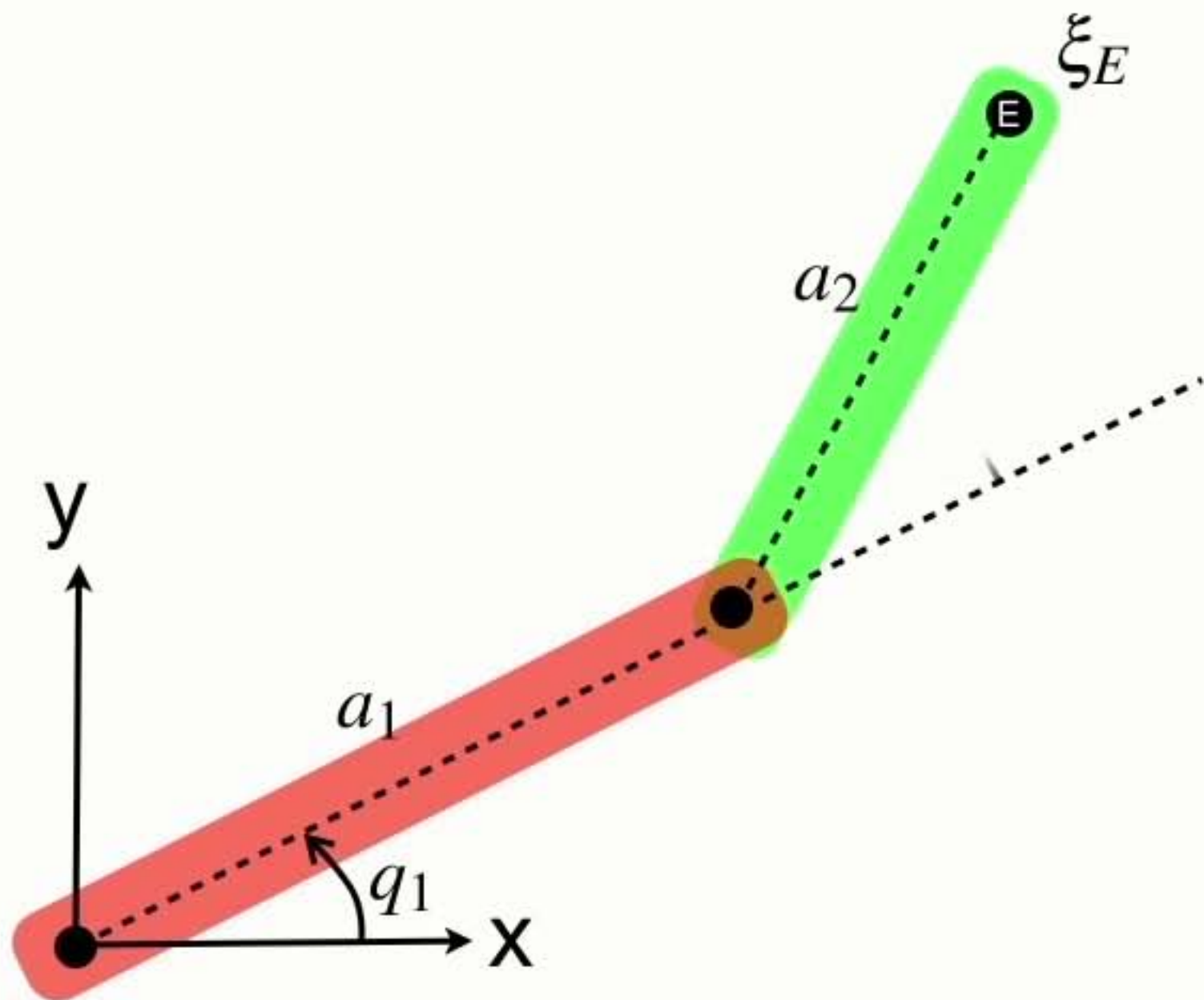


Tool tip pose



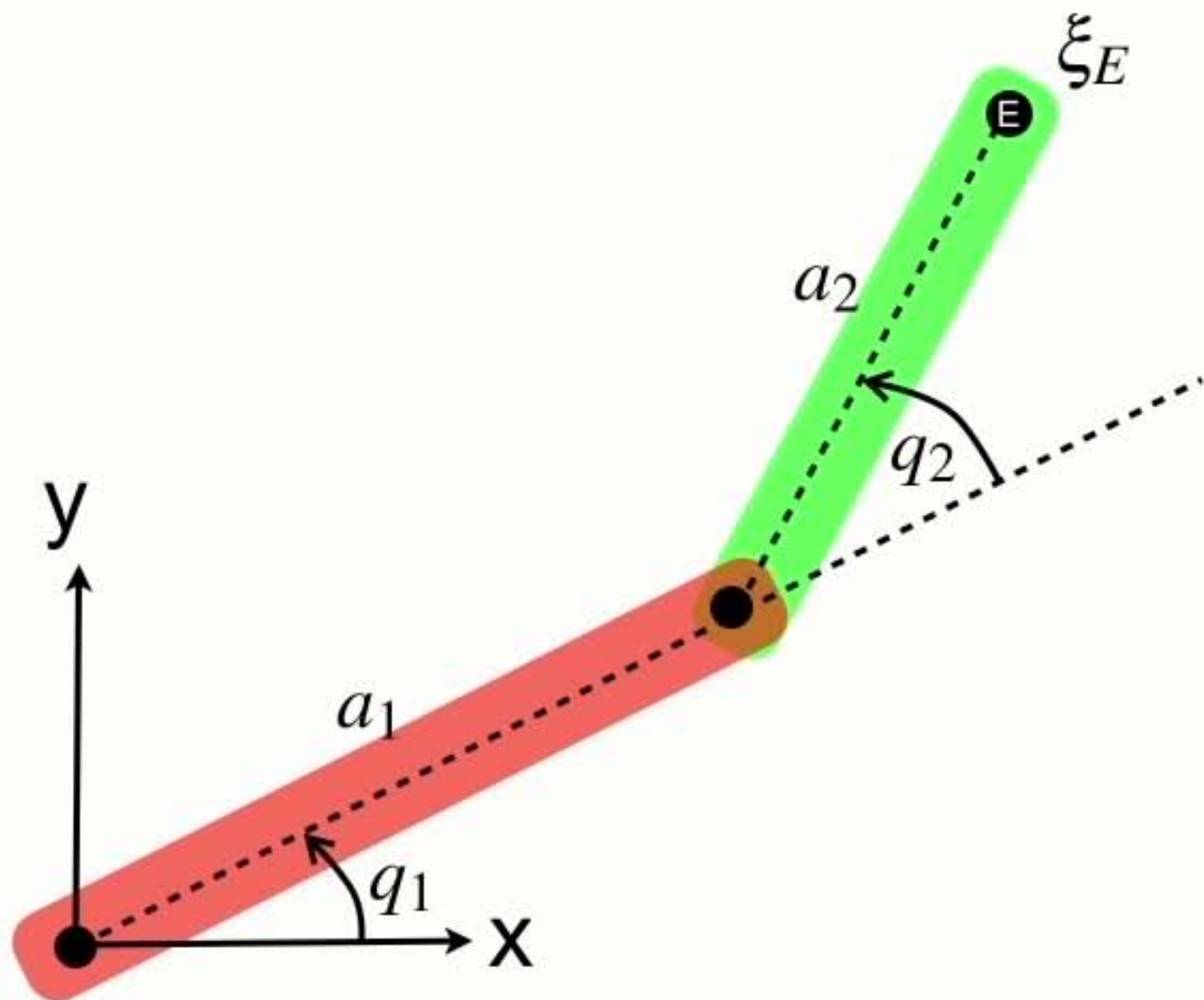
■ These are all 2D transforms $\mathbf{E} \in \text{SE}(2)$

Tool tip pose



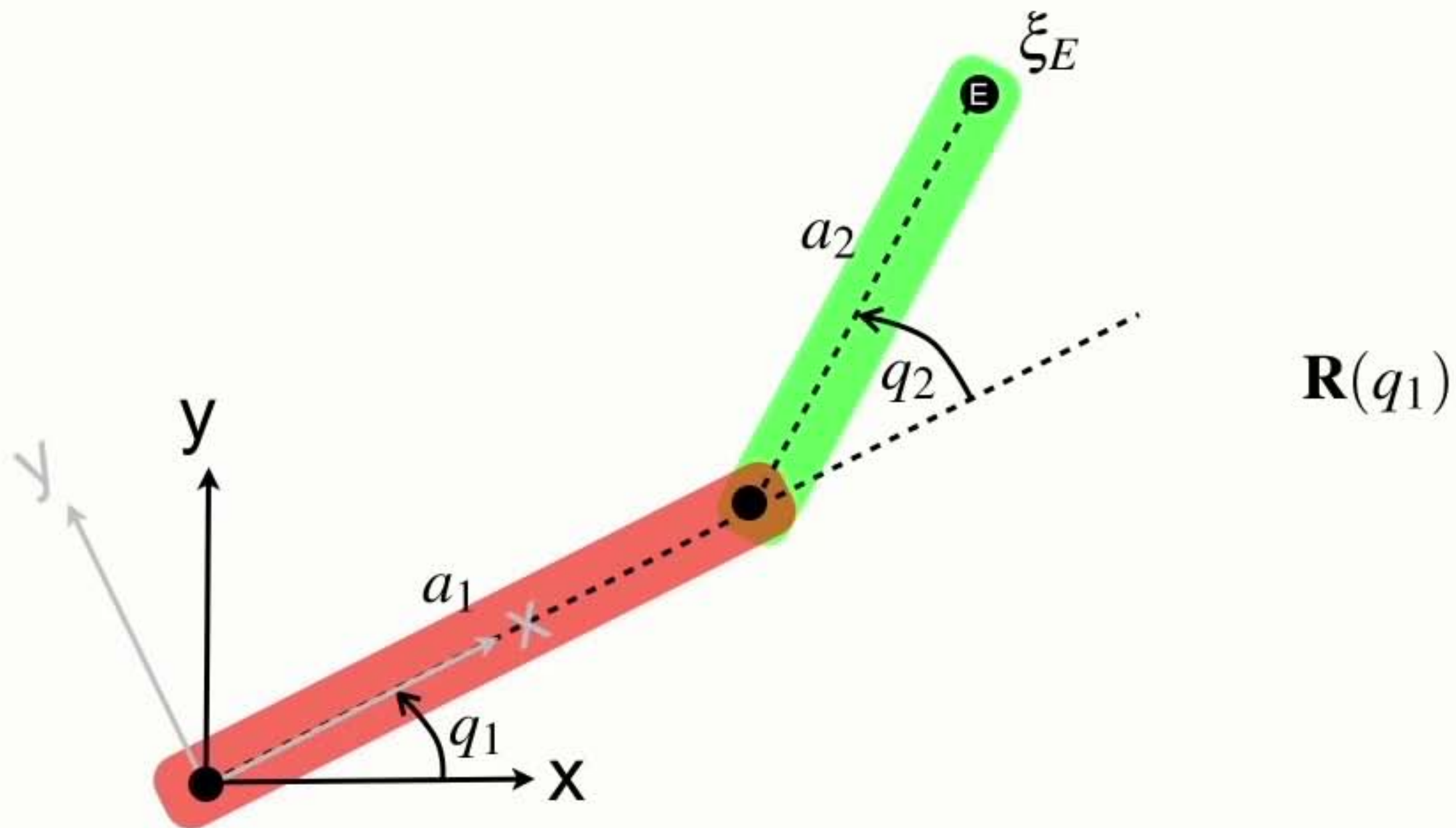
- These are all 2D transforms $\mathbf{E} \in \text{SE}(2)$

Tool tip pose



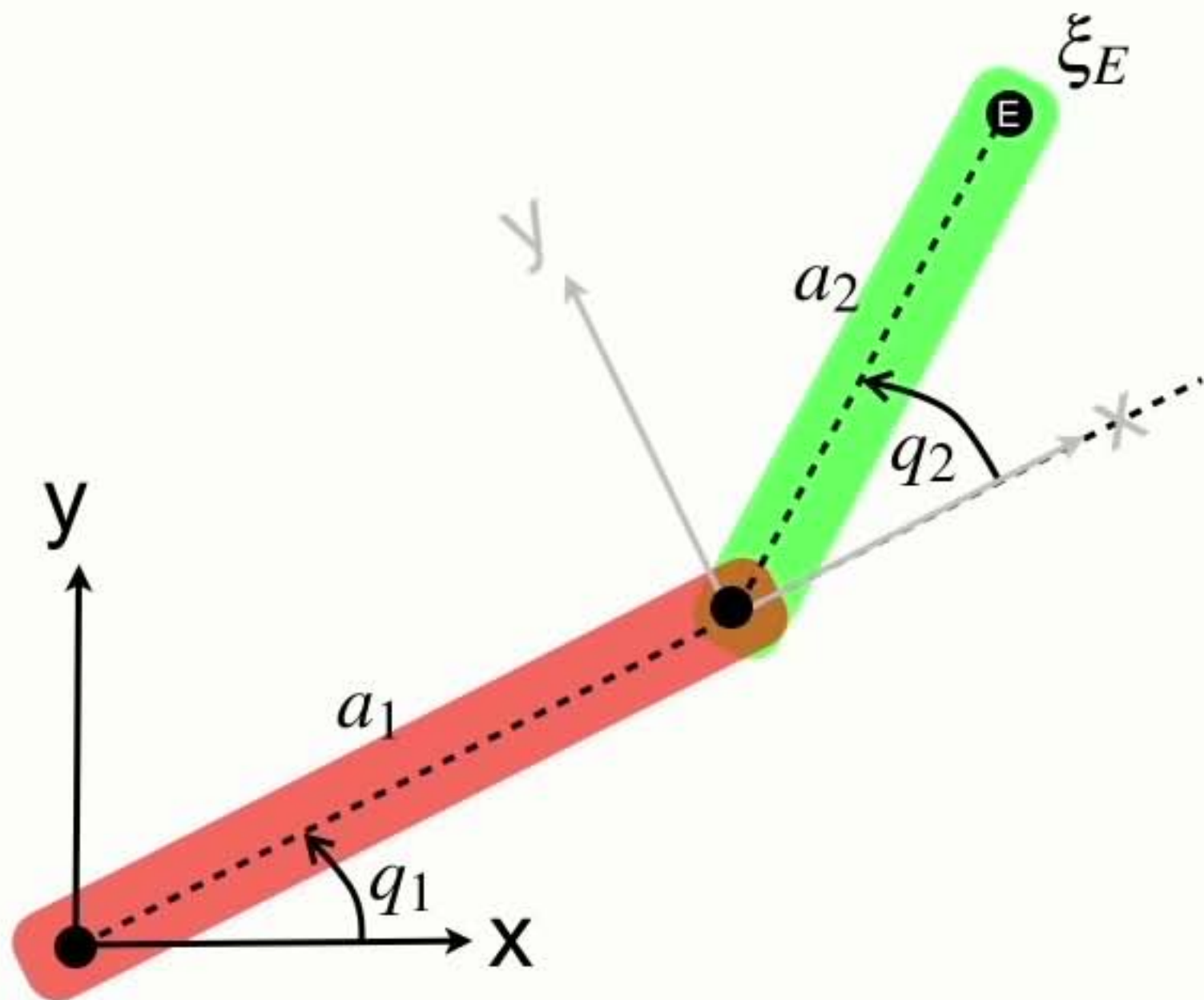
- These are all 2D transforms $\mathbf{E} \in \text{SE}(2)$

Tool tip pose



■ These are all 2D transforms $\mathbf{E} \in \text{SE}(2)$

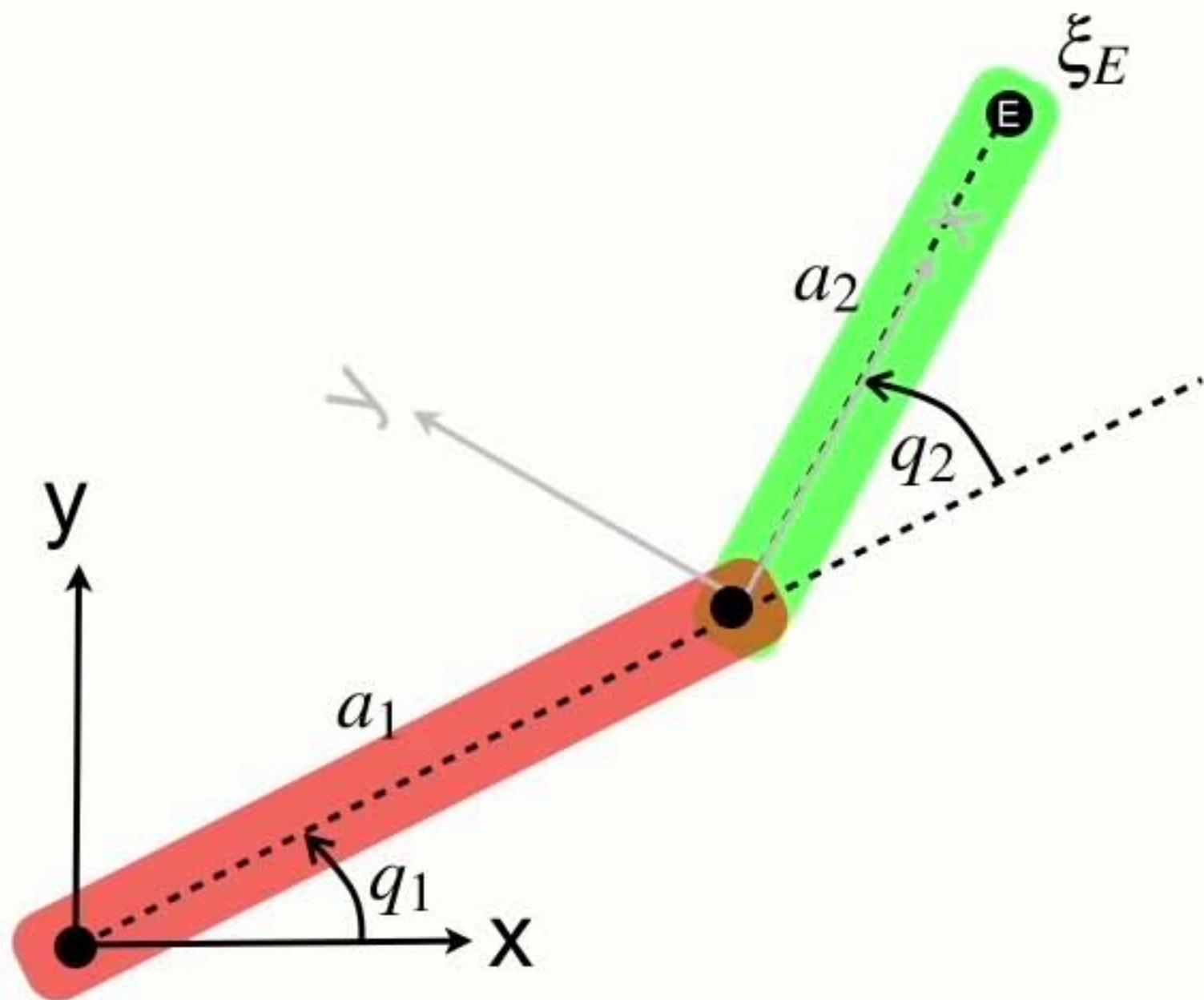
Tool tip **pose**



$$\mathbf{R}(q_1)\mathbf{T}_x(a_1)$$

■ These are all 2D transforms $\mathbf{E} \in \text{SE}(2)$

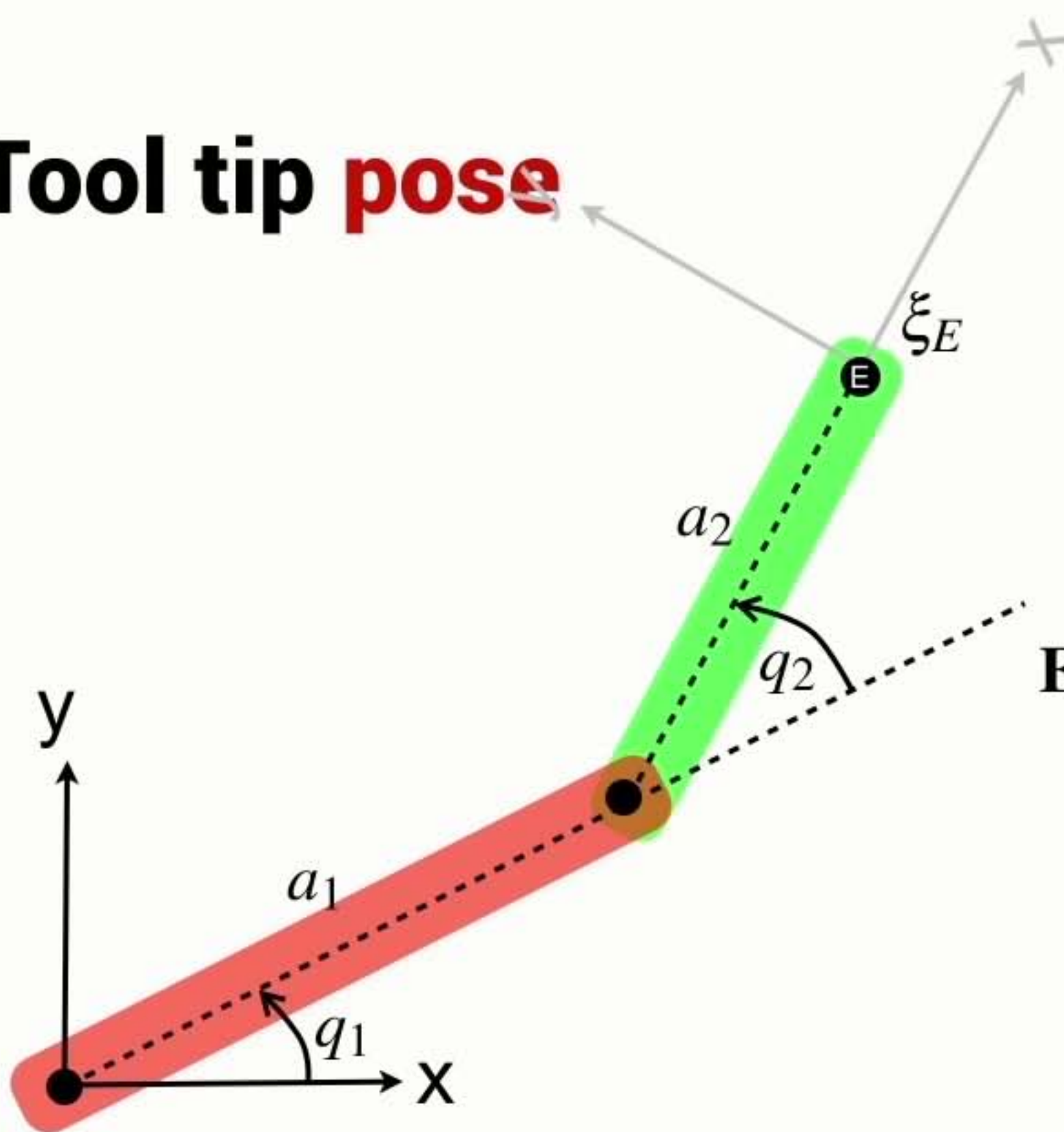
Tool tip pose



$$\mathbf{R}(q_1)\mathbf{T}_x(a_1)\mathbf{R}(q_2)$$

■ These are all 2D transforms $\mathbf{E} \in \text{SE}(2)$

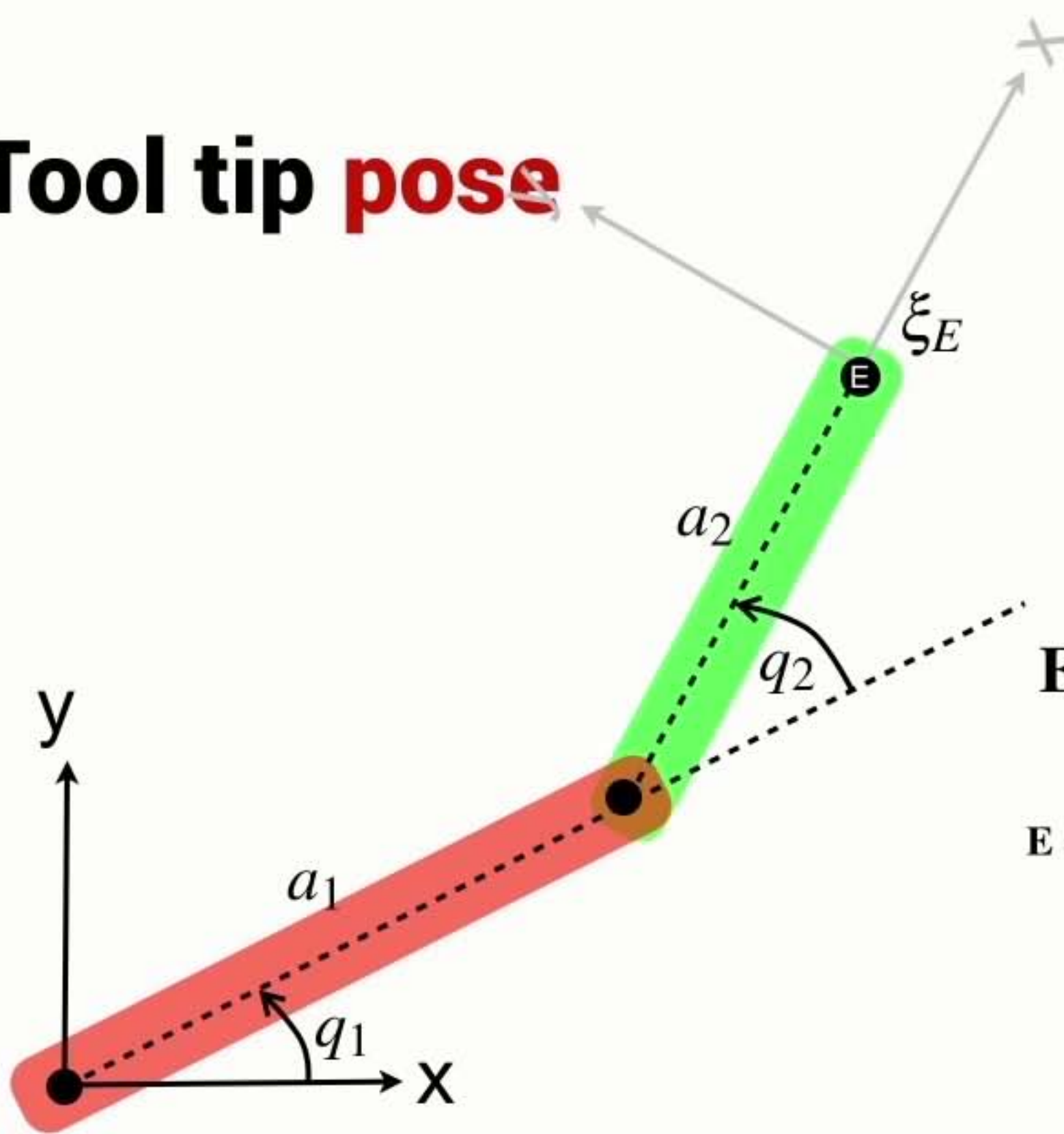
Tool tip pose



$$\mathbf{E} = \mathbf{R}(q_1) \mathbf{T}_x(a_1) \mathbf{R}(q_2) \mathbf{T}_x(a_2)$$

■ These are all 2D transforms $\mathbf{E} \in \text{SE}(2)$

Tool tip pose

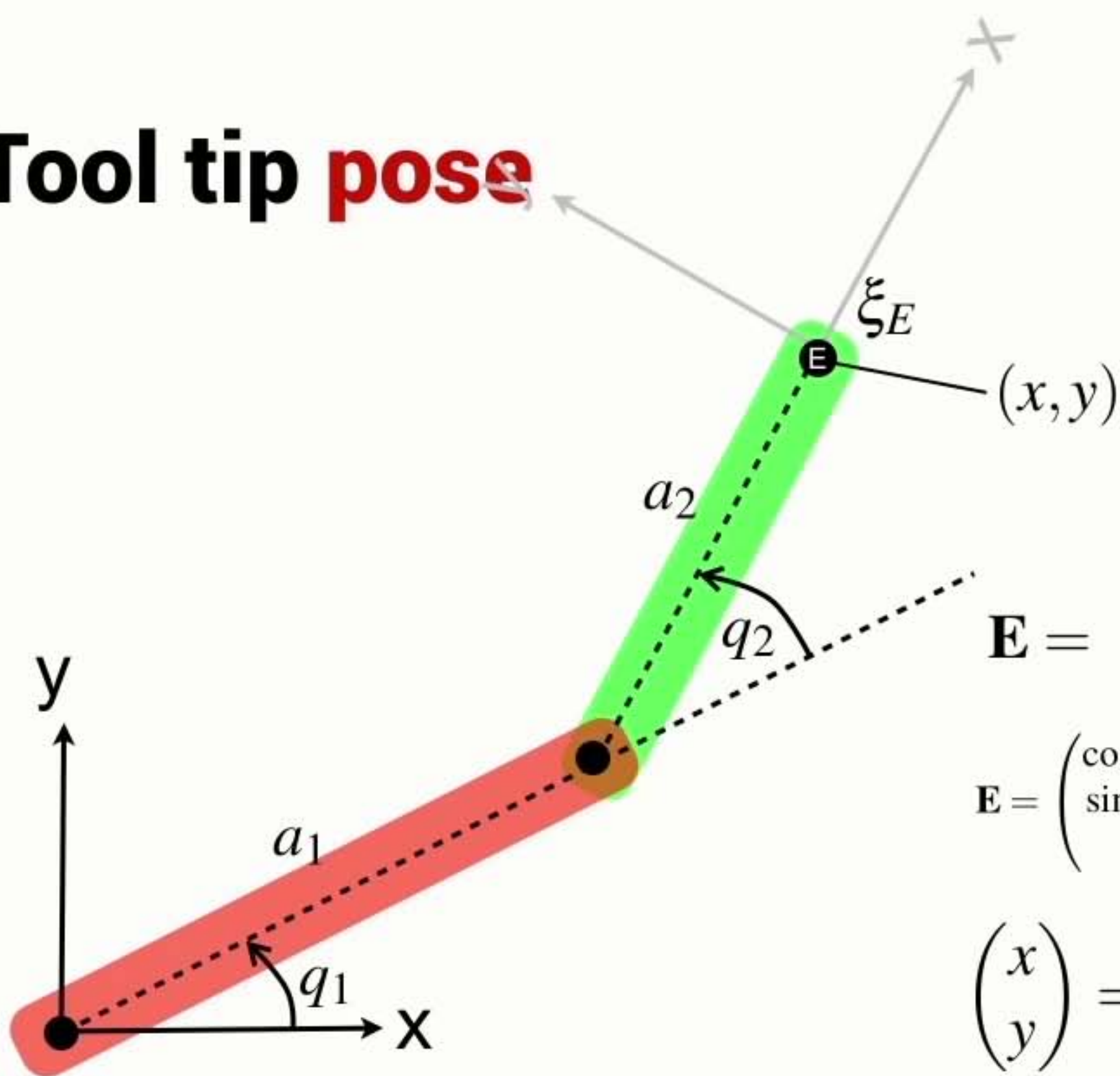


$$\mathbf{E} = \mathbf{R}(q_1) \mathbf{T}_x(a_1) \mathbf{R}(q_2) \mathbf{T}_x(a_2)$$

$$\mathbf{E} = \begin{pmatrix} \cos(q_1 + q_2) & -\sin(q_1 + q_2) & a_2 \cos(q_1 + q_2) + a_1 \cos q_1 \\ \sin(q_1 + q_2) & \cos(q_1 + q_2) & a_2 \sin(q_1 + q_2) + a_1 \sin q_1 \\ 0 & 0 & 1 \end{pmatrix}$$

■ These are all 2D transforms $\mathbf{E} \in \text{SE}(2)$

Tool tip pose



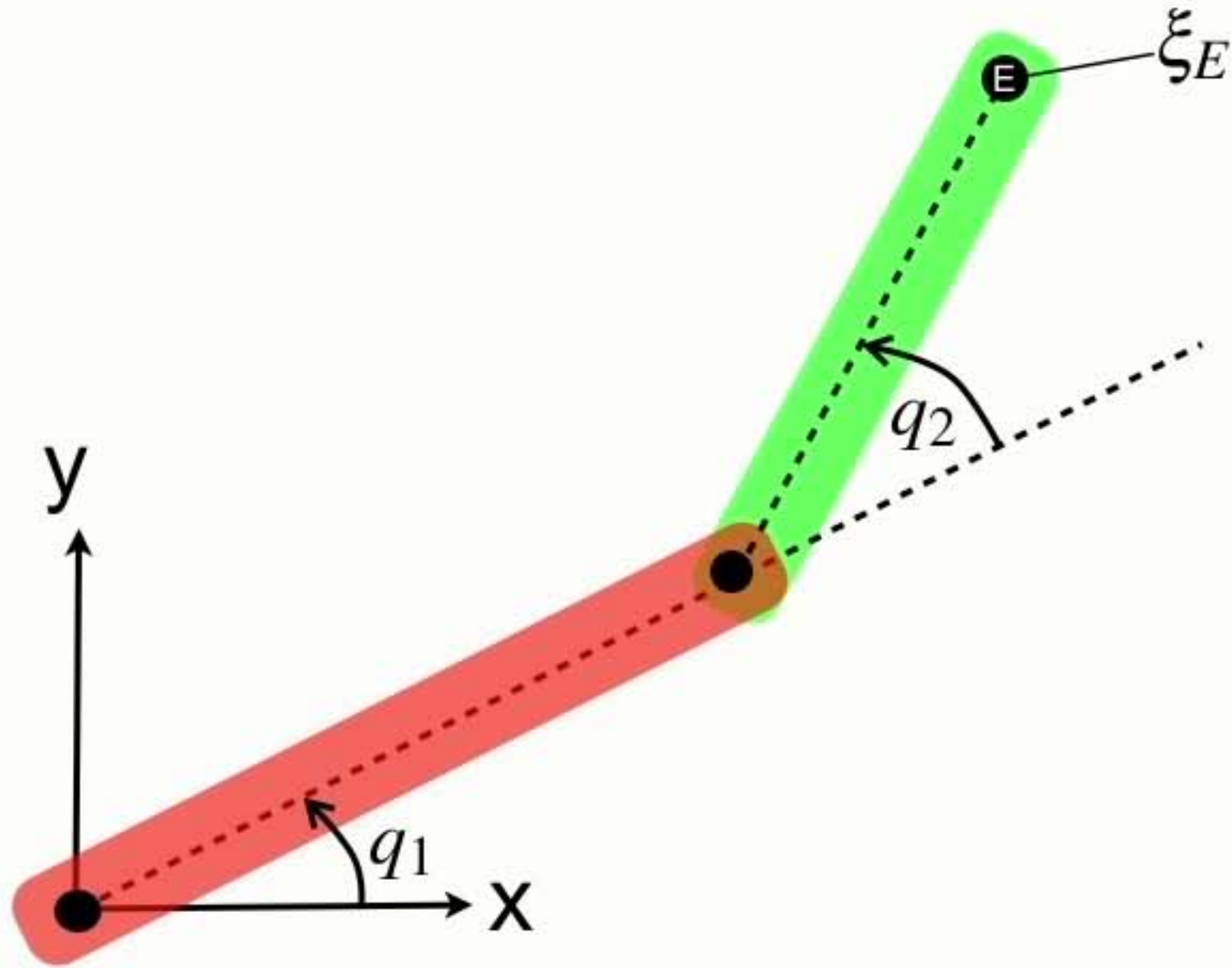
$$\mathbf{E} = \mathbf{R}(q_1) \mathbf{T}_x(a_1) \mathbf{R}(q_2) \mathbf{T}_x(a_2)$$

$$\mathbf{E} = \begin{pmatrix} \cos(q_1 + q_2) & -\sin(q_1 + q_2) & a_2 \cos(q_1 + q_2) + a_1 \cos q_1 \\ \sin(q_1 + q_2) & \cos(q_1 + q_2) & a_2 \sin(q_1 + q_2) + a_1 \sin q_1 \\ 0 & 0 & 1 \end{pmatrix}$$

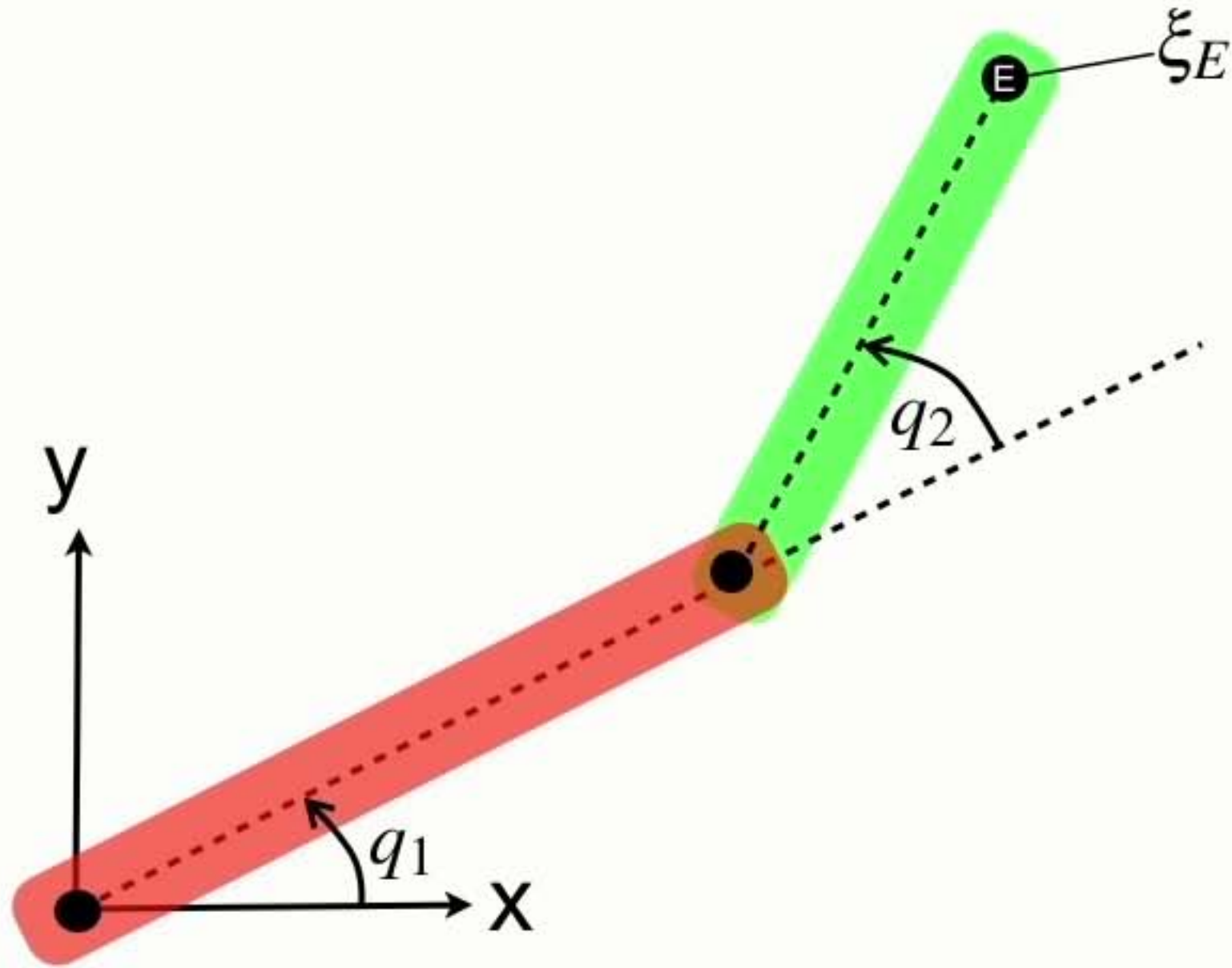
$$\begin{pmatrix} x \\ y \end{pmatrix} = \begin{pmatrix} a_2 \cos(q_1 + q_2) + a_1 \cos q_1 \\ a_2 \sin(q_1 + q_2) + a_1 \sin q_1 \end{pmatrix}$$

■ These are all 2D transforms $\mathbf{E} \in \text{SE}(2)$

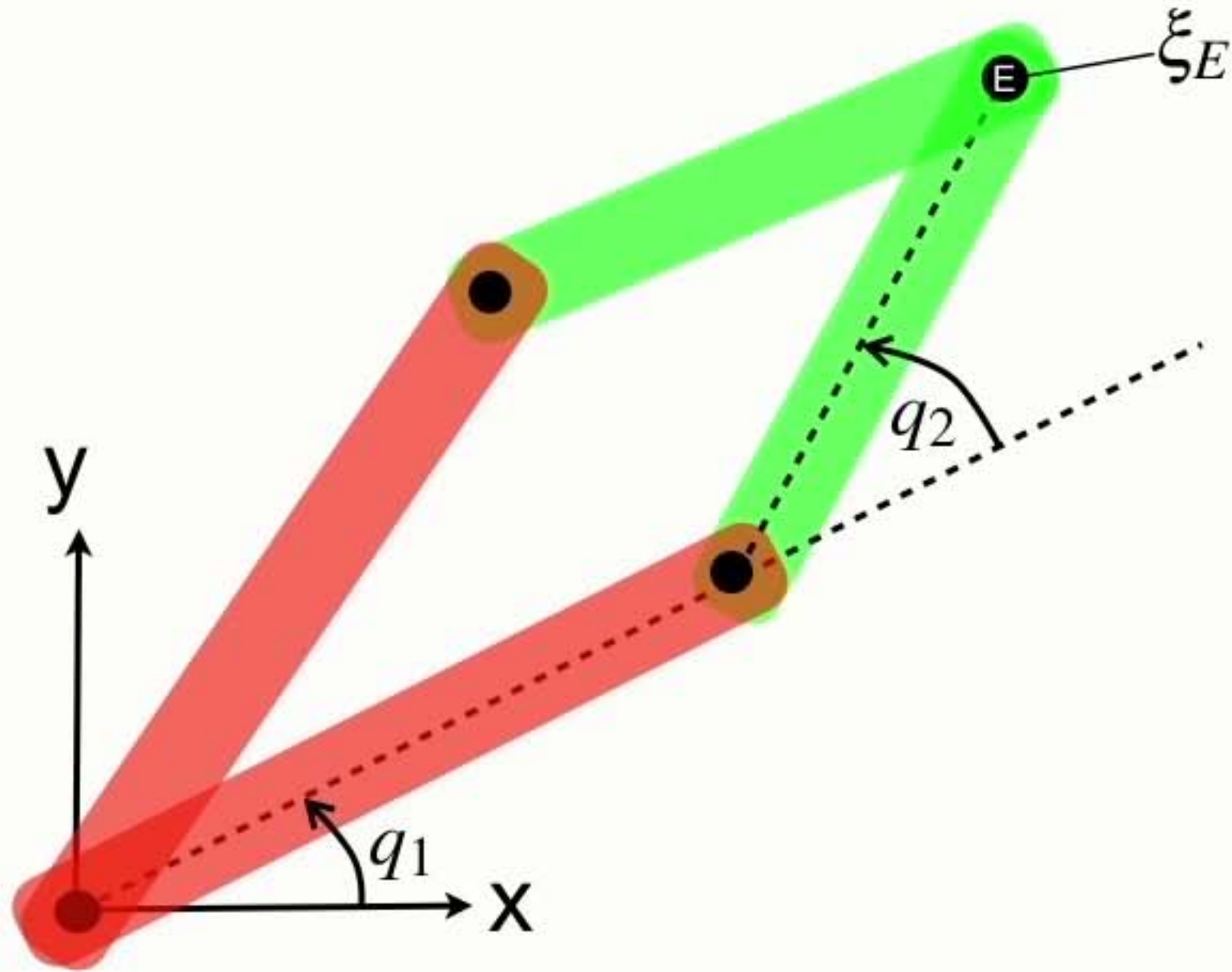
Configurations are not **unique**



Configurations are not **unique**



Configurations are not **unique**



Command Window

```

>> a1 = 1

a1 =

     1

>> a2 = 1

a2 =

     1

>> q1 = 0.2

q1 =

    0.2000

>> q2 = 0.3

q2 =

    0.3000

fx >> |
  
```

Workspace

Name ▲	Value
a1	1
a2	1
q1	0.2000
q2	0.3000

Command Window

```

>> a1 = 1

a1 =

     1

>> a2 = 1

a2 =

     1

>> q1 = 0.2

q1 =

    0.2000

>> q2 = 0.3

q2 =

    0.3000

>> trchain2('R(q1) Tx(a1) R(q2) Tx(a2)', [q1 q2])

ans =

    0.8776   -0.4794    1.8576
    0.4794    0.8776    0.6781
         0         0    1.0000

fx >>
  
```

Workspace

Name ▲	Value
a1	1
a2	1
ans	[0.8776,-0.4794,...
q1	0.2000
q2	0.3000

Command Window

```

>> a1 = 1

a1 =

    1

>> a2 = 1

a2 =

    1

>> q1 = 0.2

q1 =

    0.2000

>> q2 = 0.3

q2 =

    0.3000

>> trchain2('R(q1) Tx(a1) R(q2) Tx(a2)', [q1 q2])

ans =

    0.8776    -0.4794    1.8576
    0.4794     0.8776     0.6781
         0         0     1.0000

>> syms q1 q2 a1 a2
fx >> |
  
```

Workspace

Name ▲	Value
a1	1x1 sym
a2	1x1 sym
ans	[0.8776,-0.4794,...
q1	1x1 sym
q2	1x1 sym

Command Window

```
>> q1 = 0.2

q1 =

    0.2000

>> q2 = 0.3

q2 =

    0.3000

>> trchain2('R(q1) Tx(a1) R(q2) Tx(a2)', [q1 q2])

ans =

    0.8776   -0.4794    1.8576
    0.4794    0.8776    0.6781
         0         0    1.0000

>> syms q1 q2 a1 a2
>> trchain2('R(q1) Tx(a1) R(q2) Tx(a2)', [q1 q2])

ans =

[ cos(q1)*cos(q2) - sin(q1)*sin(q2), - cos(q1)*sin(q2) - cos(q2)*sin(q1), a2*(cos(q1)*cos(q2) - sin(q1)*sin(q2)) + a1*cos(q1)]
[ cos(q1)*sin(q2) + cos(q2)*sin(q1),  cos(q1)*cos(q2) - sin(q1)*sin(q2), a2*(cos(q1)*sin(q2) + cos(q2)*sin(q1)) + a1*sin(q1)]
[                                0,                                0,                                1]
```

f5 >> |

Workspace

Name ▲	Value
a1	1x1 sym
a2	1x1 sym
ans	3x3 sym
q1	1x1 sym
q2	1x1 sym

Command Window

```
0.8776 -0.4794 1.8576
0.4794 0.8776 0.6781
0 0 1.0000
```

```
>> syms q1 q2 a1 a2
>> trchain2('R(q1) Tx(a1) R(q2) Tx(a2)', [q1 q2])
```

ans =

```
[ cos(q1)*cos(q2) - sin(q1)*sin(q2), - cos(q1)*sin(q2) - cos(q2)*sin(q1), a2*(cos(q1)*cos(q2) - sin(q1)*sin(q2)) + a1*cos(q1)]
[ cos(q1)*sin(q2) + cos(q2)*sin(q1), cos(q1)*cos(q2) - sin(q1)*sin(q2), a2*(cos(q1)*sin(q2) + cos(q2)*sin(q1)) + a1*sin(q1)]
[ 0, 0, 1]
```

```
>> mdl_planar2
```

>> |

Workspace

Name	Value
a1	1
a2	1
ans	3x3 sym
p2	1x1 SerialLink
q1	1x1 sym
q2	1x1 sym
qz	[0,0]

Command Window

```
0.0770 -0.4794 1.0000
0.4794 0.8776 0.6781
0 0 1.0000
```

```
>> syms q1 q2 a1 a2
>> trchain2('R(q1) Tx(a1) R(q2) Tx(a2)', [q1 q2])
```

ans =

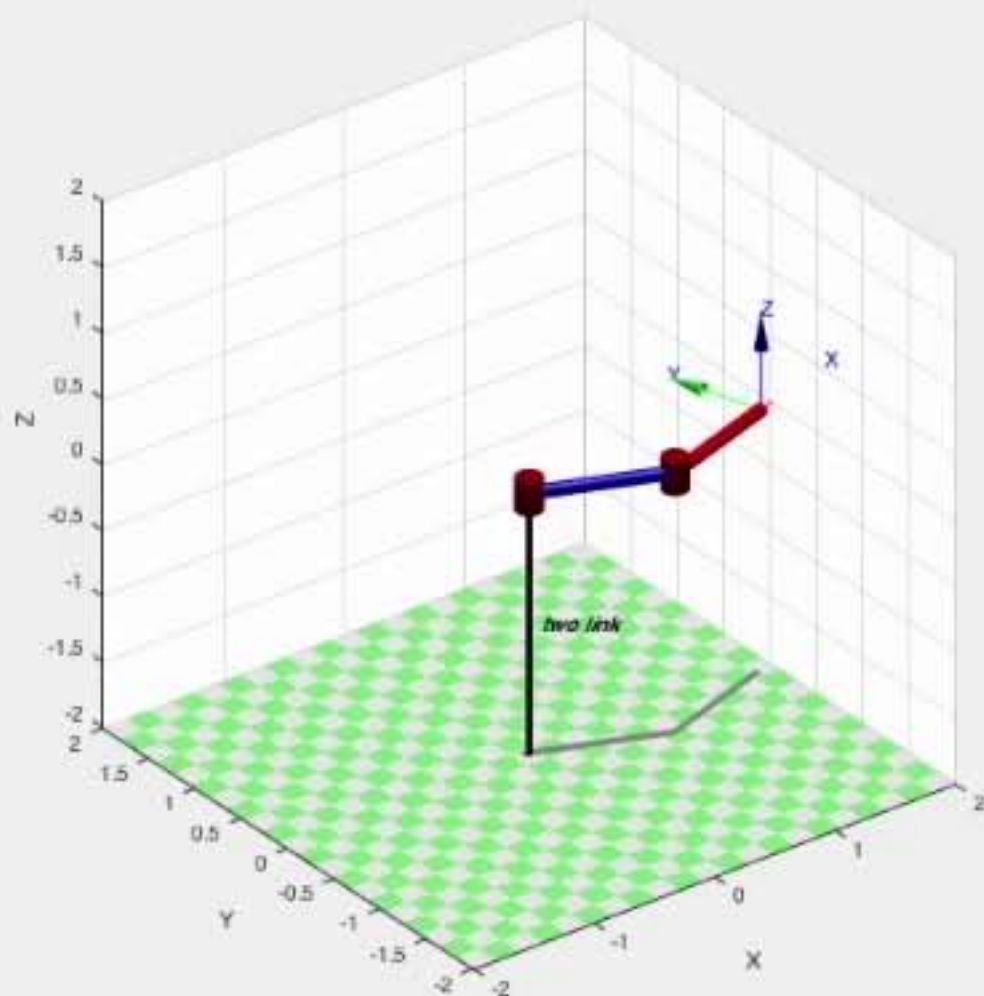
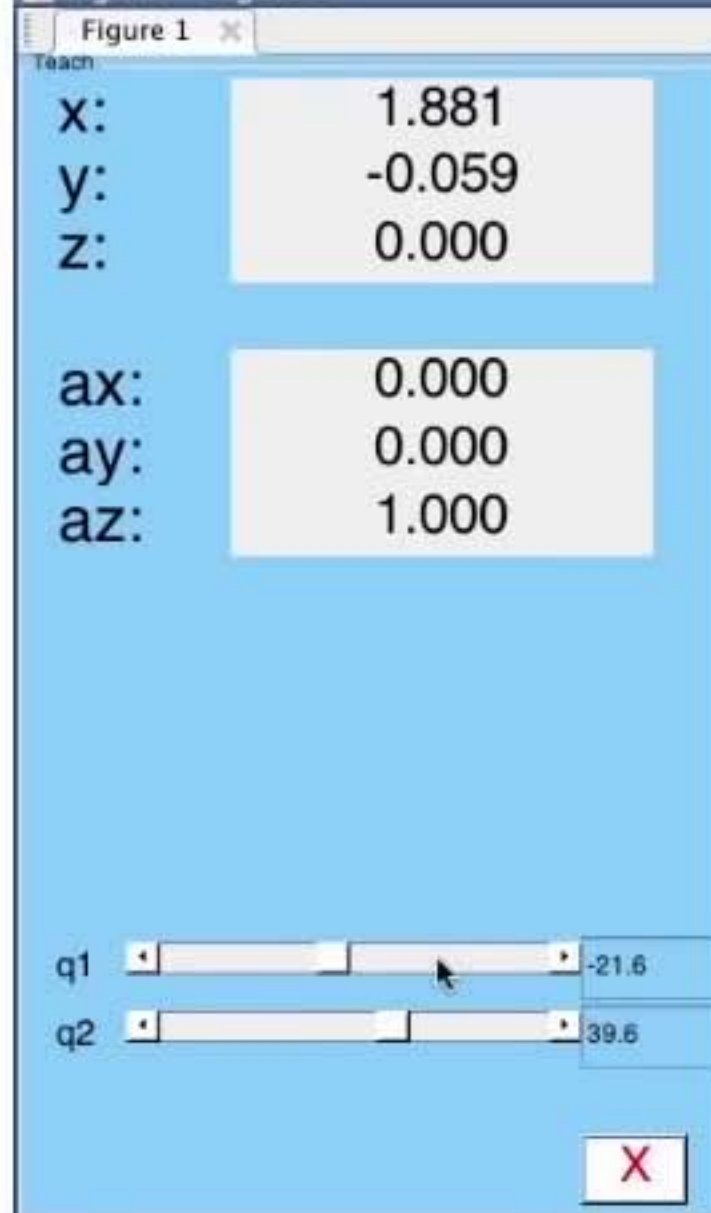
```
[ cos(q1)*cos(q2) - sin(q1)*sin(q2), -cos(q1)*sin(q2) - cos(q2)*sin(q1), a2*(cos(q1)*cos(q2) - sin(q1)*sin(q2)) + a1*cos(q1)]
[ cos(q1)*sin(q2) + cos(q2)*sin(q1), cos(q1)*cos(q2) - sin(q1)*sin(q2), a2*(cos(q1)*sin(q2) + cos(q2)*sin(q1)) + a1*sin(q1)]
[ 0, 0, 1]
```

```
>> mdl_planar2
>> p2.teach
fx >>
```

Workspace

Name	Value
a1	1
a2	1
ans	3x3 sym
p2	1x1 SerialLink
q1	1x1 sym
q2	1x1 sym
qz	[0,0]

Figures - Figure 1



```
0.4794 0.6770 0.0761
0 0 1.0000
```

```
>> syms q1 q2 a1 a2
>> trchain2('R(q1) Tx(a1) R(q2) Tx(a2)', [q1 q2])
```

ans =

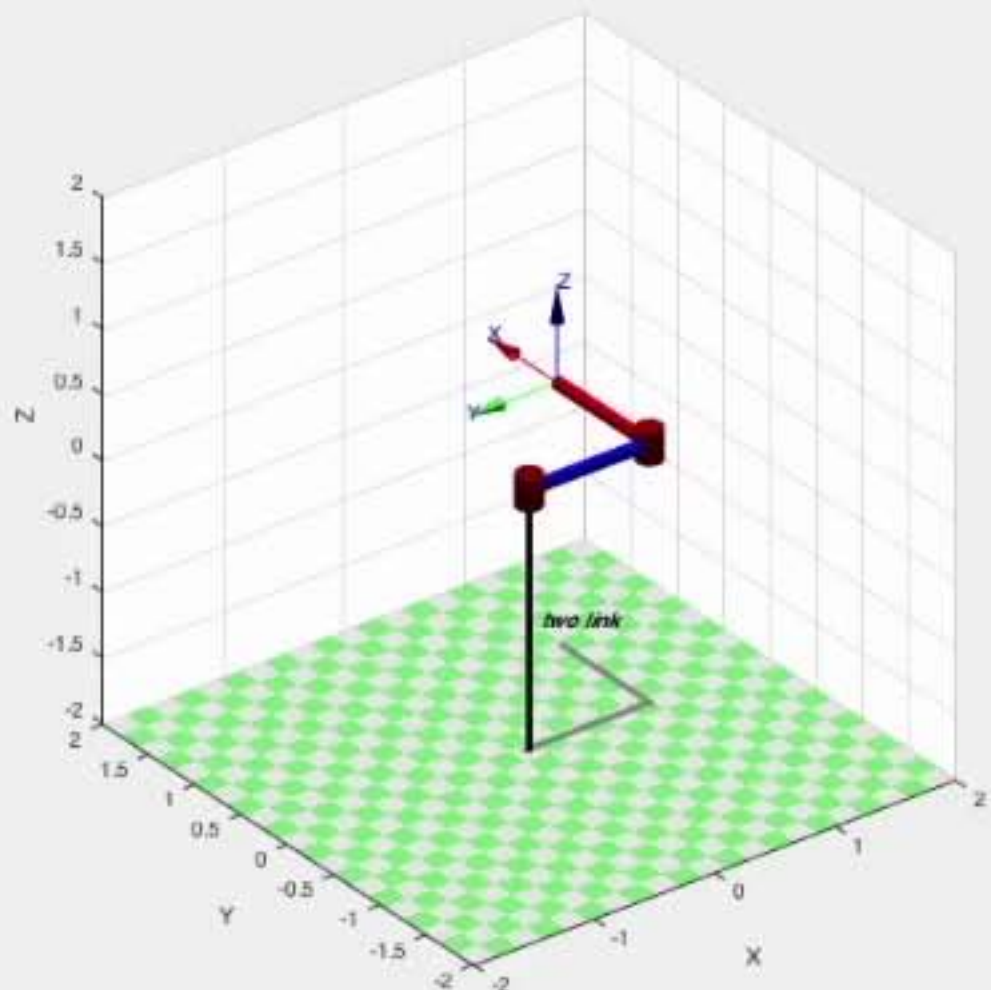
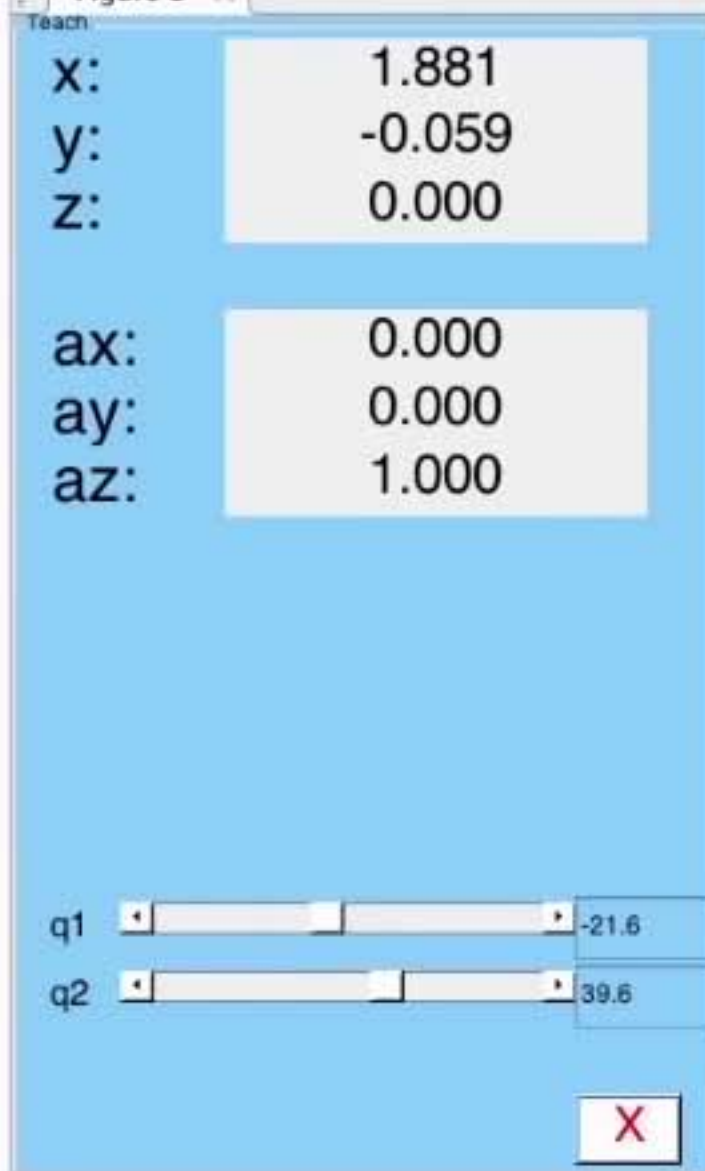
```
[ cos(q1)*cos(q2) - sin(q1)*sin(q2), -cos(q1)*sin(q2) - cos(q2)*sin(q1), a2*(cos(q1)*cos(q2) - sin(q1)*sin(q2)) + a1*cos(q1)]
[ cos(q1)*sin(q2) + cos(q2)*sin(q1), cos(q1)*cos(q2) - sin(q1)*sin(q2), a2*(cos(q1)*sin(q2) + cos(q2)*sin(q1)) + a1*sin(q1)]
[ 0, 0, 1]
```

```
>> mdl_planar2
>> p2.teach
>> p2.plot([0 pi/2])
>> p2
```

Name	Value
a1	1
a2	1
ans	3x3 sym
p2	1x1 SerialLink
q1	1x1 sym
q2	1x1 sym
qz	[0,0]

Figures - Figure 1

Figure 1



```
>> syms q1 q2 a1 a2
>> trchain2('R(q1) Tx(a1) R(q2) Tx(a2)', [q1 q2])
```

ans =

```
[ cos(q1)*cos(q2) - sin(q1)*sin(q2), - cos(q1)*sin(q2) - cos(q2)*sin(q1), a2*(cos(q1)*cos(q2) -
[ cos(q1)*sin(q2) + cos(q2)*sin(q1), cos(q1)*cos(q2) - sin(q1)*sin(q2), a2*(cos(q1)*sin(q2) +
[ 0, 0, 1]
```

```
>> mdl_planar2
```

```
>> p2.teach
```

```
>> p2.plot([0 pi/2])
```

```
>> p2.plot([pi/2 -pi/2])
```

```
>>
```

Name	Value
a1	1
a2	1
ans	3x3 sym
p2	1x1 SerialLink
q1	1x1 sym
q2	1x1 sym
qz	[0,0]

Figures - Figure 1

Figure 1

Teach

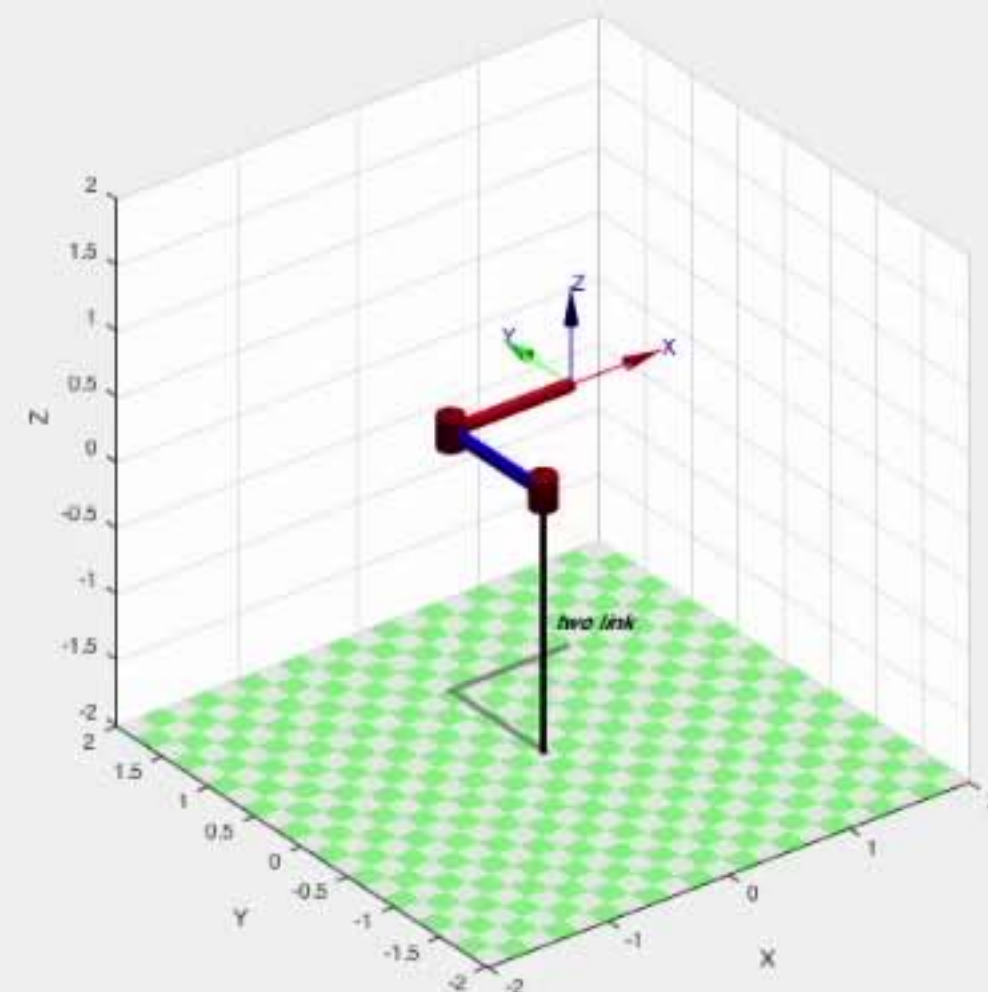
x: 1.881
y: -0.059
z: 0.000

ax: 0.000
ay: 0.000
az: 1.000

q1 -21.6

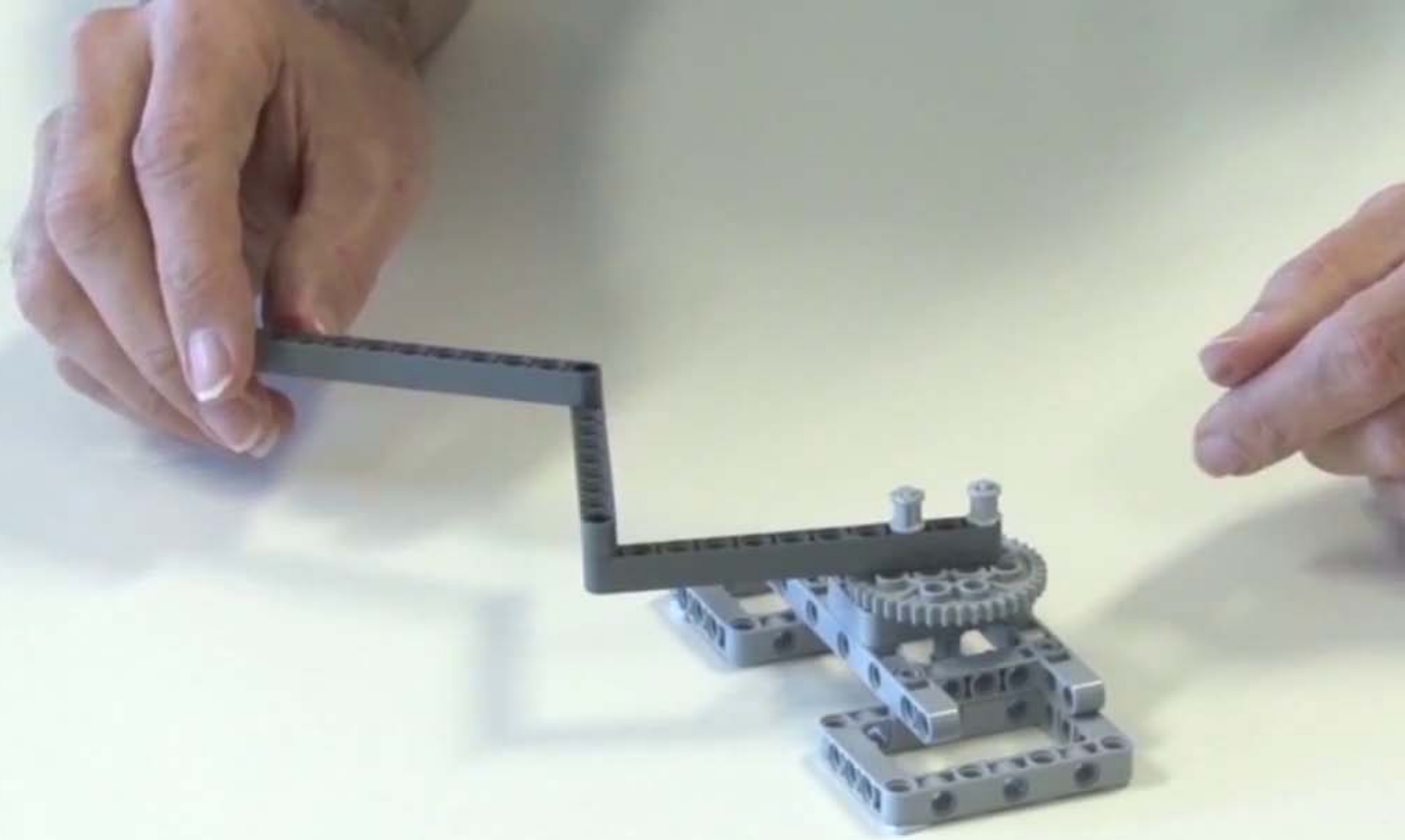
q2 39.6

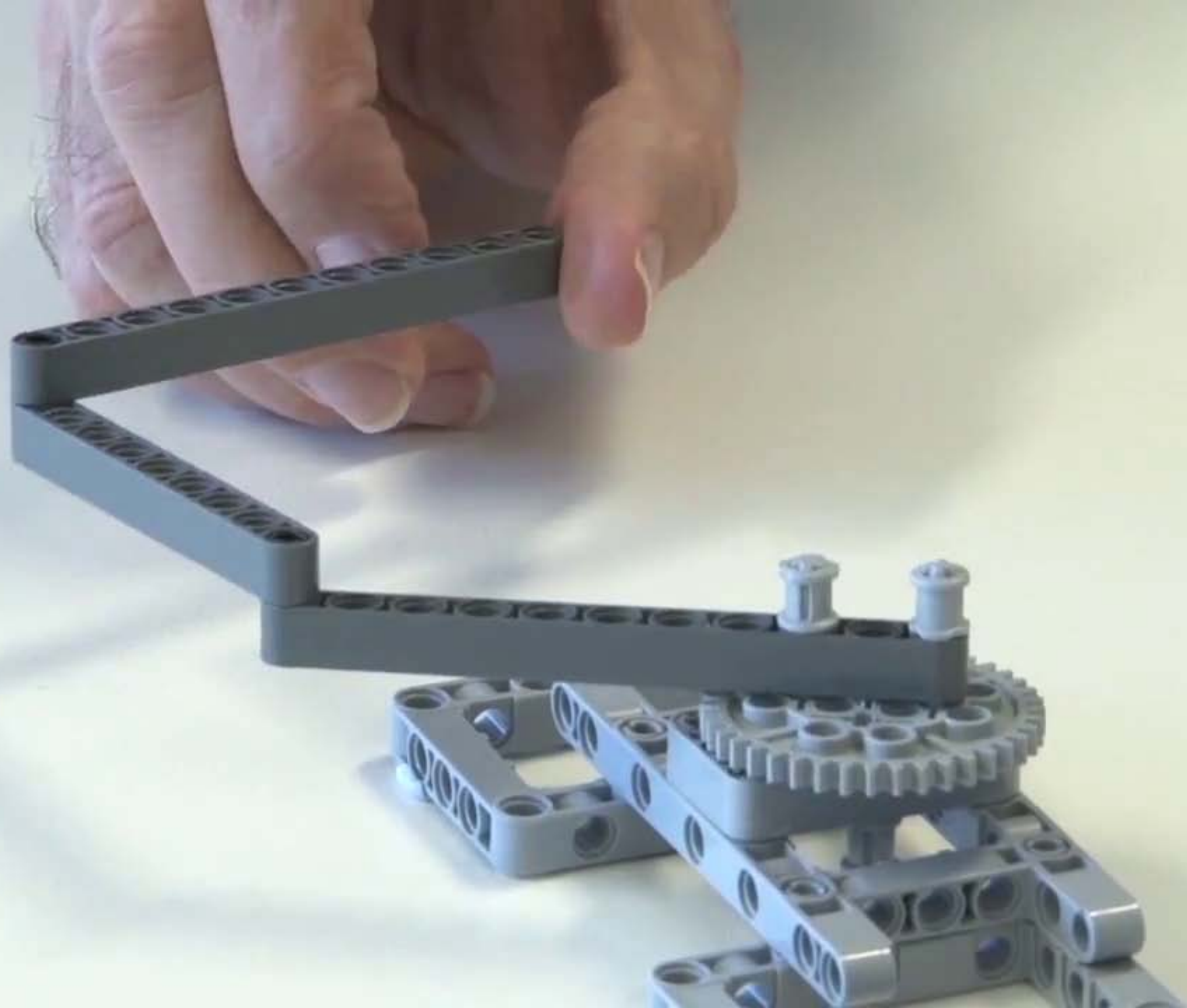
X

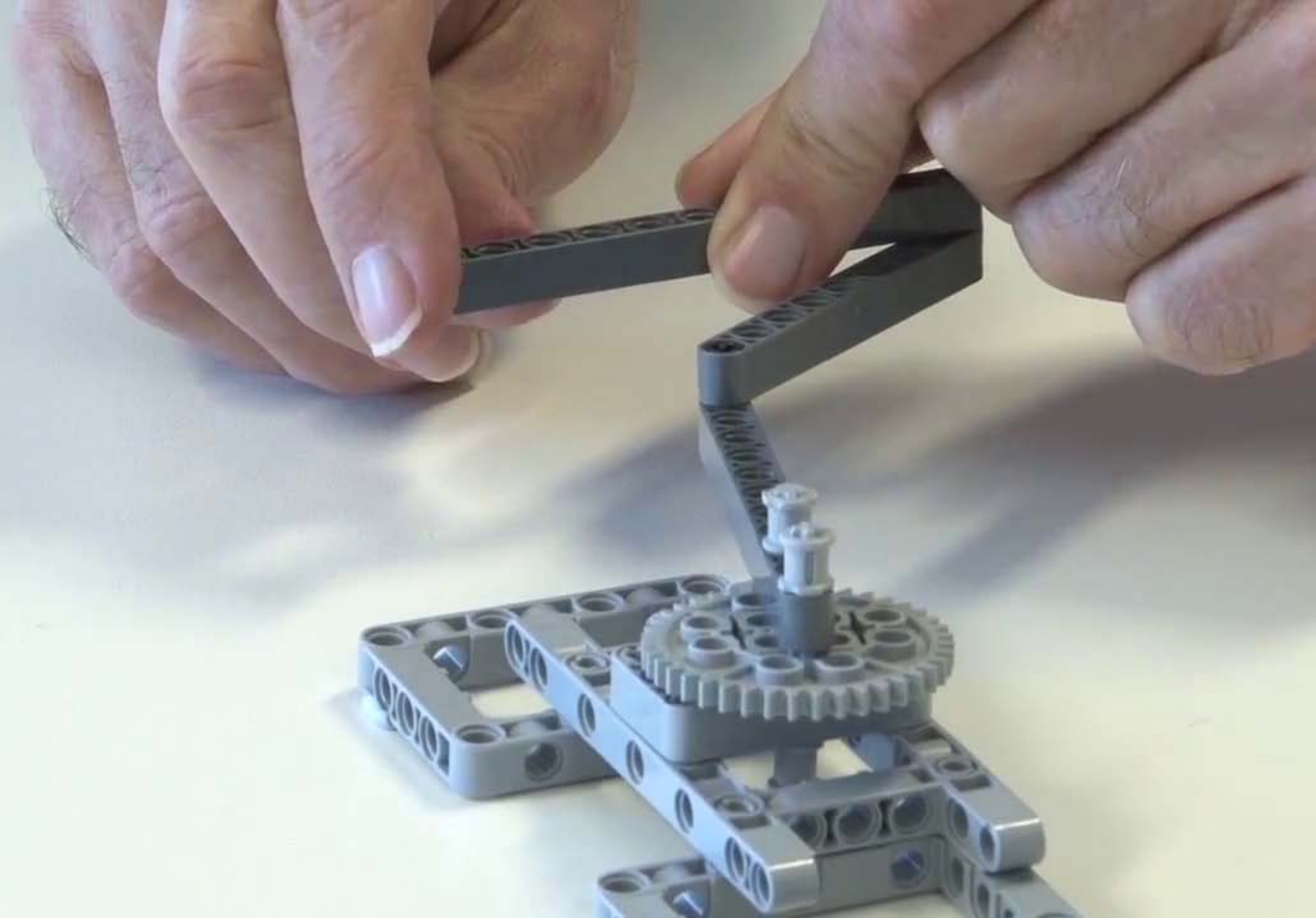


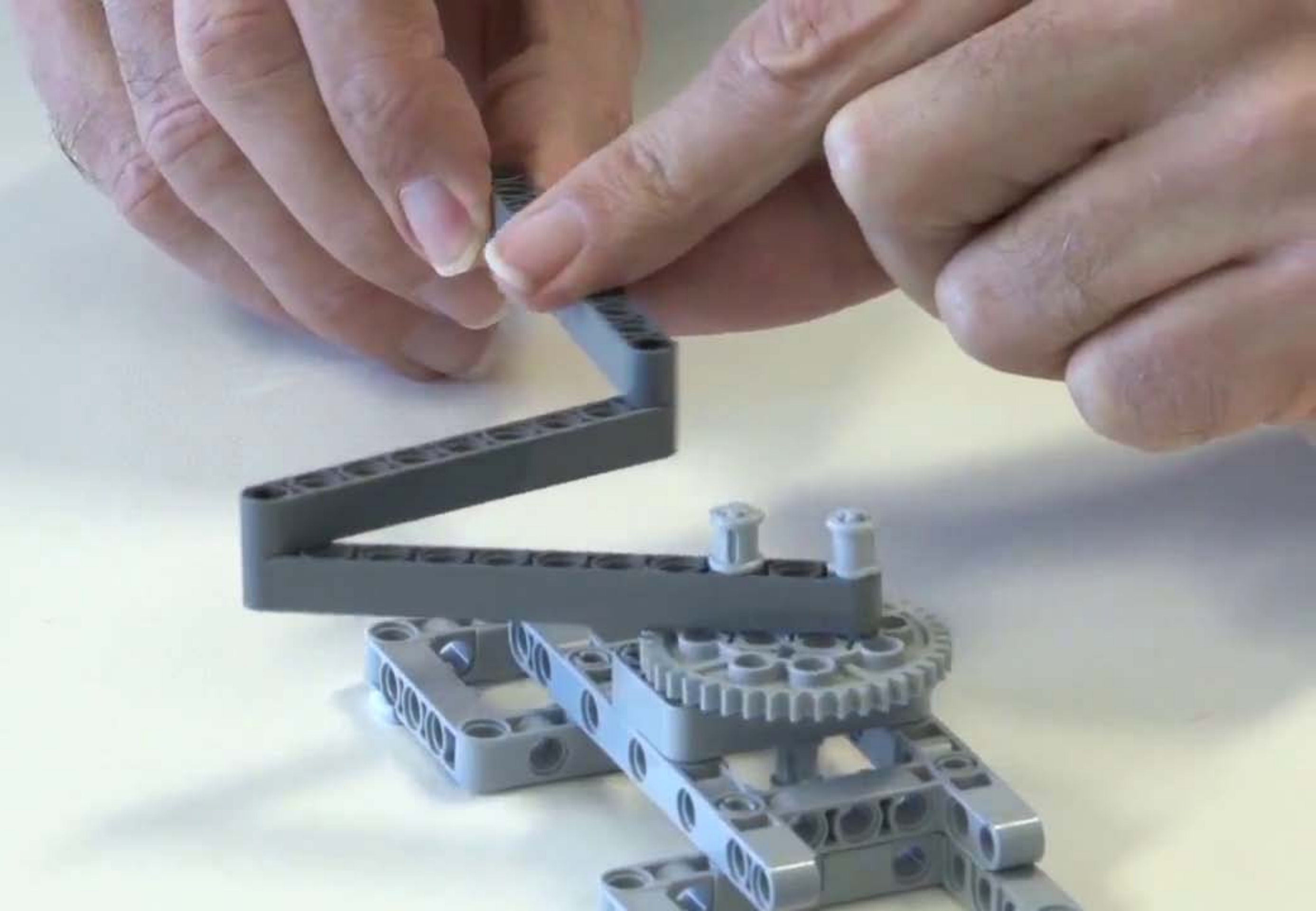
Section 4

A 3 joint planar robot

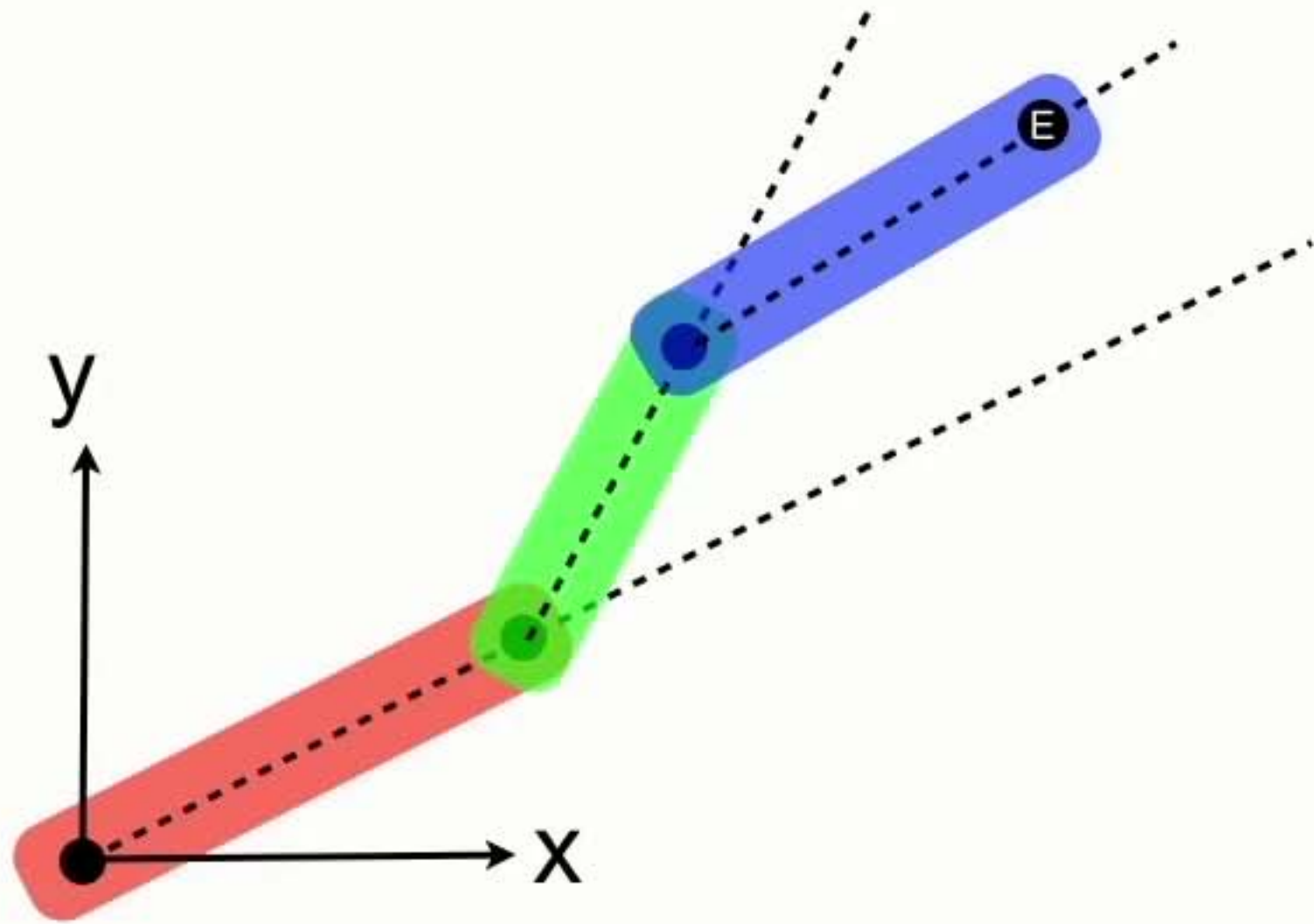




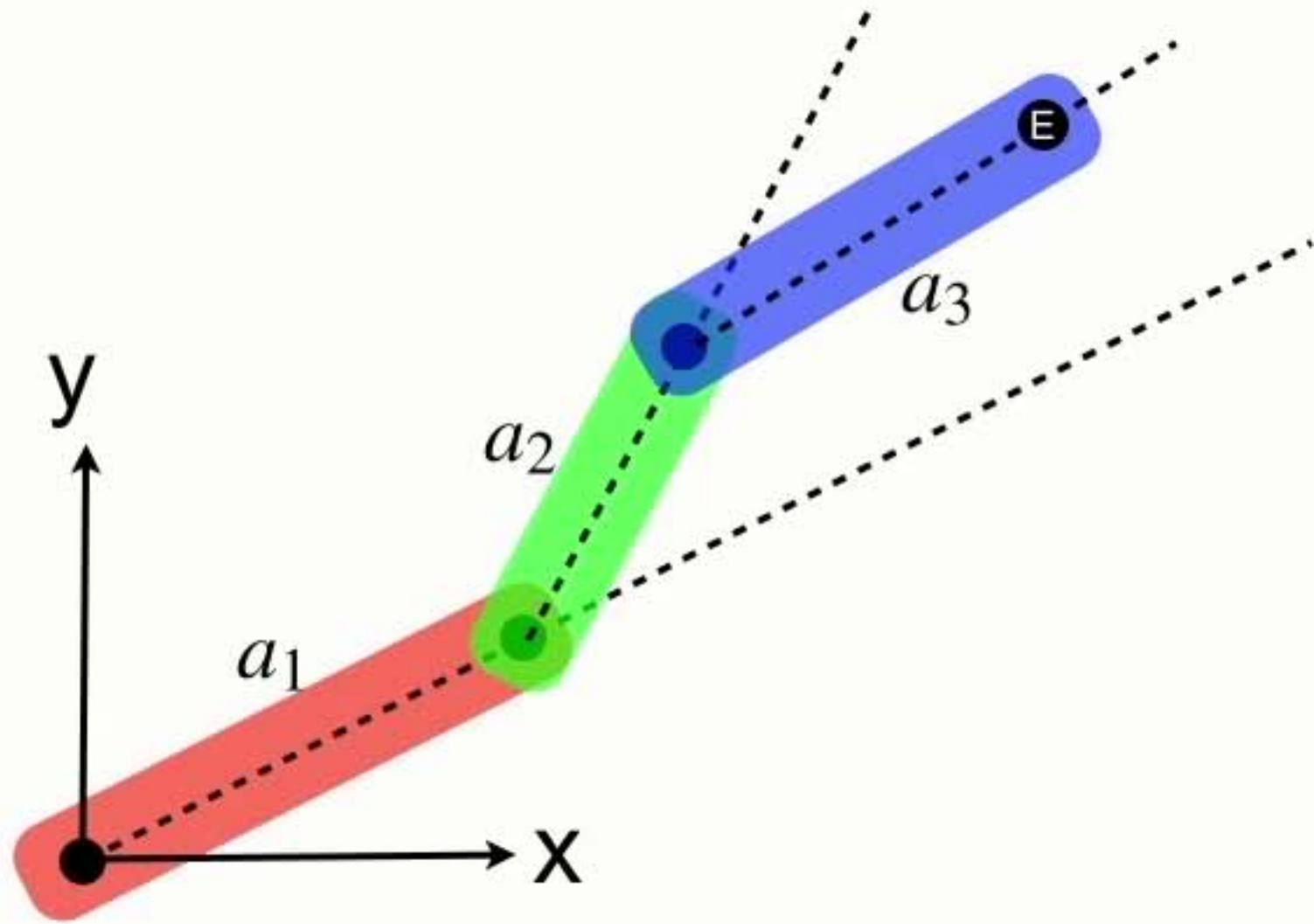




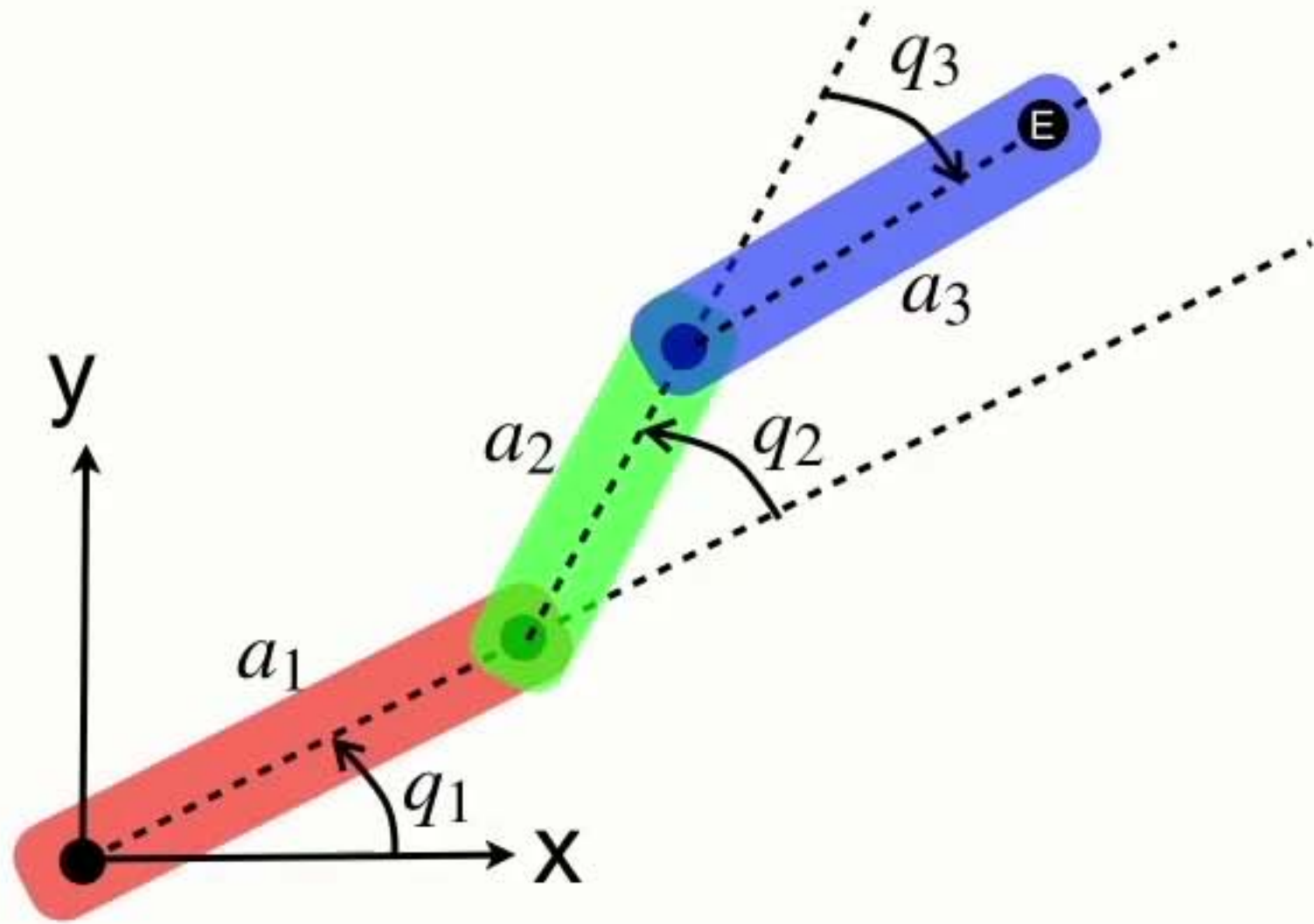
Tool tip pose



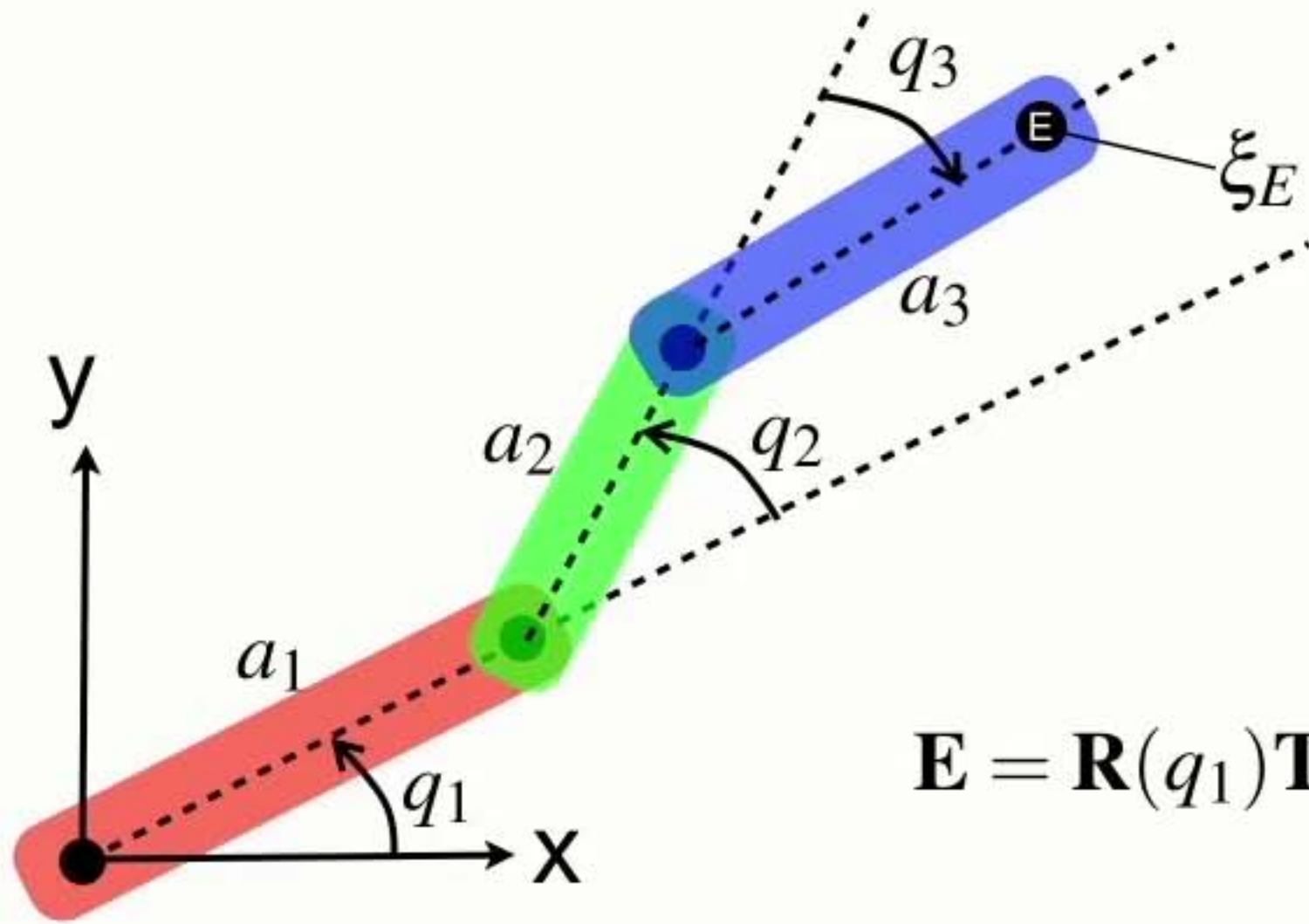
Tool tip pose



Tool tip pose



Tool tip pose



$$\mathbf{E} = \mathbf{R}(q_1)\mathbf{T}_x(a_1)\mathbf{R}(q_2)\mathbf{T}_x(a_2)\mathbf{R}(q_3)\mathbf{T}_x(a_3)$$

- These are all 2D transforms $\mathbf{E} \in \text{SE}(2)$

Command Window

```
>> syms a1 a2 a3 q1 q2 q3  
fx >>
```

Workspace

Name ▲	Value
 a1	1x1 sym
 a2	1x1 sym
 a3	1x1 sym
 q1	1x1 sym
 q2	1x1 sym
 q3	1x1 sym

Command Window

```
>> syms a1 a2 a3 q1 q2 q3
>>
>> trchain2('R(q1) Tx(a1) R(q2) Tx(a2) R(q3) Tx(a3)', [q1 q2 q3])

ans =

[ cos(q3)*(cos(q1)*cos(q2) - sin(q1)*sin(q2)) - sin(q3)*(cos(q1)*sin(q2) + cos(q2)*sin(q1)), - co
[ cos(q3)*(cos(q1)*sin(q2) + cos(q2)*sin(q1)) + sin(q3)*(cos(q1)*cos(q2) - sin(q1)*sin(q2)), co
[
0,
```

fx >>

Workspace

Name ▲	Value
a1	1x1 sym
a2	1x1 sym
a3	1x1 sym
ans	3x3 sym
q1	1x1 sym
q2	1x1 sym
q3	1x1 sym

Command Window

```
>> syms a1 a2 a3 q1 q2 q3
>>
>> trchain2('R(q1) Tx(a1) R(q2) Tx(a2) R(q3) Tx(a3)', [q1 q2 q3])

ans =

[ cos(q3)*(cos(q1)*cos(q2) - sin(q1)*sin(q2)) - sin(q3)*(cos(q1)*sin(q2) + cos(q2)*sin(q1)), - q1)*sin
[ cos(q3)*(cos(q1)*sin(q2) + cos(q2)*sin(q1)) + sin(q3)*(cos(q1)*cos(q2) - sin(q1)*sin(q2)), q1)*cos
[
0,

>> x = ans(1,3)

x =

a2*(cos(q1)*cos(q2) - sin(q1)*sin(q2)) + a1*cos(q1) + a3*(cos(q3)*(cos(q1)*cos(q2) - sin(q1)*sin(
```

Workspace

Name ▲	Value
a1	1x1 sym
a2	1x1 sym
a3	1x1 sym
ans	3x3 sym
q1	1x1 sym
q2	1x1 sym
q3	1x1 sym
x	1x1 sym

Command Window

```
>> syms a1 a2 a3 q1 q2 q3
>>
>> trchain2('R(q1) Tx(a1) R(q2) Tx(a2) R(q3) Tx(a3)', [q1 q2 q3])

ans =

[ cos(q3)*(cos(q1)*cos(q2) - sin(q1)*sin(q2)) - sin(q3)*(cos(q1)*sin(q2) + cos(q2)*sin(q1)), - d*cos(q
[ cos(q3)*(cos(q1)*sin(q2) + cos(q2)*sin(q1)) + sin(q3)*(cos(q1)*cos(q2) - sin(q1)*sin(q2)), d*sin(q
[
0,

>> x = ans(1,3)

x =

a2*(cos(q1)*cos(q2) - sin(q1)*sin(q2)) + a1*cos(q1) + a3*(cos(q3)*(cos(q1)*cos(q2) - sin(q1)*sin(

>> mdl_planar3
fx >> |
```

Workspace

Name ▲	Value
a1	1
a2	1
a3	1
ans	3x3 sym
p3	1x1 SerialLink
q1	1x1 sym
q2	1x1 sym
q3	1x1 sym
qz	[0,0,0]
x	1x1 sym

Command Window

```
ans =

[ cos(q3)*(cos(q1)*cos(q2) - sin(q1)*sin(q2)) - sin(q3)*(cos(q1)*sin(q2) + cos(q2)*sin(q1)), - cos(q3)*
[ cos(q3)*(cos(q1)*sin(q2) + cos(q2)*sin(q1)) + sin(q3)*(cos(q1)*cos(q2) - sin(q1)*sin(q2)), cos(q3)*
[
0,

>> x = ans(1,3)

x =

a2*(cos(q1)*cos(q2) - sin(q1)*sin(q2)) + a1*cos(q1) + a3*(cos(q3)*(cos(q1)*cos(q2) - sin(q1)*sin(q2)) -

>> mdl_planar3
>> p3.teach
fx >>
```

Workspace

Name	Value
a1	1
a2	1
a3	1
ans	3x3 sym
p3	1x1 SerialLink
q1	1x1 sym
q2	1x1 sym
q3	1x1 sym
qz	[0,0,0]
x	1x1 sym

Figures - Figure 1

Figure 1

Teach

x: 3.000

y: 0.000

z: 0.000

ax: 0.000

ay: 0.000

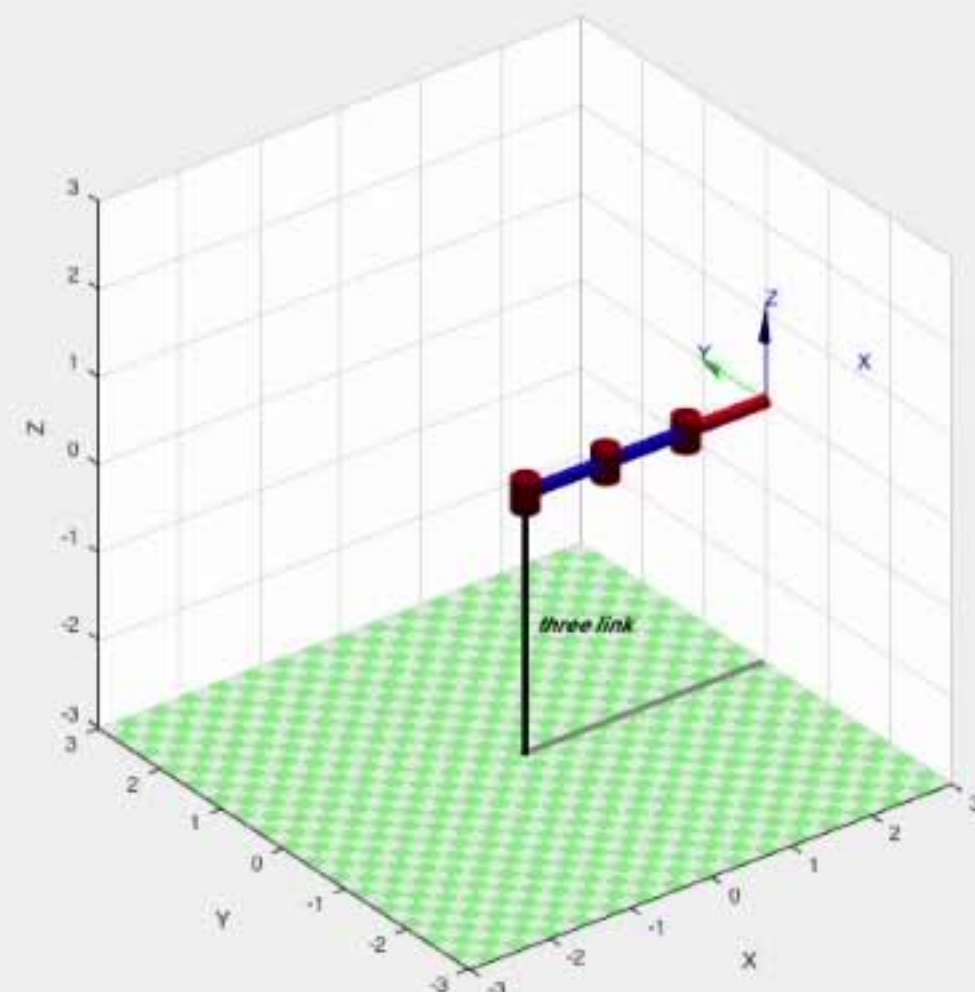
az: 1.000

q1 0

q2 0

q3 0

X



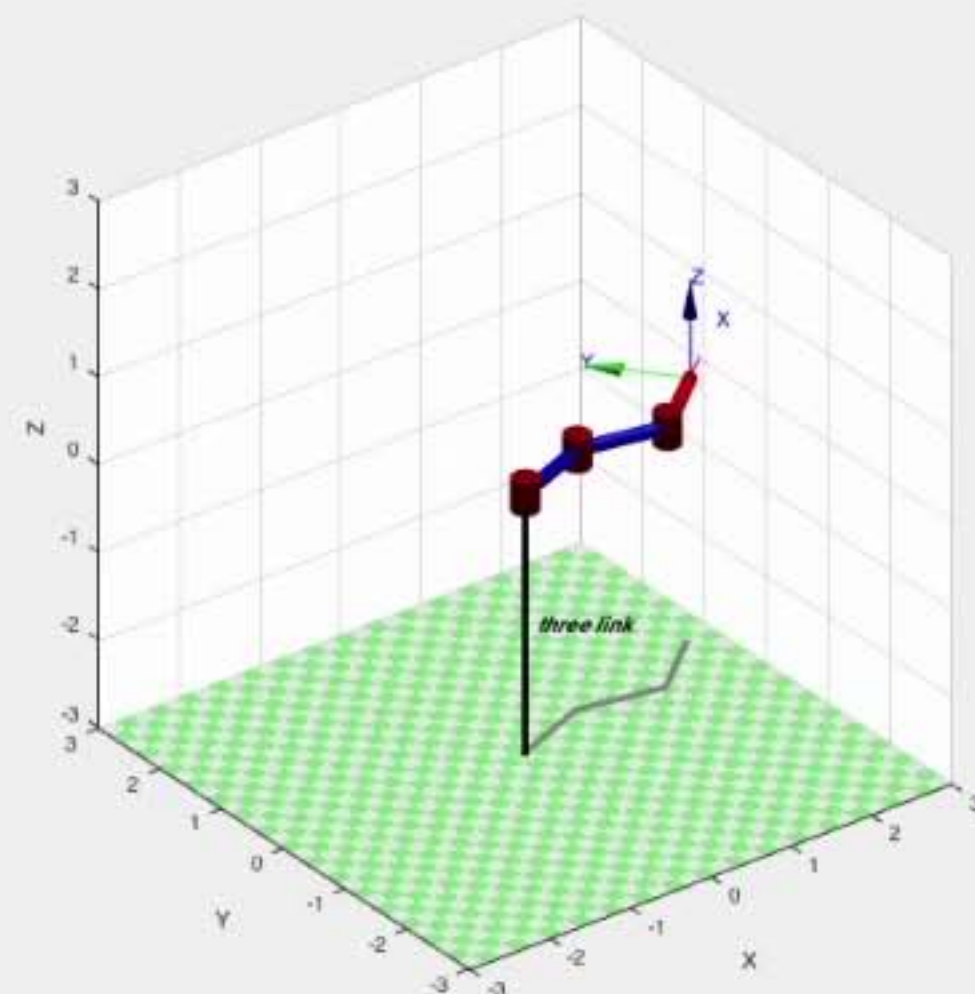
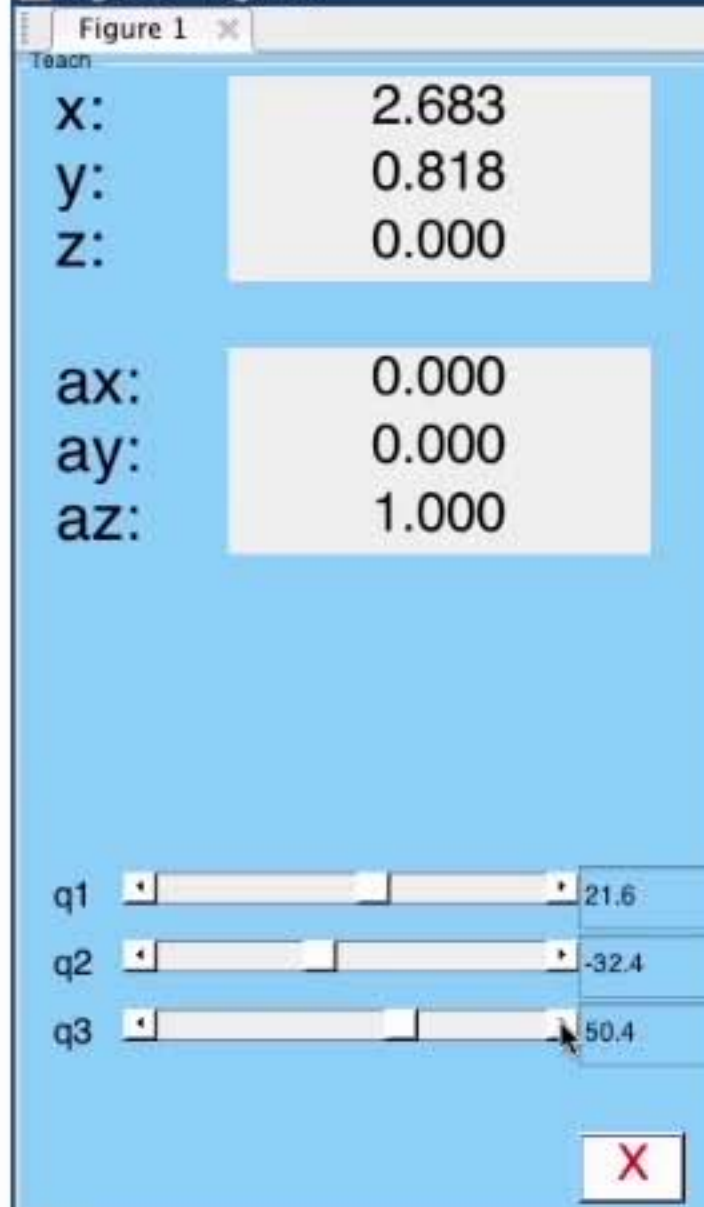
Command Window

```
ans =  
[ cos(q3)*(cos(q1)*cos(q2) - sin(q1)*sin(q2)) - sin(q3)*(cos(q1)*sin(q2) + cos(q2)*sin(q1)), - cos(q3)*  
[ cos(q3)*(cos(q1)*sin(q2) + cos(q2)*sin(q1)) + sin(q3)*(cos(q1)*cos(q2) - sin(q1)*sin(q2)), cos(q3)*  
[  
0,  
  
>> x = ans(1,3)  
  
x =  
  
a2*(cos(q1)*cos(q2) - sin(q1)*sin(q2)) + a1*cos(q1) + a3*(cos(q3)*(cos(q1)*cos(q2) - sin(q1)*sin(q2)) -  
  
>> mdl_planar3  
>> p3.teach  
fx >>
```

Workspace

Name	Value
a1	1
a2	1
a3	1
ans	3x3 sym
p3	1x1 SerialLink
q1	1x1 sym
q2	1x1 sym
q3	1x1 sym
qz	[0,0,0]
x	1x1 sym

Figures - Figure 1



Section 5

A 3D robot





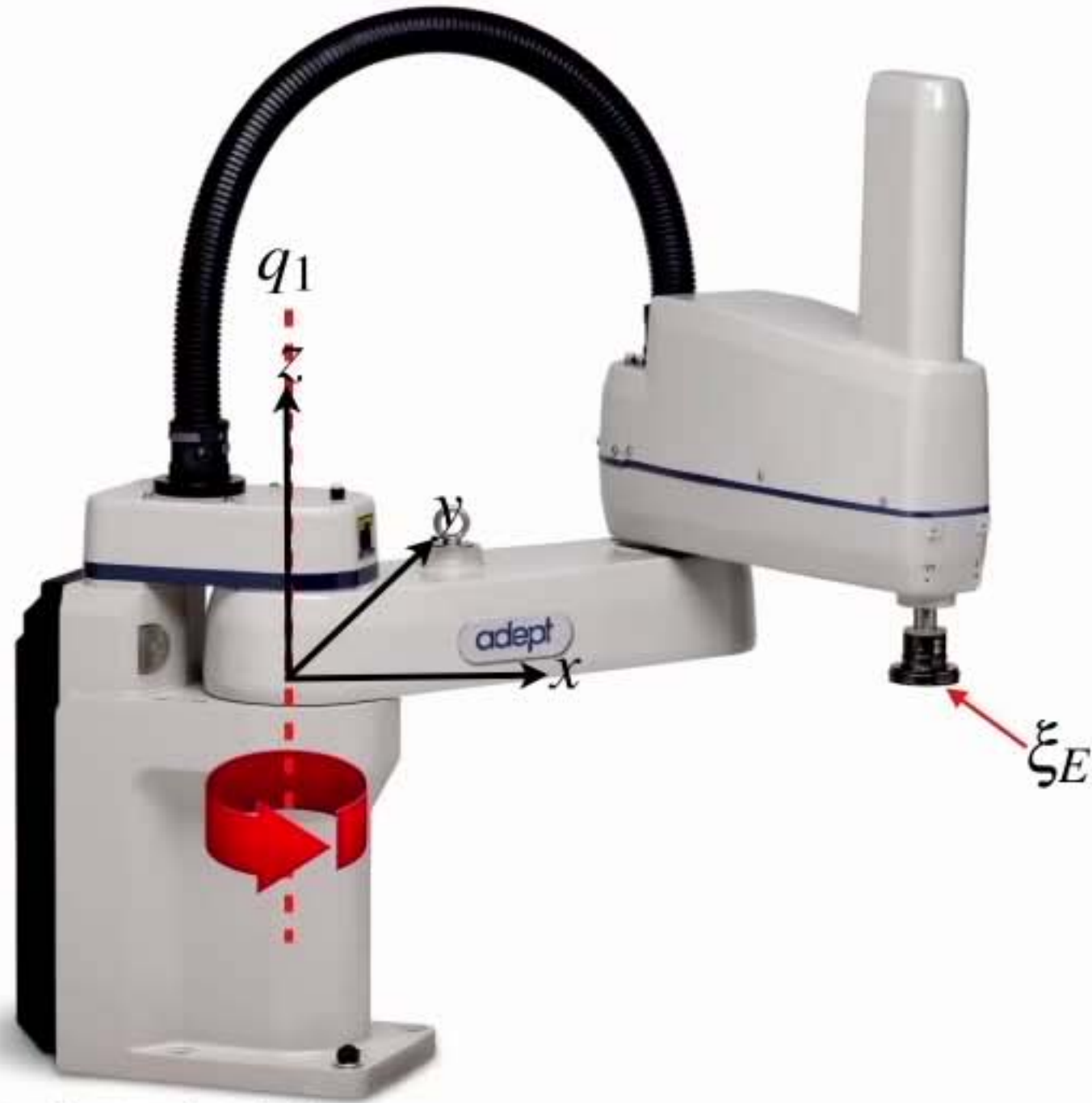
SCARA robot

Copyright © 2014 Adept Technology, Inc.
The usage of this image in no manner is to be construed as
Adept's endorsement of QUT or any QUT product or service



SCARA robot

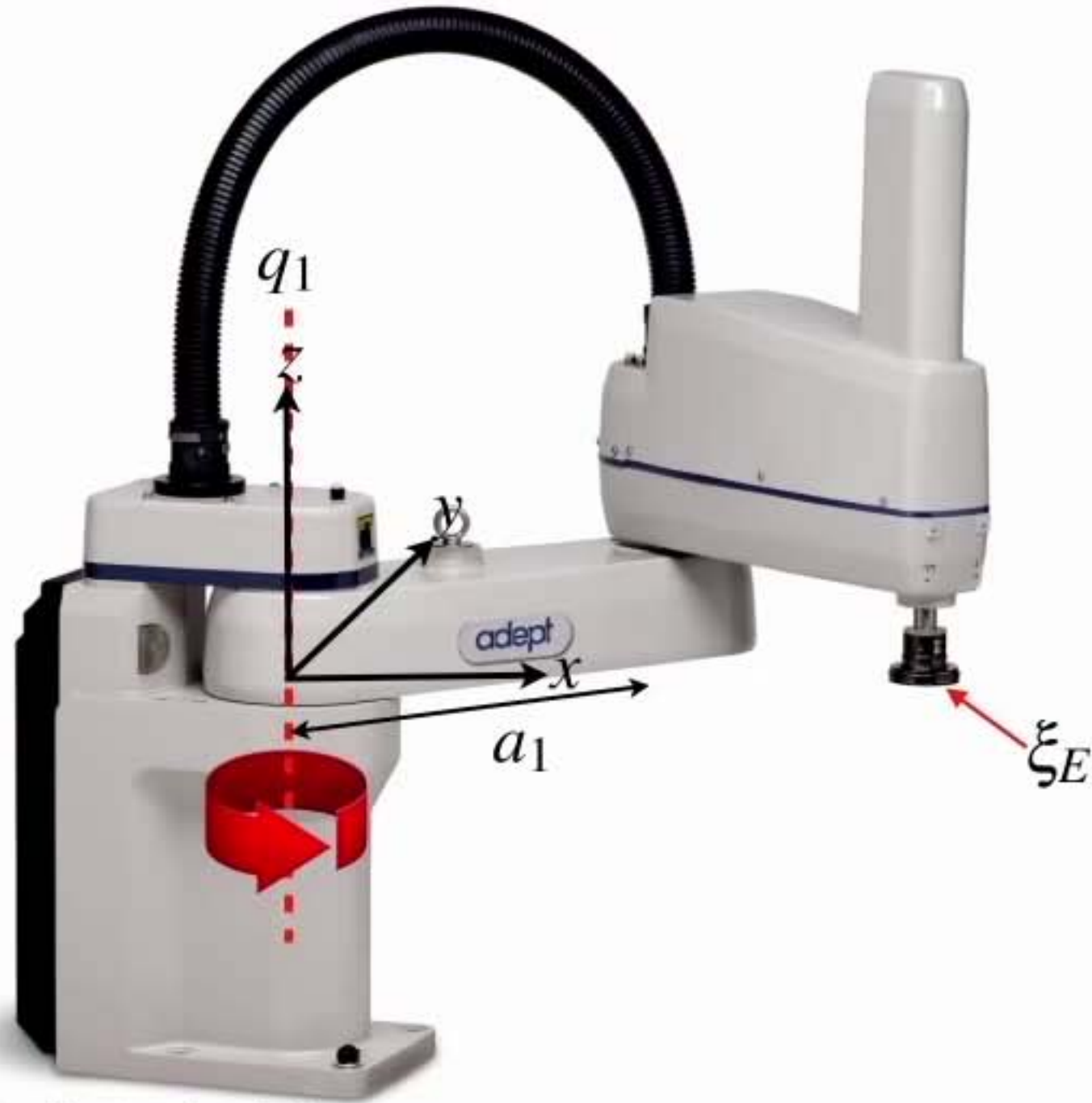
Copyright © 2014 Adept Technology, Inc.
The usage of this image in no manner is to be construed as
Adept's endorsement of QUT or any QUT product or service



SCARA robot

Copyright © 2014 Adept Technology, Inc.
 The usage of this image in no manner is to be construed as
 Adept's endorsement of QUT or any QUT product or service

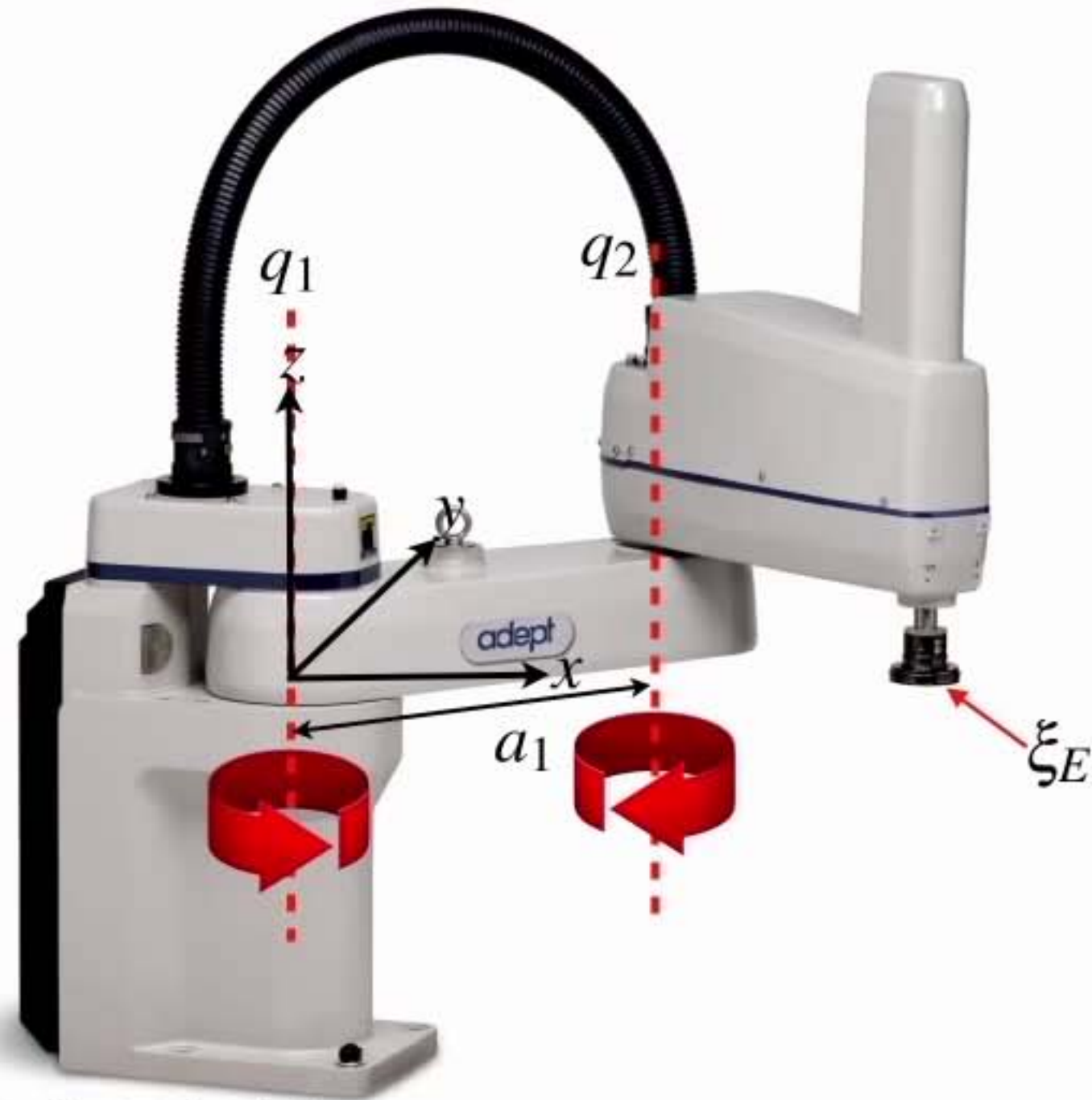
$$\mathbf{R}_z(q_1)$$



SCARA robot

Copyright © 2014 Adept Technology, Inc.
 The usage of this image in no manner is to be construed as
 Adept's endorsement of QUT or any QUT product or service

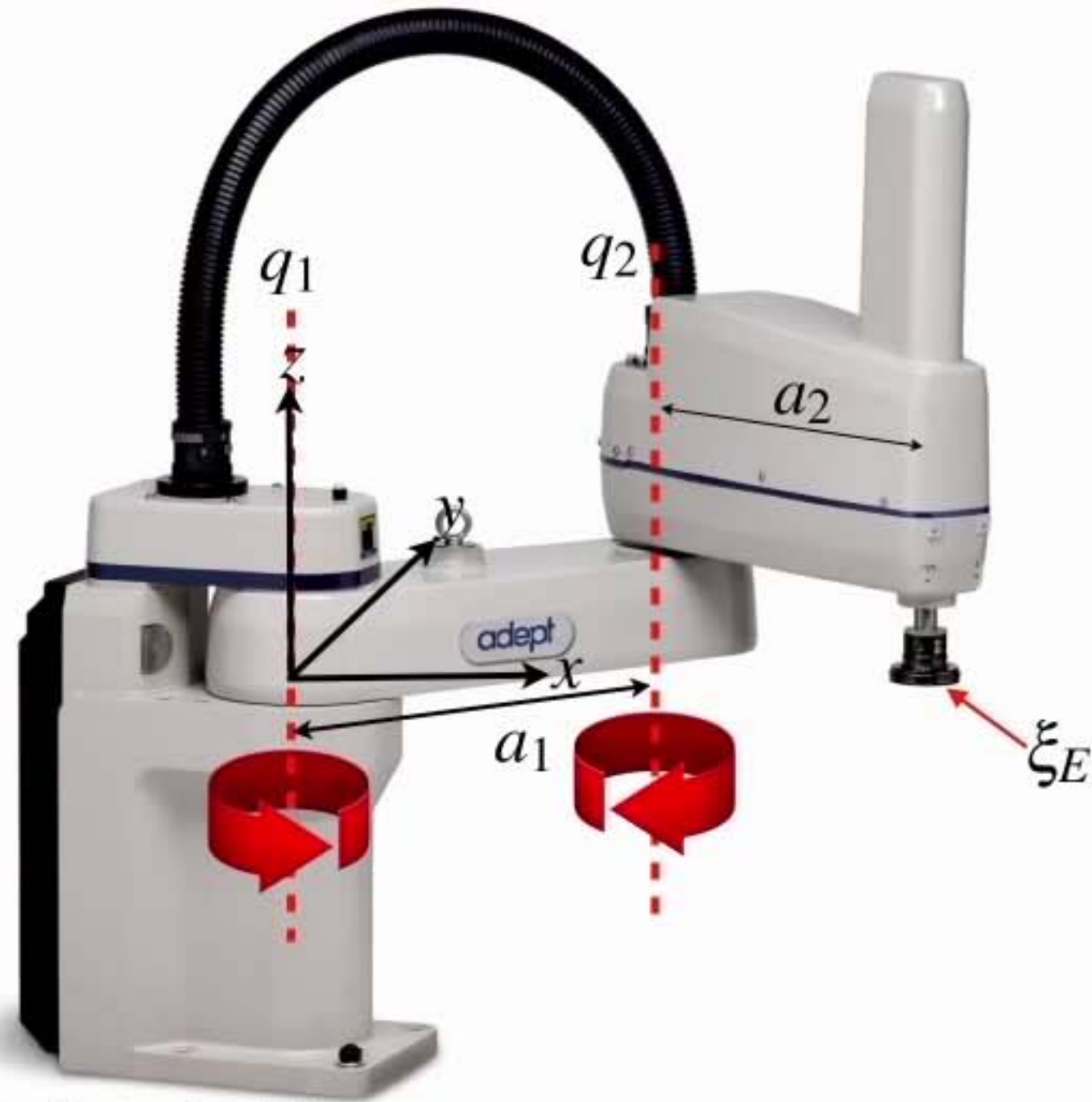
$$\mathbf{R}_z(q_1) \mathbf{T}_x(a_1)$$



SCARA robot

Copyright © 2014 Adept Technology, Inc.
 The usage of this image in no manner is to be construed as
 Adept's endorsement of QUT or any QUT product or service

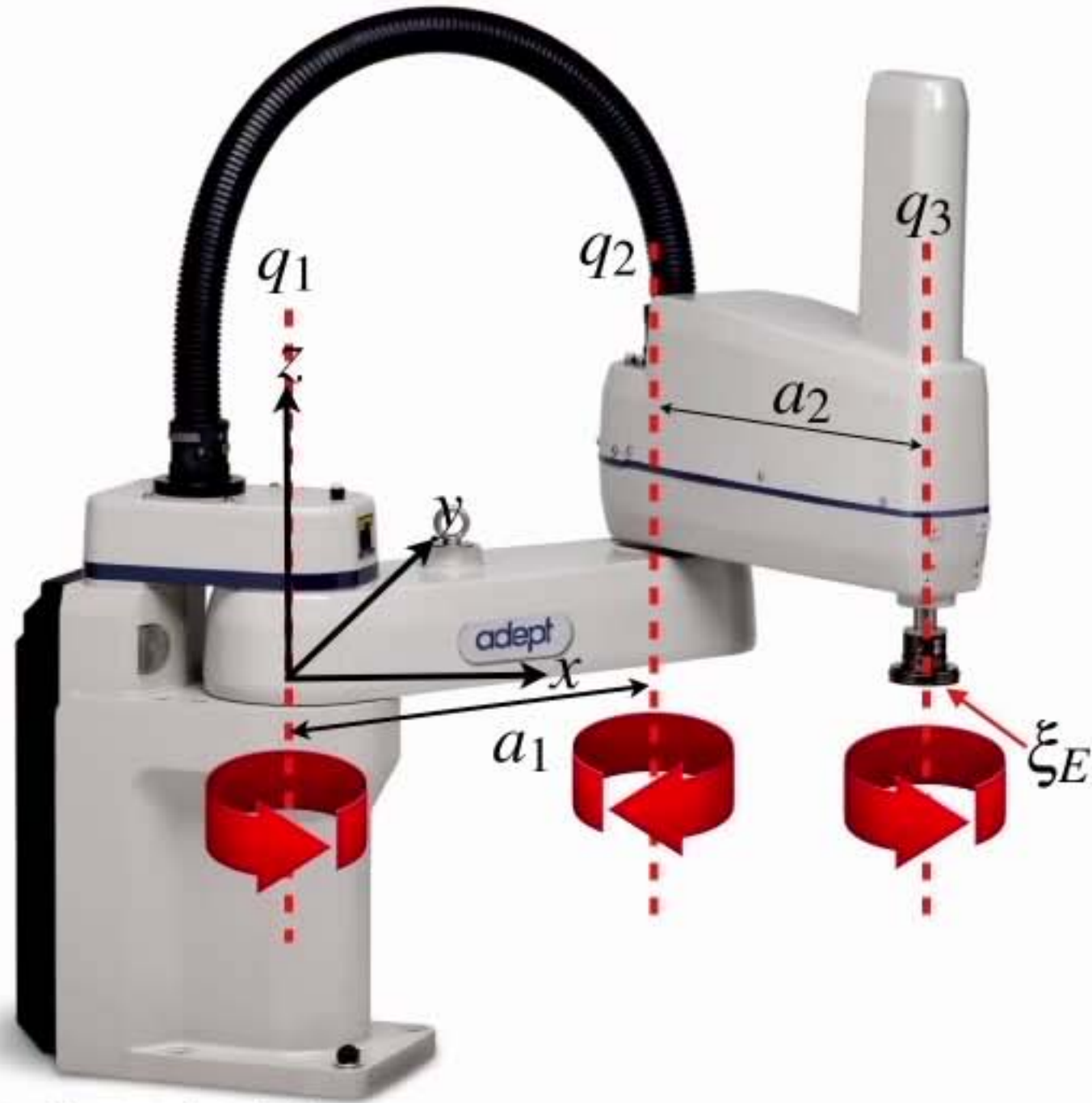
$$\mathbf{R}_z(q_1) \mathbf{T}_x(a_1) \mathbf{R}_z(q_2)$$



SCARA robot

Copyright © 2014 Adept Technology, Inc.
 The usage of this image in no manner is to be construed as
 Adept's endorsement of QUT or any QUT product or service

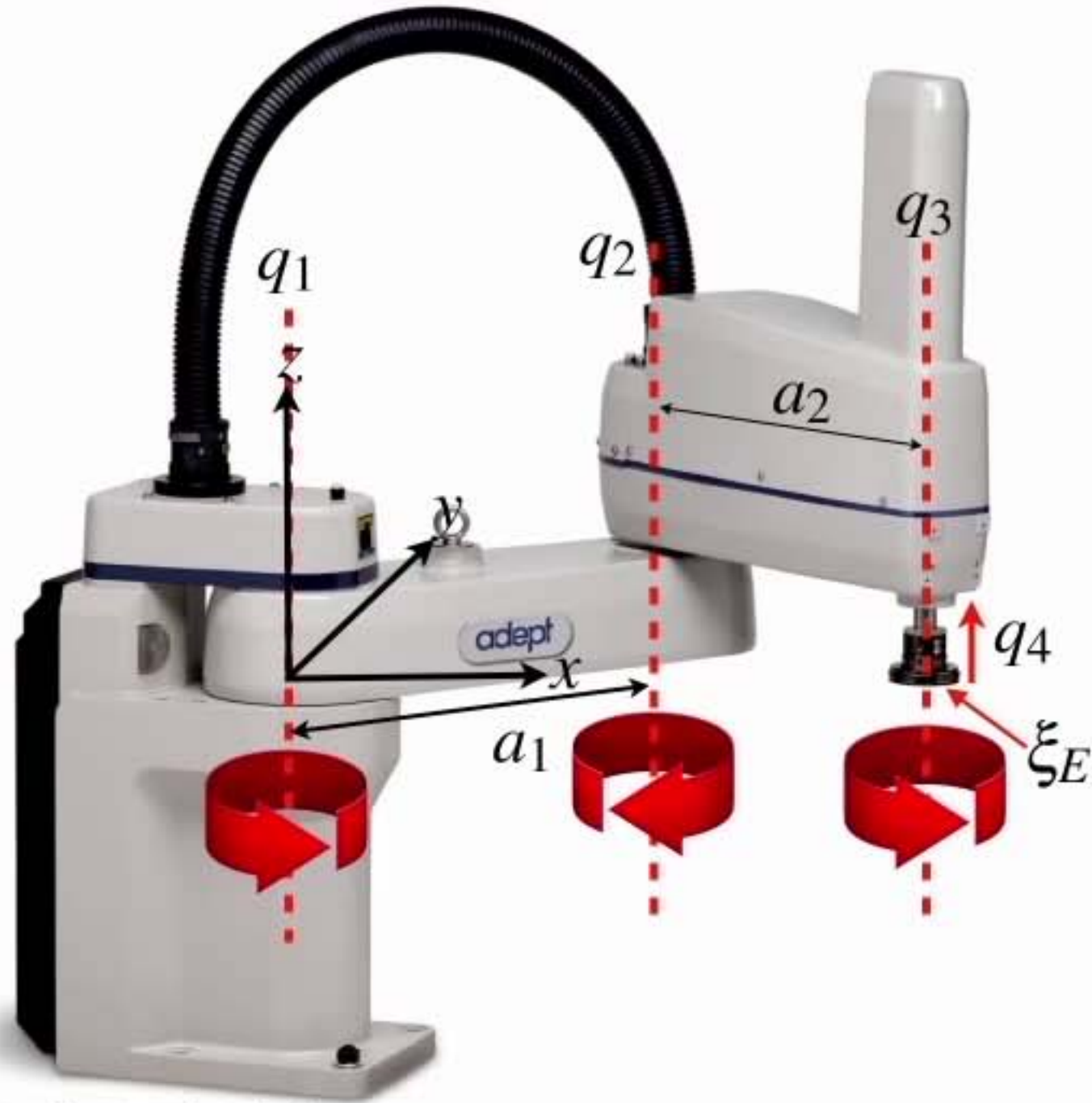
$$\mathbf{R}_z(q_1) \mathbf{T}_x(a_1) \mathbf{R}_z(q_2) \mathbf{T}_x(a_2)$$



SCARA robot

Copyright © 2014 Adept Technology, Inc.
The usage of this image in no manner is to be construed as
Adept's endorsement of QUT or any QUT product or service

$$\mathbf{R}_z(q_1) \mathbf{T}_x(a_1) \mathbf{R}_z(q_2) \mathbf{T}_x(a_2) \mathbf{R}_z(q_3)$$

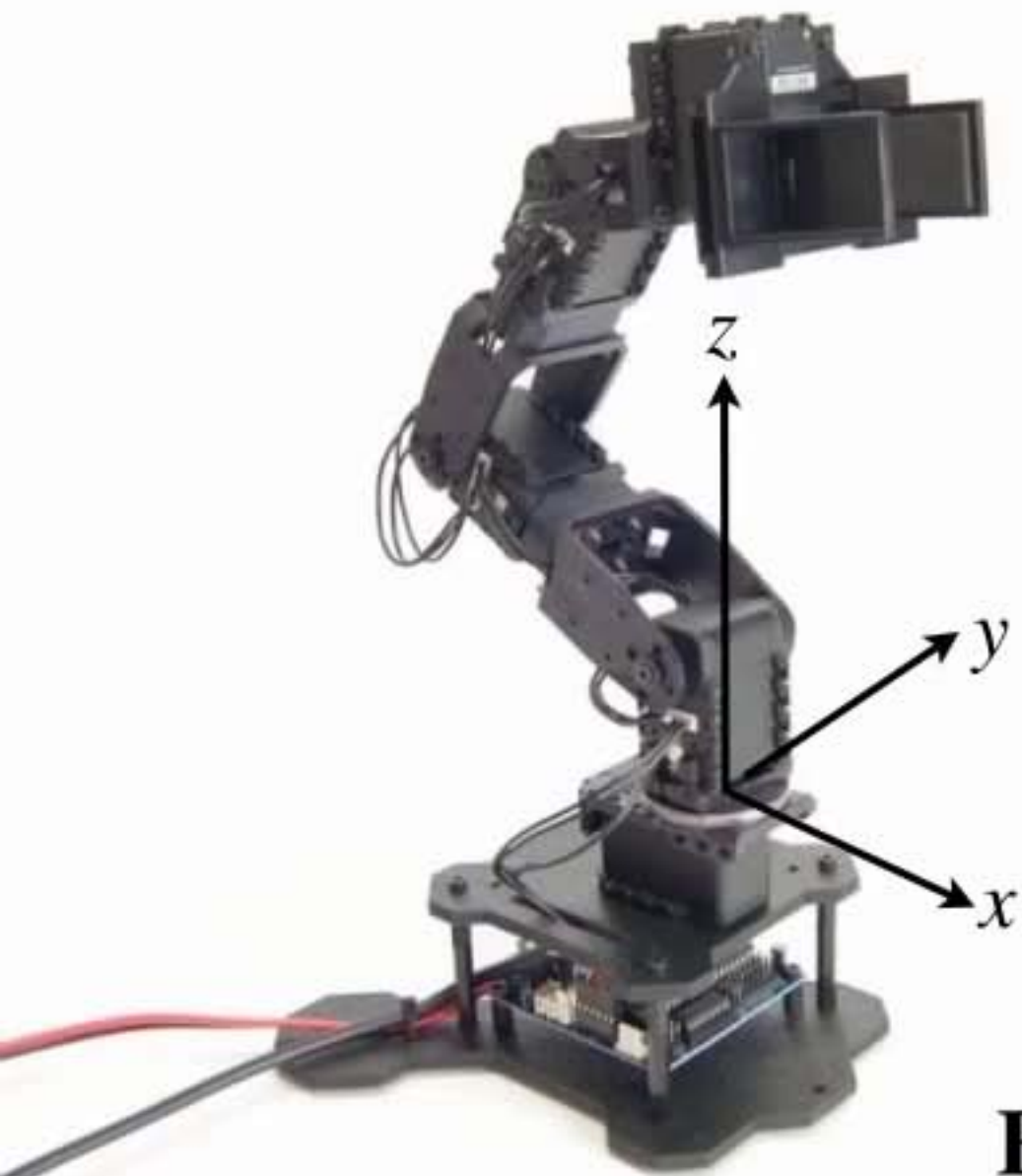


SCARA robot

Copyright © 2014 Adept Technology, Inc.
The usage of this image in no manner is to be construed as
Adept's endorsement of QUT or any QUT product or service

$$\mathbf{E} = \mathbf{R}_z(q_1) \mathbf{T}_x(a_1) \mathbf{R}_z(q_2) \mathbf{T}_x(a_2) \mathbf{R}_z(q_3) \mathbf{T}_z(q_4)$$

- These are all 3D transforms $\mathbf{E} \in \text{SE}(3)$



PhantomX Pincher Robot Arm 2014
Trossen Robotics | Used with permission

$$\mathbf{E} = \mathbf{R}_z(q_1)\mathbf{T}_z(a_1)\mathbf{R}_y(q_2)\mathbf{T}_z(a_2)\mathbf{R}_y(q_3)\mathbf{T}_z(a_3)\mathbf{R}_y(q_4)\mathbf{T}_z(a_4)$$

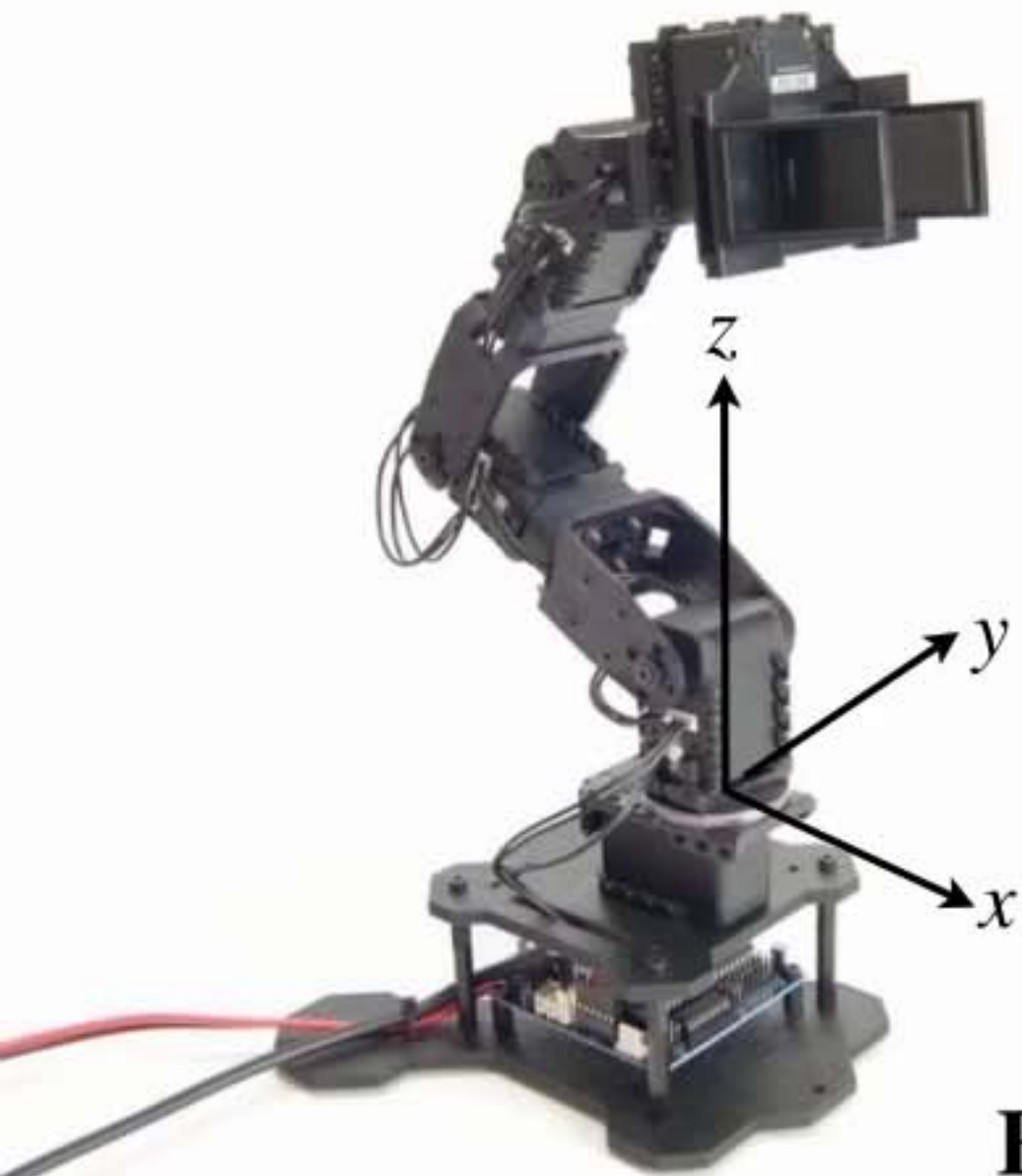
- These are all 3D transforms $\mathbf{E} \in \text{SE}(3)$











PhantomX Pincher Robot Arm 2014
Trossen Robotics | Used with permission

$$\mathbf{E} = \mathbf{R}_z(q_1)\mathbf{T}_z(a_1)\mathbf{R}_y(q_2)\mathbf{T}_z(a_2)\mathbf{R}_y(q_3)\mathbf{T}_z(a_3)\mathbf{R}_y(q_4)\mathbf{T}_z(a_4)$$

- These are all 3D transforms $\mathbf{E} \in \text{SE}(3)$

Command Window

```
>> syms a1 a2 a3 a4 q1 q2 q3 q4  
fx >>
```

Workspace

Name ▲	Value
 a1	1x1 sym
 a2	1x1 sym
 a3	1x1 sym
 a4	1x1 sym
 q1	1x1 sym
 q2	1x1 sym
 q3	1x1 sym
 q4	1x1 sym

I

Command Window

```
>> syms a1 a2 a3 a4 q1 q2 q3 q4
>> trchain('Rz(q1)Tz(a1)Ry(q2)Tz(a2)Ry(q3)Tz(a3)Ry(q4)Tz(a4)', [q1 q2 q3 q4])
```

ans =

```
[ cos(q4)*(cos(q1)*cos(q2)*cos(q3) - cos(q1)*sin(q2)*sin(q3)) - sin(q4)*(cos(q1)*cos(q2)*sin(q3) + cos(q1)*cos(q3)*sin(q2)
[ - cos(q4)*(sin(q1)*sin(q2)*sin(q3) - cos(q2)*cos(q3)*sin(q1)) - sin(q4)*(cos(q2)*sin(q1)*sin(q3) + cos(q3)*sin(q1)*sin(q2)
[      - cos(q4)*(cos(q2)*sin(q3) + cos(q3)*sin(q2)) - sin(q4)*(cos(q2)*cos(q3) - sin(q2)*sin(q3)
[
```

fx >> |

Workspace

Name ▲	Value
a1	1x1 sym
a2	1x1 sym
a3	1x1 sym
a4	1x1 sym
ans	4x4 sym
q1	1x1 sym
q2	1x1 sym
q3	1x1 sym
q4	1x1 sym

Command Window

```
>> syms a1 a2 a3 a4 q1 q2 q3 q4
>> trchain('Rz(q1)Tz(a1)Ry(q2)Tz(a2)Ry(q3)Tz(a3)Ry(q4)Tz(a4)', [q1 q2 q3 q4])

ans =

[ cos(q4)*(cos(q1)*cos(q2)*cos(q3) - cos(q1)*sin(q2)*sin(q3)) - sin(q4)*(cos(q1)*cos(q2)*sin(q3) + cos(q1)*cos(q3)*sin(q2)
[ - cos(q4)*(sin(q1)*sin(q2)*sin(q3) - cos(q2)*cos(q3)*sin(q1)) - sin(q4)*(cos(q2)*sin(q1)*sin(q3) + cos(q3)*sin(q1)*sin(q2)
[
- cos(q4)*(cos(q2)*sin(q3) + cos(q3)*sin(q2)) - sin(q4)*(cos(q2)*cos(q3) - sin(q2)*sin(q3)
[

>> ans(1,4)

ans =

a4*(cos(q4)*(cos(q1)*cos(q2)*sin(q3) + cos(q1)*cos(q3)*sin(q2)) + sin(q4)*(cos(q1)*cos(q2)*cos(q3) - cos(q1)*sin(q2)*sin(q3)
```

fx >>

Workspace

Name	Value
a1	1x1 sym
a2	1x1 sym
a3	1x1 sym
a4	1x1 sym
ans	1x1 sym
q1	1x1 sym
q2	1x1 sym
q3	1x1 sym
q4	1x1 sym

Section 6

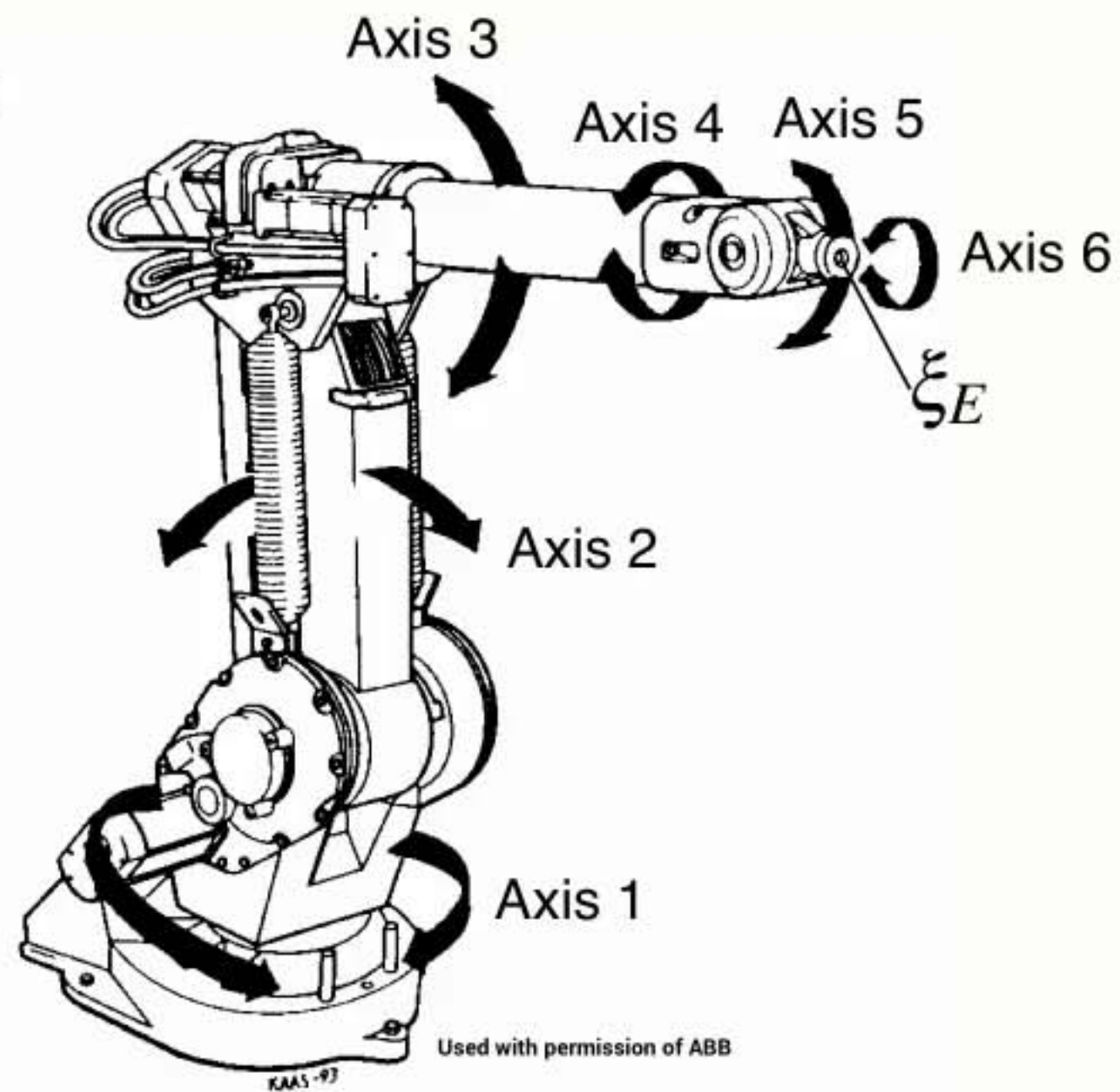
A general 3D robot



Used with permission of ABB



Unimate Puma 500
Oussama Khatib | Used with permission



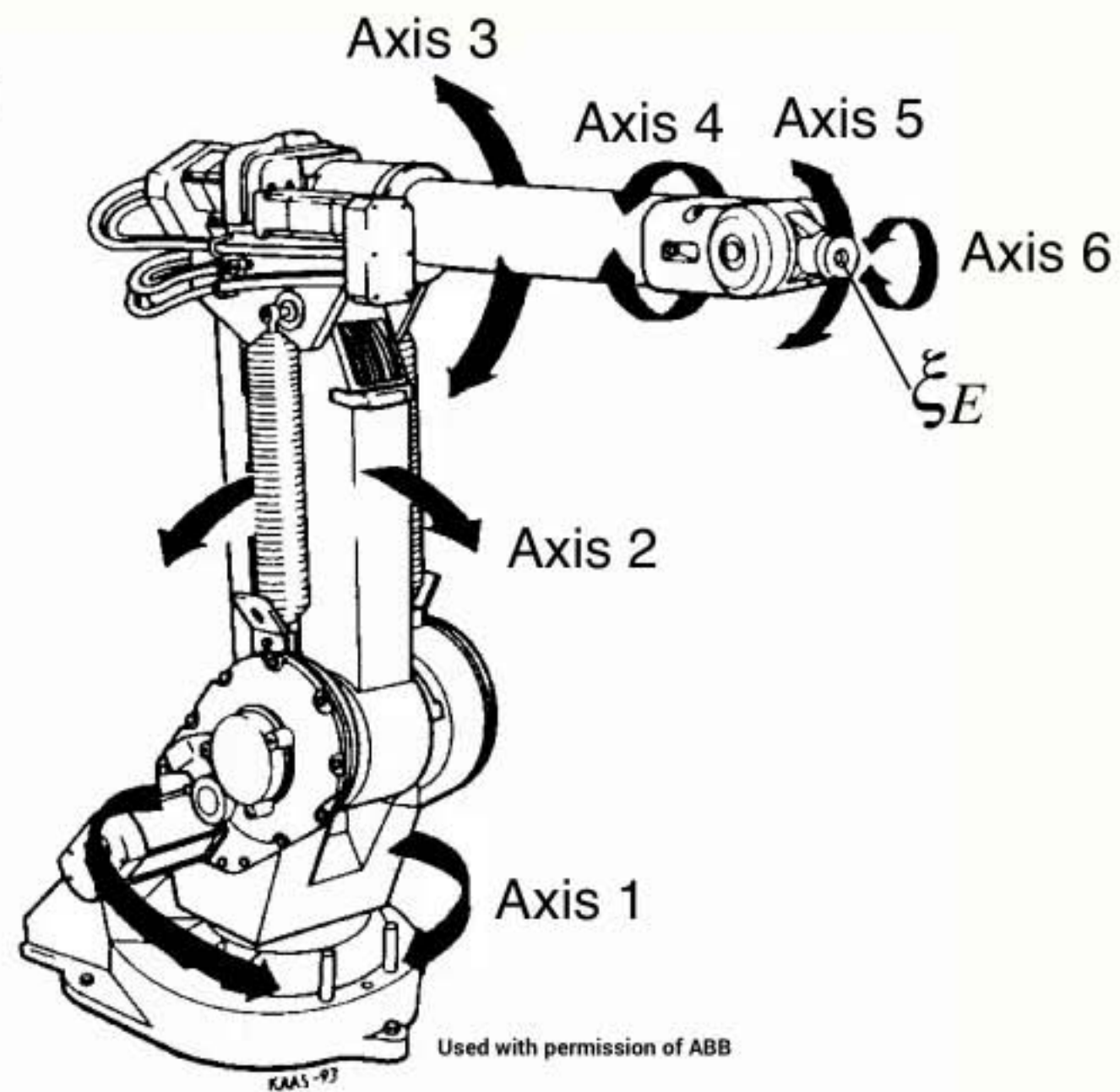
Used with permission of ABB



Used with permission of ABB



Unimate Puma 500
Oussama Khatib | Used with permission



Used with permission of ABB

Section 7

Task and configuration space

Configuration **space**

- Robot **configuration** is described by a vector of **generalised** joint coordinates

Joint
configuration

Joint
coordinate

$$\mathbf{q} = \{q_j, j \in [1 \cdots N]\}$$

Configuration **space**

- Robot **configuration** is described by a vector of **generalised** joint coordinates
- Each coordinate can be:
 - ➔ an angle, for a rotational (**revolute**) joint
 - ➔ a length, for a sliding (**prismatic**) joint

Joint
configuration

Joint
coordinate

Number of
joints

$$\mathbf{q} = \{q_j, j \in [1 \cdots N]\}$$

Configuration **space**

- Robot **configuration** is described by a vector of **generalised** joint coordinates
- Each coordinate can be:
 - ➔ an angle, for a rotational (**revolute**) joint
 - ➔ a length, for a sliding (**prismatic**) joint

Joint
configuration

Joint
coordinate

Number of
joints

Space of all
possible
configurations

$$\mathbf{q} = \{q_j, j \in [1 \cdots N]\} \in \mathcal{C}$$

Configuration space $\mathcal{C} \subset \mathbb{R}^N$

Task **space**

- The space of all possible **end-effector poses**

$$\xi_E \in \mathcal{T}$$



Space of
all possible
end-effector poses

Task **space**

- The space of all possible **end-effector poses**

$$\xi_E \in \mathcal{T}$$



Space of
all possible
end-effector poses

- In 2D $\mathcal{T} \subset \text{SE}(2)$

Task **space**

- The space of all possible **end-effector poses**

$$\xi_E \in \mathcal{T}$$



Space of
all possible
end-effector poses

- In 2D $\mathcal{T} \subset \text{SE}(2)$

$$\xi_E \sim (x, y)$$

Task **space**

- The space of all possible **end-effector poses**

$$\xi_E \in \mathcal{T}$$



Space of
all possible
end-effector poses

- In 2D $\mathcal{T} \subset \text{SE}(2)$

$$\xi_E \sim (x, y)$$

$$\xi_E \sim (x, y, \theta)$$

Task **space**

- The space of all possible **end-effector poses**

$$\xi_E \in \mathcal{T}$$



Space of
all possible
end-effector poses

- In 2D $\mathcal{T} \subset \text{SE}(2)$

$$\xi_E \sim (x, y)$$

$$\xi_E \sim (x, y, \theta)$$

- In 3D $\mathcal{T} \subset \text{SE}(3)$

Task **space**

- The space of all possible **end-effector poses**

$$\xi_E \in \mathcal{T}$$

Space of
all possible
end-effector poses

- In 2D $\mathcal{T} \subset \text{SE}(2)$

$$\xi_E \sim (x, y)$$

$$\xi_E \sim (x, y, \theta)$$

- In 3D $\mathcal{T} \subset \text{SE}(3)$

$$\xi_E \sim (x, y, z)$$

Task **space**

- The space of all possible **end-effector poses**

$$\xi_E \in \mathcal{T}$$

Space of
all possible
end-effector poses

- In 2D $\mathcal{T} \subset \text{SE}(2)$

$$\xi_E \sim (x, y)$$

$$\xi_E \sim (x, y, \theta)$$

- In 3D $\mathcal{T} \subset \text{SE}(3)$

$$\xi_E \sim (x, y, z)$$

$$\xi_E \sim (x, y, z, \theta_p)$$

Task **space**

- The space of all possible **end-effector poses**

$$\xi_E \in \mathcal{T}$$

Space of
all possible
end-effector poses

- In 2D $\mathcal{T} \subset \text{SE}(2)$

$$\xi_E \sim (x, y)$$

$$\xi_E \sim (x, y, \theta)$$

- In 3D $\mathcal{T} \subset \text{SE}(3)$

$$\xi_E \sim (x, y, z)$$

$$\xi_E \sim (x, y, z, \theta_p)$$

$$\xi_E \sim (x, y, z, \theta_r, \theta_p, \theta_y)$$

Task and configuration space **dimensions**

Task and configuration space **dimensions**

- Robot degrees of freedom (number of joints) $\dim \mathcal{C}$

Task and configuration space **dimensions**

- Robot degrees of freedom (number of joints) $\dim \mathcal{C}$
- Task space degrees of freedom $\dim \mathcal{T}$

Task and configuration space **dimensions**

- Robot degrees of freedom (number of joints) $\dim \mathcal{C}$
- Task space degrees of freedom $\dim \mathcal{T}$

$$\xi_E \sim (x, y) \quad \rightarrow \dim \mathcal{T} = 2$$

Task and configuration space **dimensions**

- Robot degrees of freedom (number of joints) $\dim \mathcal{C}$
- Task space degrees of freedom $\dim \mathcal{T}$

$$\xi_E \sim (x, y) \quad \rightarrow \dim \mathcal{T} = 2$$

$$\xi_E \sim (x, y, \theta) \quad \rightarrow \dim \mathcal{T} = 3$$

Task and configuration space **dimensions**

- Robot degrees of freedom (number of joints) $\dim \mathcal{C}$
- Task space degrees of freedom $\dim \mathcal{T}$

$$\xi_E \sim (x, y) \quad \rightarrow \dim \mathcal{T} = 2$$

$$\xi_E \sim (x, y, \theta) \quad \rightarrow \dim \mathcal{T} = 3$$

$$\xi_E \sim (x, y, z) \quad \rightarrow \dim \mathcal{T} = 3$$

Task and configuration space **dimensions**

- Robot degrees of freedom (number of joints) $\dim \mathcal{C}$
- Task space degrees of freedom $\dim \mathcal{T}$

$$\xi_E \sim (x, y) \quad \rightarrow \dim \mathcal{T} = 2$$

$$\xi_E \sim (x, y, \theta) \quad \rightarrow \dim \mathcal{T} = 3$$

$$\xi_E \sim (x, y, z) \quad \rightarrow \dim \mathcal{T} = 3$$

$$\xi_E \sim (x, y, z, \theta_p) \quad \rightarrow \dim \mathcal{T} = 4$$

Task and configuration space **dimensions**

- Robot degrees of freedom (number of joints) $\dim \mathcal{C}$
- Task space degrees of freedom $\dim \mathcal{T}$

$$\xi_E \sim (x, y) \rightarrow \dim \mathcal{T} = 2$$

$$\xi_E \sim (x, y, \theta) \rightarrow \dim \mathcal{T} = 3$$

$$\xi_E \sim (x, y, z) \rightarrow \dim \mathcal{T} = 3$$

$$\xi_E \sim (x, y, z, \theta_p) \rightarrow \dim \mathcal{T} = 4$$

$$\xi_E \sim (x, y, z, \theta_r, \theta_p, \theta_y) \rightarrow \dim \mathcal{T} = 6$$



Maximum
possible
value

Task and configuration space **dimensions**

- Robot degrees of freedom (number of joints) $\dim \mathcal{C}$
- Task space degrees of freedom $\dim \mathcal{T}$

$$\xi_E \sim (x, y) \rightarrow \dim \mathcal{T} = 2$$

$$\xi_E \sim (x, y, \theta) \rightarrow \dim \mathcal{T} = 3$$

$$\xi_E \sim (x, y, z) \rightarrow \dim \mathcal{T} = 3$$

$$\xi_E \sim (x, y, z, \theta_p) \rightarrow \dim \mathcal{T} = 4$$

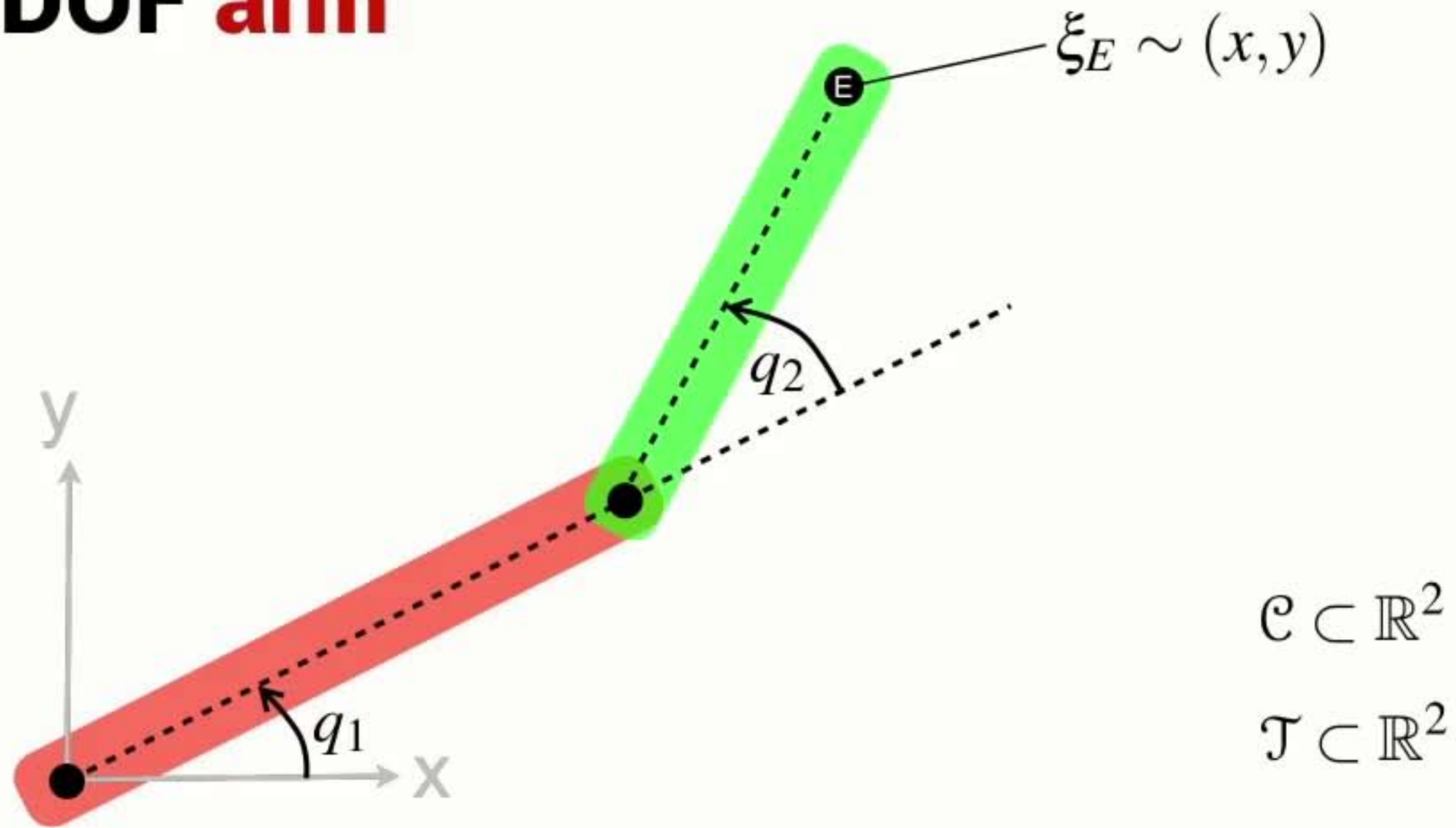
$$\xi_E \sim (x, y, z, \theta_r, \theta_p, \theta_y) \rightarrow \dim \mathcal{T} = 6$$



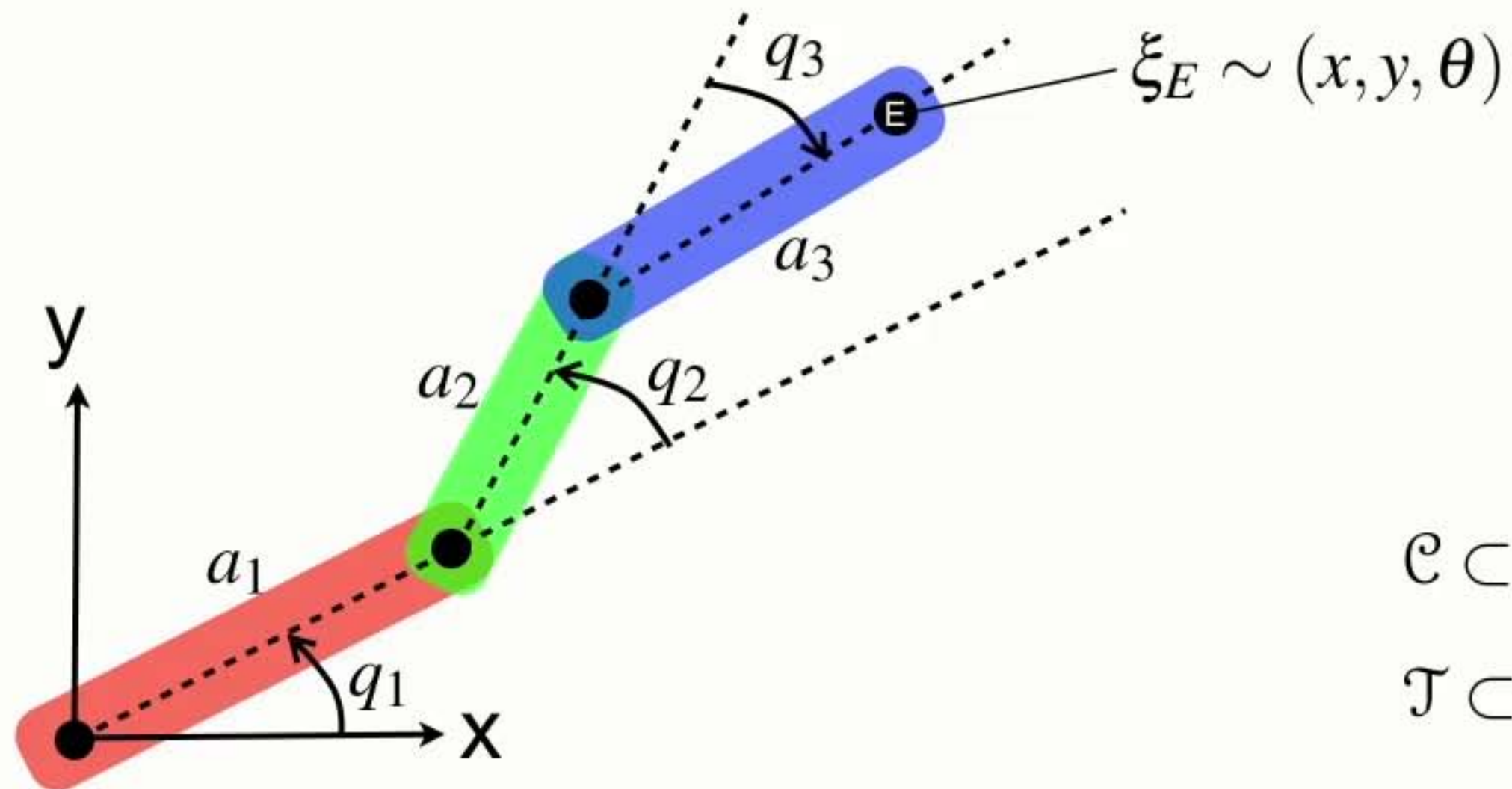
Maximum
possible
value

- **To reach all of the task space** $\dim \mathcal{C} \geq \dim \mathcal{T}$

2-DOF arm



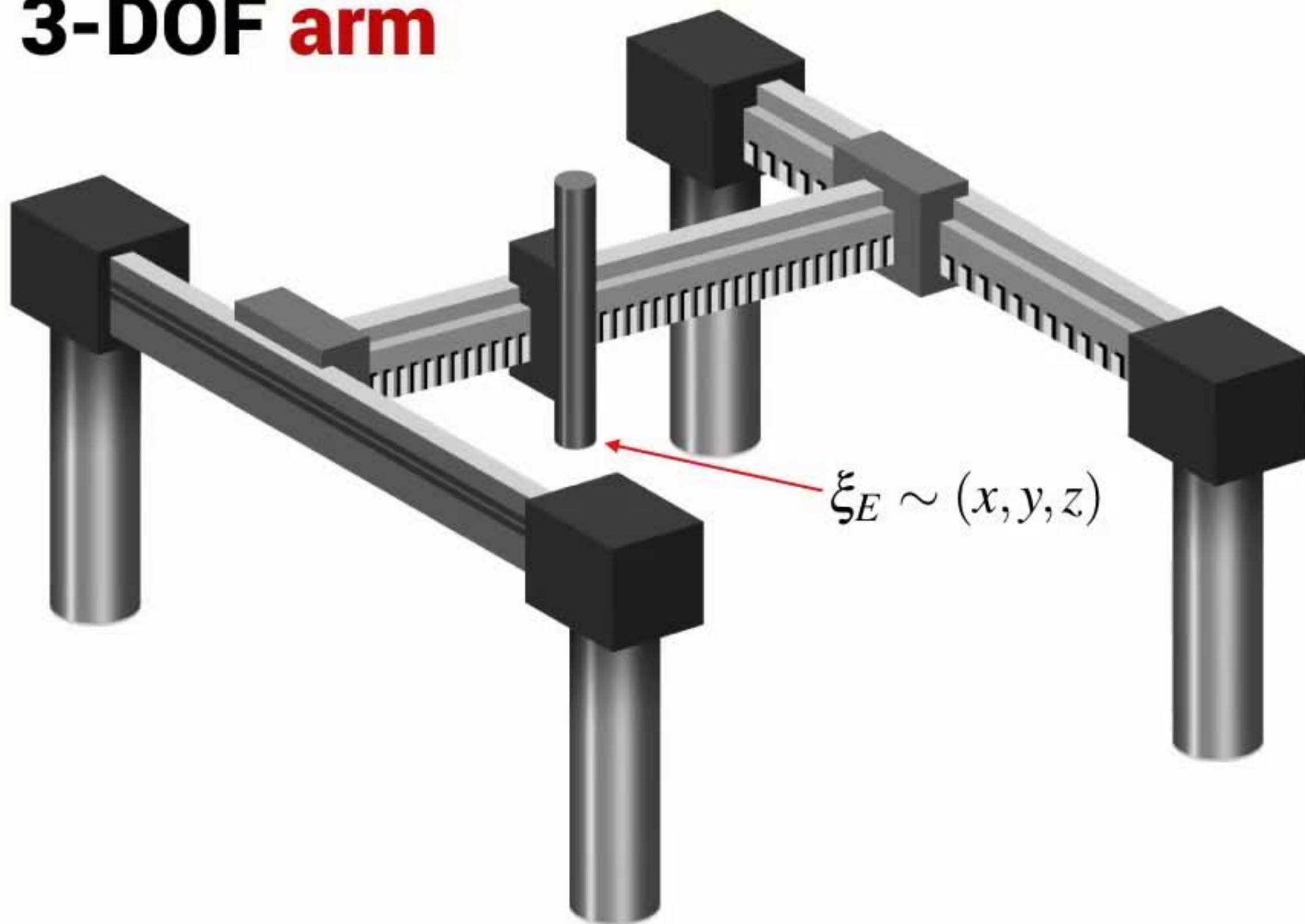
3-DOF arm



$$\mathcal{C} \subset \mathbb{R}^3$$

$$\mathcal{T} \subset \mathbb{R}^2 \times \mathbb{S}$$

3-DOF arm



$$\xi_E \sim (x, y, z)$$

$$\mathcal{C} \subset \mathbb{R}^3$$

$$\mathcal{T} \subset \mathbb{R}^3$$

Gantry robot

4-DOF arm



$$\xi_E \sim (x, y, z, \theta_y)$$

$$\mathcal{C} \subset \mathbb{R}^4$$

$$\mathcal{T} \subset \mathbb{R}^3 \times \mathbb{S}$$

SCARA robot

Copyright © 2014 Adept Technology, Inc.
The usage of this image in no manner is to be construed as
Adept's endorsement of QUT or any QUT product or service

4-DOF arm



$$\mathcal{C} \subset \mathbb{R}^4$$

$$\mathcal{T} \subset \mathbb{R}^3 \times \mathbb{S}$$

$$\xi_E \sim (x, y, z, \theta_y)$$

Used with permission of ABB

Parallel link robot

4-DOF arm



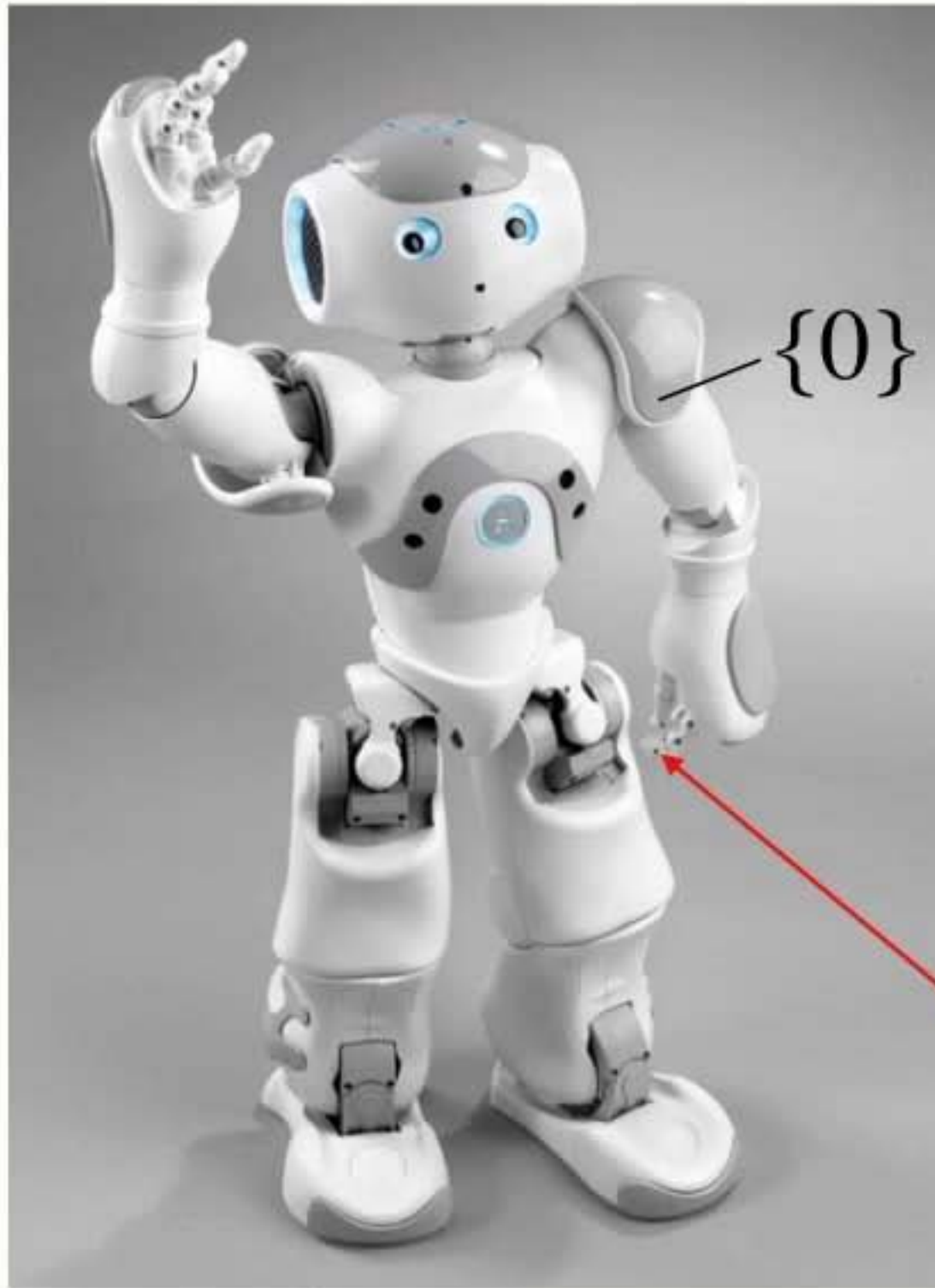
$$\xi_E \sim (x, y, z, \theta_p)$$

$$\mathcal{C} \subset \mathbb{R}^4$$

$$\mathcal{T} \subset \mathbb{R}^3 \times \mathbb{S}$$

PhantomX Pincher Robot Arm 2014
Trossen Robotics | Used with permission

4-DOF arm



By Aldebaran Robotics via Wikimedia Commons

$$\mathcal{C} \subset \mathbb{R}^4$$

$$\mathcal{T} \subset \mathbb{R}^3 \times \mathbb{S}$$

$$\xi_E \sim (x, y, z, \theta_p)$$

6-DOF **arm**

- Six joints is sufficient to achieve **any** position and orientation

$$\mathcal{C} \subset \mathbb{R}^6$$

$$\mathcal{T} \subset \mathbb{R}^3 \times \text{SO}(3)$$

$$\mathcal{T} \subset \text{SE}(3)$$



Used with permission of ABB



Stanford Robotic Arm
Oussama Khatib | Used with permission

$$\xi_E \sim (x, y, z, \theta_r, \theta_p, \theta_y)$$

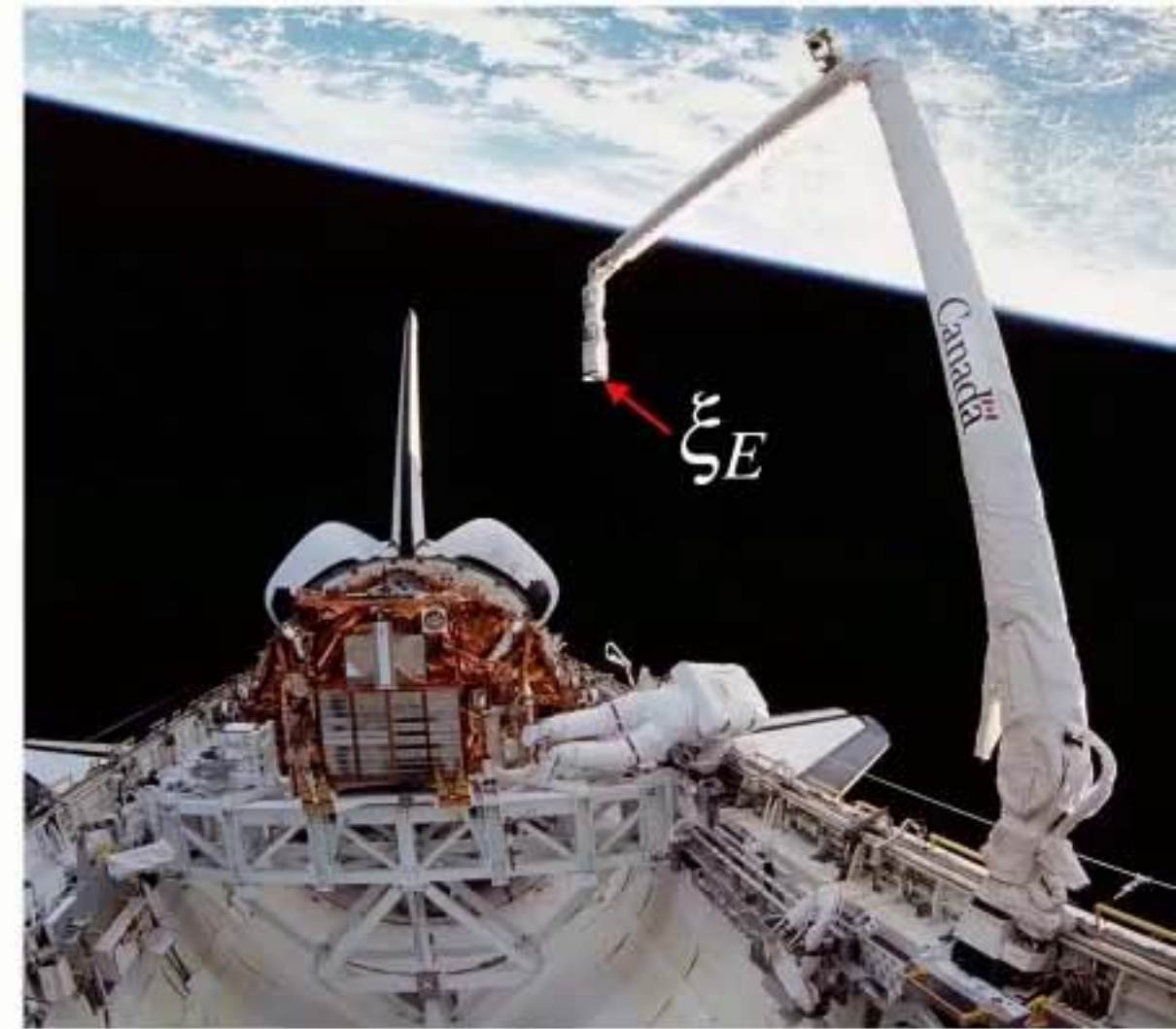


Image: NASA

Very high DOF **robot**

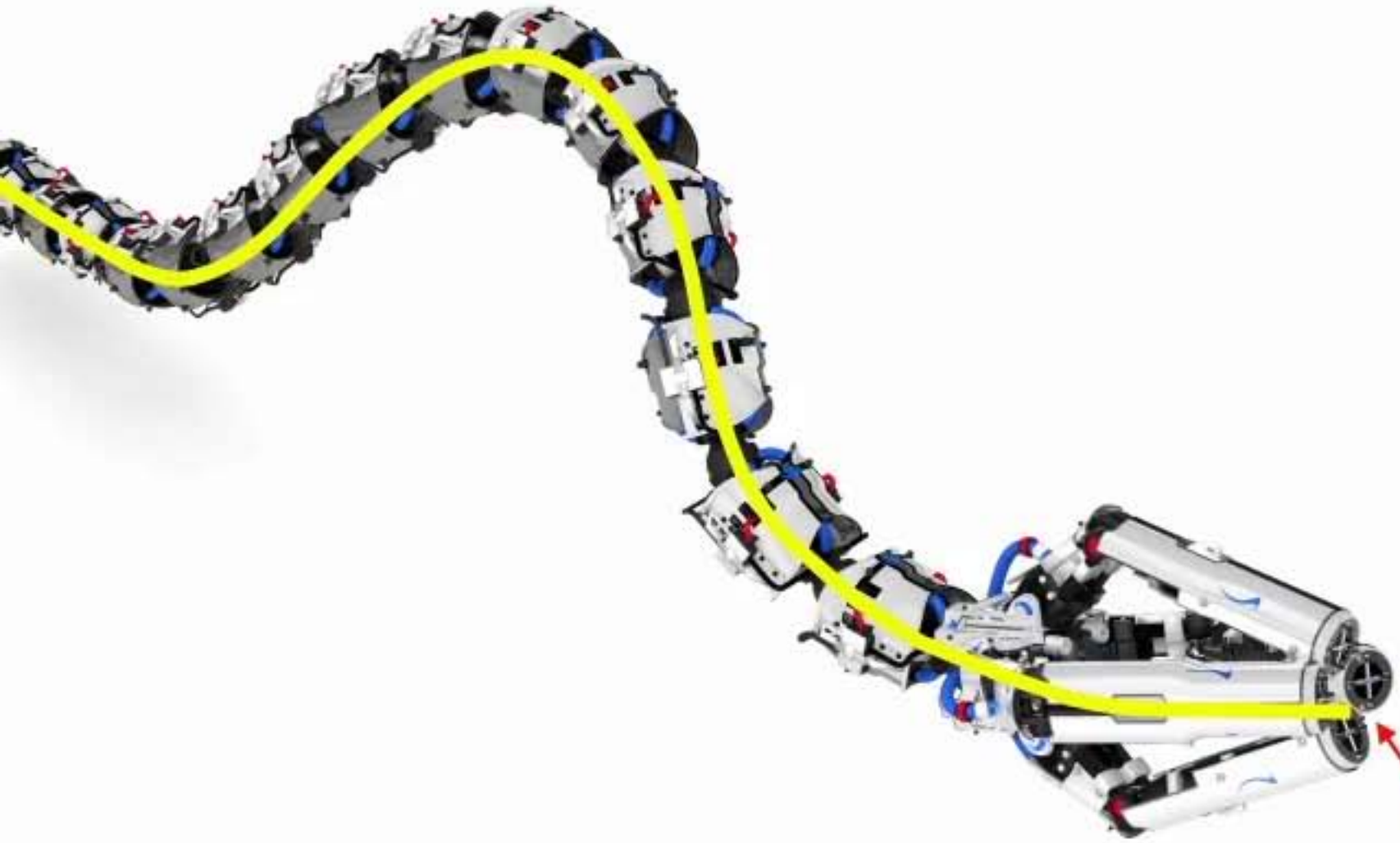


$$\mathcal{C} \subset \mathbb{R}^{100}$$

$$\mathcal{T} \subset \text{SE}(3)$$

$$\xi_E \sim (x, y, z, \theta_r, \theta_p, \theta_y)$$

Very high DOF **robot**



$$\mathcal{C} \subset \mathbb{R}^{100}$$

$$\mathcal{T} \subset \text{SE}(3)$$

configuration
space is much
bigger

task space is the
same as the Puma
robot

$$\xi_E \sim (x, y, z, \theta_r, \theta_p, \theta_y)$$

- Redundant robot
- Can achieve **any position and orientation** as well as controlling the shape of the arm

Configuration **string**



PhantomX Pincher Robot Arm 2014
Trossen Robotics | Used with permission

RRRR



Unimate Puma 500
Oussama Khatib | Used with permission

RRRRRR



Stanford Robotic Arm
Oussama Khatib | Used with permission

RRPRRR

- A string with one character describing each joints in order from base to tip:
 - ➡ R for a revolute joint
 - ➡ P for a prismatic joint

Section 8 Denavit-Hartenberg parameterisation

Denavit & Hartenberg

- Developed a general theory to describe an articulated sequence of joints
- Each joint in the robot is described by **four parameters**



Jacques Denavit

1930–2012

Denavit, Jaques. (DMG-Lib.org, n.d.). CC BY-NC-ND 3.0.



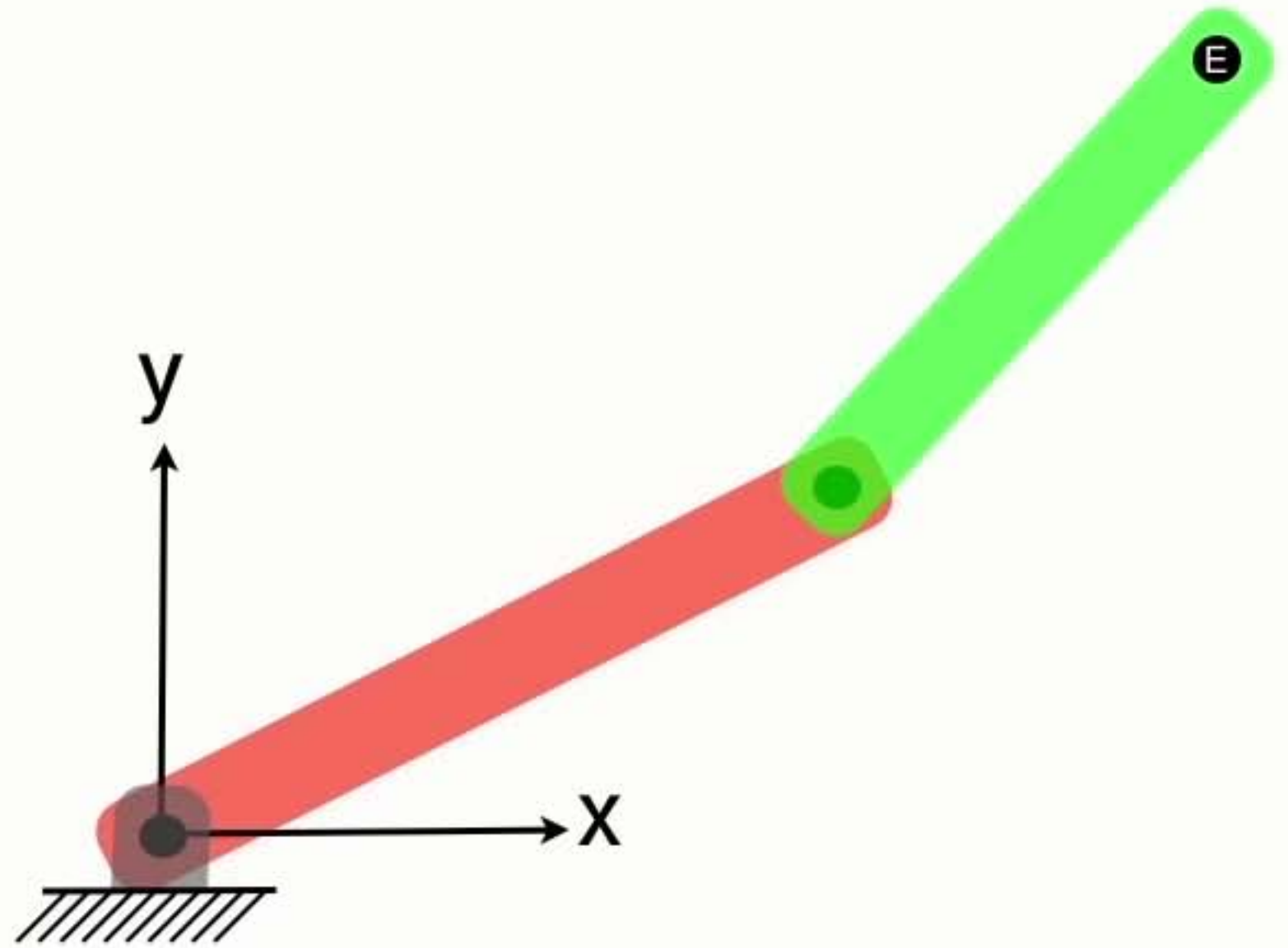
Richard Hartenberg

1907–1997

Hartenberg, Richard Scheunemann. (DMG-Lib.org, 1909-1997). CC BY-NC-ND 3.0

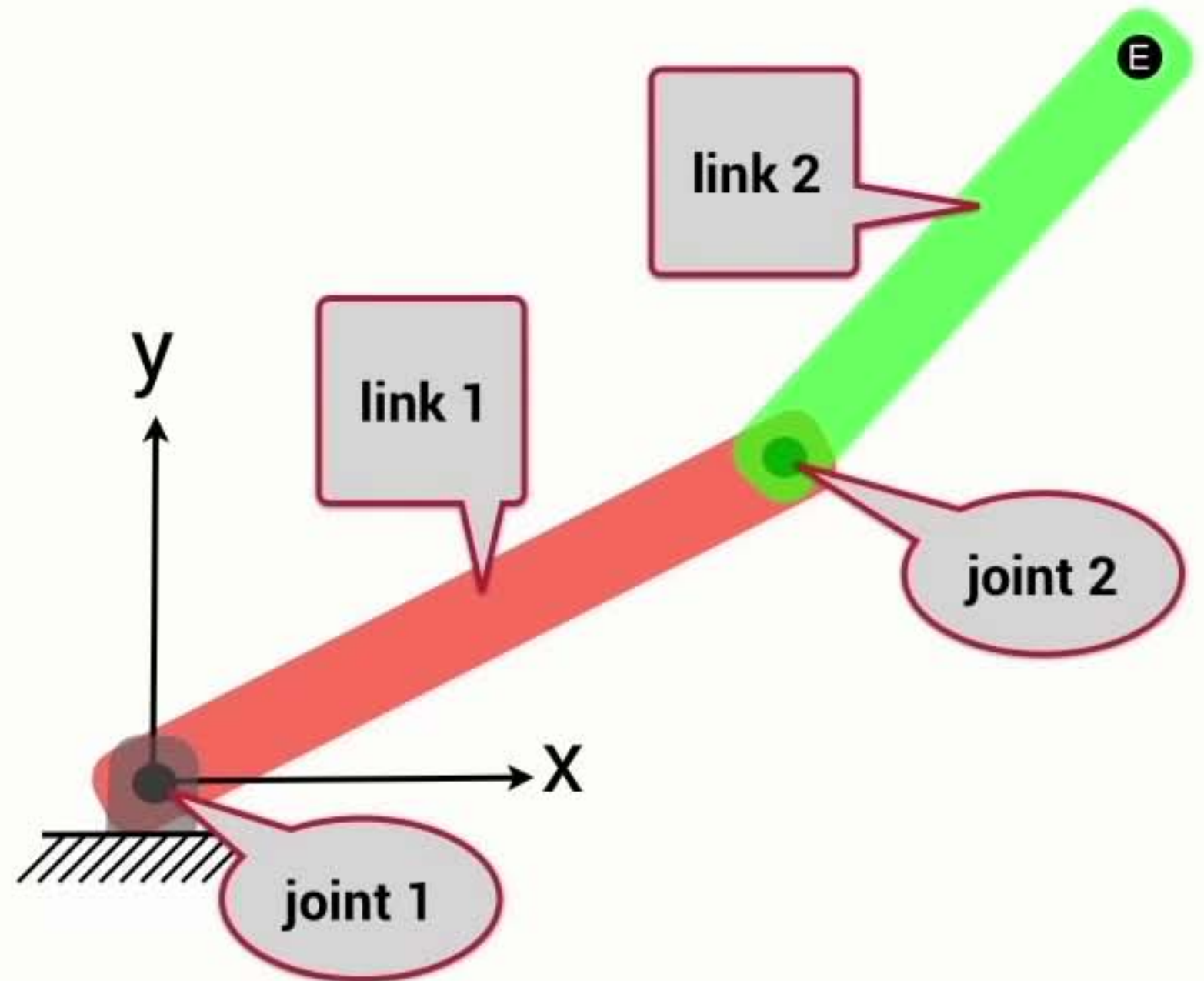
1955

Serial-link manipulator



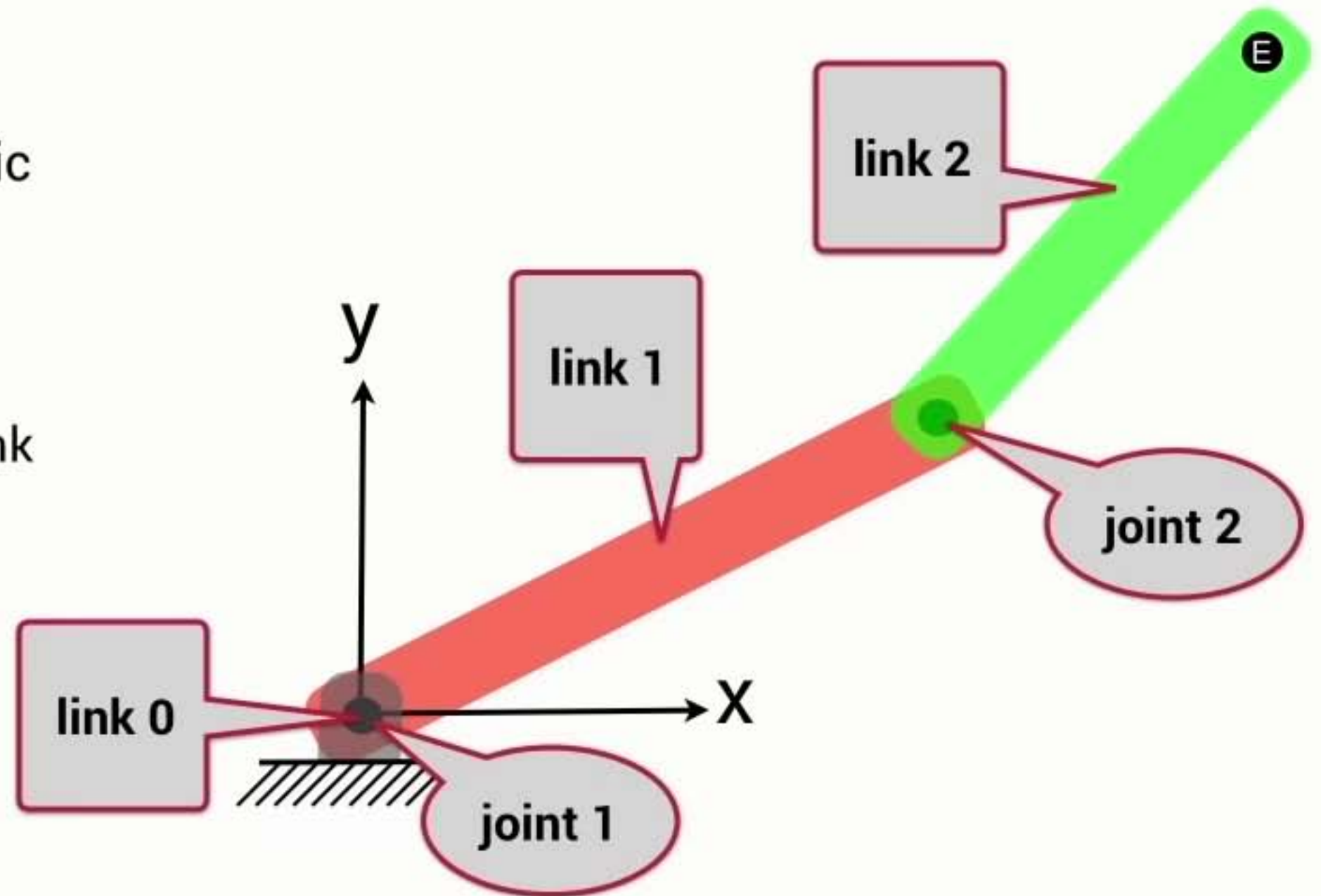
Serial-link **manipulator**

- Each joint attached, via a link, to the previous joint
 - ➔ the links are rigid
 - ➔ the joints can move
 - either revolute or prismatic



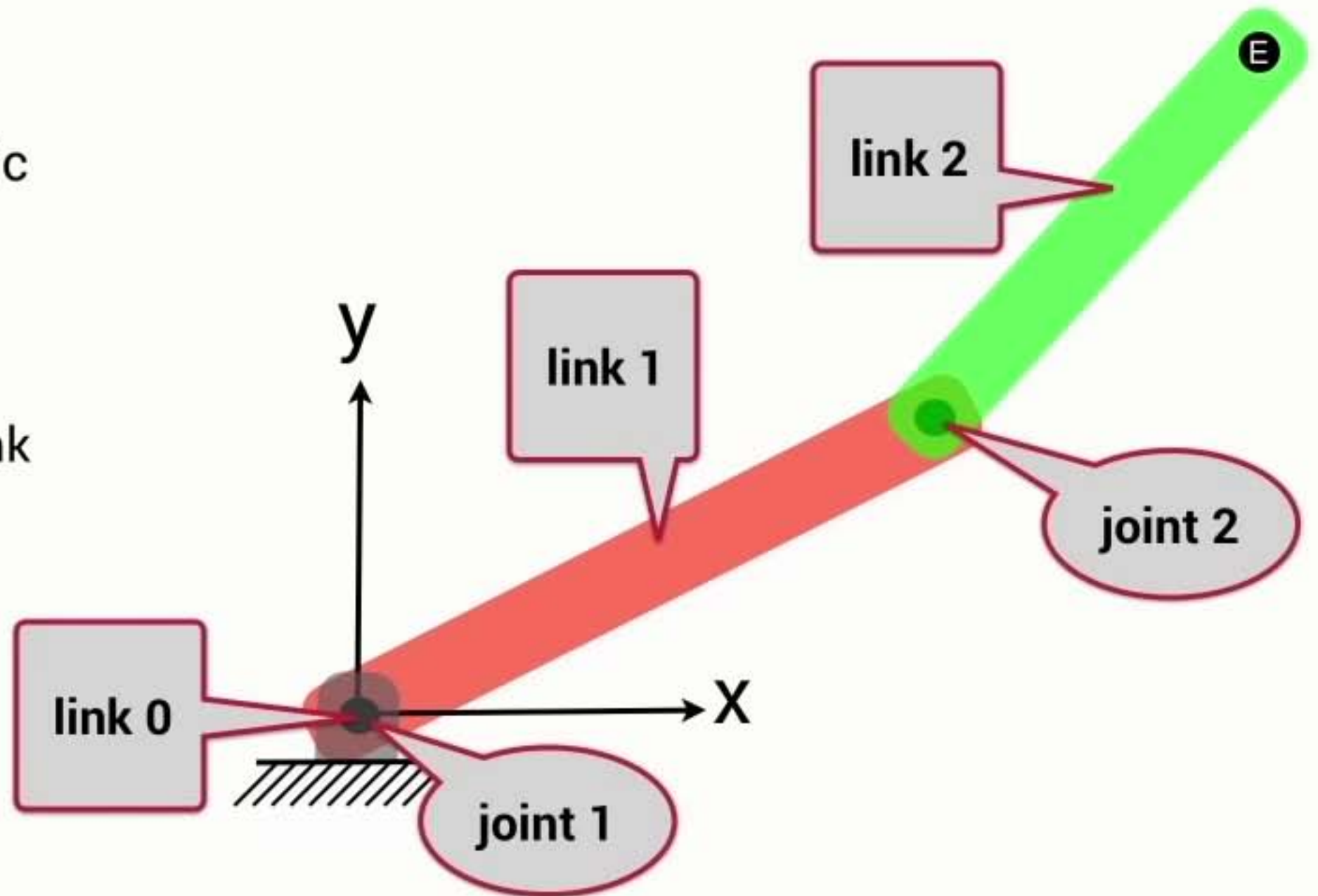
Serial-link **manipulator**

- Each joint attached, via a link, to the previous joint
 - ➔ the links are rigid
 - ➔ the joints can move
 - either revolute or prismatic
- Every joint connects two links
- Every link connects two joints
 - ➔ except the first and the last link



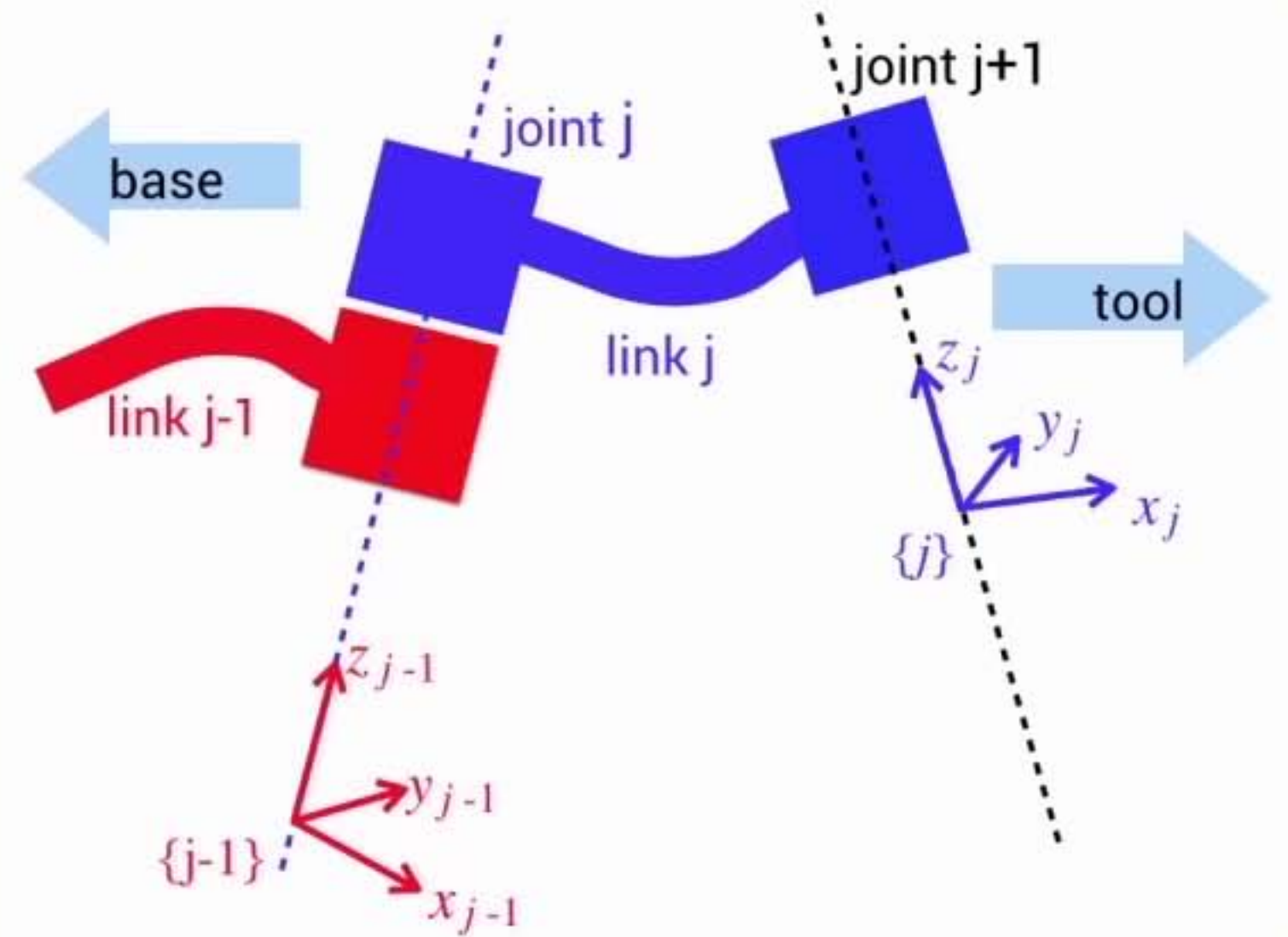
Serial-link **manipulator**

- Each joint attached, via a link, to the previous joint
 - ➔ the links are rigid
 - ➔ the joints can move
 - either revolute or prismatic
- Every joint connects two links
- Every link connects two joints
 - ➔ except the first and the last link
- N joints
- N+1 links



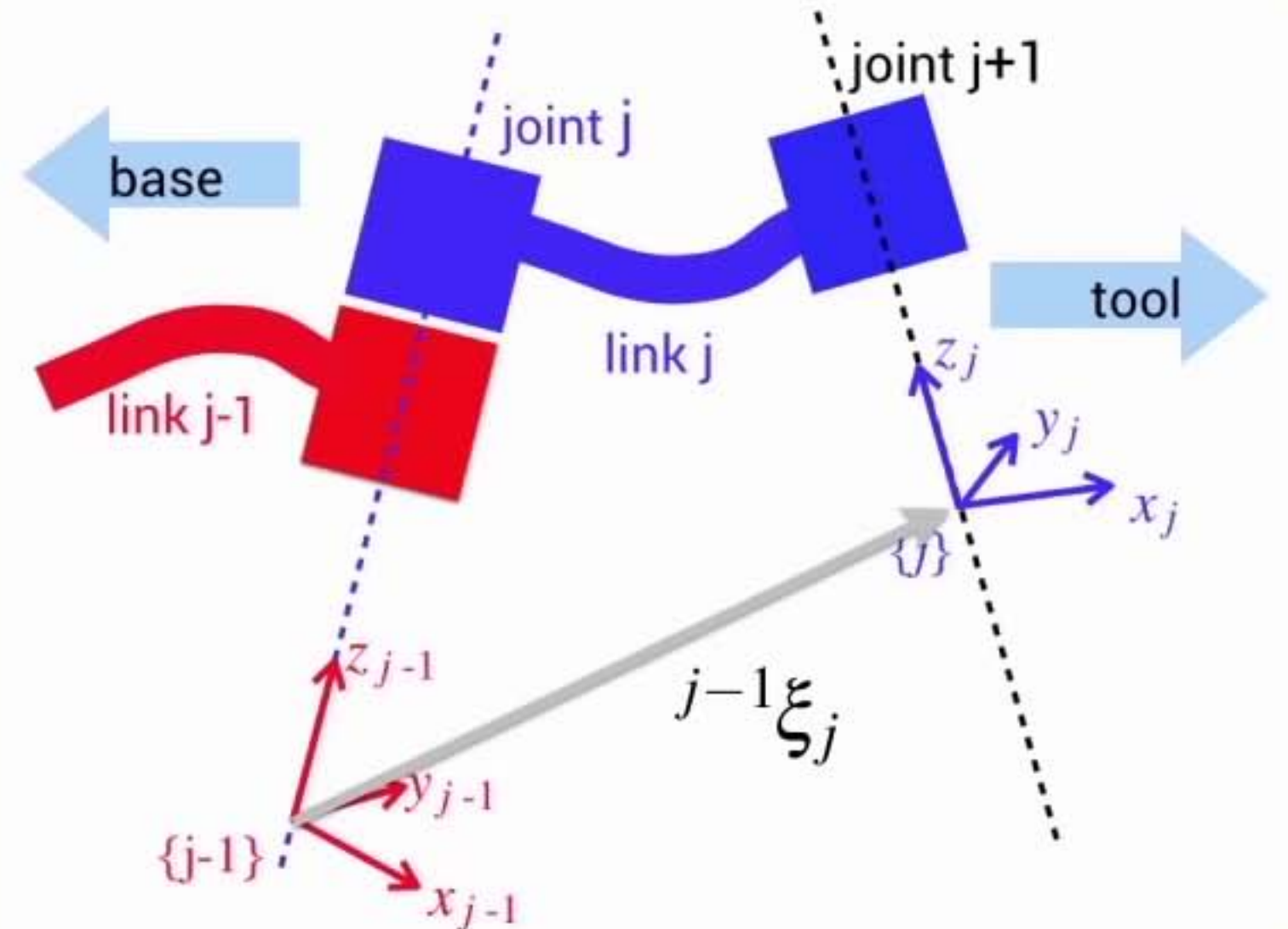
DH parameters

- Attach a coordinate frame to the far (distal) end of every link



DH parameters

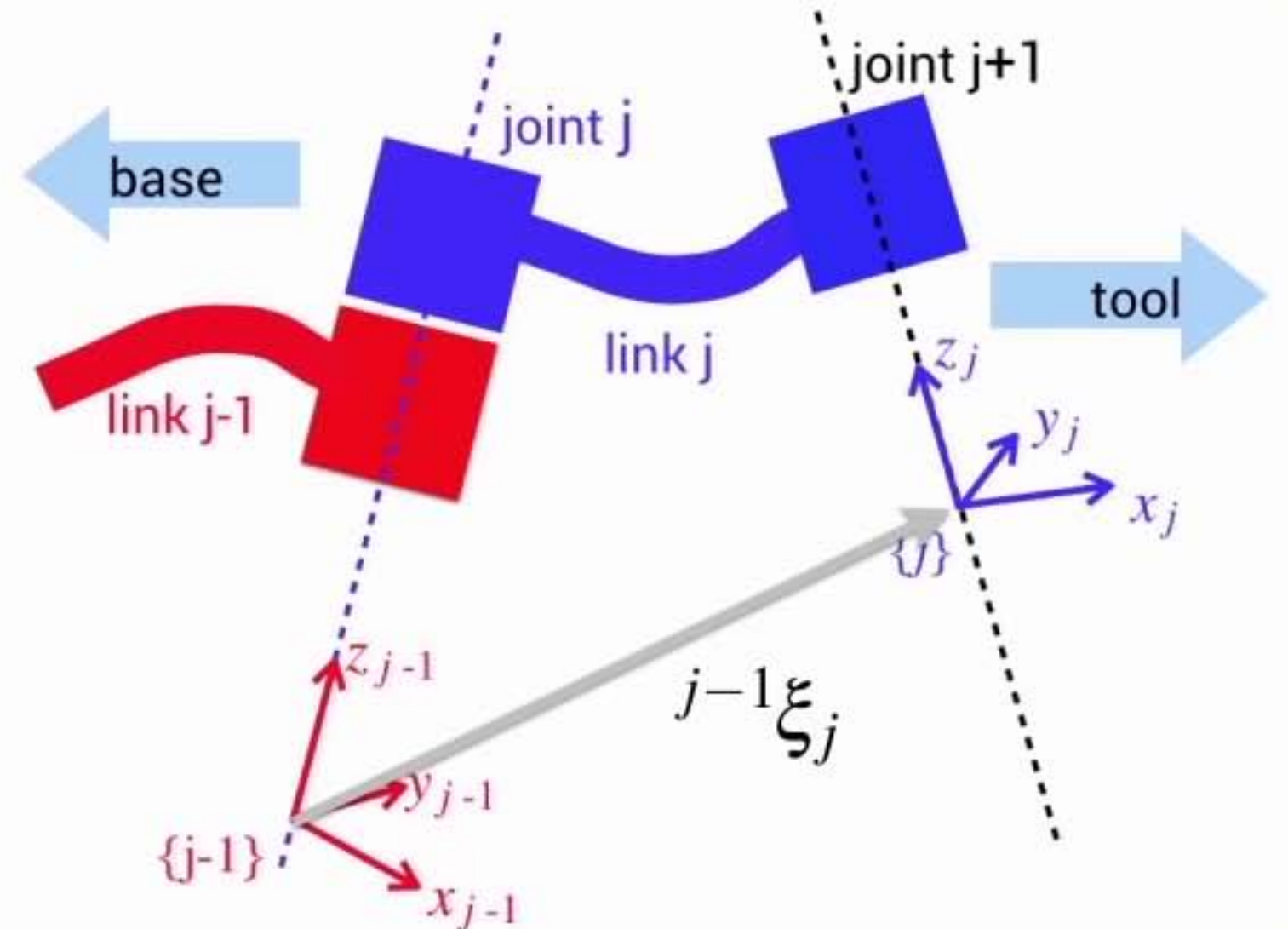
- Attach a coordinate frame to the far (distal) end of every link
- Describe the pose of a link frame with respect to the previous link frame

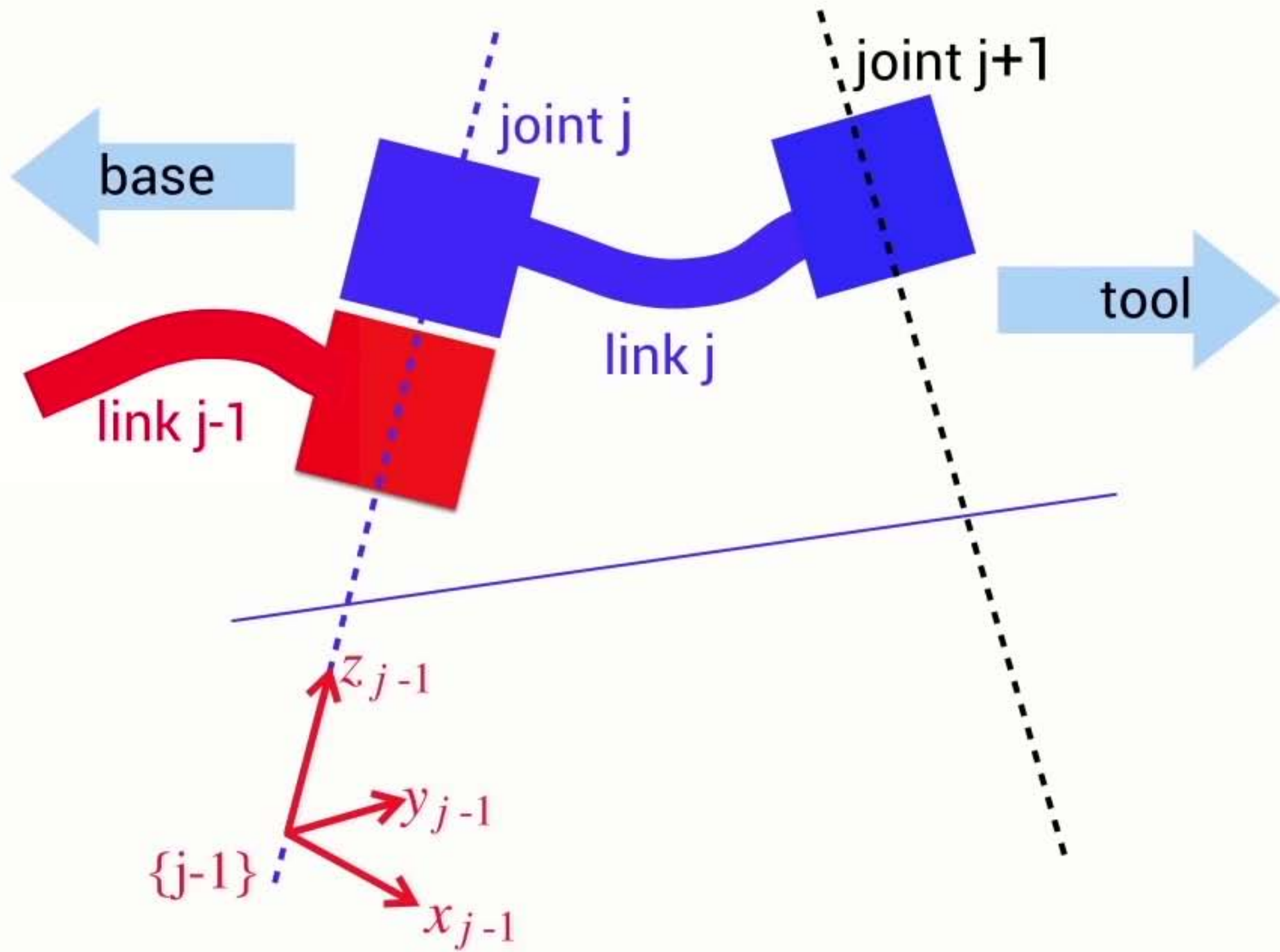


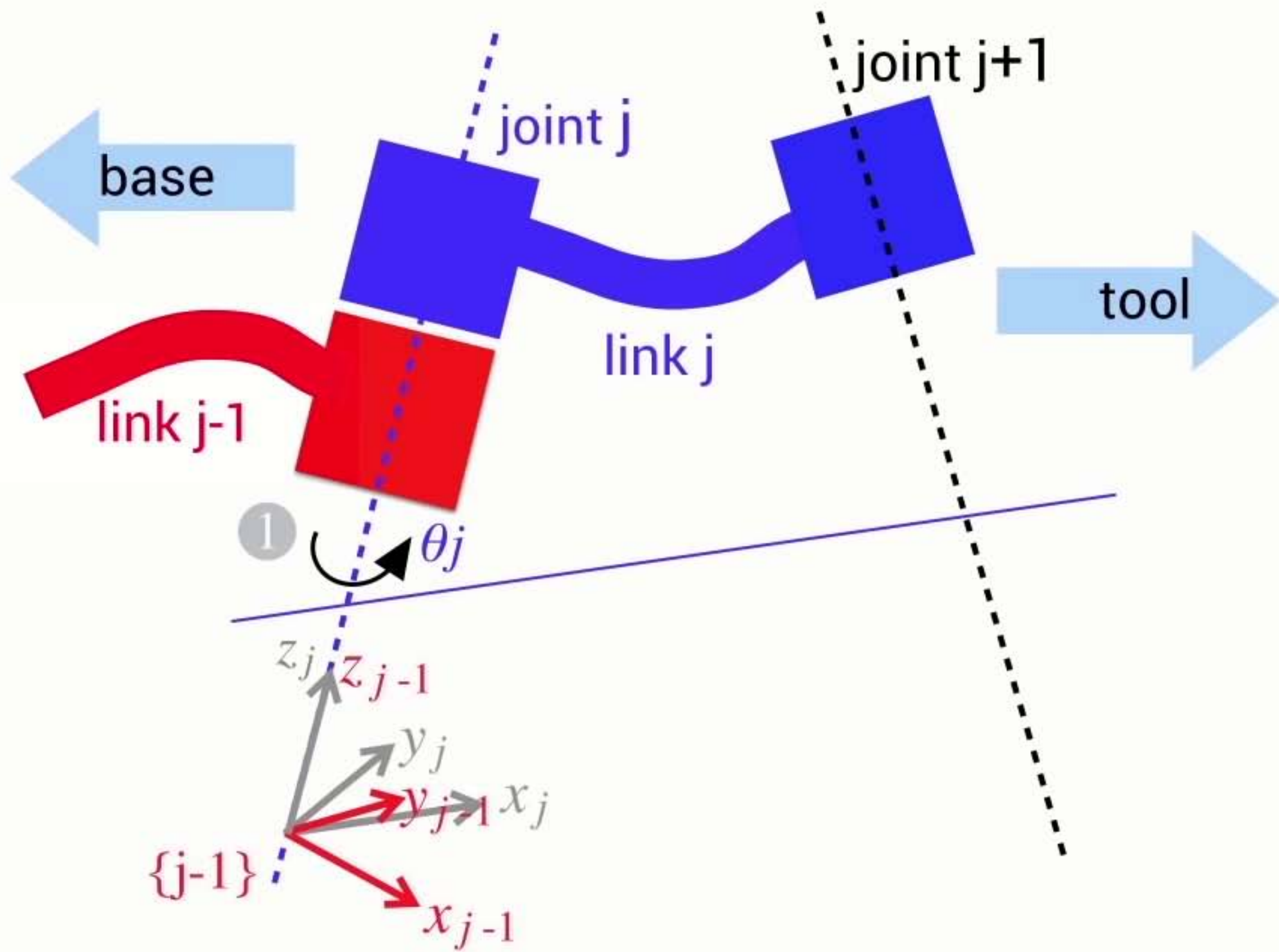
DH parameters

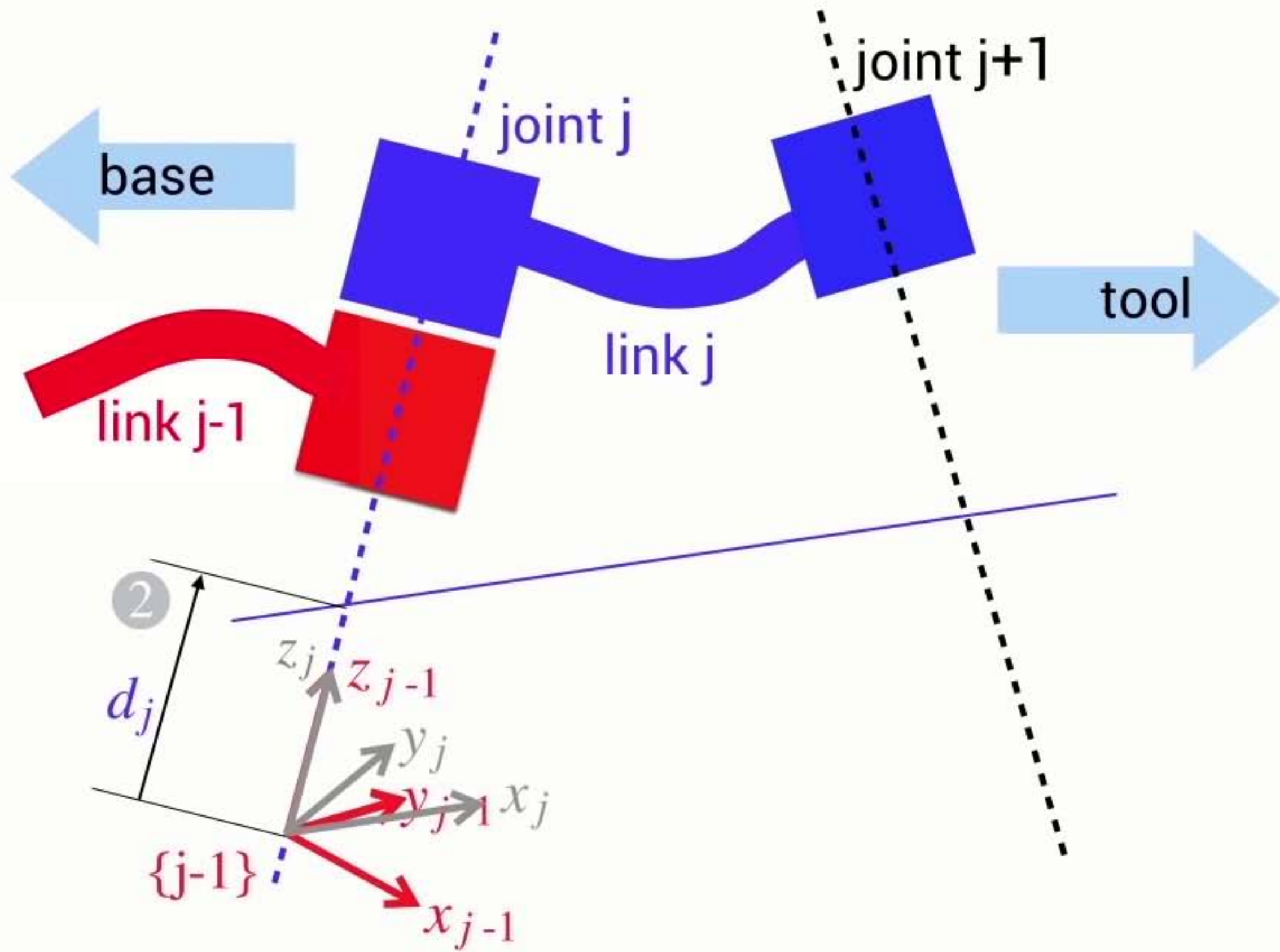
$${}^{j-1}\xi_j \sim \mathbf{A} = \mathbf{R}_z(\theta_j)\mathbf{T}_z(d_j)\mathbf{T}_x(a_j)\mathbf{R}_x(\alpha_j)$$

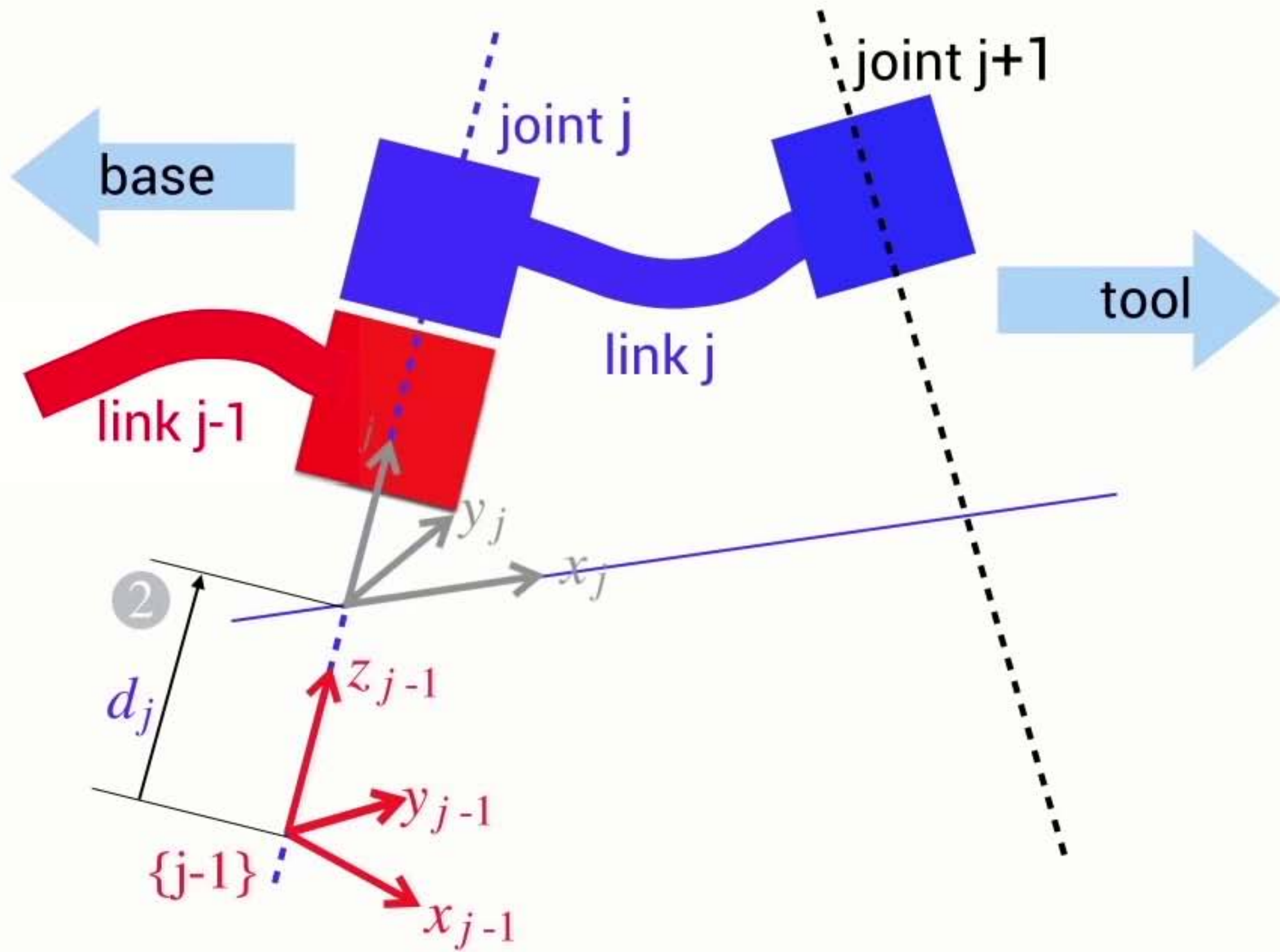
- Attach a coordinate frame to the far (distal) end of every link
- Describe the pose of a link frame with respect to the previous link frame
- Very concise
 - ➡ Only four parameters: θ , d , a , α

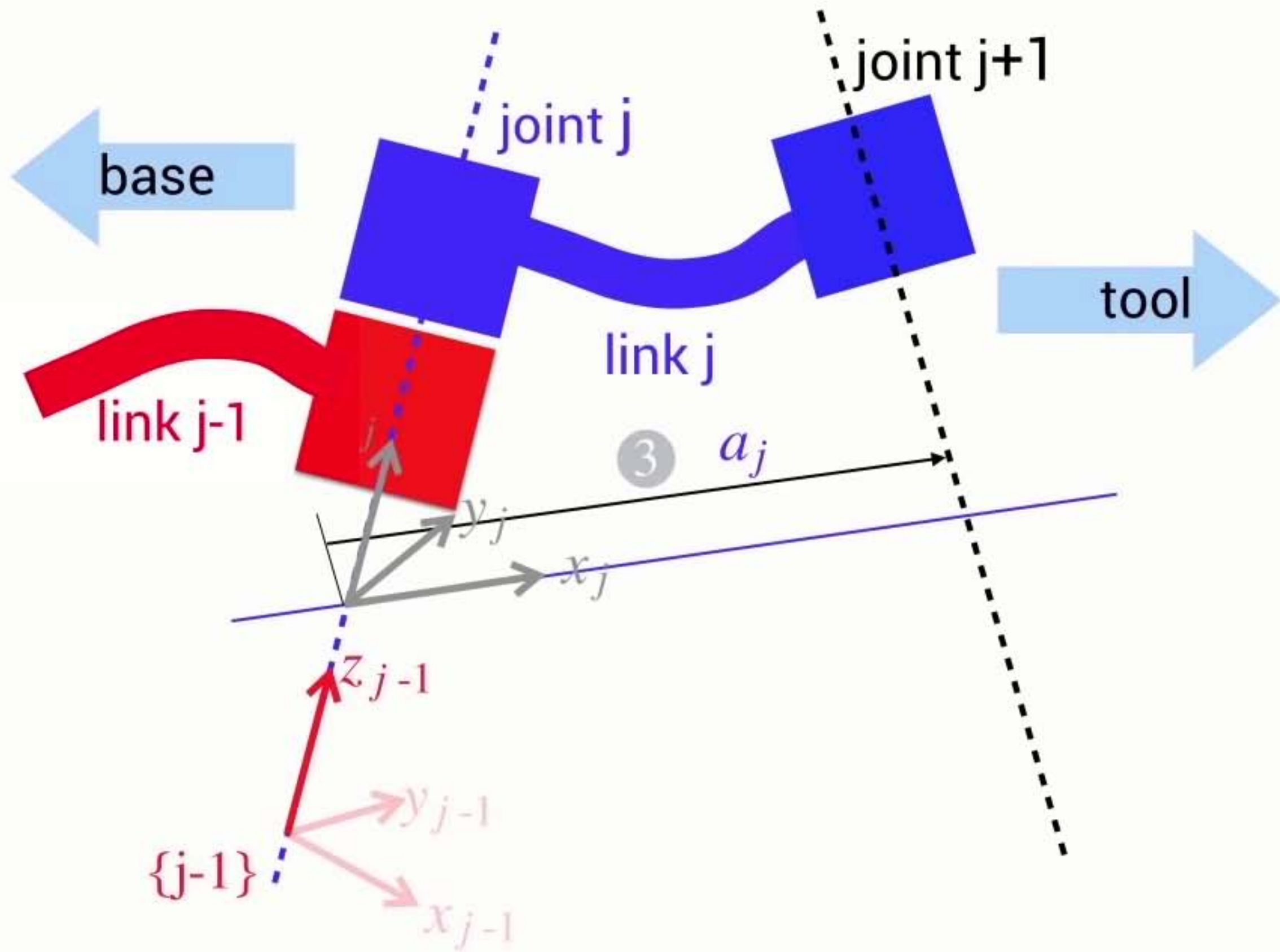


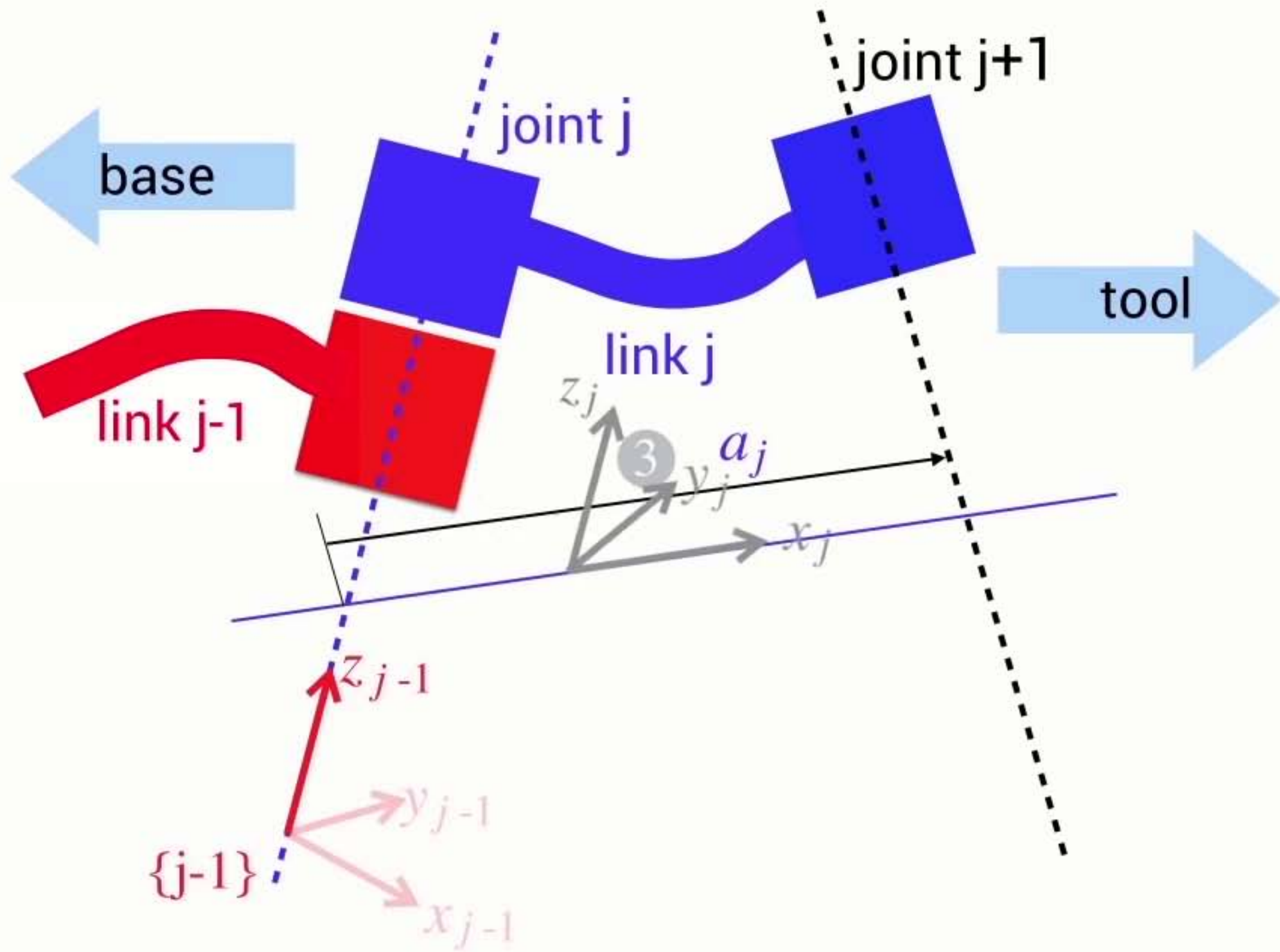


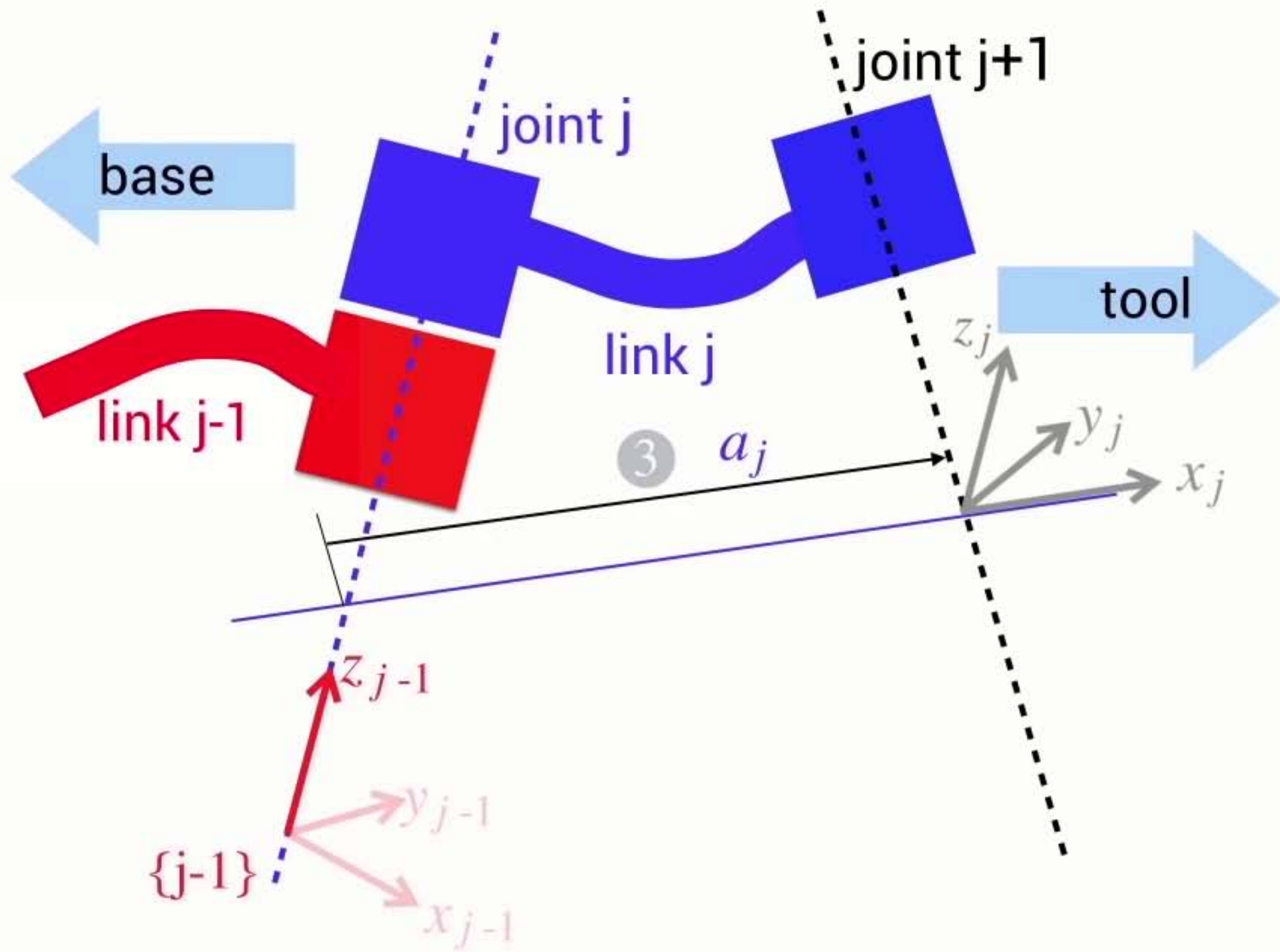


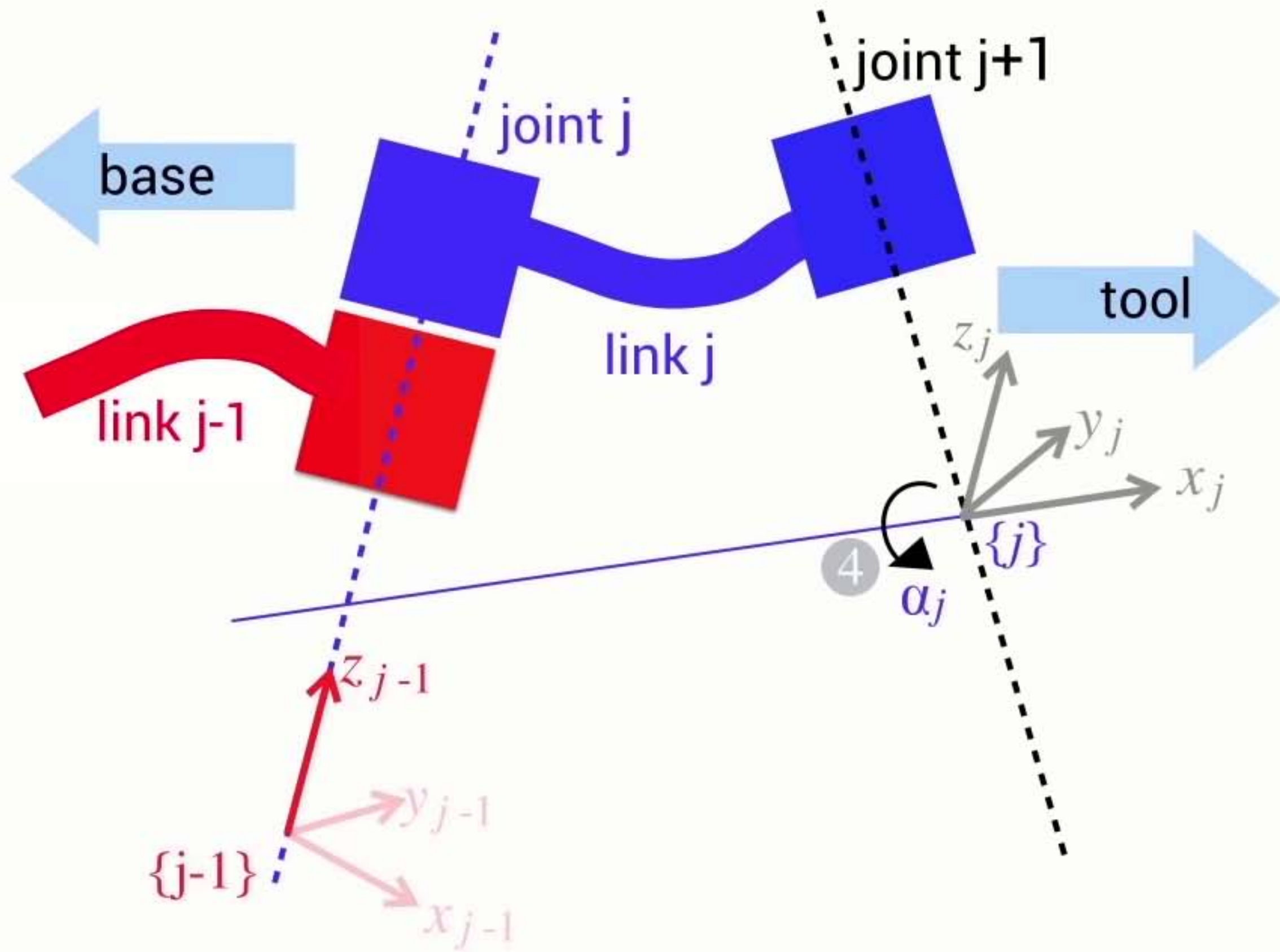


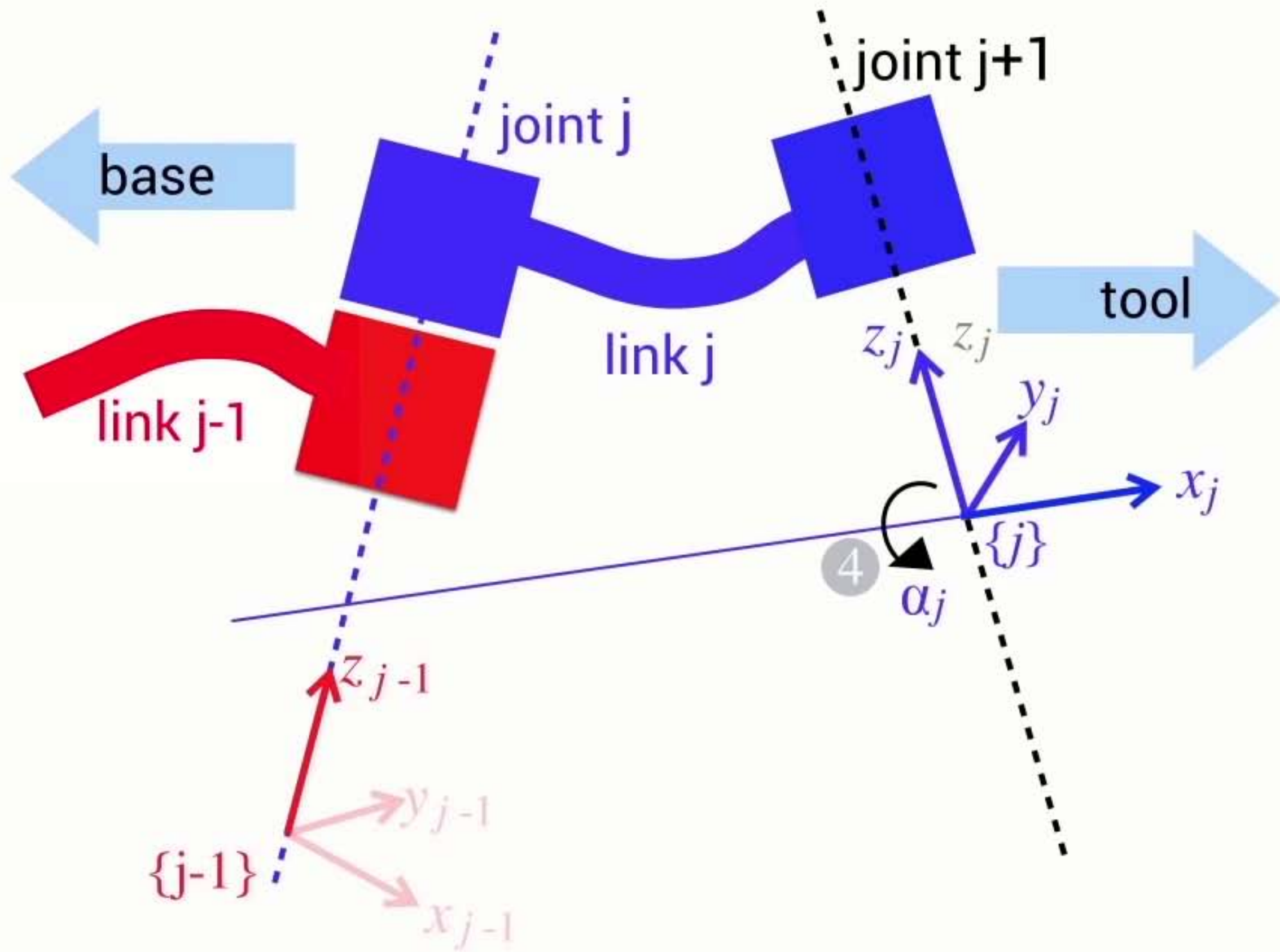






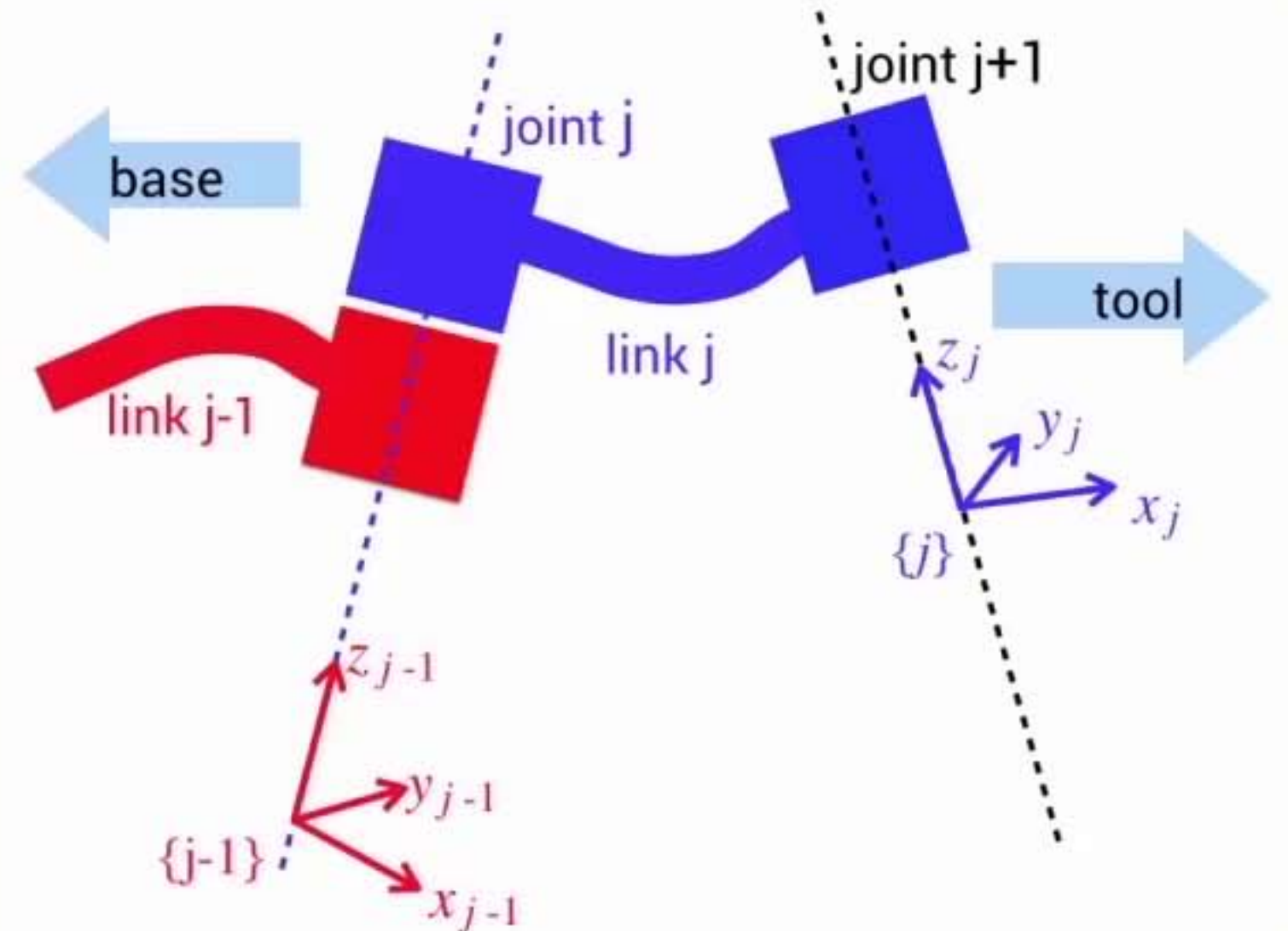






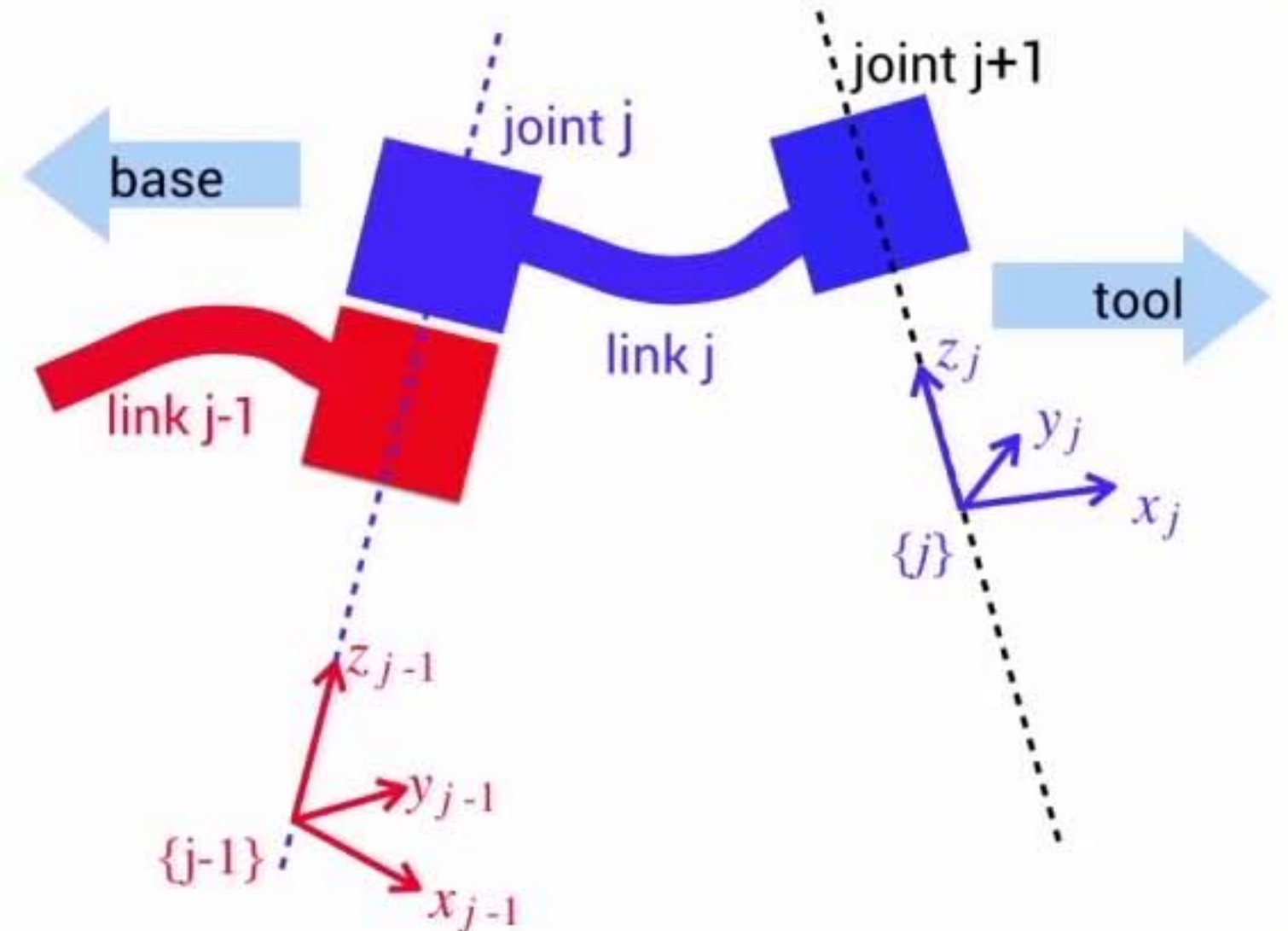
Why only 4 **parameters**?

- The relationship between two coordinate frames (a pose) has 6 parameters
 - ➔ 3 translation, 3 rotation



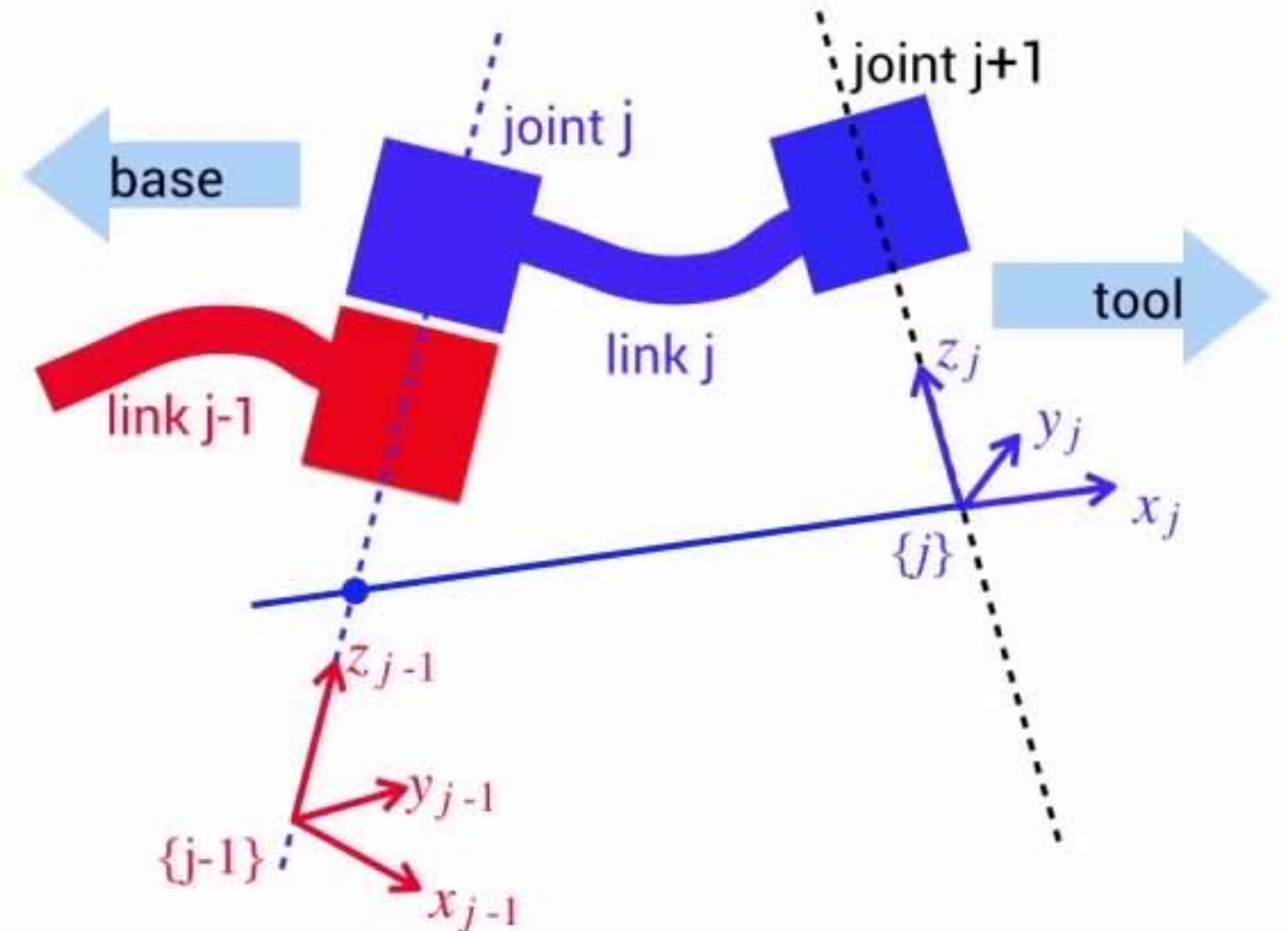
Why only 4 **parameters**?

- The relationship between two coordinate frames (a pose) has 6 parameters
 - ➔ 3 translation, 3 rotation
- For Denavit-Hartenberg notation the coordinate frames have constraints



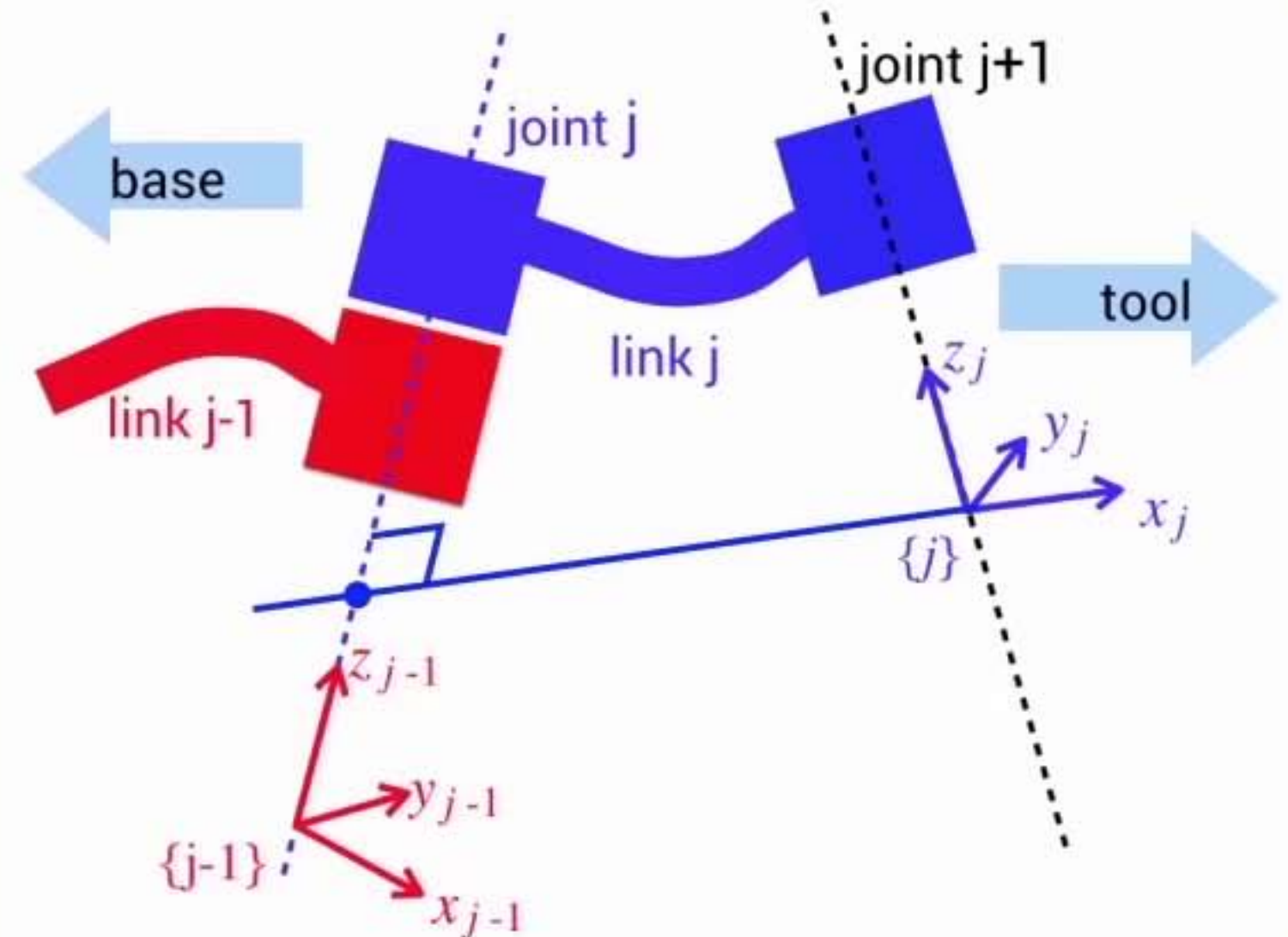
Why only 4 **parameters**?

- The relationship between two coordinate frames (a pose) has 6 parameters
 - ➔ 3 translation, 3 rotation
- For Denavit-Hartenberg notation the coordinate frames have constraints
 - ➔ Axis x_j **intersects** z_{j-1}



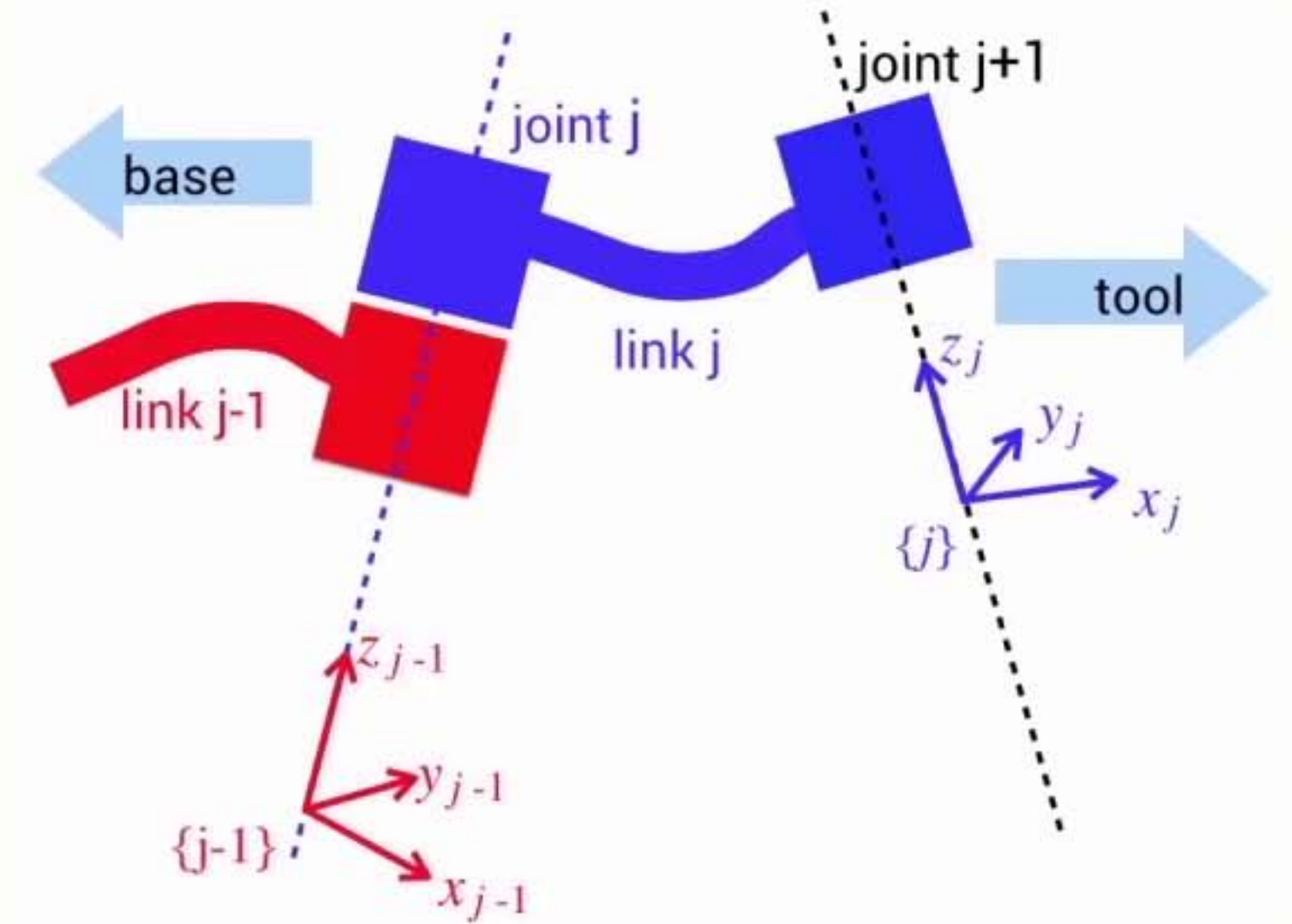
Why only 4 **parameters**?

- The relationship between two coordinate frames (a pose) has 6 parameters
 - ➔ 3 translation, 3 rotation
- For Denavit-Hartenberg notation the coordinate frames have constraints
 - ➔ Axis x_j **intersects** z_{j-1}
 - ➔ Axis x_j is **perpendicular** to axis z_{j-1}
- The result is only 4 parameters, not 6
 - ➔ Consequence: The link frame is not necessarily on the link!



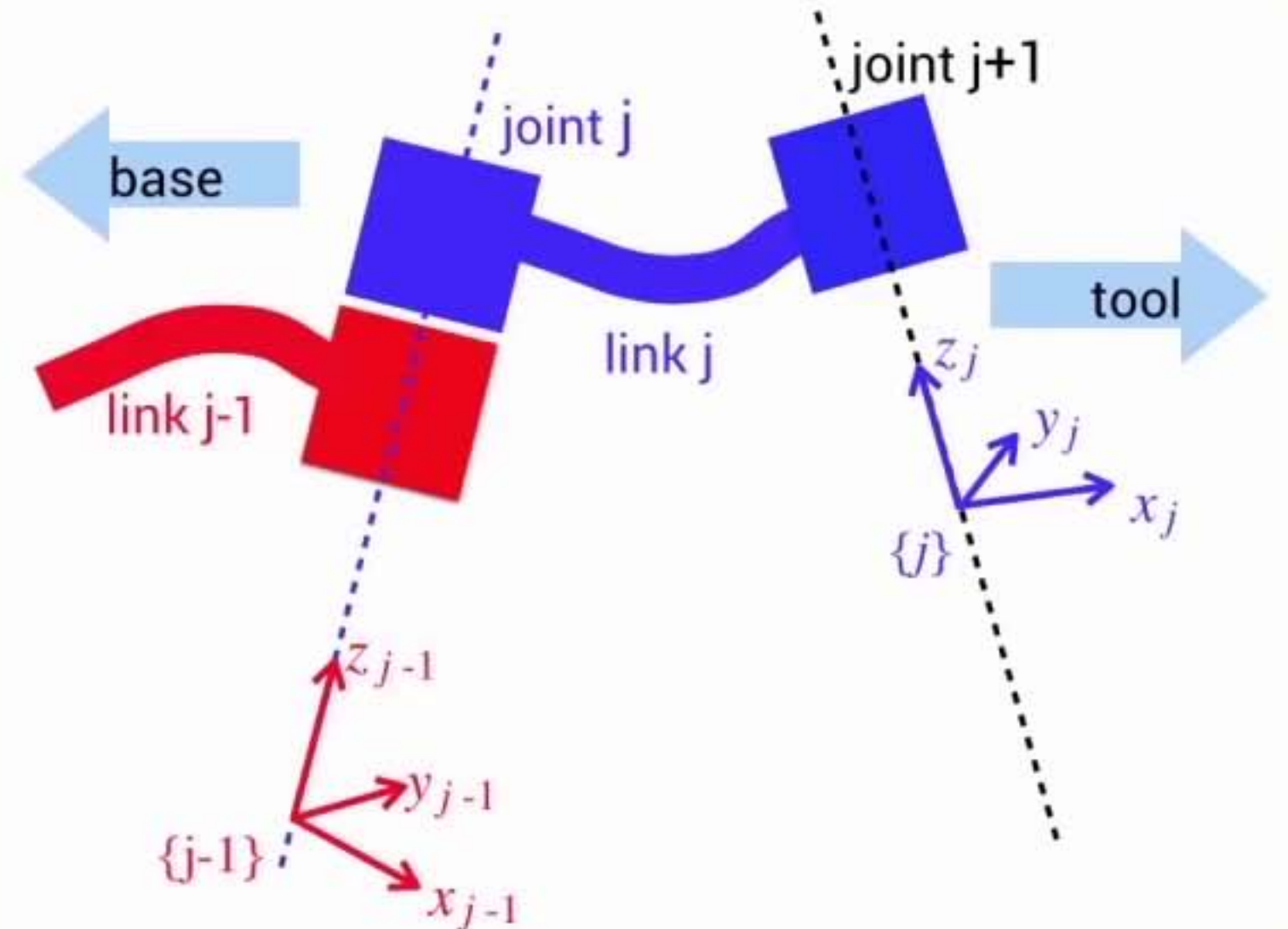
The joint **axis**

- The axis of each joint is defined by the z-axis of a coordinate frame



The joint **axis**

- The axis of each joint is defined by the z-axis of a coordinate frame
 - ➔ the joint **rotates** about the z-axis (revolute joint), or
 - ➔ **translates** along the z-axis (prismatic joint)



Stack the link transforms **end to end**

$$\mathbf{T}_E = \mathbf{R}_z(\theta_1)\mathbf{T}_z(d_1)\mathbf{T}_x(a_1)\mathbf{R}_x(\alpha_1) \mathbf{R}_z(\theta_2)\mathbf{T}_z(d_2)\mathbf{T}_x(a_2)\mathbf{R}_x(\alpha_2) \cdots \mathbf{R}_z(\theta_N)\mathbf{T}_z(d_N)\mathbf{T}_x(a_N)\mathbf{R}_x(\alpha_N)$$

Stack the link transforms **end to end**

- All revolute joints (RRRRRRR)

$$\mathbf{T}_E = \underbrace{\mathbf{R}_z(\theta_1) \mathbf{T}_z(d_1) \mathbf{T}_x(a_1) \mathbf{R}_x(\alpha_1)}_{\text{revolute joint } \mathbf{q}_1} \underbrace{\mathbf{R}_z(\theta_2) \mathbf{T}_z(d_2) \mathbf{T}_x(a_2) \mathbf{R}_x(\alpha_2)}_{\text{revolute joint } \mathbf{q}_2} \cdots \underbrace{\mathbf{R}_z(\theta_N) \mathbf{T}_z(d_N) \mathbf{T}_x(a_N) \mathbf{R}_x(\alpha_N)}_{\text{revolute joint } \mathbf{q}_n}$$

Stack the link transforms **end to end**

- All revolute joints (RRRRRRR)

$$\mathbf{T}_E = \underbrace{\mathbf{R}_z(\theta_1) \mathbf{T}_z(d_1) \mathbf{T}_x(a_1) \mathbf{R}_x(\alpha_1)}_{\text{revolute joint}} \underbrace{\mathbf{R}_z(\theta_2) \mathbf{T}_z(d_2) \mathbf{T}_x(a_2) \mathbf{R}_x(\alpha_2)}_{\text{revolute joint}} \cdots \underbrace{\mathbf{R}_z(\theta_N) \mathbf{T}_z(d_N) \mathbf{T}_x(a_N) \mathbf{R}_x(\alpha_N)}_{\text{revolute joint}}$$

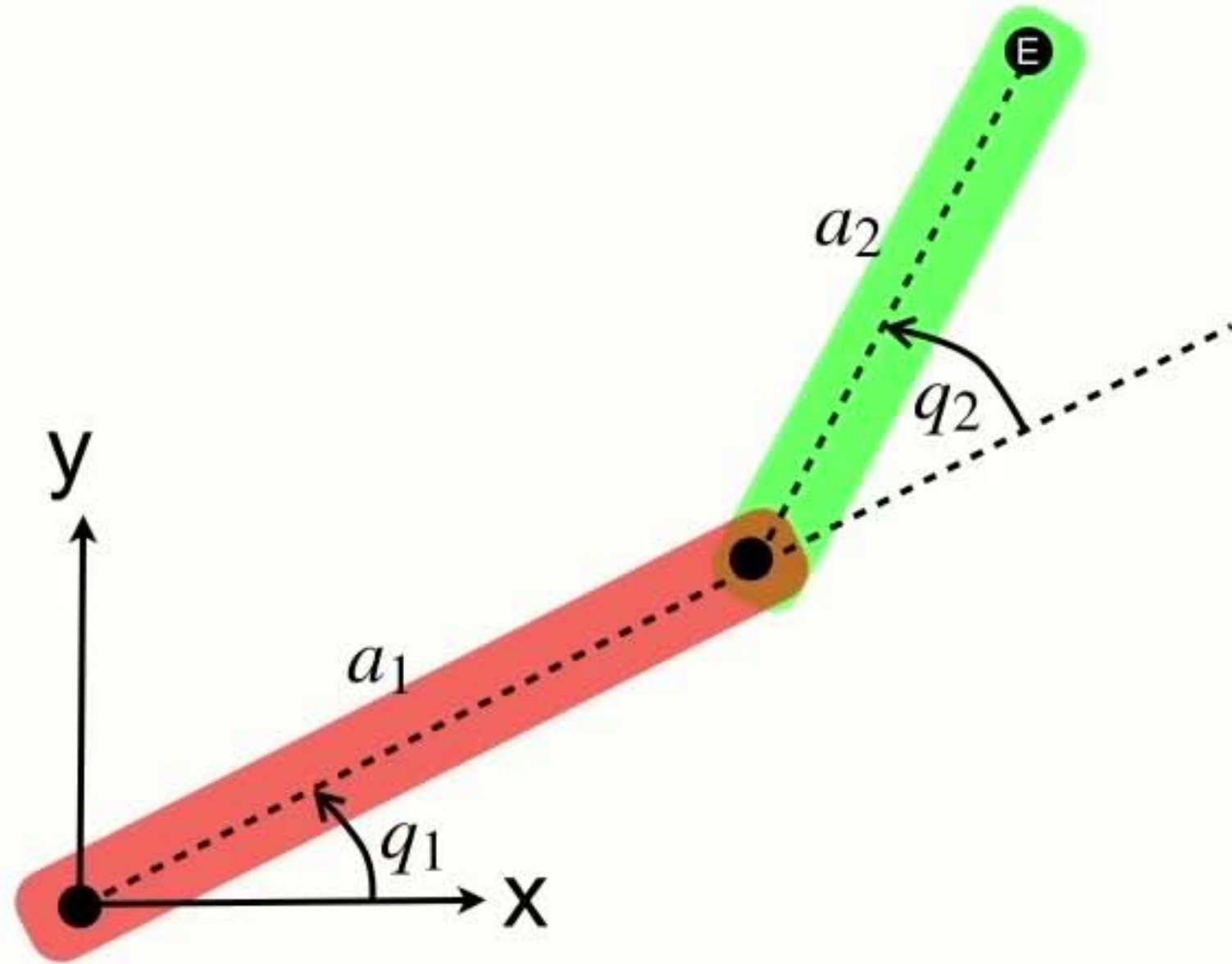
Diagram illustrating the stacking of link transforms for a robot arm with all revolute joints (RRRRRRR). The equation shows the end effector transform $\mathbf{T}_E$ as a product of individual link transforms. Each link transform is represented by a red box containing the sequence of Denavit-Hartenberg (DH) parameters: $\mathbf{R}_z(\theta_i) \mathbf{T}_z(d_i) \mathbf{T}_x(a_i) \mathbf{R}_x(\alpha_i)$. The joint variables θ_i are highlighted in speech bubbles above the first, second, and last links, labeled q_1 , q_2 , and q_n respectively. The labels "revolute joint" are placed below each red box.

- Revolute and prismatic joints (RPRRRR)

$$\mathbf{T}_E = \underbrace{\mathbf{R}_z(\theta_1) \mathbf{T}_z(d_1) \mathbf{T}_x(a_1) \mathbf{R}_x(\alpha_1)}_{\text{revolute joint}} \underbrace{\mathbf{R}_z(\theta_2) \mathbf{T}_z(d_2) \mathbf{T}_x(a_2) \mathbf{R}_x(\alpha_2)}_{\text{prismatic joint}} \cdots \underbrace{\mathbf{R}_z(\theta_N) \mathbf{T}_z(d_N) \mathbf{T}_x(a_N) \mathbf{R}_x(\alpha_N)}_{\text{revolute joint}}$$

Diagram illustrating the stacking of link transforms for a robot arm with revolute and prismatic joints (RPRRRR). The equation shows the end effector transform $\mathbf{T}_E$ as a product of individual link transforms. The first and last link transforms are in red boxes and labeled "revolute joint" below them. The middle link transform is in a green box and labeled "prismatic joint" below it. The joint variables θ_i are highlighted in speech bubbles above the first and last links, labeled q_1 and q_n respectively.

Denavit-Hartenberg parameters for **two-link**



θ_j	d_j	a_j	α_j
q_1	0	a_1	0
q_2	0	a_2	0

- This table completely describes the kinematics of the robot

Command Window

```
>> dh = [  
0 0 1 0  
0 0 1 0  
]
```

```
dh =
```

```
    0    0    1    0  
    0    0    1    0
```

```
fx >>
```

Workspace

Name ▲	Value
dh	[0,0,1,0;0,0,1,0]

Command Window

```
>> dh = [
0 0 1 0
0 0 1 0
]
```

dh =

```
    0    0    1    0
    0    0    1    0
```

```
>> r = SerialLink(dh)
```

r =

noname (2 axis, RR, stdDH, fastRNE)

j	theta	d	a	alpha
1	q1	0	1	0
2	q2	0	1	0

```
grav =    0    base = 1  0  0  0    tool = 1  0  0  0
          0          0  1  0  0          0  1  0  0
        9.81        0  0  1  0          0  0  1  0
                0  0  0  1          0  0  0  1
```

fx >>

Workspace

Name	Value
dh	[0,0,1,0;0,0,1,0]
r	1x1 SerialLink

Command Window

```
noname \z axis, RR, SLDON, FASTLINE/
```

j	theta	d	a	alpha
1	q1	0	1	0
2	q2	0	1	0

```
grav = 0 base = 1 0 0 0 tool = 1 0 0 0
      0      0 1 0 0      0 1 0 0
      9.81    0 0 1 0      0 0 1 0
           0 0 0 1      0 0 0 1
```

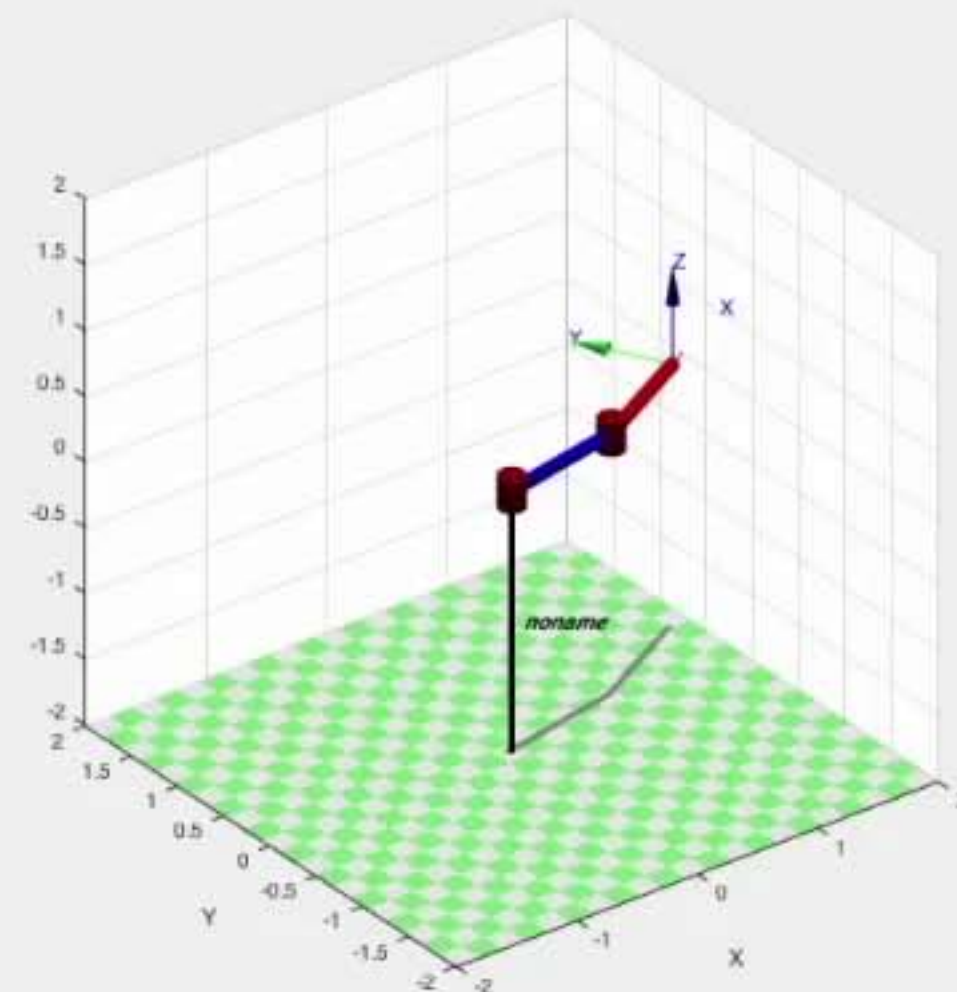
```
>> r.plot([0.2 0.3])
```

```
fx >>
```

Name	Value
dh	[0,0,1,0;0,0,1,0]
r	1x1 SerialLink

Figures - Figure 1

Figure 1



Command Window

j	theta	d	a	alpha
1	q1	0	1	0
2	q2	0	1	0

```
grav = 0 base = 1 0 0 0 tool = 1 0 0 0
      0      0 1 0 0      0 1 0 0
      9.81    0 0 1 0      0 0 1 0
           0 0 0 1      0 0 0 1
```

```
>> r.plot([0.2 0.3])
>> r.teach
```

fx >>

Name	Value
dh	[0,0,1,0;0,0,1,0]
r	1x1 SerialLink

Figures - Figure 1

Figure 1

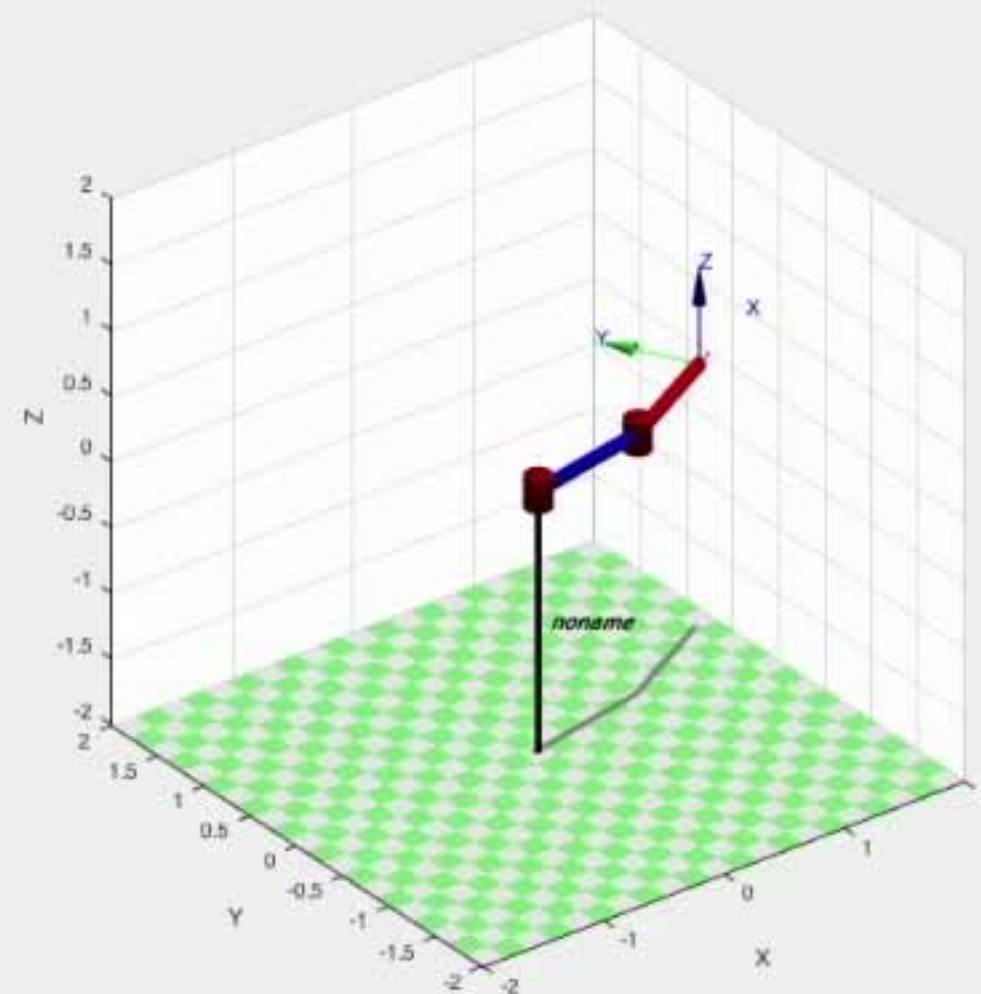
Teach

x: 1.858
y: 0.678
z: 0.000

ax: 0.000
ay: 0.000
az: 1.000

q1 11.5

q2 17.2



Command Window

```

0 0 1 0 0 0 1 0 0
9.81 0 0 1 0 0 0 1 0
0 0 0 1

```

```

>> r.plot([0.2 0.3])
>> r.teach
>> r.fkine([0.2 0.3])

```

ans =

```

0.8776 -0.4794 0 1.8576
0.4794 0.8776 0 0.6781
0 0 1.0000 0
0 0 0 1.0000

```

fx >>

Name	Value
ans	4x4 double
dh	[0,0,1,0;0,0,1,0]
r	1x1 SerialLink

Figures - Figure 1

Figure 1

Teach

x: 1.568

y: 0.256

z: 0.000

ax: 0.000

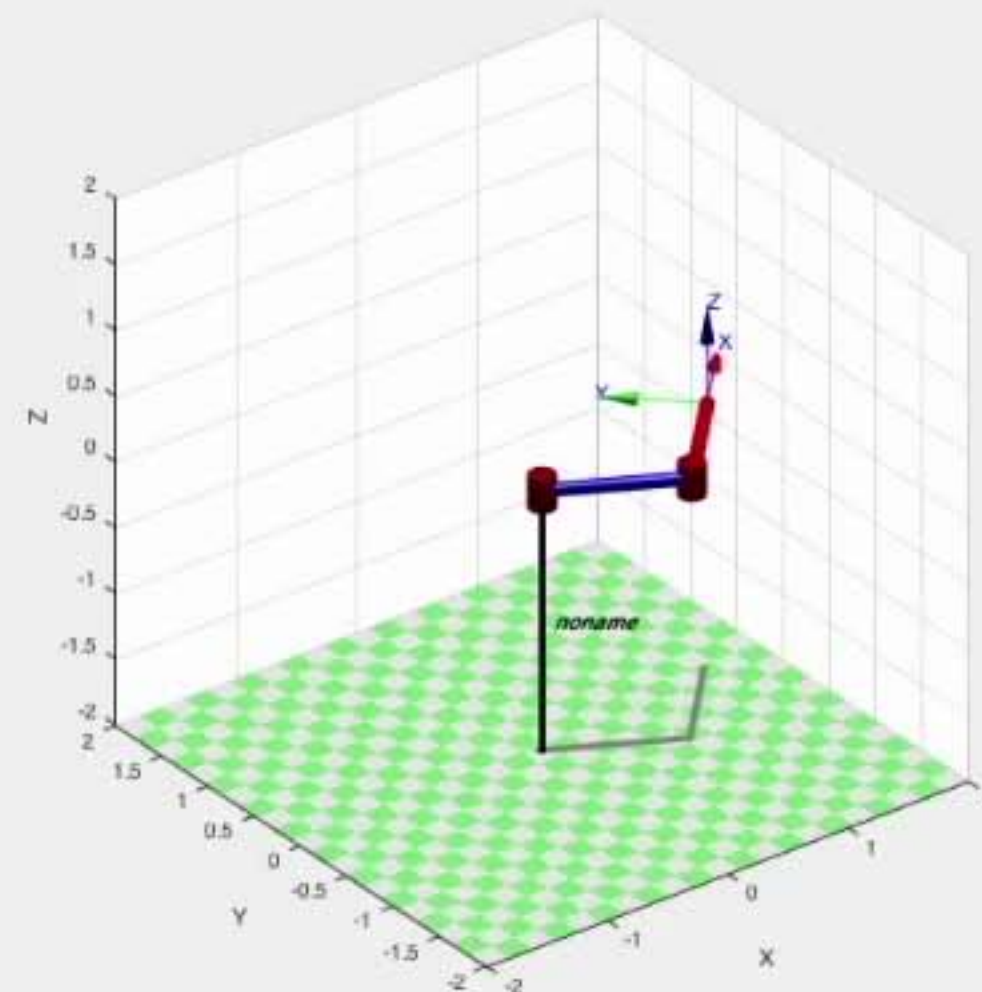
ay: 0.000

az: 1.000

q1 -28.1

q2 74.8

X



Denavit-Hartenberg parameters for **Puma**



Unimate Puma 500
Oussama Khatib | Used with permission

θ_j	d_j	a_j	α_j
q_1	0.0000	0.0000	$\pi/2$
q_2	0.0000	0.4318	0
q_3	0.1500	0.0203	$-\pi/2$
q_4	0.4318	0.0000	$\pi/2$
q_5	0.0000	0.0000	$-\pi/2$
q_6	0.0000	0.0000	0

Command Window

```
>> mdl_puma560  
fx >> |
```

Workspace

Name ▲	Value
deg	0.0175
p560	1x1 SerialLink
qn	[0,0.7854,3.1416...
qr	[0,1.5708,-1.570...
qs	[0,0,-1.5708,0,0,0]
qz	[0,0,0,0,0,0]

Command Window

```
>> mdl_puma560
>> p560
```

p560 =

Puma 560 (6 axis, RRRRRR, stdDH, fastRNE)
viscous friction; params of 8/95;

j	theta	d	a	alpha
1	q1	0	0	1.571
2	q2	0	0.4318	0
3	q3	0.15	0.0203	-1.571
4	q4	0.4318	0	1.571
5	q5	0	0	-1.571
6	q6	0	0	0

```
grav = 0 base = 1 0 0 0 tool = 1 0 0 0
      0      0 1 0 0      0 1 0 0
      9.81    0 0 1 0      0 0 1 0
           0 0 0 1      0 0 0 1
```

fx >>

Workspace

Name	Value
deg	0.0175
p560	1x1 SerialLink
qn	[0,0.7854,3.1416...
qr	[0,1.5708,-1.570...
qs	[0,0,-1.5708,0,0,0]
qz	[0,0,0,0,0,0]

Command Window

1	q1	0	0	1.571
2	q2	0	0.4318	0
3	q3	0.15	0.0203	-1.571
4	q4	0.4318	0	1.571
5	q5	0	0	-1.571
6	q6	0	0	0

```
grav = 0 base = 1 0 0 0 tool = 1 0 0 0
      0      0 1 0 0      0 1 0 0
      9.81 0 0 1 0      0 0 1 0
           0 0 0 1      0 0 0 1
```

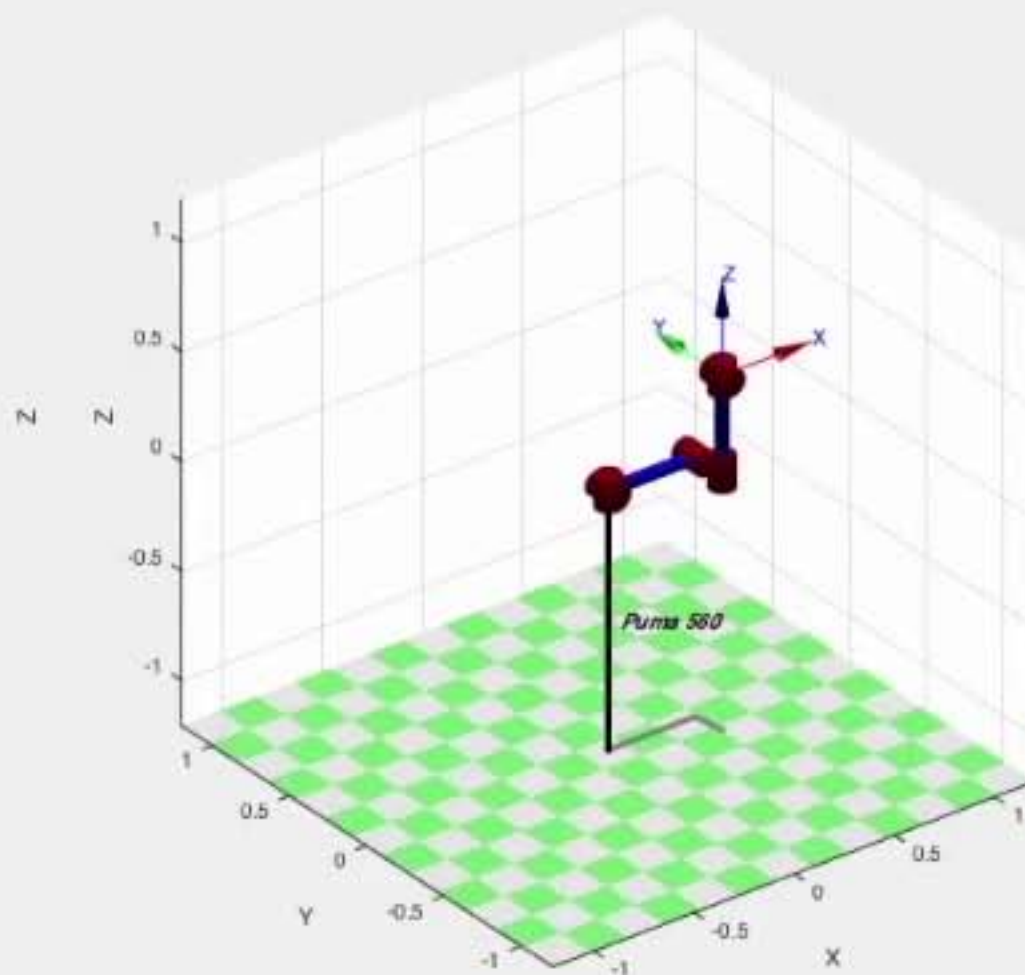
```
>> p560.plot(qz)
```

```
fx >>
```

Name	Value
deg	0.0175
p560	1x1 SerialLink
qn	[0,0.7854,3.1416...
qr	[0,1.5708,-1.570...
qs	[0,0,-1.5708,0,0,0]
qz	[0,0,0,0,0,0]

Figures - Figure 1

Figure 1



Command Window

2	q2	0	0.4318	0
3	q3	0.15	0.0203	-1.571
4	q4	0.4318	0	1.571
5	q5	0	0	-1.571
6	q6	0	0	0

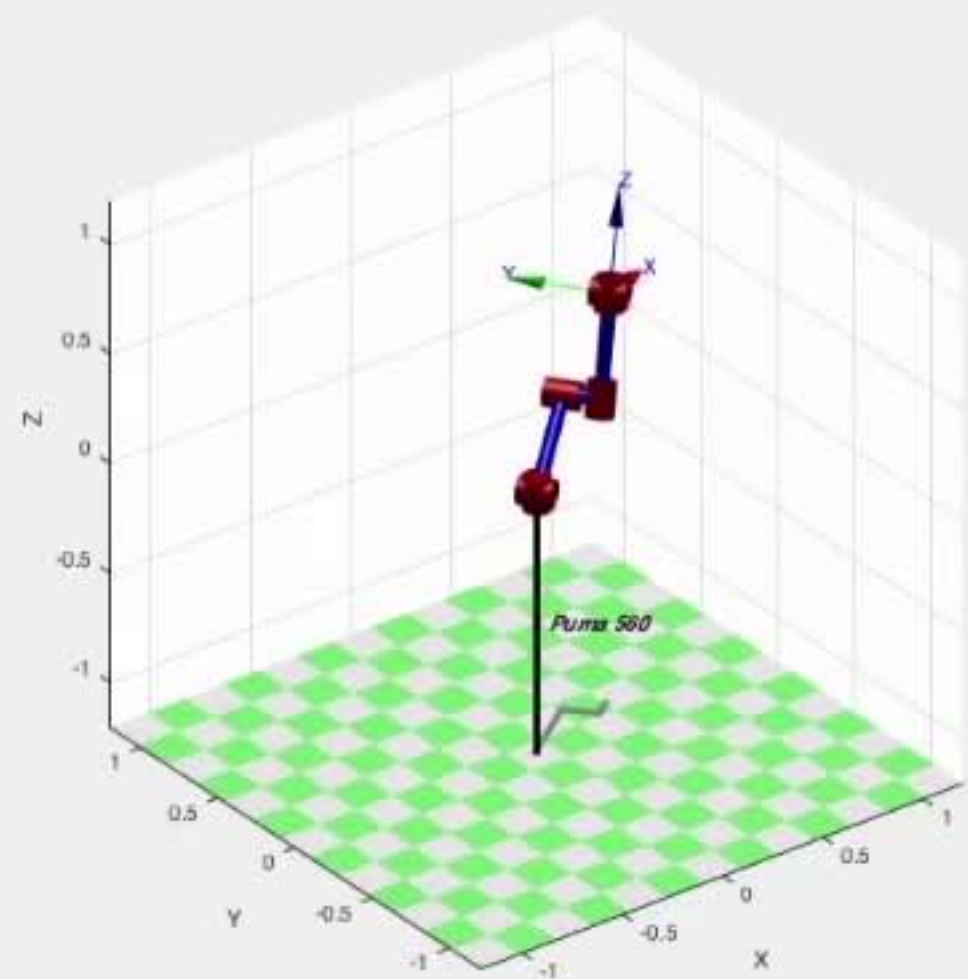
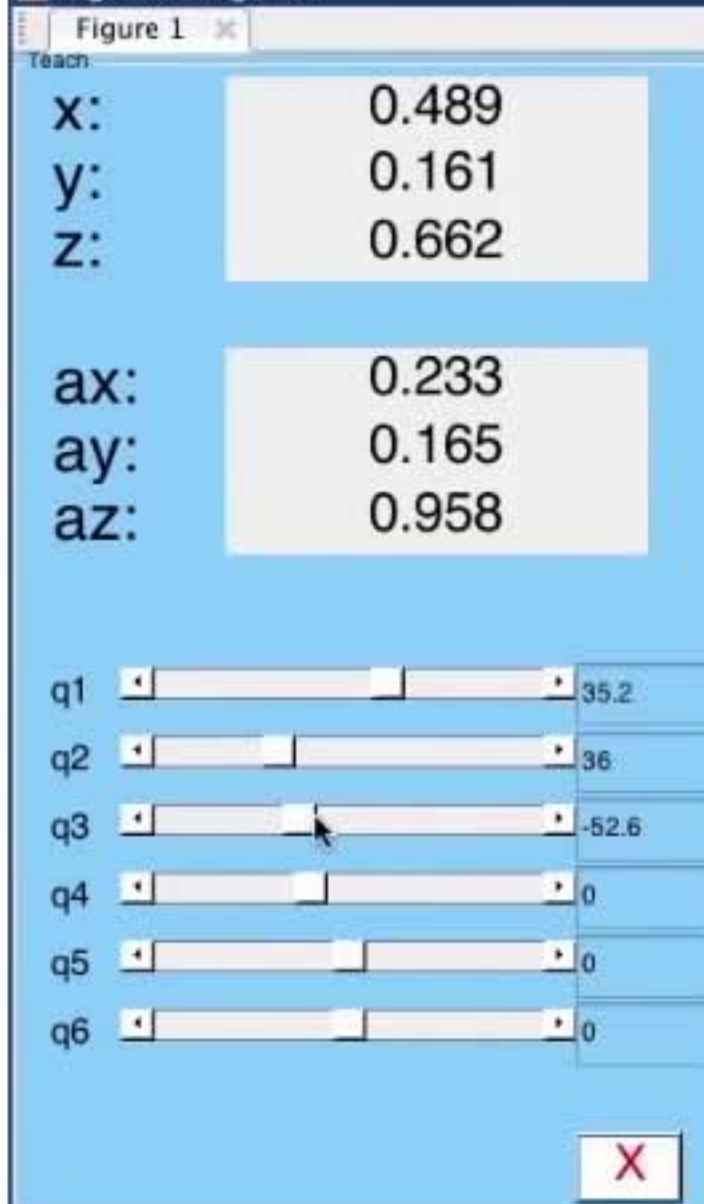
```
grav = 0 base = 1 0 0 0 tool = 1 0 0 0
      0      0 1 0 0      0 1 0 0
      9.81 0 0 1 0      0 0 1 0
           0 0 0 1      0 0 0 1
```

```
>> p560.plot(qz)
>> p560.plot(qr)
>> p560.teach
```

fx >>

Name	Value
deg	0.0175
p560	1x1 SerialLink
qn	[0,0.7854,3.1416...
qr	[0,1.5708,-1.570...
qs	[0,0,-1.5708,0,0,0]
qz	[0,0,0,0,0,0]

Figures - Figure 1



Command Window

```

9.01      0 0 1 0      0 0 1 0
          0 0 0 1      0 0 0 1

>> p560.plot(qz)
>> p560.plot(qr)
>> p560.teach
>> p560.fkine([0.1 0.2 0.3 0 0 0])

```

ans =

```

0.8732 -0.0998 -0.4770 0.2478
0.0876 0.9950 -0.0479 -0.1259
0.4794 -0.0000 0.8776 0.4745
0      0      0      1.0000

```

fx >> |

I

Name	Value
ans	4x4 double
deg	0.0175
p560	1x1 SerialLink
qn	[0,0.7854,3.1416...
qr	[0,1.5708,-1.570...
qs	[0,0,-1.5708,0,0,0]
qz	[0,0,0,0,0,0]

Figures - Figure 1

Figure 1

Teach

x: 0.219

y: -0.699

z: 0.027

ax: 0.614

ay: -0.781

az: -0.114

q1 -60.8

q2 36

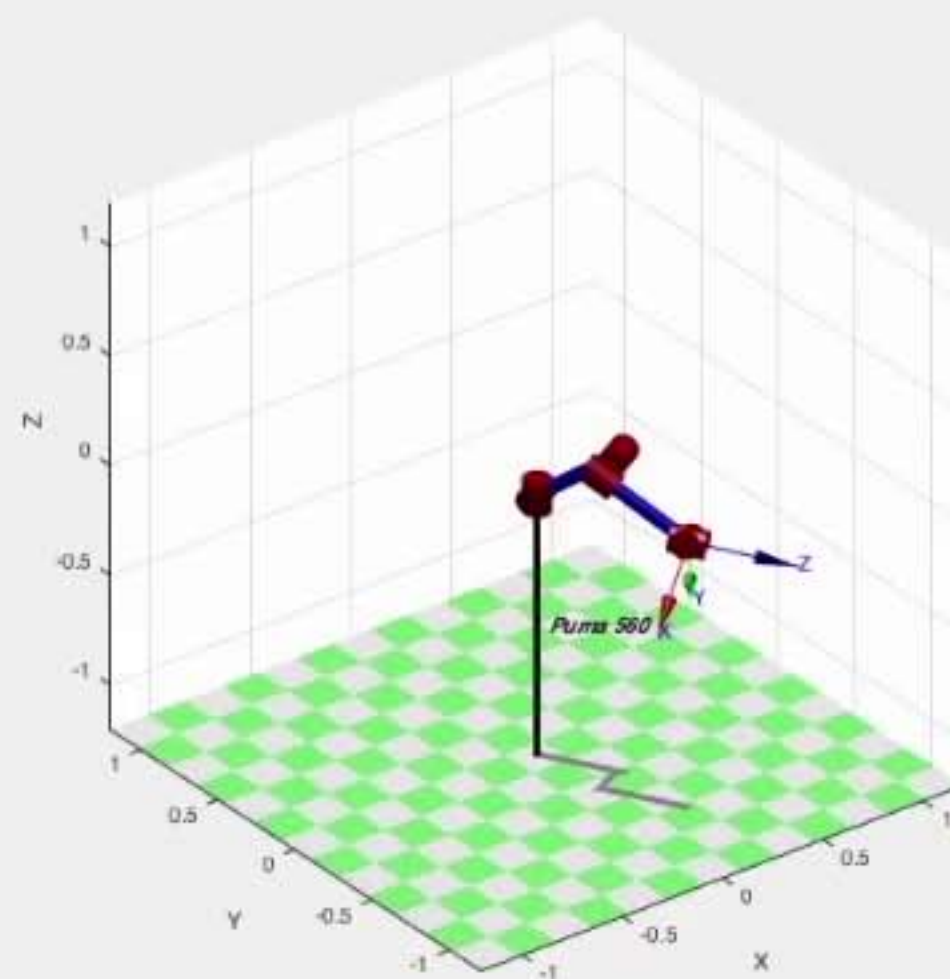
q3 -155

q4 -22.4

q5 24

q6 -42.6

X



General **form**

pose

$$\xi_N = \mathcal{K}(\boldsymbol{q}; \theta, \boldsymbol{d}, \boldsymbol{a}, \alpha, \sigma)$$

joint
configuration

General **form**

The diagram illustrates the general form of a kinematic chain mapping function. The function is represented as $\xi_N = \mathcal{K}(q; \theta, d, a, \alpha, \sigma)$. Each parameter is associated with a callout bubble explaining its physical meaning:

- pose**: points to the output ξ_N
- joint angles**: points to θ
- link offsets**: points to d
- link lengths**: points to a
- link twists**: points to α
- joint types**: points to σ
- joint configuration**: points to the input q

if $\sigma_j = \begin{cases} R \rightarrow \theta_j = q_j \\ P \rightarrow d_j = q_j \end{cases}$

General **form**

The diagram illustrates the general form of a kinematic chain mapping function. The function is represented as $\xi_N = \mathcal{K}(q; \theta, d, a, \alpha, \sigma)$. Each parameter is associated with a callout bubble explaining its physical meaning:

- pose**: points to the output ξ_N
- joint angles**: points to θ
- link offsets**: points to d
- link lengths**: points to a
- link twists**: points to α
- joint types**: points to σ
- joint configuration**: points to the input q

if $\sigma_j = \begin{cases} R \rightarrow \theta_j = q_j \\ P \rightarrow d_j = q_j \end{cases}$

Section 9

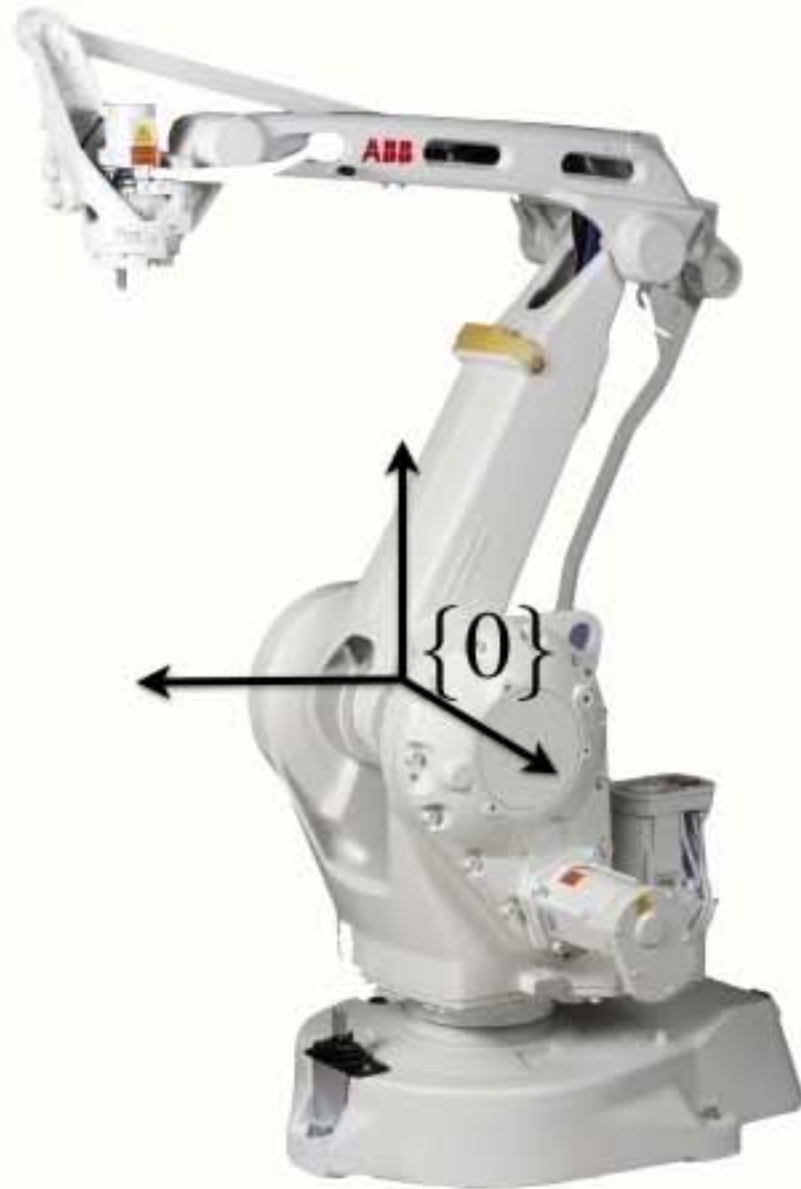
Base and tool transforms

Base transform



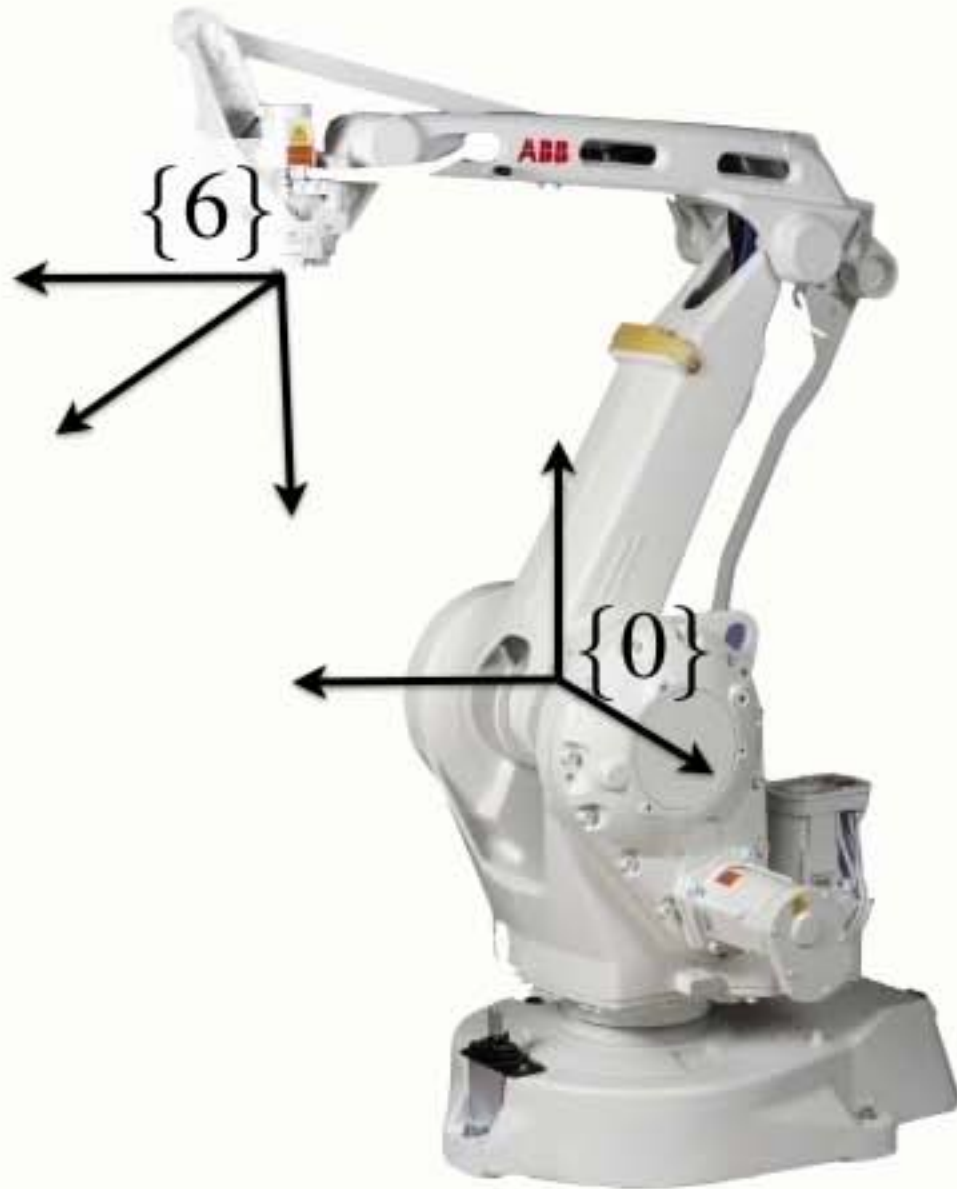
Used with permission of ABB

Base transform



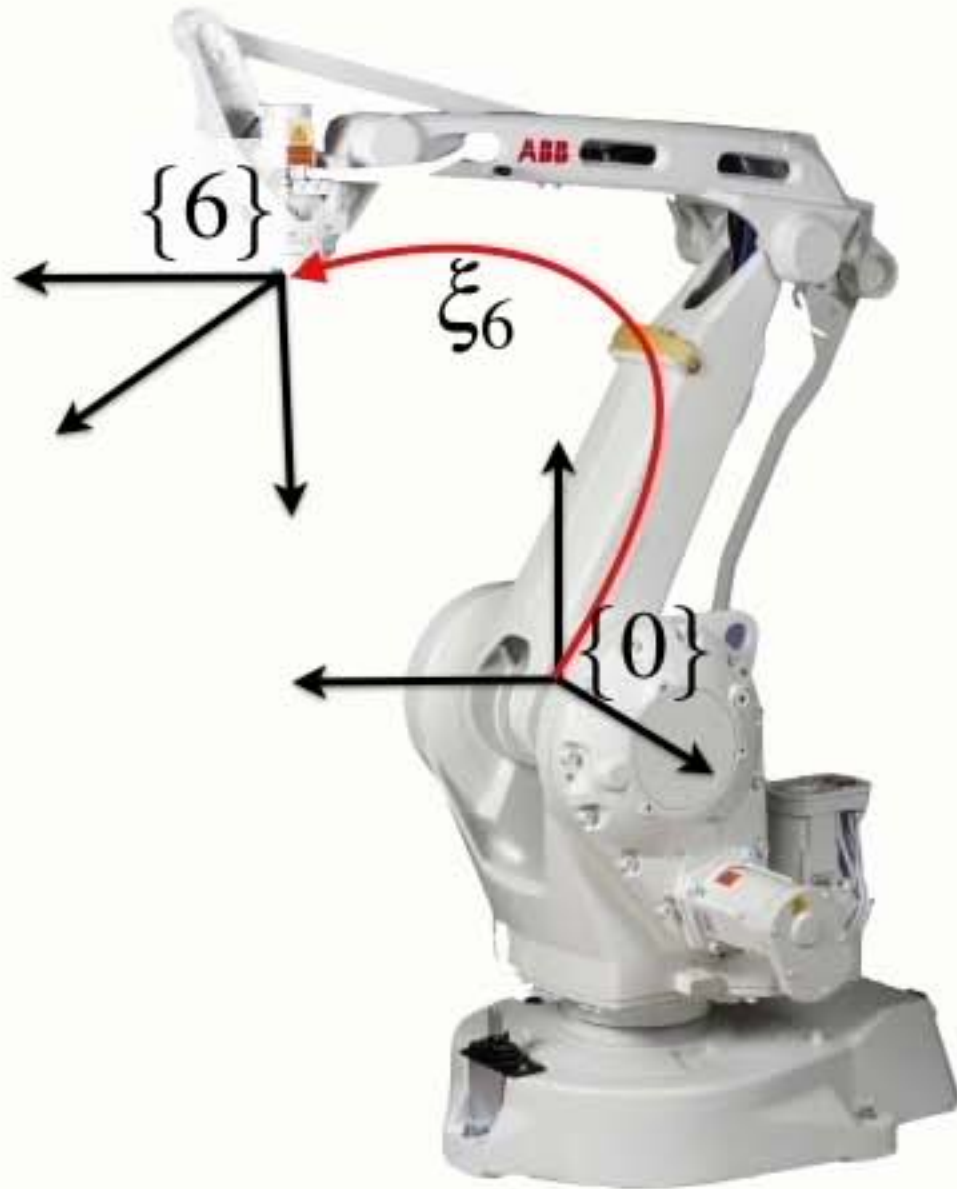
Used with permission of ABB

Base transform



Used with permission of ABB

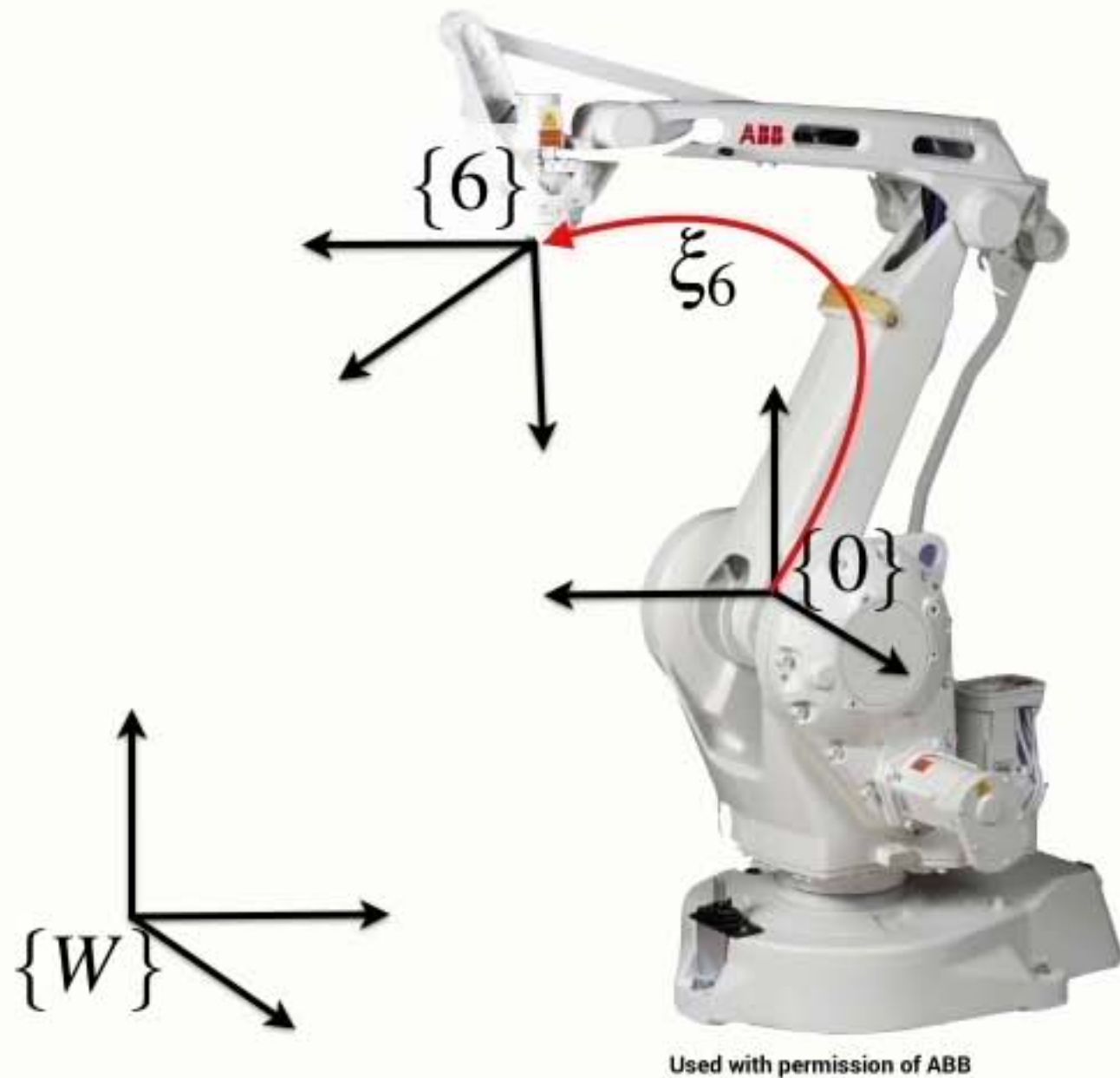
Base transform



Used with permission of ABB

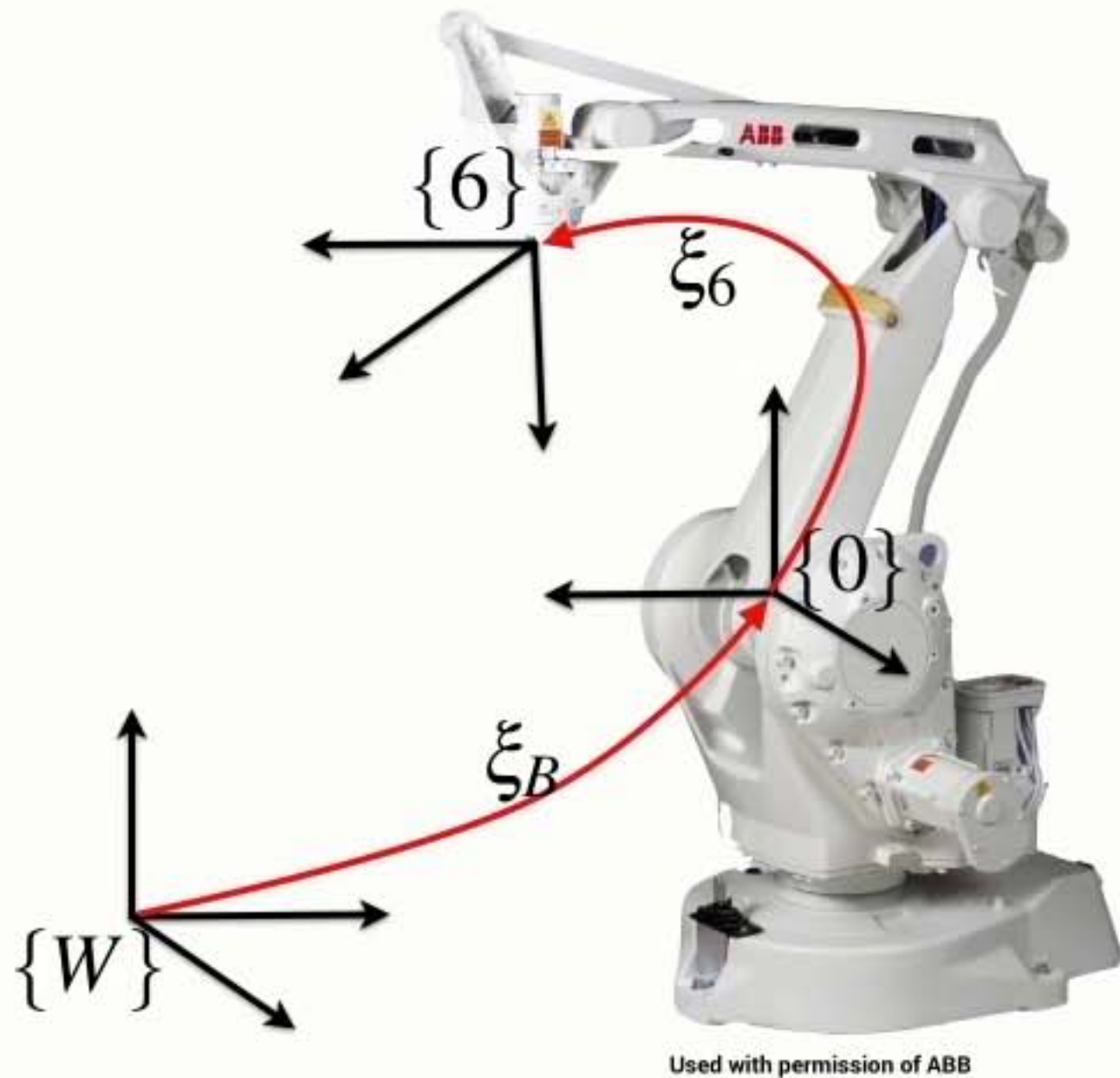
$${}^0\xi_6 = \mathcal{K}(q_1 \cdots q_6)$$

Base transform



$${}^0\xi_6 = \mathcal{K}(q_1 \cdots q_6)$$

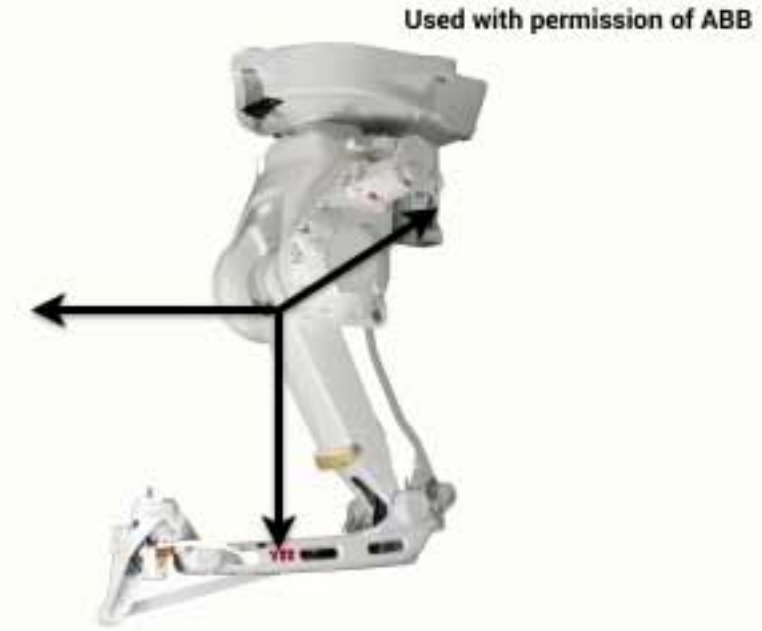
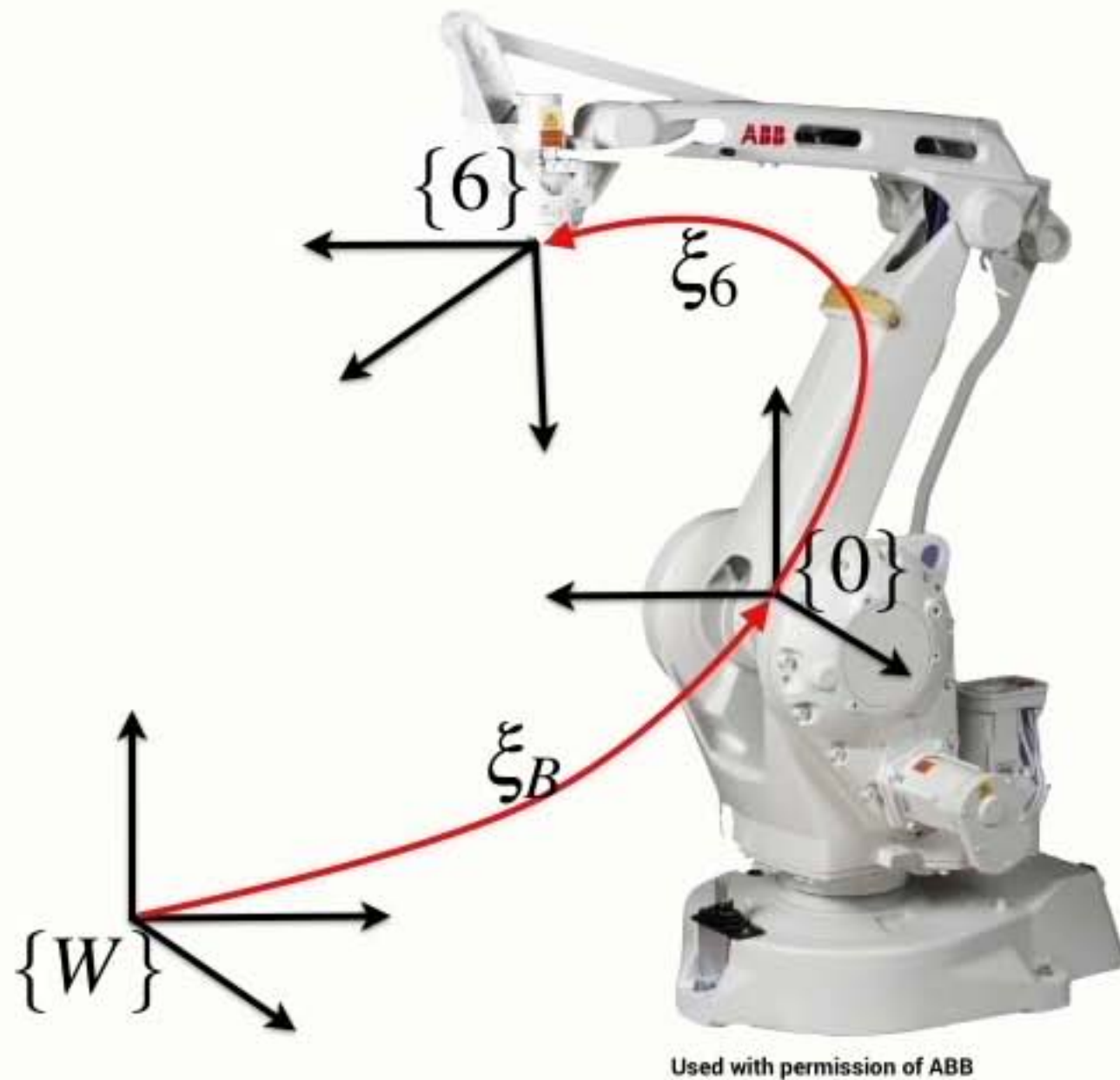
Base transform



$${}^0\xi_6 = \mathcal{K}(q_1 \cdots q_6)$$

$${}^W\xi_6 = \xi_B \oplus \mathcal{K}(q_1 \cdots q_6)$$

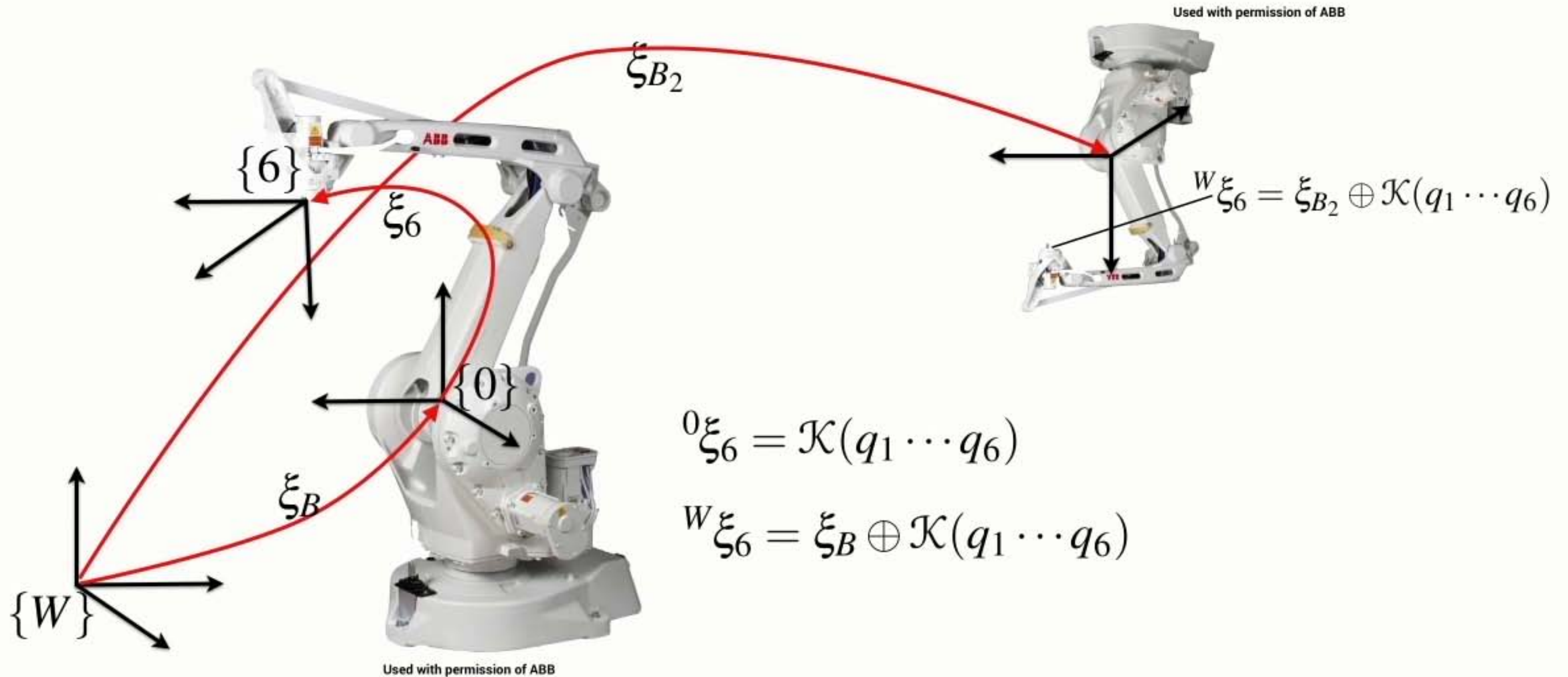
Base transform



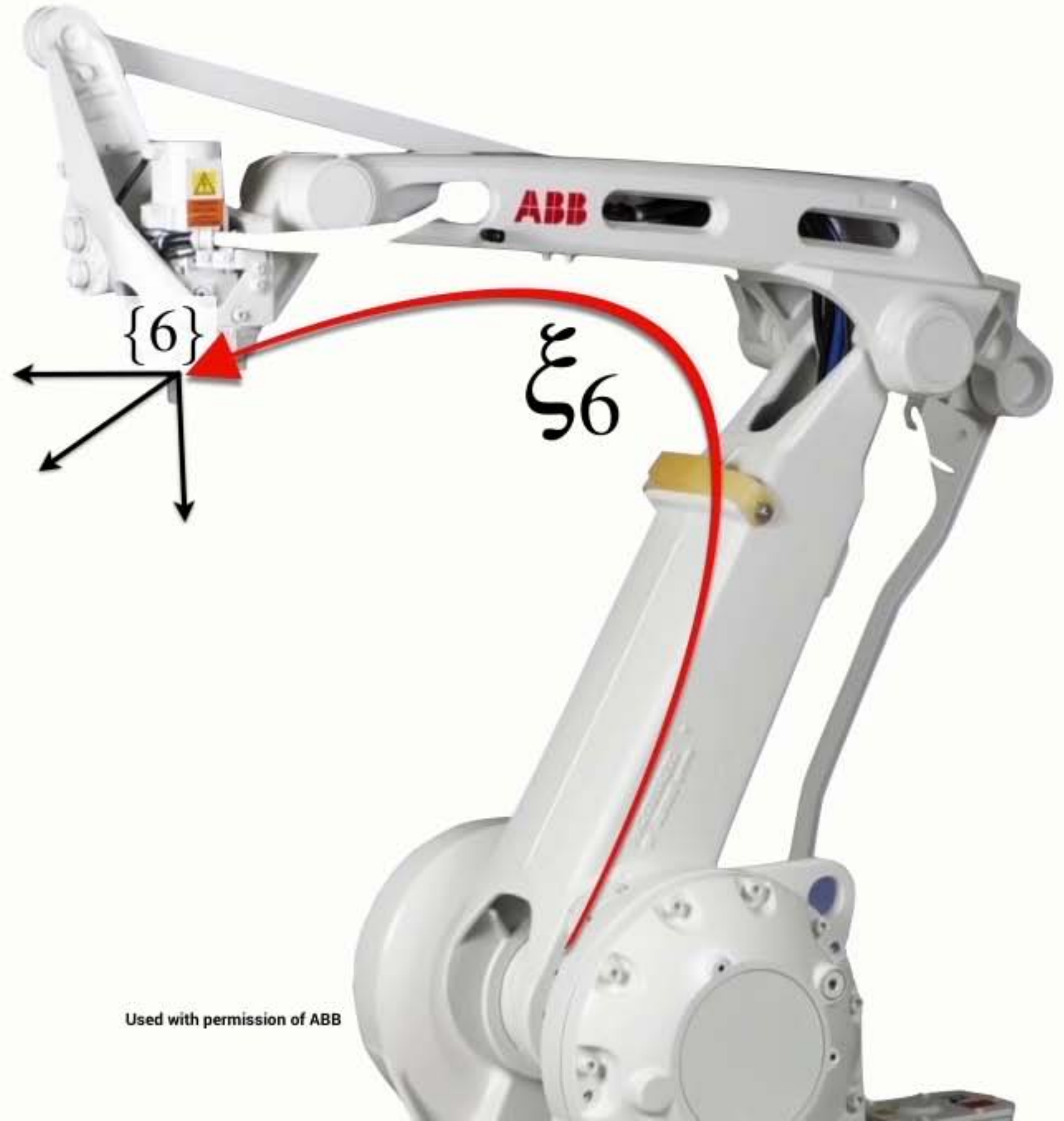
$${}^0\xi_6 = \mathcal{K}(q_1 \cdots q_6)$$

$${}^W\xi_6 = \xi_B \oplus \mathcal{K}(q_1 \cdots q_6)$$

Base transform



Tool transform



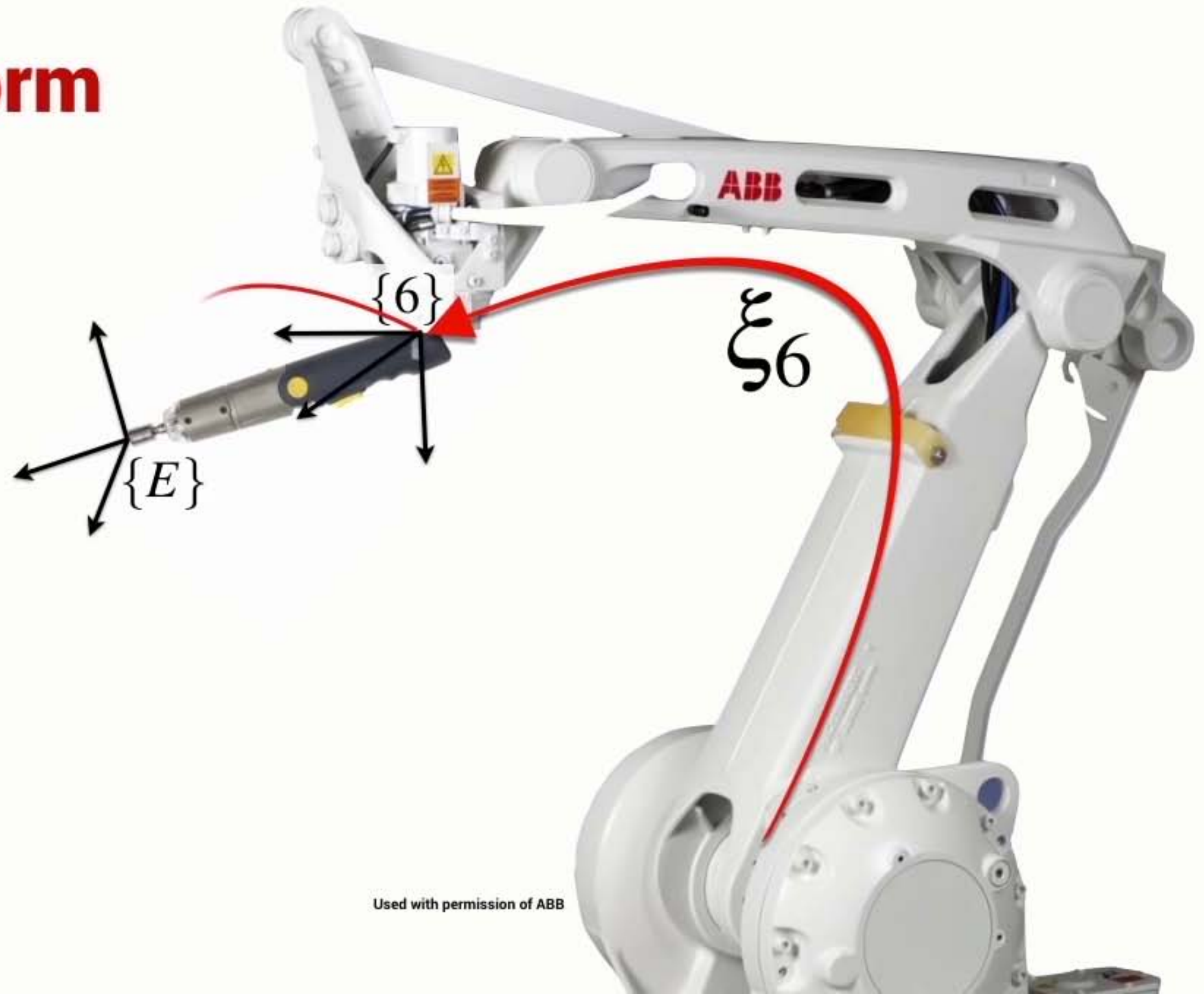
Used with permission of ABB

Tool transform



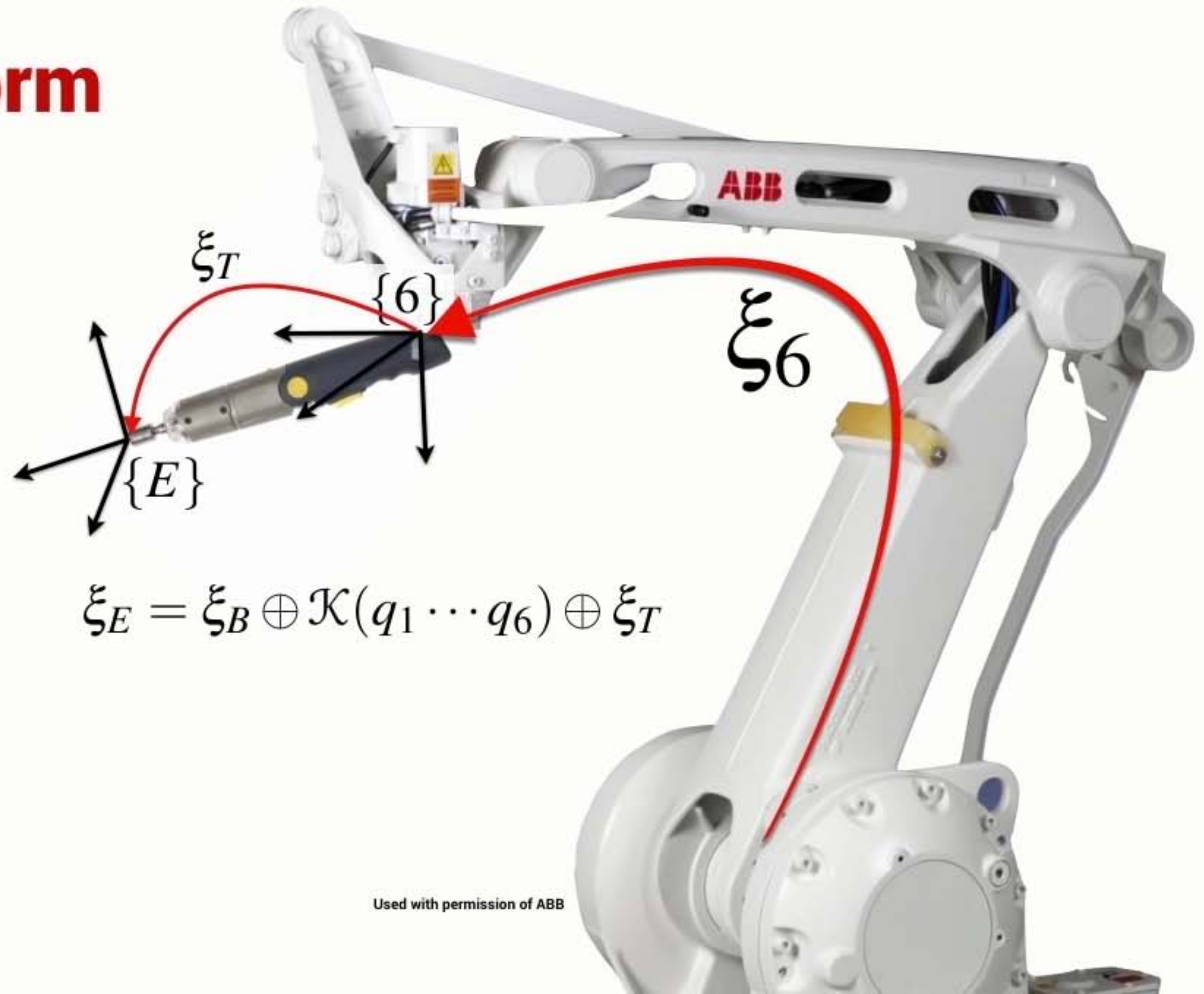
Used with permission of ABB

Tool transform



Used with permission of ABB

Tool transform



$$\xi_E = \xi_B \oplus \mathcal{K}(q_1 \cdots q_6) \oplus \xi_T$$

Used with permission of ABB

Command Window

```
>> mdl_puma560
```

```
>> p560
```

```
p560 =
```

```
Puma 560 (6 axis, RRRRRR, stdDH, fastRNE)
```

```
viscous friction; params of 8/95;
```

j	theta	d	a	alpha
1	q1	0	0	1.571
2	q2	0	0.4318	0
3	q3	0.15	0.0203	-1.571
4	q4	0.4318	0	1.571
5	q5	0	0	-1.571
6	q6	0	0	0

```
grav =    0  base = 1  0  0  0  tool = 1  0  0  0
          0          0  1  0  0          0  1  0  0
        9.81        0  0  1  0          0  0  1  0
                0  0  0  1          0  0  0  1
```

```
>> p560.fkine([0.1 0.2 0.3 0 0 0])
```

```
ans =
```

```
0.8732 -0.0998 -0.4770 0.2478
0.0876 0.9950 -0.0479 -0.1259
0.4794 -0.0000 0.8776 0.4745
0 0 0 1.0000
```

fx >> |

Workspace

Name ▲	Value
ans	4x4 double
deg	0.0175
p560	1x1 SerialLink
qn	[0,0.7854,3.1416...
qr	[0,1.5708,-1.570...
qs	[0,0,-1.5708,0,0,0]
qz	[0,0,0,0,0,0]

Command Window

```

0.8732 -0.0998 -0.4770 0.2478
0.0876 0.9950 -0.0479 -0.1259
0.4794 -0.0000 0.8776 0.4745
0 0 0 1.0000

```

```
>> p560.base = transl(10, 15, 2)
```

```
p560 =
```

```

Puma 560 (6 axis, RRRRRR, stdDH, fastRNE)
viscous friction; params of 8/95;

```

j	theta	d	a	alpha
1	q1	0	0	1.571
2	q2	0	0.4318	0
3	q3	0.15	0.0203	-1.571
4	q4	0.4318	0	1.571
5	q5	0	0	-1.571
6	q6	0	0	0

```

grav = 0 base = 1 0 0 10 tool = 1 0 0 0
0 0 1 0 15 0 1 0 0
9.81 0 0 1 2 0 0 1 0
0 0 0 1 0 0 0 1

```

fx >>

Workspace

Name	Value
ans	4x4 double
deg	0.0175
p560	1x1 SerialLink
qn	[0,0.7854,3.1416...
qr	[0,1.5708,-1.570...
qs	[0,0,-1.5708,0,0,0]
qz	[0,0,0,0,0,0]

Command Window

```

0.8732 -0.0998 -0.4770 0.2478
0.0876 0.9950 -0.0479 -0.1259
0.4794 -0.0000 0.8776 0.4745
0 0 0 1.0000

```

```
>> p560.base = transl(10, 15, 2)
```

```
p560 =
```

```

Puma 560 (6 axis, RRRRRR, stdDH, fastRNE)
viscous friction; params of 8/95;

```

j	theta	d	a	alpha
1	q1	0	0	1.571
2	q2	0	0.4318	0
3	q3	0.15	0.0203	-1.571
4	q4	0.4318	0	1.571
5	q5	0	0	-1.571
6	q6	0	0	0

```

grav = 0 base = 1 0 0 10 tool = 1 0 0 0
      0      0 1 0 15      0 1 0 0
      9.81    0 0 1 2      0 0 1 0
            0 0 0 1      0 0 0 1

```

```
fx >> p560.fkine([0.1 0.2 0.3 0 0 0])
```

Workspace

Name ▲	Value
ans	4x4 double
deg	0.0175
p560	1x1 SerialLink
qn	[0,0.7854,3.1416...
qr	[0,1.5708,-1.570...
qs	[0,0,-1.5708,0,0,0]
qz	[0,0,0,0,0,0]

Command Window

1	q1	0	0	1.571
2	q2	0	0.4318	0
3	q3	0.15	0.0203	-1.571
4	q4	0.4318	0	1.571
5	q5	0	0	-1.571
6	q6	0	0	0

```
grav = 0 base = 1 0 0 10 tool = 1 0 0 0
      0      0 1 0 15      0 1 0 0
      9.81    0 0 1 2      0 0 1 0
           0 0 0 1      0 0 0 1
```

```
>> p560.fkine([0.1 0.2 0.3 0 0 0])
```

```
ans =
```

```
0.8732 -0.0998 -0.4770 10.2478
0.0876 0.9950 -0.0479 14.8741
0.4794 -0.0000 0.8776 2.4745
0      0      0      1.0000
```

```
fx >> |
```

Workspace

Name ▲	Value
ans	4x4 double
deg	0.0175
p560	1x1 SerialLink
qn	[0,0.7854,3.1416...
qr	[0,1.5708,-1.570...
qs	[0,0,-1.5708,0,0,0]
qz	[0,0,0,0,0,0]

```
>> p560.base = transl(10, 15, 2) * trotx(pi)
```

```
p560 =
```

```
Puma 560 (6 axis, RRRRRR, stdDH, fastRNE)
```

```
viscous friction; params of 8/95;
```

j	theta	d	a	alpha
1	q1	0	0	1.571
2	q2	0	0.4318	0
3	q3	0.15	0.0203	-1.571
4	q4	0.4318	0	1.571
5	q5	0	0	-1.571
6	q6	0	0	0

```
grav =      0  base = 1   0   0  10  tool = 1   0   0   0
           0      0  -1   0  15      0   1   0   0
        9.81      0   0  -1   2      0   0   1   0
           0      0   0   1      0   0   0   1
```

```
fx >>
```

Name ▲	Value
ans	4x4 double
deg	0.0175
p560	1x1 SerialLink
qn	[0,0.7854,3.1416...
qr	[0,1.5708,-1.570...
qs	[0,0,-1.5708,0,0,0]
qz	[0,0,0,0,0,0]

Command Window

1	q1	0	0	1.571
2	q2	0	0.4318	0
3	q3	0.15	0.0203	-1.571
4	q4	0.4318	0	1.571
5	q5	0	0	-1.571
6	q6	0	0	0

```
grav = 0 base = 1 0 0 10 tool = 1 0 0 0
      0      0 -1 0 15      0 1 0 0
      9.81    0 0 -1 2      0 0 1 0
           0 0 0 1      0 0 0 1
```

```
>> p560.fkine([0.1 0.2 0.3 0 0 0])
```

```
ans =
```

```
0.8732 -0.0998 -0.4770 10.2478
-0.0876 -0.9950 0.0479 15.1259
-0.4794 0.0000 -0.8776 1.5255
0 0 0 1.0000
```

fx >>

Workspace

Name ▲	Value
ans	4x4 double
deg	0.0175
p560	1x1 SerialLink
qn	[0,0.7854,3.1416...
qr	[0,1.5708,-1.570...
qs	[0,0,-1.5708,0,0,0]
qz	[0,0,0,0,0,0]

```
>> p560.tool = transl(0, 0, 0.2)
```

```
p560 =
```

```
Puma 560 (6 axis, RRRRRR, stdDH, fastRNE)
```

```
viscous friction; params of 8/95;
```

j	theta	d	a	alpha
1	q1	0	0	1.571
2	q2	0	0.4318	0
3	q3	0.15	0.0203	-1.571
4	q4	0.4318	0	1.571
5	q5	0	0	-1.571
6	q6	0	0	0

```
grav =      0  base = 1      0      0  10  tool = 1      0      0      0
          0      0     -1      0  15      0      1      0      0
        9.81      0      0     -1      2      0      0      1      0.2
              0      0      0      1      0      0      0      0      1
```

```
fx >>
```

Name ▲	Value
ans	4x4 double
deg	0.0175
p560	1x1 SerialLink
qn	[0,0.7854,3.1416...
qr	[0,1.5708,-1.570...
qs	[0,0,-1.5708,0,0,0]
qz	[0,0,0,0,0,0]

Command Window

1	q1	0	0	1.571
2	q2	0	0.4318	0
3	q3	0.15	0.0203	-1.571
4	q4	0.4318	0	1.571
5	q5	0	0	-1.571
6	q6	0	0	0

```
grav = 0 base = 1 0 0 10 tool = 1 0 0 0
      0      0 -1 0 15      0      1 0 0
      9.81  0 0 -1 2      0      0 1 0.2
           0 0 0 1      0      0 0 1
```

```
>> p560.fkine([0.1 0.2 0.3 0 0 0])
```

```
ans =
```

```
0.8732 -0.0998 -0.4770 10.1524
-0.0876 -0.9950 0.0479 15.1355
-0.4794 0.0000 -0.8776 1.3500
0 0 0 1.0000
```

fx >>

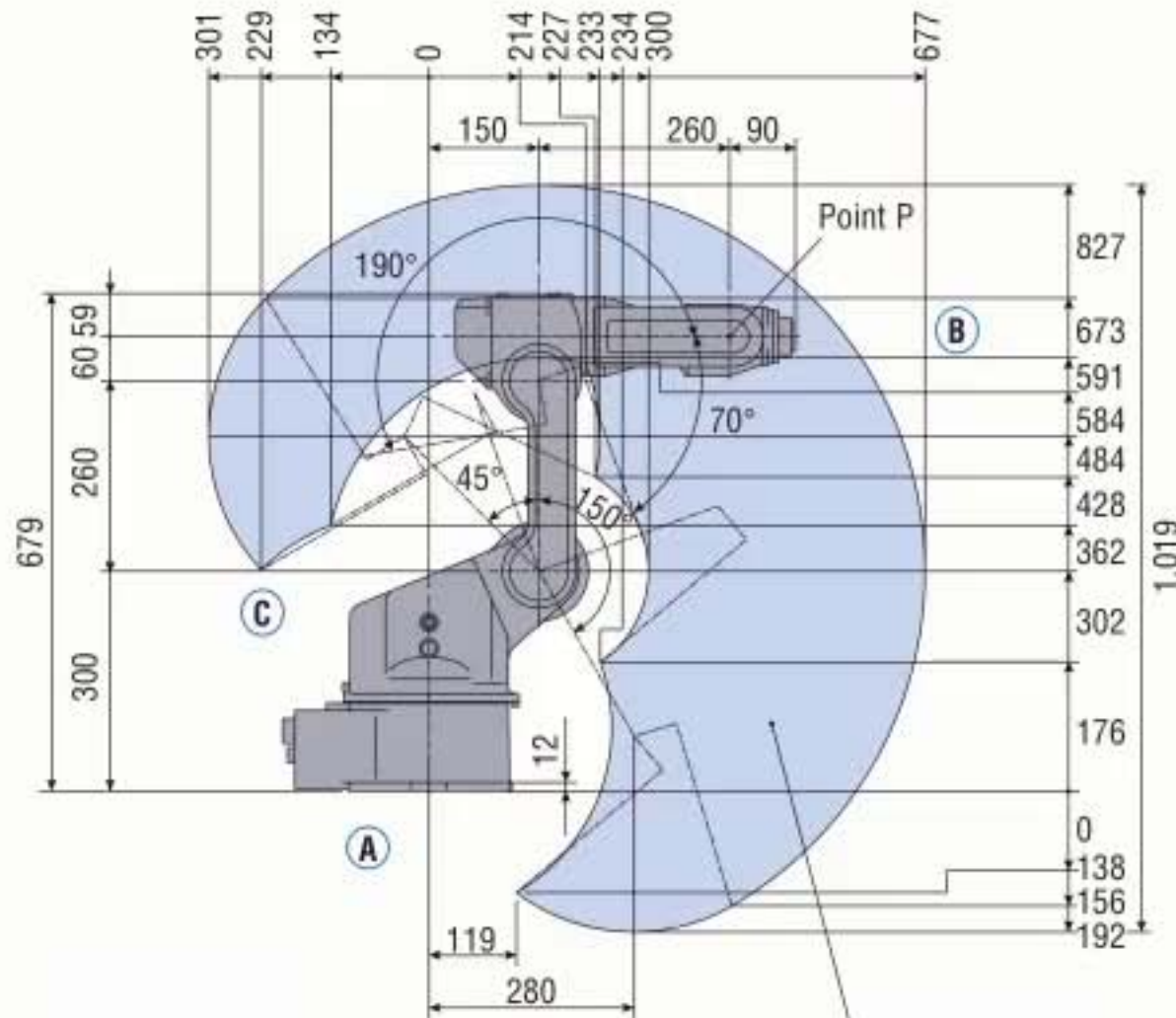
Workspace

Name ▲	Value
ans	4x4 double
deg	0.0175
p560	1x1 SerialLink
qn	[0,0.7854,3.1416...
qr	[0,1.5708,-1.570...
qs	[0,0,-1.5708,0,0,0]
qz	[0,0,0,0,0,0]

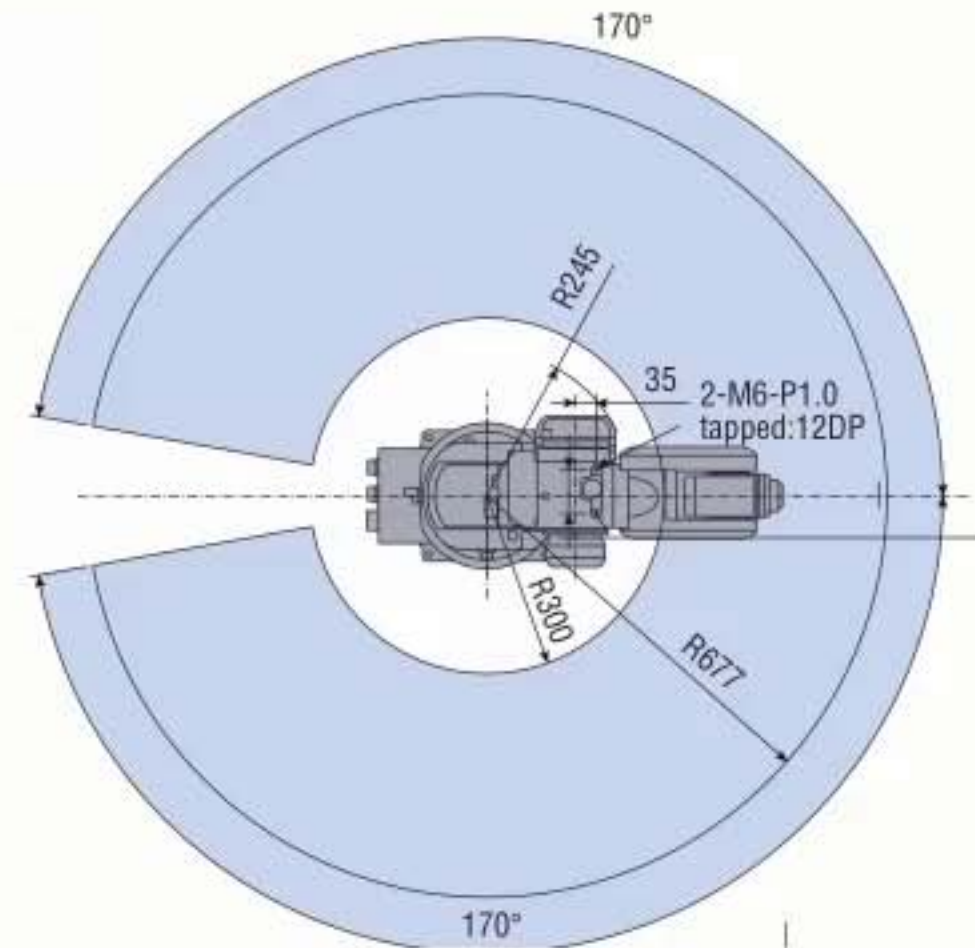
Section10 | Workspace

Workspace

- The set of all reachable end point positions



Yaskawa America, Inc., Motoman Robotics Division



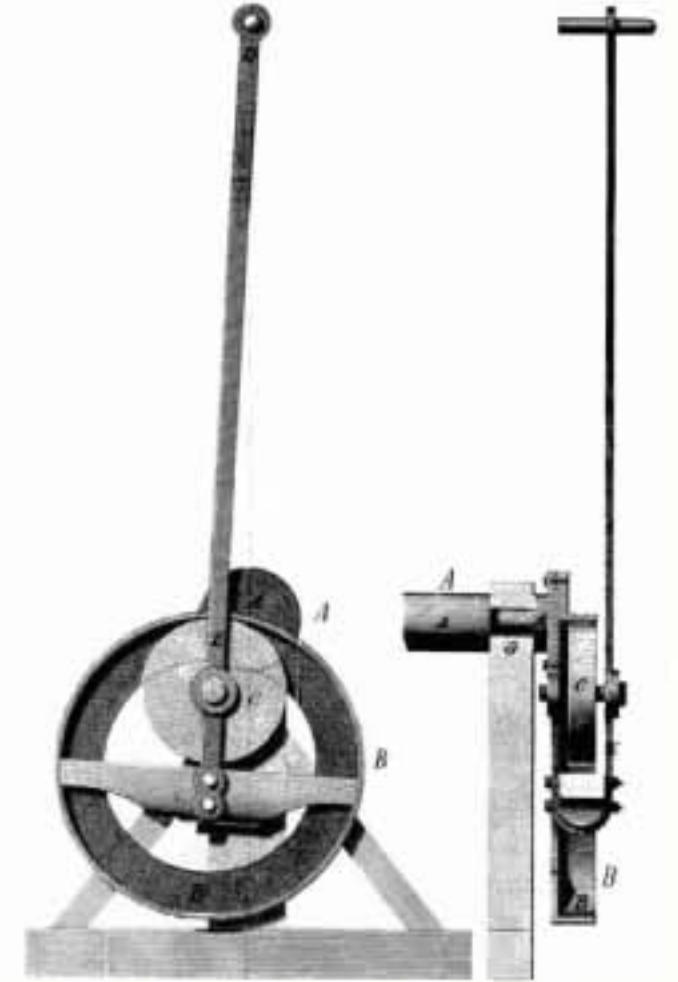
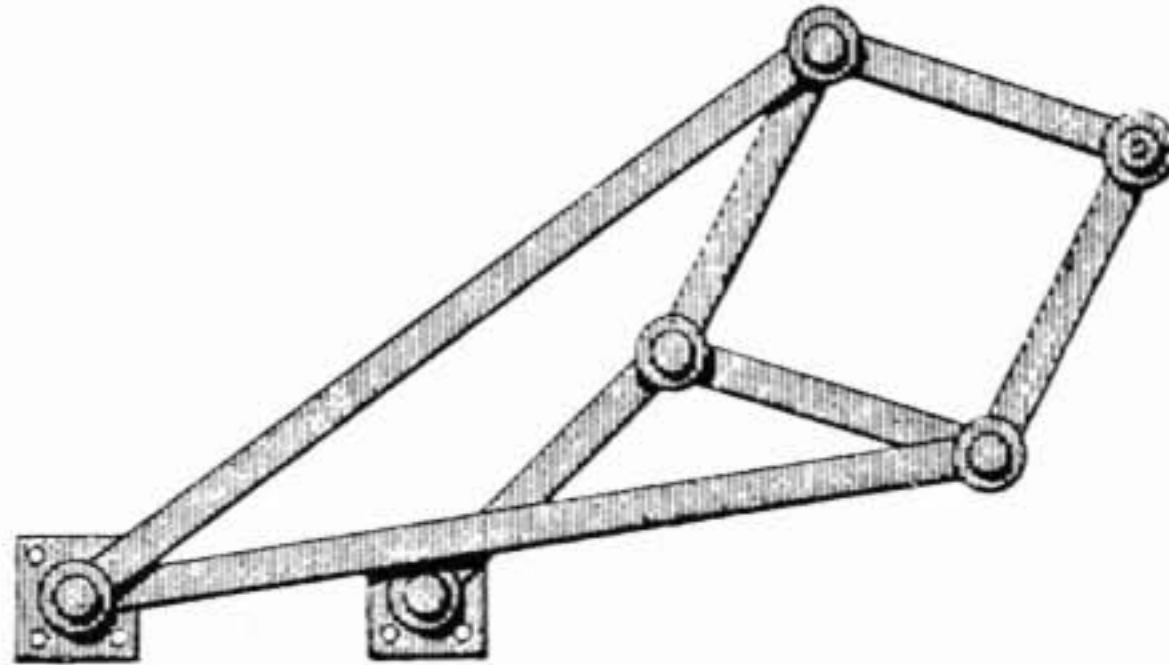
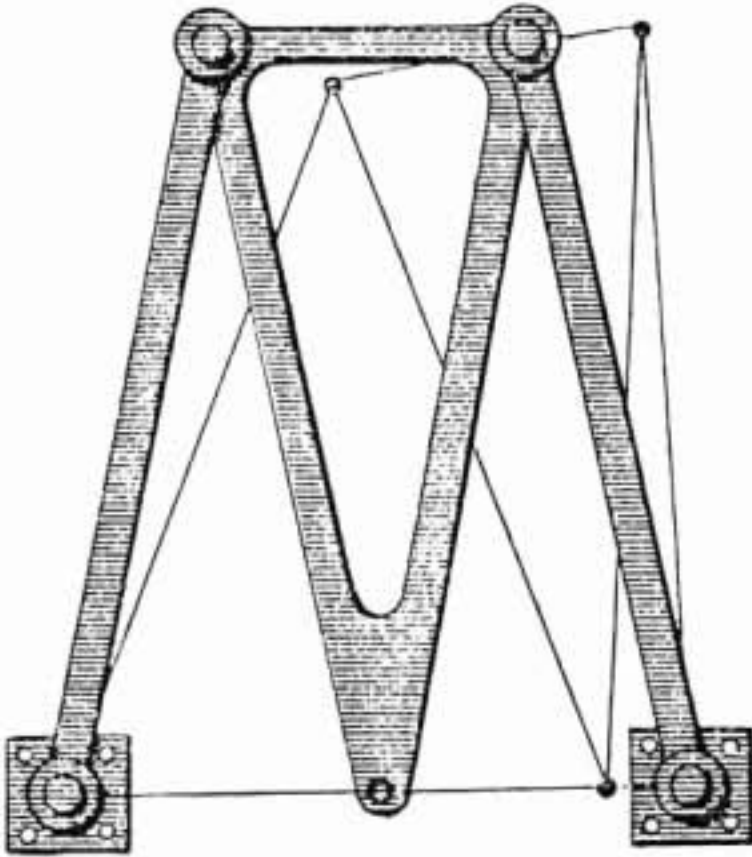
Yaskawa America, Inc., Motoman Robotics Division

Yaskawa Motoman SV3X

Section 11 Kinematics

Kinematics

- A branch of **mathematics** that studies the **motion** of a body, or a system of bodies
- Concerned with **positions** (and angles) and **velocities** (translational and angular)
- Not concerned with forces and moments



Robot **Kinematics**

- We consider **two kinematic problems** in robotics

Robot **Kinematics**

- We consider **two kinematic problems** in robotics
 - ➔ Forward kinematic problem
 - given the joint angles, where is the robot's tool tip?

Robot **Kinematics**

- We consider **two kinematic problems** in robotics
 - ➔ Forward kinematic problem
 - given the joint angles, where is the robot's tool tip?
 - ➔ Inverse kinematic problem
 - given the pose of the robot's tool tip, what joint angles are required?

Section 12 Summary

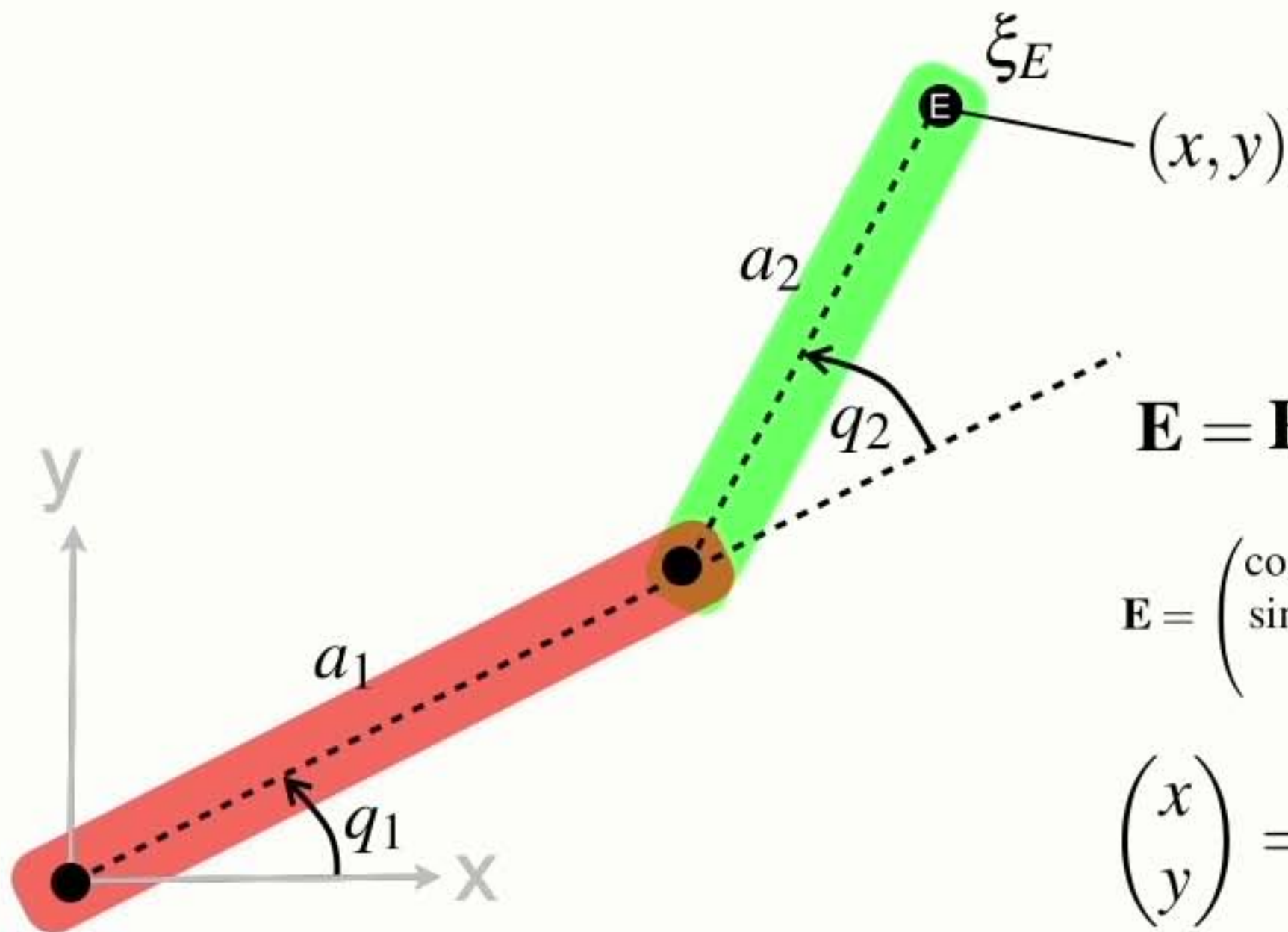
Forward **Kinematics**

- Robot tip pose given a set of joint coordinates

$$\xi_N = \mathcal{K}(\boldsymbol{q})$$

$$\boldsymbol{q} = \{q_j, j \in [1 \cdots N]\}$$

Finding the tool tip **pose**

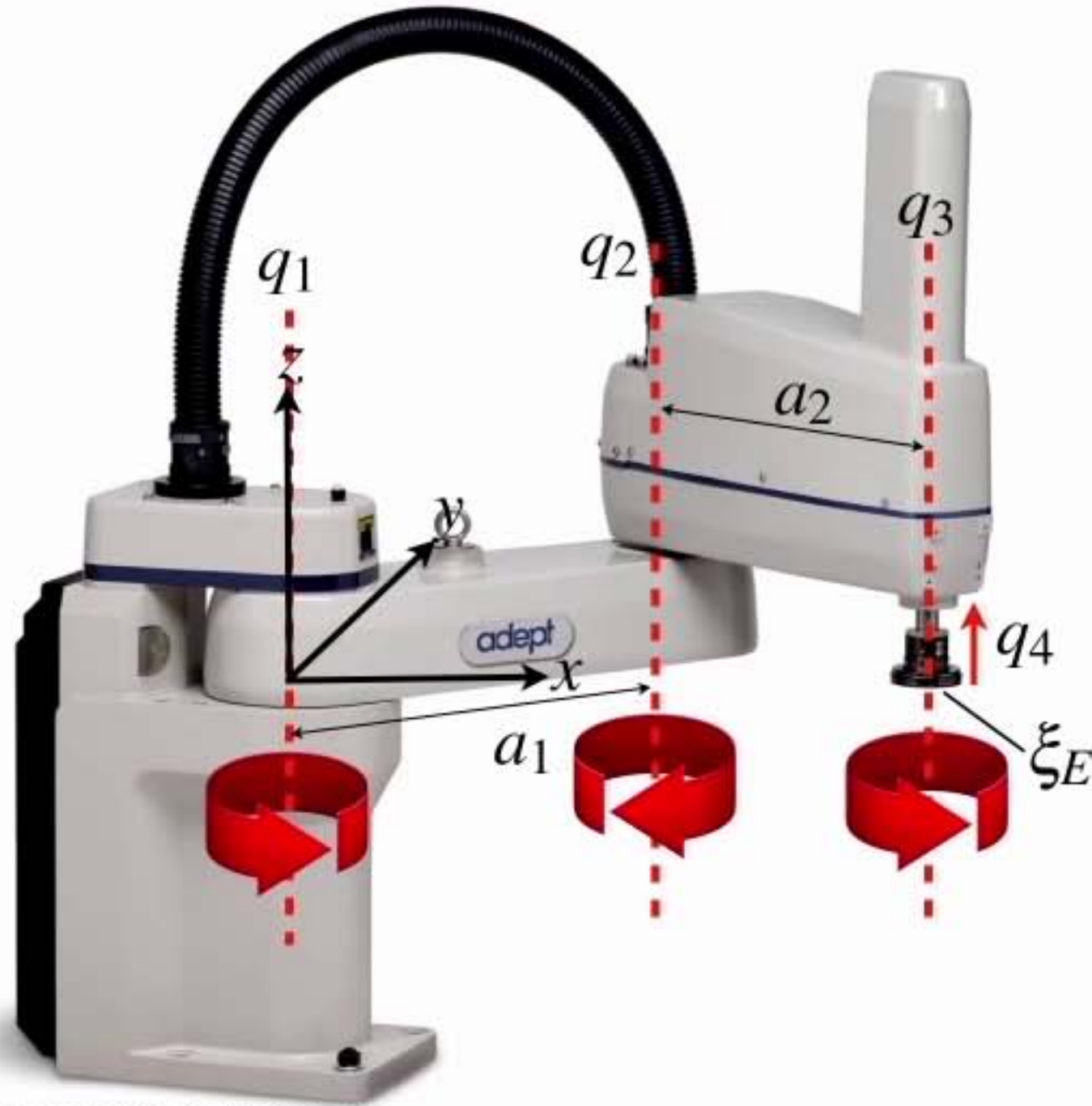


$$\mathbf{E} = \mathbf{R}_x(q_1) \mathbf{T}_x(a_1) \mathbf{R}_x(q_2) \mathbf{T}_x(a_2)$$

$$\mathbf{E} = \begin{pmatrix} \cos(q_1 + q_2) & -\sin(q_1 + q_2) & a_2 \cos(q_1 + q_2) + a_1 \cos q_1 \\ \sin(q_1 + q_2) & \cos(q_1 + q_2) & a_2 \sin(q_1 + q_2) + a_1 \sin q_1 \\ 0 & 0 & 1 \end{pmatrix}$$

$$\begin{pmatrix} x \\ y \end{pmatrix} = \begin{pmatrix} a_2 \cos(q_1 + q_2) + a_1 \cos q_1 \\ a_2 \sin(q_1 + q_2) + a_1 \sin q_1 \end{pmatrix}$$

■ These are all 2D transforms $\mathbf{E} \in \text{SE}(2)$



SCARA robot

Copyright © 2014 Adept Technology, Inc.
The usage of this image in no manner is to be construed as
Adept's endorsement of QUT or any QUT product or service

$$\mathbf{E} = \mathbf{R}_z(q_1) \mathbf{T}_x(a_1) \mathbf{R}_z(q_2) \mathbf{T}_x(a_2) \mathbf{R}_z(q_3) \mathbf{T}_z(q_4)$$

- These are all 3D transforms $\mathbf{E} \in \text{SE}(3)$

Configuration **space**

- Robot **configuration** is described by a vector of **generalised** joint coordinates
- Each coordinate can be:
 - ➔ an angle, for a rotational (**revolute**) joint
 - ➔ a length, for a sliding (**prismatic**) joint

Joint
configuration

Joint
coordinate

Number of
joints

Space of all
possible
configurations

$$\mathbf{q} = \{q_j, j \in [1 \cdots N]\} \in \mathcal{C}$$

Configuration space $\mathcal{C} \subset \mathbb{R}^N$

Task **space**

- The space of all possible **end-effector poses**

$$\xi_E \in \mathcal{T}$$

Space of all
possible end-
effector poses

- In 2D $\mathcal{T} \subset \text{SE}(2)$

$$\xi_E \sim (x, y)$$

$$\xi_E \sim (x, y, \theta)$$

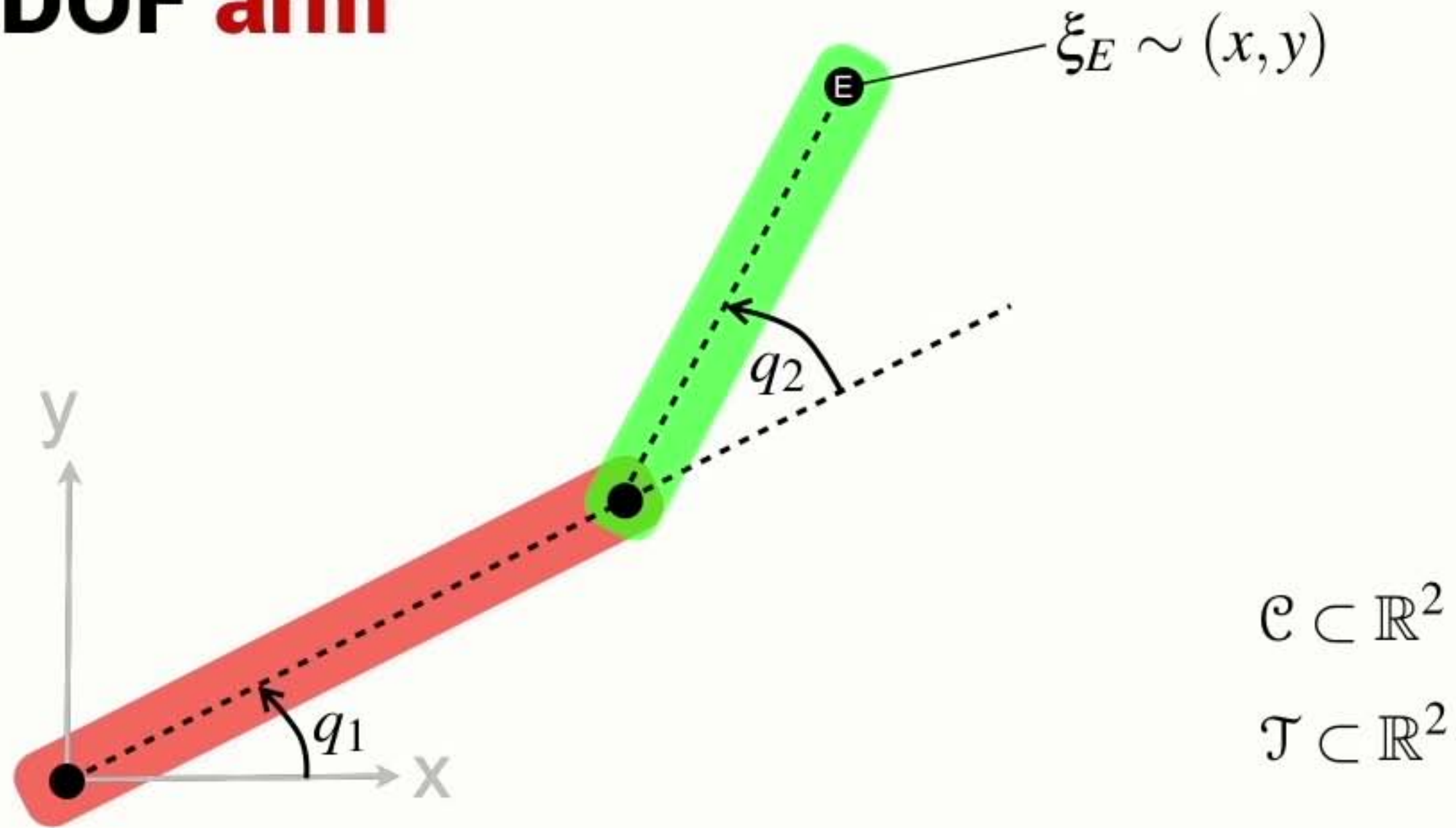
- In 3D $\mathcal{T} \subset \text{SE}(3)$

$$\xi_E \sim (x, y, z)$$

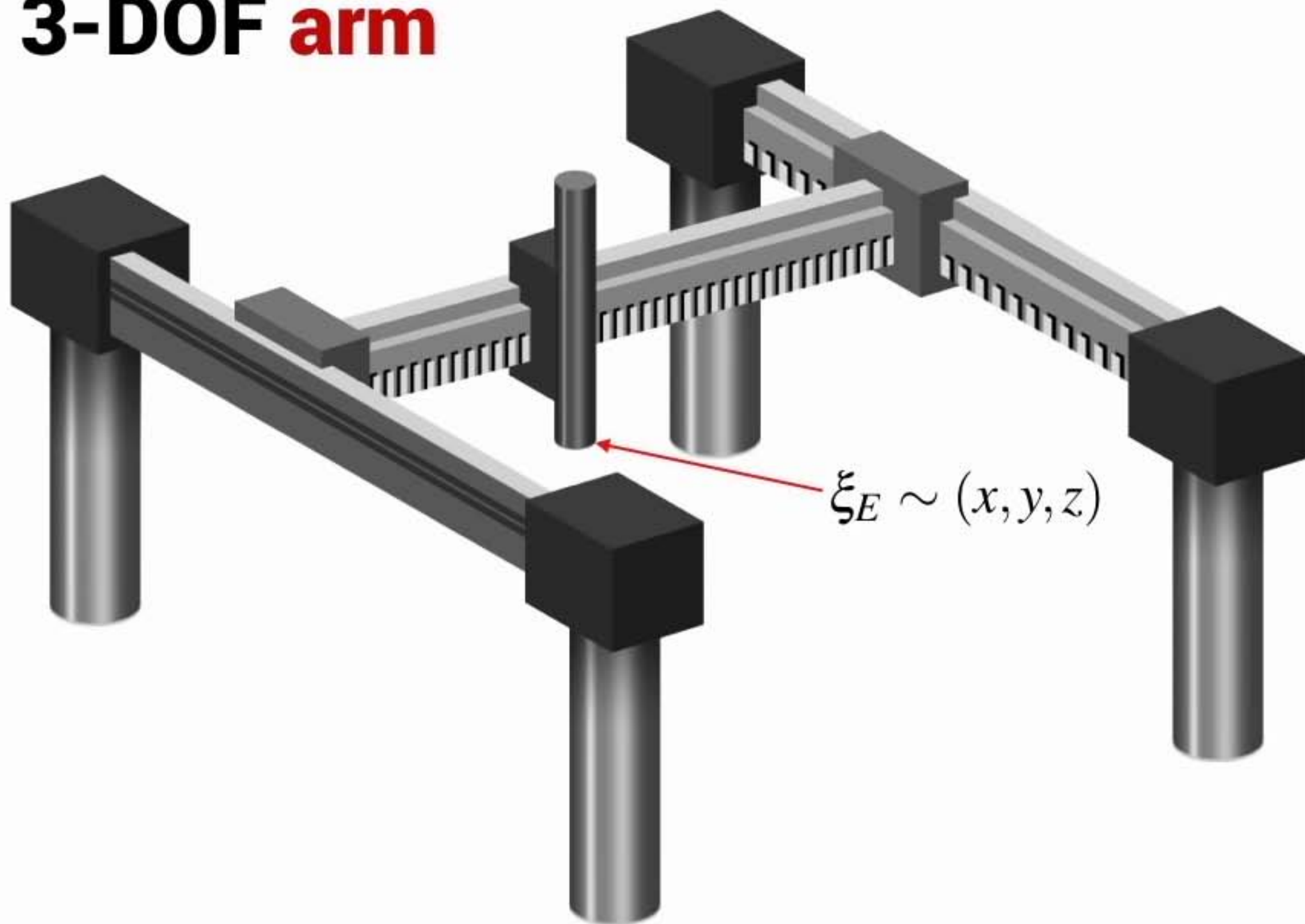
$$\xi_E \sim (x, y, z, \theta_p)$$

$$\xi_E \sim (x, y, z, \theta_r, \theta_p, \theta_y)$$

2-DOF arm



3-DOF arm



$$\xi_E \sim (x, y, z)$$

$$\mathcal{C} \subset \mathbb{R}^3$$

$$\mathcal{T} \subset \mathbb{R}^3$$

Gantry robot

4-DOF arm



$$\xi_E \sim (x, y, z, \theta_y)$$

$$\mathcal{C} \subset \mathbb{R}^4$$

$$\mathcal{T} \subset \mathbb{R}^3 \times \mathbb{S}$$

SCARA robot

Copyright © 2014 Adept Technology, Inc.
The usage of this image in no manner is to be construed as
Adept's endorsement of QUT or any QUT product or service

6-DOF **arm**

- Six joints is sufficient to achieve **any** position and orientation

$$\mathcal{C} \subset \mathbb{R}^6$$

$$\mathcal{T} \subset \mathbb{R}^3 \times \text{SO}(3)$$

$$\mathcal{T} \subset \text{SE}(3)$$



Used with permission of ABB



Stanford Robotic Arm
Oussama Khatib | Used with permission

$$\xi_E \sim (x, y, z, \theta_r, \theta_p, \theta_y)$$

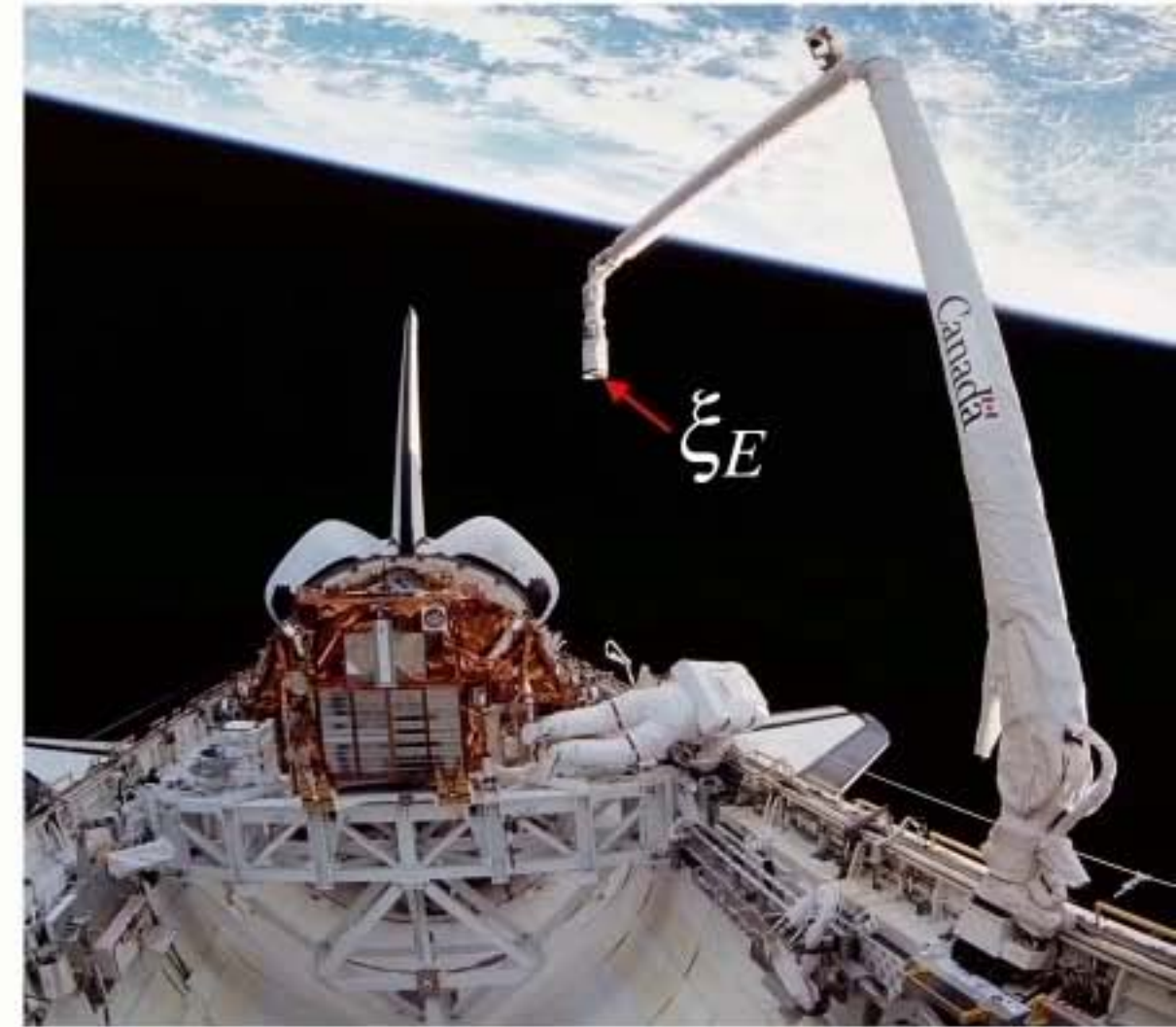
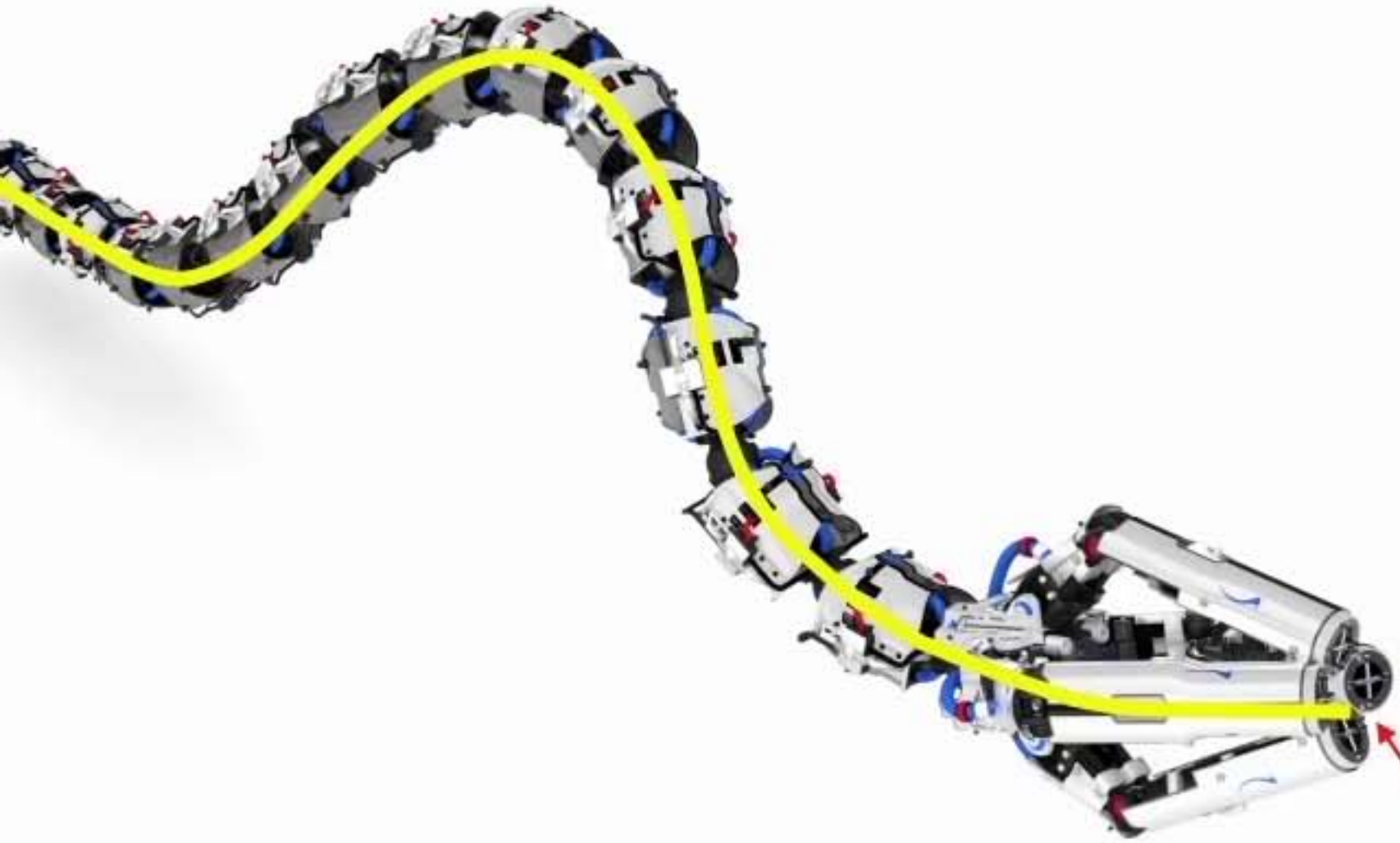


Image: NASA

Very high DOF **robot**



$$\mathcal{C} \subset \mathbb{R}^{100}$$

$$\mathcal{T} \subset \text{SE}(3)$$

configuration
space is much
bigger

task space is the
same as the Puma
robot

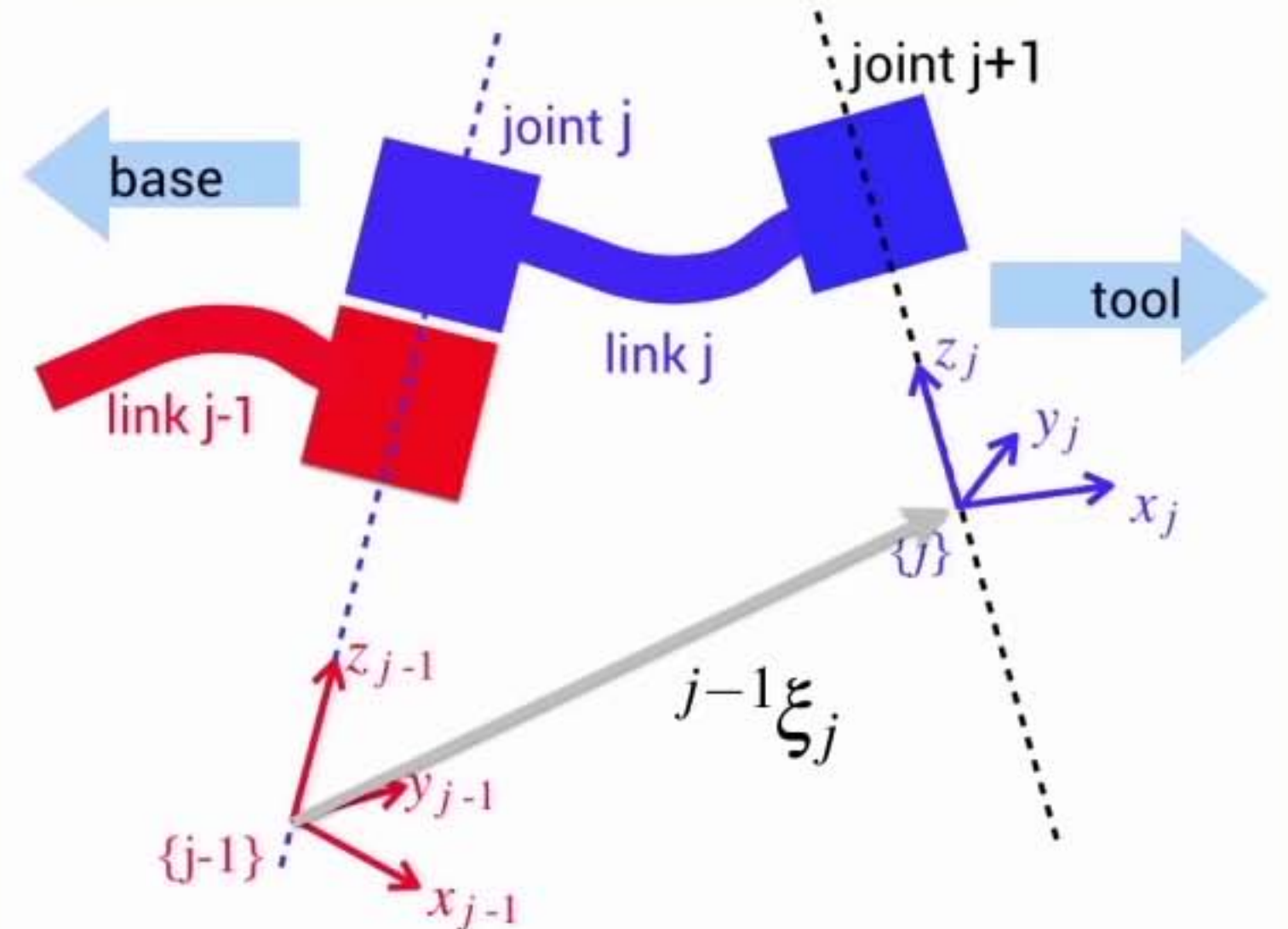
$$\xi_E \sim (x, y, z, \theta_r, \theta_p, \theta_y)$$

- Redundant robot
- Can achieve **any position and orientation** as well as controlling the shape of the arm

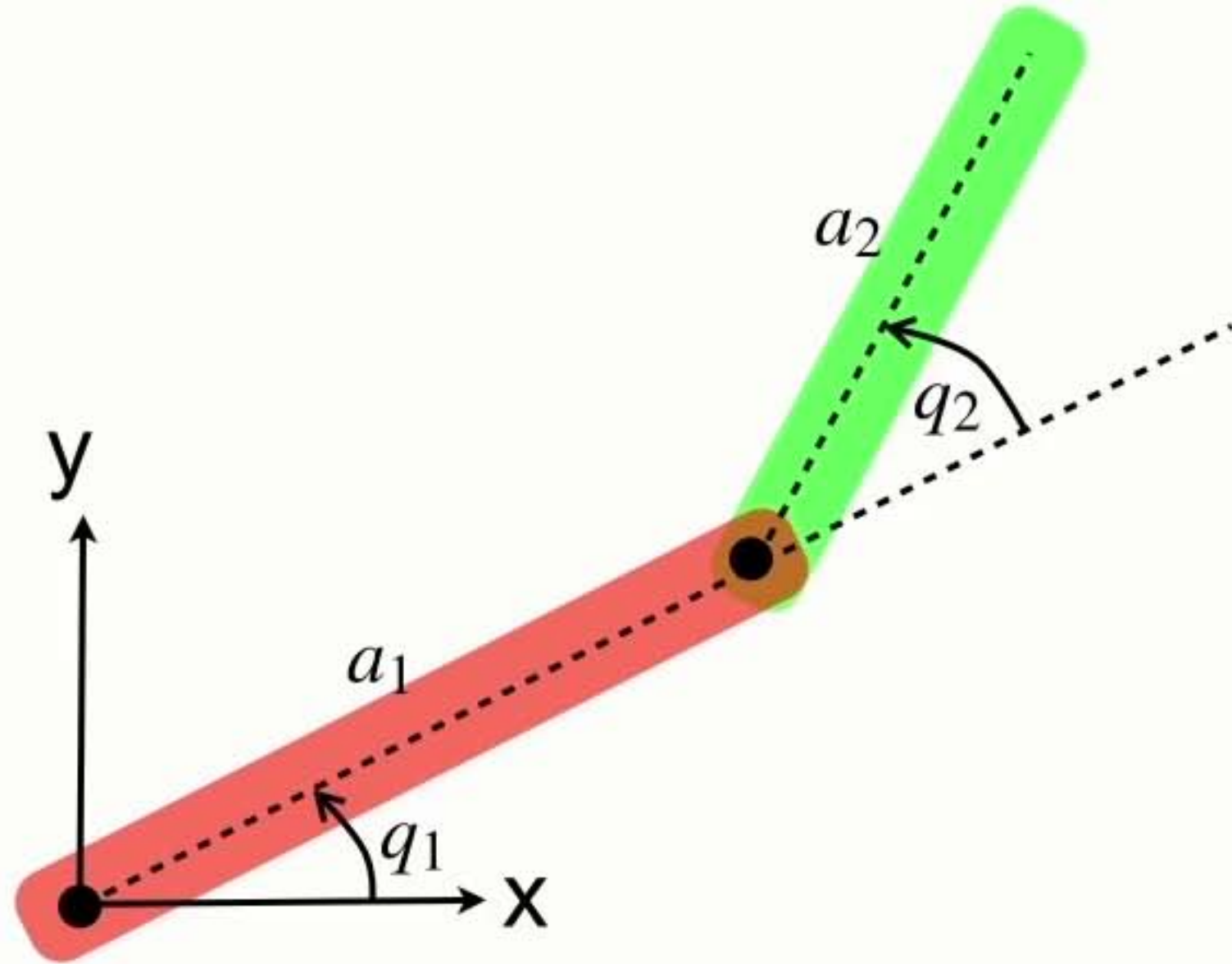
DH parameters

$${}^{j-1}\xi_j \sim \mathbf{A} = \mathbf{R}_z(\theta_j)\mathbf{T}_z(d_j)\mathbf{T}_x(a_j)\mathbf{R}_x(\alpha_j)$$

- Attach a coordinate frame to the far (distal) end of every link
- Describe the pose of a link frame with respect to the previous link frame
- Very concise
 - ➡ Only four parameters: θ , d , a , α



Denavit-Hartenberg parameters for **two-link**



θ_j	d_j	a_j	α_j
q_1	0	a_1	0
q_2	0	a_2	0

- This table completely describes the kinematics of the robot

Denavit-Hartenberg parameters for **Puma**



Unimate Puma 500
Oussama Khatib | Used with permission

θ_j	d_j	a_j	α_j
q_1	0.0000	0.0000	$\pi/2$
q_2	0.0000	0.4318	0
q_3	0.1500	0.0203	$-\pi/2$
q_4	0.4318	0.0000	$\pi/2$
q_5	0.0000	0.0000	$-\pi/2$
q_6	0.0000	0.0000	0

General **form**

$\xi_N = \mathcal{K}(q; \theta, d, a, \alpha, \sigma)$

if $\sigma_j = \begin{cases} R \rightarrow \theta_j = q_j \\ P \rightarrow d_j = q_j \end{cases}$

Diagram illustrating the general form of a kinematic chain mapping function, with callouts identifying the parameters:

- pose
- joint angles
- link offsets
- link lengths
- link twists
- joint types
- joint configuration

Stack the link transforms **end to end**

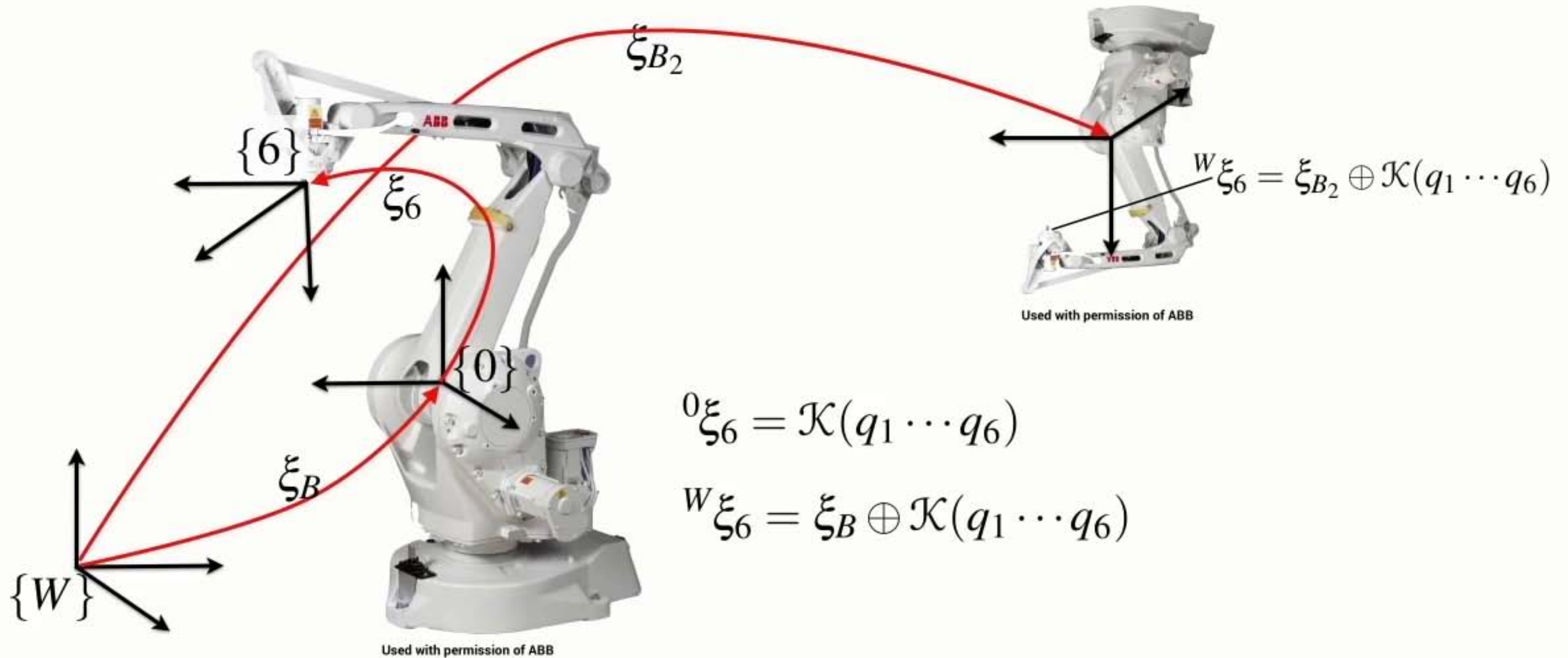
- All revolute joints (RRRRRRR)

$$\mathbf{T}_E = \underbrace{\mathbf{R}_z(\theta_1)\mathbf{T}_z(d_1)\mathbf{T}_x(a_1)\mathbf{R}_x(\alpha_1)}_{\text{revolute joint}} \underbrace{\mathbf{R}_z(\theta_2)\mathbf{T}_z(d_2)\mathbf{T}_x(a_2)\mathbf{R}_x(\alpha_2)}_{\text{revolute joint}} \cdots \underbrace{\mathbf{R}_z(\theta_N)\mathbf{T}_z(d_N)\mathbf{T}_x(a_N)\mathbf{R}_x(\alpha_N)}_{\text{revolute joint}}$$

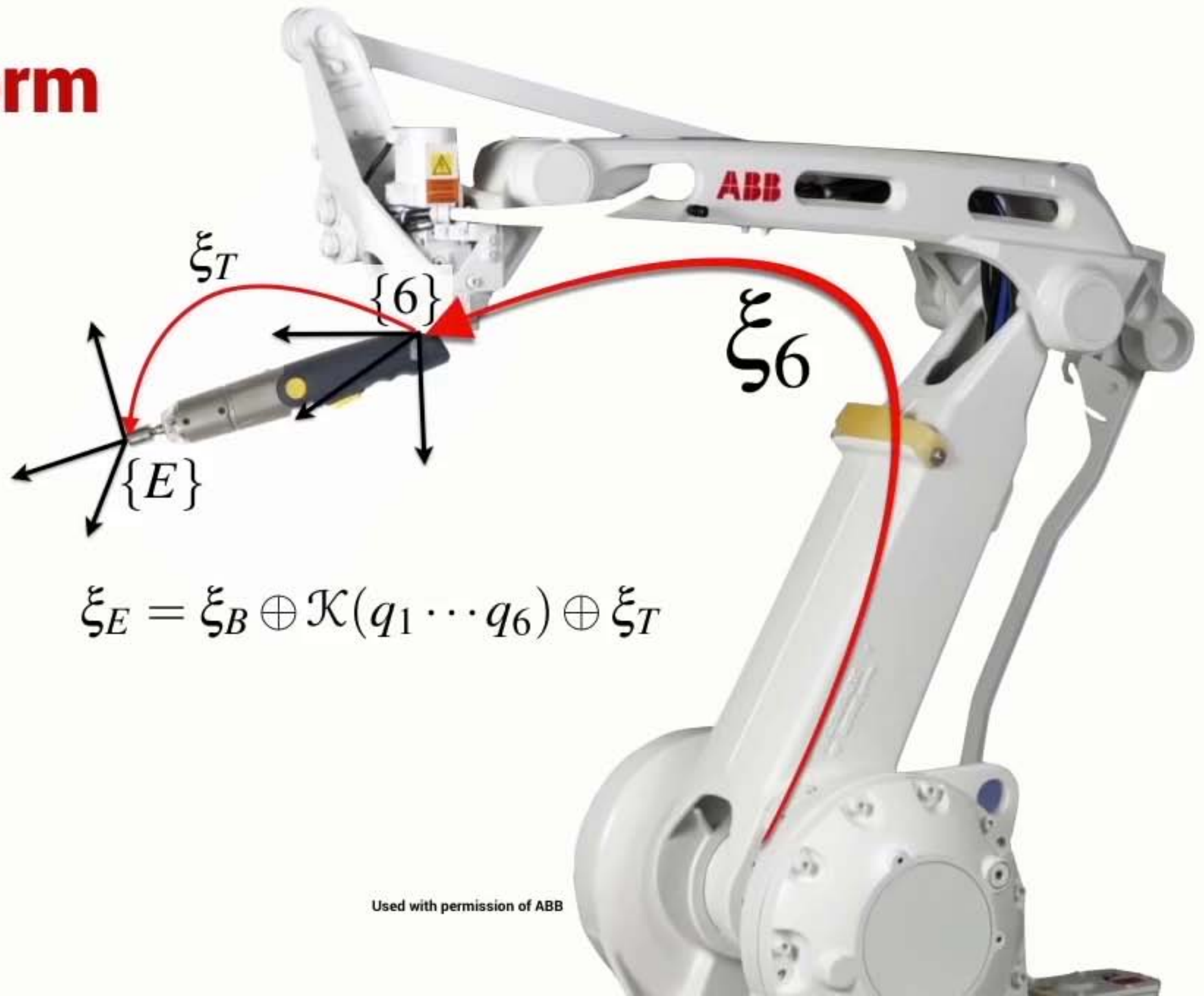
- Revolute and prismatic joints (RPRRRR)

$$\mathbf{T}_E = \underbrace{\mathbf{R}_z(\theta_1)\mathbf{T}_z(d_1)\mathbf{T}_x(a_1)\mathbf{R}_x(\alpha_1)}_{\text{revolute joint}} \underbrace{\mathbf{R}_z(\theta_2)\mathbf{T}_z(d_2)\mathbf{T}_x(a_2)\mathbf{R}_x(\alpha_2)}_{\text{prismatic joint}} \cdots \underbrace{\mathbf{R}_z(\theta_N)\mathbf{T}_z(d_N)\mathbf{T}_x(a_N)\mathbf{R}_x(\alpha_N)}_{\text{revolute joint}}$$

Base transform



Tool transform

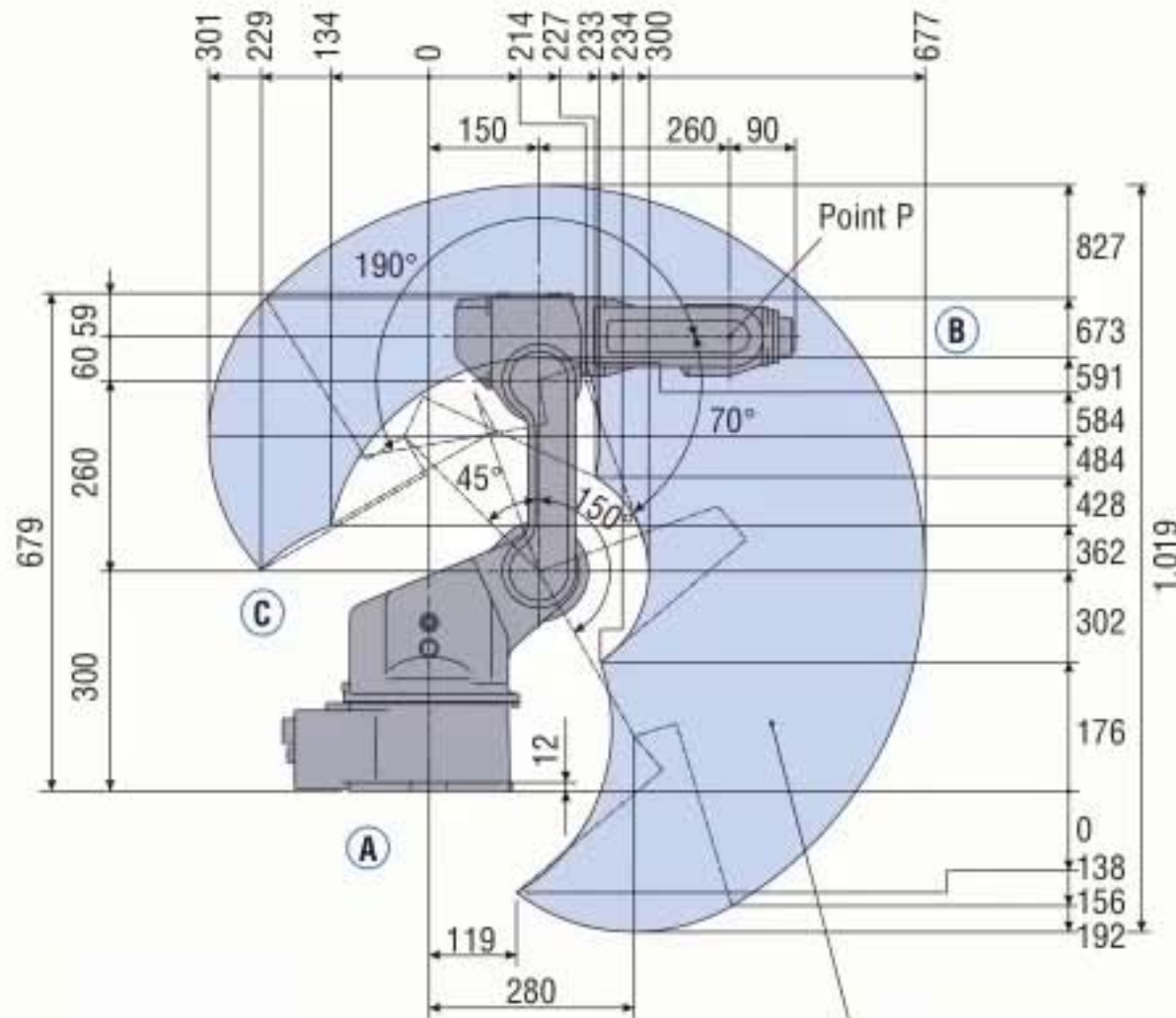


$$\xi_E = \xi_B \oplus \mathcal{K}(q_1 \cdots q_6) \oplus \xi_T$$

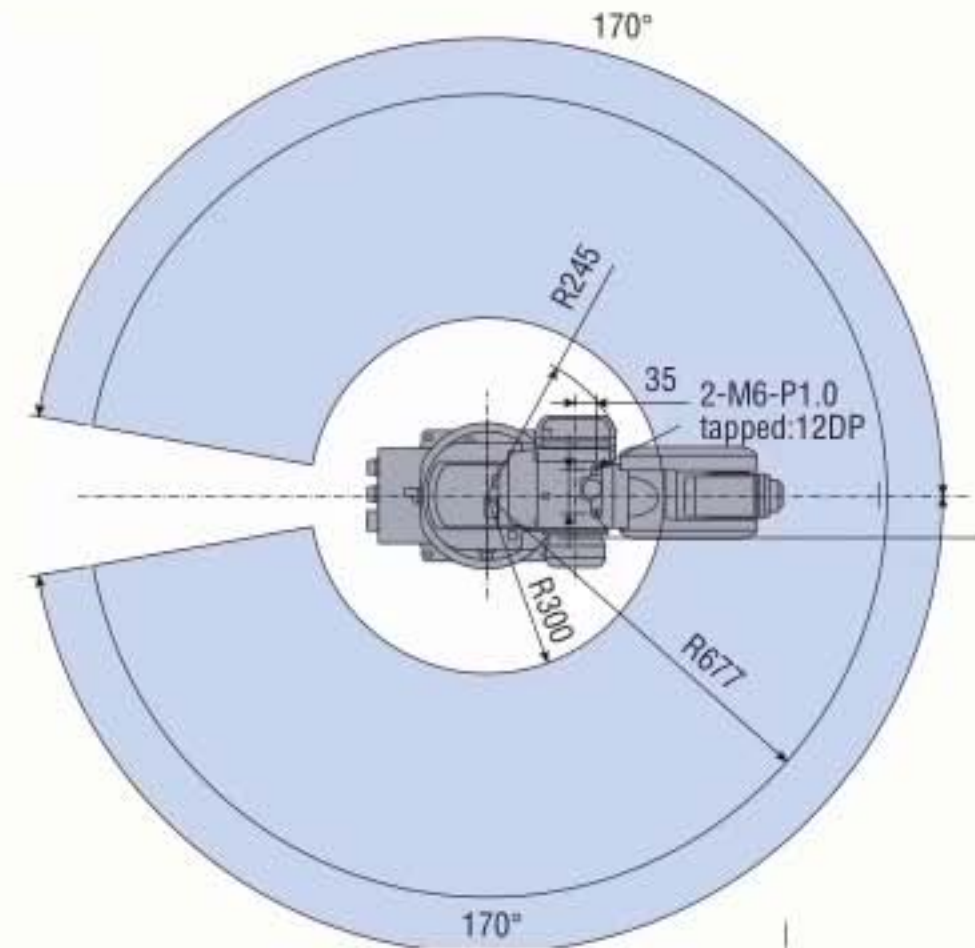
Used with permission of ABB

Workspace

- The set of all reachable end point positions



Yaskawa America, Inc., Motoman Robotics Division



Yaskawa America, Inc., Motoman Robotics Division

Yaskawa Motoman SV3X

Robot **Kinematics**

- We consider **two kinematic problems** in robotics
 - ➡ Forward kinematic problem
 - given the joint angles, where is the robot's tool tip?
 - ➡ Inverse kinematic problem
 - given the pose of the robot's tool tip, what joint angles are required?